

《最全面 C++ 面试题》 ---<套餐 6/7 学员专用>

必读说明：

(1).红色高亮：内容极为重要，务必掌握

(2).绿色高亮：内容重要，需要理解并应用

(3).无颜色：基础知识，了解即可

学习重点：

(1).语法：首要学习重点，掌握基本编程语法

(2).数据结构与算法：重要性最高，大厂必考

(3).计算机网络：理解网络概念和通信机制，结合项目灵活运用，重点明白各种协议的特点

(4).操作系统：重点关注进程和线程的相关原理，特别是生产者消费者模型

学习策略

为了有效地掌握知识，避免无效的死记硬背，老秦建议采取以下学习策略：

(1).理解为主，记忆为辅：在学习任何新概念时，首先理解其背后的原理和逻辑。只有深入理解了，记忆才能持久且有用（多问为什么）。

(2).逐层深入：从基础概念开始，逐步深入到复杂的知识点。不要试图一开始就解决所有难题，而是应循序渐进，逐步构建知识体系（很重要，构建整个计算机体系）。

(3).实践与应用：尽量将学到的理论知识应用于实际编程或项目中。通过实践，可以加深对知识的理解和记忆，同时发现并填补理解中的漏洞（只有在实际操作的时候，才会接触到一些细节点，以及会遇到的实际问题）。

(4).教与学结合：尝试将学到的内容用自己的话解释给他人听。教学可以帮助巩固和检验自己的理解，同时可能获得新的视角和反馈（能把学会的东西完整的讲出来，你才能让面试官明白你懂这些）。

(5).定期复习：定期回顾已学的内容，特别是重要的概念和原理。复习可以防止遗忘，加深印象（人是会遗忘的，需要重复记忆）。

(6).解决问题：面对问题时，先独立思考尝试解决，遇到难题不要急于求解答。探索解决问题的过程能够增强解决实际问题的能力（多思考，才会懂的更多更深层次）。

(7).利用多种资源：结合书籍、在线课程、视频教程、论坛和社群等多种学习资源。不同的视角和解释方式有助于全面理解复杂的概念（拒绝一成不变的方式）。

第一章 c++ 基础

1.C++三大特性？

图示（方便记忆）：

表. C++三大特性

性质	描述	访问修饰符	示例应用
封装	将数据（属性）和代码（方法）封装在一起，隐藏内部实现，提高安全性和数据保护。	public, private, protected	创建类时，用私有成员变量来保护数据，通过公有方法提供访问接口。
继承	允许新的类（派生类）继承现有类（基类）的属性和方法。可以重写或扩展基类功能。	使用 public, protected 继承	定义通用类（如车辆），然后定义特定的子类（如汽车、自行车）。
多态	允许同一接口呈现不同的行为。支持接口重用和行为修改，使得代码更加灵活。	主要通过虚函数和覆盖实现	定义一个基类接口（如绘图函数），让不同的派生类实现具体的绘图行为。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <vector>
using namespace std;
// 基类 Animal 使用封装来隐藏数据，使用多态来定义通用接口
class Animal {
protected:
    string name; // 封装：数据隐藏
    int age;
public:
    Animal(string n, int a) : name(n), age(a) {} // 构造函数初始化数据
    // 多态：使用虚函数定义接口以实现在派生类中的不同表现
    virtual void speak() const {
        cout << "Some animal sound" << endl;
    }
    virtual void info() const {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
    virtual ~Animal() {} // 虚析构函数以保证派生类对象的正确销毁;
};
// Dog 类继承自 Animal
class Dog : public Animal {
public:
    Dog(string n, int a) : Animal(n, a) {}
    void speak() const override {
        cout << "Woof!" << endl;
    }
    void info() const override {
        cout << "Dog -> ";
        Animal::info();
    }
};
// Cat 类继承自 Animal
```

```

class Cat : public Animal {
public:
    Cat(string n, int a) : Animal(n, a) {}
    void speak() const override {
        cout << "Meow!" << endl;
    }
    void info() const override {
        cout << "Cat -> ";
        Animal::info();
    }
}

int main() {
    vector<Animal*> zoo; // 存储不同类型的动物
    zoo.push_back(new Dog("Buddy", 5));
    zoo.push_back(new Cat("Whiskers", 3));
    for (auto animal : zoo) {
        animal->info(); // 多态调用
        animal->speak(); // 多态调用
    }
    // 清理内存
    for (auto animal : zoo) {
        delete animal;
    }
    zoo.clear();
    return 0;
}

```

2.C++中 struct 和 class 的区别？

图示（方便记忆）：

表. C++中 struct 和 class 对比表格

特性	Struct	Class
默认访问修饰符	默认公开 (public)	默认私有 (private)
继承用途功能兼容性	默认公开继承 (public) 传统用于数据结构简单对象 支持成员函数、继承、多态 与 C 语言结构兼容	默认私有继承 (private) 用于数据封装和行为的对象 支持成员函数、继承、多态 不与 C 语言结构兼容

总结（老秦推荐必看，方便理解）：

在 C++ 中，struct 和 class 非常相似，它们都可以用来定义包含数据和函数的类类型。主要区别在于默认访问修饰符和继承类型。struct 默认的成员访问权限和继承都是公开的，这使得它更适合用于那些主要用于数据存储的简单数据结构。相反，class 默认的成员访问和继承权限是私有的，这鼓励了更加严格的封装和抽象，使其成为定义复杂行为和交互的理想选择。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
```

```

struct Person {
    std::string name;
    int age;
    void display() {
        std::cout << "Name: " << name << ", Age: " << age << std::endl; }
};

class Person01 {
private:
    std::string name;
    int age;
public:
    void setName(std::string n) { name = n; }
    void setAge(int a) { age = a; }
    void display() {
        std::cout << "Name: " << name << ", Age: " << age << std::endl; }
};

int main() {
    Person person;
    person.name = "Alice";
    person.age = 30;
    person.display(); // 直接访问和调用
    Person01 person01;
    person01.setName("Bob");
    person01.setAge(25);
    person.display(); // 不能直接访问`name`和`age`, 必须通过公开的方法
    return 0;
}

```

3. 结构体 struct 和联合体 union 的区别

图示（方便记忆）：

表. 结构体 struct 和联合体 union 的区别表格

特性	struct	union
内存分配	每个成员都有自己的内存空间	所有成员共享同一块内存区域
大小	大小为所有成员的大小之和	大小为最大成员的大小
使用场景	存储不同的但相关的数据	存储多种类型的数据，但一次只用一种
访问	可以同时访问多个成员	只能访问当前存储的成员
数据类型	需要存储多种不同数据类型的结构	节省内存的场景，多个成员互斥使用

代码示例（老秦建议：自己动手实践一下）

```

#include <iostream>
#include <cstring>
struct StructExample {
    int intValue;
}

```

```

float floatValue;
char str[20];};
union UnionExample {
    int intValue;
    float floatValue;
    char str[20];};
int main() {
    StructExample s;
    s.intValue = 10;
    s.floatValue = 3.14;
    strcpy(s.str, "Hello");
    std::cout << "Struct Example:" << std::endl;
    std::cout << "intValue: " << s.intValue << std::endl;
    std::cout << "floatValue: " << s.floatValue << std::endl;
    std::cout << "str: " << s.str << std::endl;
    UnionExample u;
    u.intValue = 10;
    std::cout << "Union Example:" << std::endl;
    std::cout << "intValue: " << u.intValue << std::endl;
    u.floatValue = 3.14;
    std::cout << "floatValue: " << u.floatValue << std::endl;
    strcpy(u.str, "Hello");
    std::cout << "str: " << u.str << std::endl;
    return 0;
}

```

4.指针和引用的区别？

图示（方便记忆）：

表. 指针与引用对比表

特性	指针	引用
定义	存储另一个变量的内存地址的变量	另一个变量的别名，必须在定义时绑定且不能重新绑定
初始化	可以初始化为 nullptr 或其他指针的值	必须在定义时初始化，不能为 nullptr
多级性	可以形成多级指针（如 int **p）	不存在多级引用，因为它们不具有独立的内存地址
空值	可以是 nullptr，表示不指向任何地址	不能是 nullptr，且必须立即初始化
可变性	可以改变指向另一个不同的地址	一旦绑定某个变量，就不能改变
sizeof	得到的是指针本身的大小（通常为 4 或 8 字节）	得到的是引用所绑定的原始变量的大小

作为函数参数	传递地址，内部修改不会影响实参 除非操作指向的值	相当于直接操作实参，内部修改会影响实参
使用场景	动态内存管理、数组操作、指针算术等	函数参数传递（尤其是大型对象或不可复制对象）、多级间接引用等场景
安全性	灵活但容易出错（野指针、内存泄漏等）	通常更安全，限制性较强，避免了某些可能的错误操作

总结（老秦推荐必看，方便理解）：

指针：指针就像一把钥匙，它能够打开存储数据的房间（内存地址）。这把**钥匙（指针）**可以复制给其他人（**复制指针**），也可以**改变打开的房间（重新赋值指针）**。但是，如果不小心弄丢了这把钥匙，或者用它打开了错误的房间，就可能遇到麻烦（野指针或内存泄漏）。

引用：引用则像是一条秘密通道，一旦建立，就**永远指向一个特定的房间（变量）**，并且不能改变方向。这个通道非常安全，因为它确保了您总是访问正确的房间。但是，它必须在建造时就知道要通往哪里，且一旦建成，就不能改变路径（在定义时必须初始化且不能重新赋值）

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
int main() {
    int x = 10;
    int y = 20;
    // 指针的使用
    int* ptr = &x; // ptr 指向 x 的地址
    std::cout << "Pointer to x: " << *ptr << std::endl; // 输出 x 的值，应为 10
    ptr = &y; // 改变 ptr，使其指向 y
    std::cout << "Pointer now points to y: " << *ptr << std::endl; // 输出 y 的值，应为 20
    // 引用的使用
    int& ref = x; // ref 是 x 的引用
    std::cout << "Reference to x: " << ref << std::endl; // 输出 x 的值，应为 10
    ref = y; // 不是改变引用的指向，而是将 y 的值赋给 x
    std::cout << "Reference after assigning y to ref: " << ref << std::endl; // 输出 x 的新值，应为 20
    std::cout << "Value of x after ref assigned y: " << x << std::endl; // 再次输出 x 的值，确认其变为 20
    return 0;
}
```

5. 空类能实例化吗？

总结（老秦推荐必看，方便理解）：

是的，**空类可以实例化**。尽管空类没有任何成员变量或成员函数，但它仍然可以实例

化。每个实例会占用最小的内存（通常是 1 字节），以确保每个对象有一个唯一的地址。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class Empty {};
int main() {
    Empty e1, e2;
    std::cout << "Size of empty class: " << sizeof(Empty) << std::endl;
    std::cout << "Addresses: " << &e1 << " " << &e2 << std::endl;
    return 0;
}
```

6. 当一个类中没有任何成员变量和成员函数，这时 sizeof(A) 的 A 值是多少？

总结（老秦推荐必看，方便理解）：

其实就是空类，即使一个类中没有任何成员变量和成员函数，其 sizeof（）值也为 1。这是因为编译器必须为每个对象分配一个独特的地址，以便在程序中区分不同的对象实例，因此会占用 1 字节的空间。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class A {};
int main() {
    std::cout << "Size of empty class A: " << sizeof(A) << std::endl;
    return 0;
}
```

7. 子类不能继承父类的函数有哪些？

图示（方便记忆）：

表. 子类不能继承的父类函数表格

函数类型	描述
构造函数	子类不能继承父类的构造函数
析构函数	子类不能继承父类的析构函数
拷贝构造函数	子类不能继承父类的拷贝构造函数
赋值运算符	子类不能继承父类的赋值运算符
私有成员函数	子类不能继承父类的私有成员函数

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
```



```

class Base {
private:
    void privateFunction() {
        std::cout << "Base private function" << std::endl;
    }
public:
    Base() { std::cout << "Base constructor" << std::endl; }
    Base(const Base&) { std::cout << "Base copy constructor" << std::endl; }
    Base& operator=(const Base&) {
        std::cout << "Base assignment operator" << std::endl;
        return *this;
    }
    ~Base() { std::cout << "Base destructor" << std::endl; };
}
class Derived : public Base {
public:
    Derived() { std::cout << "Derived constructor" << std::endl; }
    Derived(const Derived& d) : Base(d) { std::cout << "Derived copy
constructor" << std::endl; }
    Derived& operator=(const Derived& d) {
        Base::operator=(d);
        std::cout << "Derived assignment operator" << std::endl;
        return *this;
    }
    ~Derived() { std::cout << "Derived destructor" << std::endl; };
}
int main() {
    Derived d1;
    Derived d2 = d1;
    d1 = d2;
    return 0;
}

```

8.C++类构造函数初始化列表执行顺序？

总结（老秦推荐必看，方便理解）：

执行顺序

- 基类构造函数：

- 首先调用基类的构造函数（如果存在）。
- 如果有多个基类，则按照它们在继承列表中的顺序依次调用。

- 成员变量初始化：

- 按照成员变量在类中声明的顺序进行初始化，而不是按照它们在初始化列表中的顺序。
- 这种顺序确保所有依赖关系都能正确处理，避免未初始化的成员被使用。

- 构造函数主体：

- 最后执行派生类构造函数的主体部分。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class Base {
public:
    Base() {
        std::cout << "Base constructor called" << std::endl;};
class Member {
public:
    Member() {
        std::cout << "Member constructor called" << std::endl;};
class Derived : public Base {
private:
    Member member1;
    Member member2;
public:
    Derived() : member1(), member2() {
        std::cout << "Derived constructor called" << std::endl;};
int main() {
    Derived d;
    return 0;}
```

解释：

基类构造函数：

在执行派生类 Derived 的构造函数之前，首先调用基类 Base 的构造函数，输出"Base constructor called"。

成员变量初始化：

按照声明顺序初始化成员变量 member1 和 member2。尽管在初始化列表中顺序不同，但成员变量的初始化顺序固定为声明顺序，因此先调用 Member 的构造函数初始化 member1，输出"Member constructor called"，再初始化 member2，输出同样的消息。

构造函数主体：

最后执行派生类 Derived 的构造函数主体部分，输出"Derived constructor called"。

9.构造函数的顺序，析构函数的顺序？

总结（老秦推荐必看，方便理解）：

- 构造函数的顺序

- 基类构造函数：首先调用基类的构造函数。

- 成员变量初始化：按照成员变量在类中声明的顺序进行初始化。
- 构造函数主体：最后执行派生类构造函数的主体部分。
- 析构函数的顺序
 - 析构函数主体：首先执行派生类析构函数的主体部分。
 - 成员变量销毁：按成员变量在类中声明的顺序的反向销毁成员变量。
 - 基类析构函数：最后调用基类的析构函数。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class Base {
public:
    Base() {
        std::cout << "Base constructor" << std::endl;
    }
    ~Base() {
        std::cout << "Base destructor" << std::endl;
    }
};
class Member {
public:
    Member() {
        std::cout << "Member constructor" << std::endl;
    }
    ~Member() {
        std::cout << "Member destructor" << std::endl;
    }
};
class Derived : public Base {
public:
    Member m1;
    Member m2;
    Derived() {
        std::cout << "Derived constructor" << std::endl;
    }
    ~Derived() {
        std::cout << "Derived destructor" << std::endl;
    }
};
int main() {
    Derived d;
    return 0;
}
```

运行结果

```
Base constructor
Member constructor
Member constructor
Derived constructor
Derived destructor
Member destructor
Member destructor
Base destructor
```

10.c++有哪些构造函数呢？

图示（方便记忆）：

表. 构造函数类型表格

函数类型	描述
默认构造函数	无参数的构造函数
参数化构造函数	接受参数的构造函数
拷贝构造函数	使用已有对象初始化新对象
移动构造函数	从临时对象初始化新对象
委托构造函数	一个构造函数调用另一个
显式构造函数	防止隐式类型转换和复制初始化

代码示例（老秦建议：自己动手实践一下）

```
class MyClass {
public:
    int x, y;
    int* data;
    // 默认构造函数
    MyClass() {
        // 默认构造函数
    }
    // 参数化构造函数
    MyClass(int val) : x(val) {
        // 参数化构造函数
    }
    // 拷贝构造函数
    MyClass(const MyClass& other) : x(other.x), data(new int[*other.data])
    {
        // 拷贝构造函数
    }
    // 移动构造函数
    MyClass(MyClass&& other) noexcept : data(other.data) {
        other.data = nullptr;
        // 移动构造函数
    }
    // 委托构造函数
    MyClass(int val1, int val2) : x(val1), y(val2) {
        // 参数化构造函数
    }
    MyClass(int val) : MyClass(val, 0) {
        // 委托构造函数
    }
    // 显式构造函数
    explicit MyClass(int val) : x(val) {
        // 显式构造函数
    }
    ~MyClass() {
        delete[] data;
    }
};
```

11.成员变量初始化顺序？

总结（老秦推荐必看，方便理解）：

成员变量的初始化顺序是按照它们在类中声明的顺序，而不是初始化列表中的顺序。编译器会按照声明顺序依次初始化成员变量。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class Example {
    int a;
    int b;
public:
    Example(int x, int y) : b(y), a(x) {
        std::cout << "a: " << a << ", b: " << b << std::endl;
    }
};
int main() {
    Example ex(1, 2);
    return 0;
}
```

12.全局变量和局部变量的区别？

图示（方便记忆）：

表. 全局变量和局部变量的对比表格

特性	全局变量	局部变量
作用范围	整个程序范围内有效	定义所在的代码块内有效
生命周期	程序开始到结束	所在代码块执行时生效，结束后失效
存储位置	通常存储在数据段	通常存储在栈上
初始化值	默认为零值	未初始化时为未定义值

总结（老秦推荐必看，方便理解）：

全局变量在程序的整个生命周期内有效，适用于需要在多个函数间共享数据的场合。局部变量在其定义的代码块内有效，生命周期较短，适用于临时数据存储。尽量减少全局变量的使用，以避免命名冲突和意外修改。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
int globalVar = 10; // 全局变量
void function() {
    int localVar = 20; // 局部变量
    std::cout << "Local Variable: " << localVar << std::endl;
}
int main() {
    std::cout << "Global Variable: " << globalVar << std::endl;
    function();
    return 0;
}
```

13. 常量指针和指针常量区别？

图示（方便记忆）：

表. 常量指针和指针常量区别

类型	描述	示例
常量指针	指针指向的数据不能被修改，但指针本身可以指向另一个地址。	<code>const int* ptr;</code> 指向整数的常量指针。
指针常量	指针本身的地址不能被修改，但它指向的数据内容可以被修改。	<code>int* const ptr;</code> 常量指针，指向整数。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
int main() {
    int x = 10;
    int y = 20;
    // 常量指针：指针指向的数据不能改变，但指针可以指向别处
    const int* ptr1 = &x;
    std::cout << "ptr1 points to x: " << *ptr1 << std::endl;
    ptr1 = &y; // 允许
    // *ptr1 = 25; // 错误：不能通过 ptr1 修改所指向的数据
    // 指针常量：指针指向的地址不能改变，但其指向的数据可以修改
    int* const ptr2 = &x;
    std::cout << "ptr2 points to x: " << *ptr2 << std::endl;
    *ptr2 = 30; // 允许
    // ptr2 = &y; // 错误：不能改变 ptr2 的指向
    std::cout << "After changes:" << std::endl;
    std::cout << "x: " << x << std::endl; // 输出 30 显示通过 ptr2 修改了 x 的值
    std::cout << "y: " << y << std::endl; // 输出 20，y 未改变
    return 0;}
```

14. 智能指针有什么缺点？

总结（老秦推荐必看，方便理解）：

虽然智能指针极大地简化了内存管理，但它们也带来了性能开销、循环引用导致内存泄漏以及代码复杂性增加的问题。在使用智能指针时，需要注意防止循环引用，理解其性能影响，并在必要时进行适当的优化。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <memory>
struct Node {
```

```

std::shared_ptr<Node> next;
~Node() { std::cout << "Node destroyed" << std::endl; };
int main() {
    std::shared_ptr<Node> node1 = std::make_shared<Node>();
    std::shared_ptr<Node> node2 = std::make_shared<Node>();
    node1->next = node2;
    node2->next = node1; // 循环引用导致内存泄漏
    return 0;}

```

15.shared_ptr 是不是线程安全的？

图示（方便记忆）：

表. shared_ptr 线程安全性表格

特性	描述
引用计数线程安全	对引用计数的操作是线程安全的
对象本身非线程安全	被管理对象本身不是线程安全的
读写操作	读写操作需要额外的同步机制
使用场景	适用于多线程共享同一对象的场景

总结（老秦推荐必看，方便理解）：

shared_ptr 的引用计数操作是线程安全的，确保引用计数的增减不会引发竞争条件。然而，shared_ptr 所管理的对象本身并不是线程安全的，读写操作需要额外的同步机制来保证线程安全。

代码示例（老秦建议：自己动手实践一下）

```

#include <iostream>
#include <memory>
#include <thread>
#include <vector>
void threadFunction(std::shared_ptr<int> ptr) {
    std::cout << "Thread: " << std::this_thread::get_id() << ", Value: " <<
*ptr << std::endl;}
int main() {
    std::shared_ptr<int> sharedPtr = std::make_shared<int>(100);
    std::vector<std::thread> threads;
    for (int i = 0; i < 10; ++i) {
        threads.push_back(std::thread(threadFunction, sharedPtr));}
    for (auto& th : threads) {
        th.join();}
    return 0;}

```

16.什么时候用智能指针？什么时候用裸指针？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)智能指针：

- 1) 适用于需要自动管理资源生命周期的场景。
- 2) 适用于需要防止内存泄漏的场景。
- 3) 适用于需要管理复杂对象的所有权和生命周期的场景。
- 4) 适用于需要确保异常情况下资源能被正确释放的场景。

(2)裸指针：

- 1) 适用于简单的、临时的、不涉及所有权的对象。
- 2) 适用于对性能开销要求高的场景。
- 3) 适用于不需要管理资源生命周期的场景。

17.shared_ptr与weak_ptr的区别与联系？

图示（方便记忆）：

表. shared_ptr与weak_ptr对比表格

特性	shared_ptr	weak_ptr
所有权	拥有资源的共享所有权	不拥有资源的所有权
引用计数	增加引用计数	不增加引用计数

生命周期控制	控制资源的生命周期	不控制资源的生命周期
循环引用	可能导致循环引用导致内存泄漏	可以打破循环引用，防止内存泄漏

总结（老秦推荐必看，方便理解）：

shared_ptr 拥有资源的共享所有权并控制资源的生命周期，增加引用计数，而 weak_ptr 仅提供对资源的弱引用，不控制资源的生命周期，也不增加引用计数。weak_ptr 常用于打破循环引用，防止内存泄漏。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <memory>
class Node {
public:
    std::shared_ptr<Node> next;
    ~Node() { std::cout << "Node destroyed" << std::endl; };
void createCycle() {
    auto node1 = std::make_shared<Node>();
    auto node2 = std::make_shared<Node>();
    node1->next = node2;
    node2->next = node1; // 循环引用导致内存泄漏}
void createCycleWithWeakPtr() {
    auto node1 = std::make_shared<Node>();
    auto node2 = std::make_shared<Node>();
    node1->next = node2;
    node2->next = std::weak_ptr<Node>(node1); // 打破循环引用}
int main() {
    createCycle();
    createCycleWithWeakPtr();
    return 0;}
```

18. 引用作为函数参数在 C++ 中有以下几个好处？

图示（方便记忆）：

表. 引用作为函数参数的好处表格

特性	描述
避免拷贝开销	直接使用对象，避免了不必要的拷贝操作
支持修改参数	通过引用可以修改函数参数的值
一致性	传递引用与直接使用对象在语法上保持一致
支持大对象传递	对于大对象，使用引用传递效率更高
安全性	避免空指针问题，提高代码的安全性

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <vector>
#include <string>
// 避免拷贝开销和支持大对象传递
void processLargeVector(const std::vector<int>& vec) {
    std::cout << "Vector size: " << vec.size() << std::endl;
}
// 支持修改参数
void updateValue(int& x) {
    x = 42;
}
// 一致性和安全性
void printValue(const int& x) {
    std::cout << "Value: " << x << std::endl;
}
int main() {
    int a = 10;
    std::cout << "Original value: " << a << std::endl;
    updateValue(a);
    std::cout << "Updated value: " << a << std::endl;
    printValue(a);
    std::vector<int> largeVector(1000000, 1);
    processLargeVector(largeVector); // 使用引用避免拷贝大对象
    std::string largeString(1000000, 'a');
    printValue(largeString.length()); // 打印大字符串的长度
    return 0;
}
```

19.模板是在什么时候进行实例化？

图示（方便记忆）：

表. 模板实例化时机表格

情况	描述
编译时实例化	模板在编译器遇到模板的具体使用时实例化，如调用或使用
显式实例化	开发者可以显式指定模板实例化
链接时	链接时确保所有模板实例化的定义都可见

总结（老秦推荐必看，方便理解）：

模板在编译时实例化，当编译器遇到模板的具体使用时，例如调用模板函数或创建模板类实例时，编译器会生成相应的代码。这一过程称为模板的实例化。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
template <typename T>
class MyClass {
public:
    T data;
    MyClass(T d) : data(d) {}
    void display() const {
        std::cout << "Data: " << data << std::endl;
    }
};

int main() {
    MyClass<int> obj1(42); // 编译时实例化
    MyClass<double> obj2(3.14);
    obj1.display();
    obj2.display();
    return 0;
}
```

20.为什么模板类一般都是放在一个 h 文件中？

图示（方便记忆）：

表. 模板实例化时机表格

特性	描述
编译时实例化	模板的实例化发生在编译时，编译器需要看到模板的定义和实现
链接问题	如果模板定义和实现分开，编译器可能无法在链接时找到实现

总结（老秦推荐必看，方便理解）：

(1)模板类在编译期实例化：模板类的实例化发生在编译期，编译器需要在实例化

模板时看到模板的完整定义和实现。

(2)头文件包含完整定义和实现：将模板类的定义和实现都放在头文件中，确保编译器在实例化模板时能访问到完整信息，避免链接错误。

(3)避免链接错误：如果将模板类的实现放在源文件中，可能会导致链接错误，因为编译器在链接阶段无法找到模板实例化的具体实现。

21.define 宏定义和 const , static 的区别

图示（方便记忆）：

表. define 宏定义和 const , static 对比表格

特性	define 宏定义	const	static
类型检查	不进行类型检查	进行类型检查	进行类型检查
作用范围	全局	作用域内有效	作用域内有效
存储类型	编译时替换文本	编译时常量	静态存储，生命周期贯穿整个程序
调试支持	调试困难，无法设置断点	可以设置断点	可以设置断点
使用方式	定义简单常量或复杂表达式	定义常量(或者表示为只读)	定义静态变量或静态成员变量

总结（老秦推荐必看，方便理解）：

在 C++ 中，define 宏定义、const 和 static 各有其用途和特点。**define 宏定义通过预处理器在编译时进行简单的文本替换，缺乏类型检查，适用于定义简单的常量或宏。**const 用于定义常量，能够进行类型检查，具有更高的安全性。static 用于定义静态变量或成员，具有文件内或函数内的局部作用域，并在程序的生命周期内保持值的持久性。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#define PI 3.14159 // 宏定义
const int MAX_SIZE = 100; // 常量定义
static int counter = 0; // 静态变量定义
void incrementCounter() {
    static int localCounter = 0; // 静态局部变量
    localCounter++;
    counter++;
    std::cout << "Local Counter: " << localCounter << ", Global Counter: "
    << counter << std::endl;
}
int main() {
    std::cout << "PI: " << PI << std::endl; // 使用宏定义
    std::cout << "Max Size: " << MAX_SIZE << std::endl; // 使用常量
    incrementCounter();
}
```

```
incrementCounter();
incrementCounter();
return 0;}
```

22.new 和 malloc 申请的是哪里的内存？如何减少内存碎片？

图示（方便记忆）：

表. new 和 malloc 对比表格

特性	new	malloc
内存分配位置	堆区	堆区
初始化	调用构造函数进行初始化	不调用构造函数，需手动初始化
返回类型	返回指定类型的指针	返回 void*，需强制转换类型
异常处理	分配失败时抛出异常	分配失败时返回 NULL

总结（老秦推荐必看，方便理解）：

new 和 malloc 都在堆区分配内存，但 new 会调用构造函数进行初始化，并在分配失败时抛出异常，而 malloc 不调用构造函数，需手动初始化，并在分配失败时返回 NULL。减少内存碎片的策略包括：使用内存池、按需分配大块内存、避免频繁的分配和释放操作、合并相邻的空闲块。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
int main() {
    // 使用 new 分配内存
    int* p1 = new int(5);
    std::cout << "Value allocated with new: " << *p1 << std::endl;
    delete p1;
    // 使用 malloc 分配内存
    int* p2 = (int*)malloc(sizeof(int));
    if (p2 != nullptr) {
        *p2 = 10;
        std::cout << "Value allocated with malloc: " << *p2 << std::endl;
        free(p2);}
    else {
        std::cerr << "Memory allocation failed" << std::endl;}
    return 0;}
```

23.实际情况中想用 new 分配内存但又不确定大小怎么做？

总结（老秦推荐必看，方便理解）：

(1)确定最大可能的大小：预先确定一个最大可能的大小，然后用 new 分配内存。

(2)动态调整大小：使用 std::vector 或 std::unique_ptr 等动态容器，根据需要动态调整内存大小。

(3)使用类型擦除和多态：使用基类指针指向派生类对象，动态确定对象的实际类型和大小。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
void allocateMemory(int numElements) {
    if (numElements > 0) {
        int* array = new int[numElements];
        // 使用分配的内存
        for (int i = 0; i < numElements; ++i) {
            array[i] = i;
        }
        // 打印数组
        for (int i = 0; i < numElements; ++i) {
            std::cout << array[i] << " ";
        }
        std::cout << std::endl;
        // 释放内存
        delete[] array;
    }
    else {
        std::cout << "Invalid number of elements." << std::endl;
    }
}
int main() {
    int numElements;
    std::cout << "Enter number of elements: ";
    std::cin >> numElements;
    allocateMemory(numElements);
    return 0;
}
```

24.堆和栈的区别？

表. 堆与栈对比表

特性	堆 (Heap)	栈 (Stack)
内存管理	动态分配和管理。由程序员或垃圾回收机制管理。	自动管理。函数调用时自动分配，函数返回时自动释放。
存储内容	主要用于存储动态分配的对象和数据。	主要用于存储局部变量和函数调用的执行环境。

控制方式	程序员控制内存的申请和释放（或由垃圾回收机制自动管理）。	自动由编译器控制，无需程序员手动干预。
大小和扩展性	通常较大，可以动态扩展。但扩展性和限制依赖于操作系统。	大小固定，由操作系统或编程环境预设。
访问速度	相较于栈访问速度较慢。	访问速度快，因为数据通常都位于连续的内存块中。
成本	管理成本高，可能涉及复杂的内存管理和碎片整理。	管理成本低，自动处理内存分配和回收。
安全性	使用不当可能导致内存泄漏、碎片等问题。	较为安全，但可能会遇到栈溢出的问题。
生命周期	对象可以在任何时刻被创建和销毁，生命周期不固定。	对象的生命周期通常随着函数的执行开始和结束。
使用场景	适用于程序运行时需要动态分配大量内存的情况。	适用于存储函数的返回地址、参数和局部变量。

总结（老秦推荐必看，方便理解）：

堆 (Heap)：想象堆是一个繁忙的港口，船只（程序员或垃圾回收机制）不断地将货物（动态分配的对象和数据）运来运去。港口的仓库空间（内存）很大，但需要船只自己找到空位（内存地址）存放货物，并且记住位置（手动管理内存）。港口的规模（内存大小）可以根据需要进行扩展，但船只需要自己注意不要堵塞航道（内存泄漏）。

栈 (Stack)：把栈想象成一家自动化的快餐店，顾客（函数调用）到来时，服务员（编译器）会立即提供一个干净的桌子（自动分配的内存）来放置食物（局部变量和执行环境）。顾客离开时，服务员会自动清理桌子，准备给下一位顾客使用（自动释放内存）。快餐店的空间（栈的大小）是固定的，一旦坐满，就不能再接待更多顾客（栈溢出）。

25.C++栈溢出问题？

图示（方便记忆）：

表. C++栈溢出情况表格

情况	描述	例子
递归调用过深	函数递归调用过多次，超出栈的容量	计算斐波那契数列，递归无基准条件
分配过大的局部变量	局部变量（如数组）过大，超过栈的容量	分配一个非常大的局部数组
无限递归	函数递归调用自身且无终止条件	函数不断调用自身，无基准条件
过深的嵌套调用	函数嵌套调用层级过深，超出栈的容量	多层嵌套函数调用

总结（老秦推荐必看，方便理解）

C++中的栈溢出通常由递归调用过深、分配过大的局部变量、无限递归或过深的嵌套调用引起。这些情况会导致栈空间用尽，程序崩溃。

详细解释和示例代码

(1) 递归调用过深

每次递归调用都会在栈上分配一个新的栈帧，包含函数的返回地址、局部变量和参数。当递归层级太深时，栈帧会不断累积，直到栈空间用尽，导致栈溢出。

示例代码

```
#include <iostream>
void deepRecursion(int n) {
    if (n == 0) {
        return;
    }
    else {
        deepRecursion(n - 1);
    }
}
int main() {
    deepRecursion(100000); // 递归层级过深，可能导致栈溢出
    return 0;
}
```

(2) 分配过大局部变量

在栈上分配过大的局部变量（如数组）会占用大量的栈空间，导致栈空间不足，发生栈溢出。

示例代码

```
#include <iostream>
void largeArray() {
    int arr[1000000]; // 分配过大的局部数组，可能导致栈溢出
    arr[0] = 1; // 使用数组，防止编译器优化
    std::cout << "Array allocated." << std::endl;
}
int main() {
    largeArray();
    return 0;
}
```

(3) 无限递归

无限递归是函数不断调用自身且没有终止条件的情况，每次调用都会分配新的栈帧，最终导致栈空间用尽，发生栈溢出。

示例代码

```
#include <iostream>
void infiniteRecursion() {
    infiniteRecursion(); // 无限递归调用，导致栈溢出
}
int main() {
    infiniteRecursion();
    return 0;
}
```


(4) 过深的嵌套调用

过深的嵌套调用是指函数调用层级过深，每次调用都会栈上分配一个新的栈帧，最终导致栈空间用尽，发生栈溢出。

示例代码

```
#include <iostream>
void nestedCall(int depth) {
    if (depth == 0) {
        return;
    }
    else {
        nestedCall(depth - 1); // 嵌套调用层级过深，可能导致栈溢出
    }
}
int main() {
    nestedCall(100000); // 调用层级过深，导致栈溢出
    return 0;
}
```

25.1 如何防止栈溢出？

表. 防止栈溢出的措施表格

方法	描述
限制递归深度	在递归函数中添加基准条件，限制递归深度
使用堆分配大数据	对于大数组或大对象，使用 new 在堆上分配内存
优化递归	使用尾递归优化或将递归改为迭代，减少栈帧的使用
检查嵌套调用	避免过深的嵌套调用，合理设计函数调用关系

示例代码

```
#include <iostream>
// 限制递归深度
void limitedRecursion(int n) {
    if (n == 0) {
        return;
    }
    else {
        limitedRecursion(n - 1);
    }
}
// 使用堆分配大数据
void heapAllocation() {
    int* arr = new int[1000000]; // 在堆上分配大数组
    arr[0] = 1; // 使用数组，防止编译器优化
    std::cout << "Array allocated on heap." << std::endl;
    delete[] arr; // 释放堆内存
}
int main() {
    limitedRecursion(1000); // 限制递归深度
    heapAllocation(); // 使用堆分配大数据
    return 0;
}
```

26.C++内存泄露及检测工具？

图示（方便记忆）：

表. 内存泄露检测工具表格

工具	描述
Valgrind	强大的内存分析工具，检测内存泄漏、无效内存访问
AddressSanitizer	编译器支持的内存错误检测工具
LeakSanitizer	专门用于检测内存泄漏的工具
Visual Studio	内置的内存分析和检测工具

27.介绍一下 RAII 思想，他是怎么保证对象没有内存泄漏的？

图示（方便记忆）：

表. RAII 思想表格

特性	描述
资源获取即初始化	资源在对象创建时即获取
析构时释放资源	资源在对象析构时自动释放
作用范围控制	通过对象的作用范围控制资源的生命周期
保证异常安全	在异常发生时确保资源被正确释放

总结（老秦推荐必看，方便理解）：

RAII (Resource Acquisition Is Initialization) 是 C++ 中的一种重要思想，确保资源在对象的生命周期内得到正确管理。在对象创建时获取资源，在对象析构时释放资源，避免了资源泄漏和未定义行为。通过 RAII，程序能够自动管理资源，即使在异常发生时也能保证资源被正确释放。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <fstream>
class FileRAII {
    std::fstream file;
public:
    FileRAII(const std::string& filename) {
        file.open(filename, std::ios::out | std::ios::trunc);
        if (!file.is_open()) {
            throw std::runtime_error("Unable to open file");
        }
    }
    ~FileRAII() {}
};
```

```

        if (file.is_open()) {
            file.close();}
    void write(const std::string& content) {
        if (file.is_open()) {
            file << content << std::endl;}}};
int main() {
    try {
        FileRAII file("example.txt");
        file.write("Hello, RAI!");}
    catch (const std::exception& e) {
        std::cerr << "Exception: " << e.what() << std::endl;}
    return 0;}

```

28.友元函数？

图示（方便记忆）：

表. 友元函数

特性	描述
定义	类外的函数，可以访问类的私有和保护成员
声明	在类定义中使用 friend 关键字声明

代码示例（老秦建议：自己动手实践一下）

```

#include <iostream>
class MyClass {
private:
    int data;
public:
    MyClass(int d) : data(d) {}
    friend void displayData(const MyClass& obj);
void displayData(const MyClass& obj) {
    std::cout << "Data: " << obj.data << std::endl;}
int main() {
    MyClass obj(42);
    displayData(obj);
    return 0;}

```

29.友元类？

图示（方便记忆）：

表. 友元类表格

特性	描述
----	----

定义	可以访问另一个类的私有和保护成员
声明	在类定义中使用 friend class 声明

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class B; // 前向声明
class A {
private:
    int data;
public:
    A(int d) : data(d) {}
    friend class B; // B 是 A 的友元类};
class B {
public:
    void displayData(const A& obj) {
        std::cout << "Data: " << obj.data << std::endl;};
int main() {
    A objA(42);
    B objB;
    objB.displayData(objA);
    return 0;}
```

30.什么是虚函数？

总结（老秦推荐必看，方便理解）：

(1)核心概念：

1) 动态绑定：虚函数的调用不是在编译时解析的，而是在运行时解析。这意味着直到程序运行时，才决定调用哪个函数。

2) 虚函数表 (vtable)：每个含有虚函数的类都有一个虚函数表。这个表是一个函数指针数组，每个元素对应类中的一个虚函数。对象通过这个表来查找正确的函数调用。

3) 虚指针 (vptr)：每个含有虚函数的对象都自动包含一个指向其类的虚函数表的指针，称为虚指针。这个指针是对象运行时动态绑定的关键。

(2)使用虚函数的好处：

1) 提高灵活性：允许在不修改现有代码的情况下添加新的行为和功能。

2) 代码重用：通过继承和覆盖虚函数来扩展类功能，而不是重写已有代码。

3) 接口一致性：基类可定义接口（虚函数），派生类实现具体功能，使得代码更加模块化。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
```

```

class Base {
public:
    // 虚函数声明
    virtual void show() {
        std::cout << "Base class show function called." << std::endl;
    }
    virtual ~Base() {}
};
class Derived : public Base {
public:
    // 覆盖基类中的虚函数
    void show() override {
        std::cout << "Derived class show function called." << std::endl;
    }
};
void function(Base& b) {
    b.show(); // 在运行时确定调用哪个版本的 show()
}
int main() {
    Base b;
    Derived d;
    function(b); // 输出 "Base class show function called."
    function(d); // 输出 "Derived class show function called."
    return 0;
}

```

31. 什么函数不能声明为虚函数？

图示（方便记忆）：

表. 不能声明为虚函数的函数表格

函数类型	描述
构造函数	构造函数不能声明为虚函数，因为对象在构造时还未建立虚函数表
静态函数	静态函数没有 this 指针，与对象无关，不能声明为虚函数
内联函数	内联函数可以声明为虚函数，但内联特性会被编译器忽略

32. 构造函数和析构函数能不能是虚函数，说明原因？

总结（老秦推荐必看，方便理解）：

(1) 构造函数不能是虚函数：

原因：构造函数在对象创建时调用，而对象在构造过程中还没有完全构造好，虚指针（vptr）可能尚未设置或尚未指向最终的类。因此，构造函数不能是虚函数。

(2) 析构函数可以是虚函数：

原因：析构函数是对象销毁时调用的，虚指针（vptr）已经指向了正确的虚函数表。如果基类的析构函数是虚函数，当通过基类指针删除派生类对象时，会调用派生类的析构

函数，确保正确释放资源，防止内存泄漏和其他资源泄露问题。

33.虚函数表在哪个阶段被分配的？

总结（老秦推荐必看，方便理解）：

(1)编译阶段

虚函数表的创建：在编译阶段，编译器会分析类的声明和定义，确定哪些类包含虚函数，并为这些类创建虚函数表。虚函数表中包含指向类中所有虚函数的指针。

(2)链接阶段

虚函数表的分配：在链接阶段，虚函数表被分配具体的内存地址。链接器会将所有虚函数表放入全局内存空间中，确保每个虚函数表都有唯一的内存地址。

(3)运行时

虚表指针的初始化：在运行时，当类的对象被实例化时，虚表指针（vptr）被初始化，指向对应类的虚函数表。每个包含虚函数的类的对象都会有一个虚表指针，指向该类的虚函数表。

34.没有虚函数的话，C++ 如何实现多态？

图示（方便记忆）：

表. 没有虚函数实现多态表格

特性	描述
函数指针	通过函数指针实现运行时的多态性
std::function	使用 std::function 进行函数对象的存储
手动类型擦除	通过手动类型擦除实现多态
模板	使用模板实现编译时多态

总结（老秦推荐必看，方便理解）：

在没有虚函数的情况下，可以通过函数指针、std::function、手动类型擦除和模板来实现多态。每种方法都有其适用场景和局限性，需要根据具体需求选择合适的方法。

35.同一个类的不同对象的虚函数表是同一个吗？

总结（老秦推荐必看，方便理解）：

是的，同一个类的所有对象共享同一个虚函数表。每个对象中都有一个虚指针（vptr），指向该类的虚函数表。因此，同一个类的不同对象的虚函数调用会通过相同的虚函数表进行。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
```

```

class Base {
public:
    virtual void show() {
        std::cout << "Base show" << std::endl;
    }
    virtual ~Base() {
        std::cout << "Base destructor" << std::endl;
    }
};

class Derived : public Base {
public:
    void show() override {
        std::cout << "Derived show" << std::endl;
    }
    ~Derived() {
        std::cout << "Derived destructor" << std::endl;
    }
};

int main() {
    Base* b = new Derived();
    b->show(); // 调用的是 Derived 的 show 函数
    delete b; // 调用的是 Derived 的析构函数
    return 0;
}

```

虚函数表和虚指针：

Base 和 Derived 各自有一个虚函数表，存储了虚函数的地址。

当创建 Derived 对象时，虚指针（vptr）指向 Derived 的虚函数表。

虚函数调用：

通过 Base 类型的指针 b 调用 show 函数，由于 show 是虚函数，调用 Derived 类中的实现。

通过 delete 关键字销毁 b 指向的对象，由于 Base 的析构函数是虚函数，调用 Derived 类的析构函数，随后调用 Base 类的析构函数。

36.基类的虚函数表存放在内存的什么区，虚表指针

vptr 的初始化时间？

图示（方便记忆）：

表. 虚函数表的特征和存储位置表

特征	详细描述
全局共享	虚函数表是全局共享的，每个类型只有一个虚函数表。在编译时就已经构造完成。
存储内容	虚函数表存储的是虚函数的地址，它类似于一个数组。每个类对象中都存储有一个指向其虚函数表的指针（vptr）。
类比	由于虚函数表的特性类似于类中的静态成员变量，它们都是全局共享且大小固定。
动态分配	虚函数表的大小在编译时已确定，无需在运行时动态分配内存，因此不存储在堆区。
内存区域	在 Linux/Unix 系统中，虚函数表通常存放在可执行文件的只读数据段（.rodata）中。在 Windows 系统中，可能存放在常量段。

表. 虚表指针 (vptr) 的初始化时间和存储位置

特征	详细描述
初始化时机	虚表指针 vptr 在类的构造函数执行时进行初始化。它是类与虚函数密不可分的一部分。
存储位置	vptr 存在于对象内存布局的最前面，它指向对应的虚函数表。
内存区域	虚表指针作为对象的一部分，存储在对象所在的内存区域，例如如果对象是局部变量则在栈上，动态分配则在堆上。

总结（老秦推荐必看，方便理解）：

虚函数表 (vtable)：虚函数表 (vtable) 是实现多态性的关键组件。**每个包含虚函数的类都有一个全局共享的虚函数表，它在编译时构建完成并存储在程序的只读数据段 (.rodata) 中。**虚函数表类似于一个指针数组，其中每个元素指向一个具体的虚函数实现。这种机制使得程序在运行时能够根据对象的实际类型调用相应的虚函数，从而实现动态多态。

虚表指针 (vptr)：虚表指针 (vptr) 则是类实例中的一个指针，**位于对象内存布局的最前端，指向相应的虚函数表。**虚表指针的初始化发生在类构造函数的执行过程中，确保每个对象实例都正确地链接到其类型特定的虚函数表。由于虚表指针的存在，即使使用基类的指针或引用来操作对象，程序也能通过 vptr 找到正确的虚函数表，并调用到派生类中的相应虚函数，实现了类型的多态性行为。

37.虚函数表，是对象拥有还是类拥有？

总结（老秦推荐必看，方便理解）：

虚函数表 (vtable) 是类拥有的，而不是对象拥有的。每个包含虚函数的类都有一个虚函数表，表中存储了指向该类虚函数实现的指针。对象中有一个指向该虚函数表的指针（通常称为虚指针，vptr）。

38.虚函数内部调用非虚函数是调用指针类还是对象类？

总结（老秦推荐必看，方便理解）：

当一个虚函数在调用时，调用的是实际对象的虚函数实现，这是通过虚函数表 (vtable) 来实现的。然而，非虚函数的调用是静态绑定的，即在编译时就已经确定了调用哪个函数。因此，当虚函数内部调用非虚函数时，调用的是当前对象所属类的非虚函数。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class Base {
public:
    virtual void show() {
```



```

    print(); // 调用的是 Derived 类的 print
}
void print() {
    std::cout << "Base print" << std::endl;
}
};
class Derived : public Base {
public:
    void show() override {
        print(); // 调用的是 Derived 类的 print
    }
    void print() {
        std::cout << "Derived print" << std::endl;
    }
};
int main() {
    Derived d;
    Base* b = &d;
    b->show(); // 调用 Derived 的 show , 继而调用 Derived 的 print
    return 0;
}

```

分析：

调用路径：

在 main 函数中，通过 Base 类型的指针 b 调用了 show 函数。

由于 show 是虚函数，调用的是实际对象 d 的 Derived 类中的 show 实现。

内部调用：

在 Derived 类的 show 函数中调用了 print 函数。

print 不是虚函数，调用的是 Derived 类中的 print 函数。

39. 纯虚函数的实现原理？

图示（方便记忆）：

表. 纯虚函数表格

特性	描述
定义	用于定义接口，没有具体实现
虚函数表	类包含虚函数时，编译器生成虚函数表（vtable）
虚函数指针	每个对象包含一个指向虚函数表的指针（vptr）
动态绑定	运行时通过虚函数指针和虚函数表实现动态绑定

总结（老秦推荐必看，方便理解）：

纯虚函数用于定义接口，没有具体实现。含有纯虚函数的类被称为抽象类，不能实例

化。纯虚函数的实现依赖于虚函数表（vtable）和虚函数指针（vptr）。每个包含虚函数的类都有一个虚函数表，表中包含指向虚函数的指针。每个对象包含一个指向该类虚函数表的指针，运行时通过虚函数指针和虚函数表实现动态绑定。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class Base {
public:
    virtual void show() = 0; // 纯虚函数
};
class Derived : public Base {
public:
    void show() override {
        std::cout << "Derived implementation of show" << std::endl;
    }
};
int main() {
    Derived d;
    Base* b = &d;
    b->show(); // 调用派生类的实现
    return 0;
}
```

40. 纯虚函数？使用场景有哪些？

总结（老秦推荐必看，方便理解）：

纯虚函数主要用于定义接口，确保所有派生类实现特定的行为。

常见的使用场景包括：

接口定义：定义接口类，要求派生类实现特定的方法。

多态实现：通过基类指针或引用调用派生类的实现，实现多态行为。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class Shape {
public:
    virtual void draw() = 0; // 纯虚函数
};
class Circle : public Shape {
public:
    void draw() override {
        std::cout << "Drawing Circle" << std::endl;
    }
}
```

```
};
class Square : public Shape {
public:
    void draw() override {
        std::cout << "Drawing Square" << std::endl;
    }
};
void renderShape(Shape& shape) {
    shape.draw();
}
int main() {
    Circle circle;
    Square square;
    renderShape(circle);
    renderShape(square);
    return 0;
}
```

4.1. 虚继承解决什么问题？

图示（方便记忆）：

表. 虚继承解决问题表格

问题	描述
菱形继承（钻石继承）	多继承结构中，继承路径重复导致基类成员冗余
虚继承	通过虚基类的方式，确保基类成员只被继承一次

总结（老秦推荐必看，方便理解）：

虚继承用于解决菱形继承（钻石继承）问题。在多继承结构中，不使用虚继承会导致基类成员的重复继承。通过虚继承，基类成员在继承链中只存在一份，避免了冗余和潜在的歧义性。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class Base {
public:
    int value;
};
class Derived1 : virtual public Base {};
class Derived2 : virtual public Base {};
class Final : public Derived1, public Derived2 {};
int main() {
    Final f;
```

```

f.value = 10; // 通过虚继承，避免了基类成员的重复继承
std::cout << "Final value: " << f.value << std::endl;
return 0;
}

```

42.野指针和悬空指针的概念？

图示（方便记忆）：

表. 野指针和悬空指针的概念与区别

类型	定义	原因及危害
野指针	指针变量指向了一个随机的内存地址。	通常是因为没有初始化指针变量。使用野指针可能会读写到不属于程序的内存区域，导致数据损坏或程序崩溃。
悬空指针	指向了一块已经被释放的内存的指针。	如果在释放内存后没有将指针设为 nullptr，再次使用这个指针可能会导致未定义的行为，如访问已释放的内存。

总结（老秦推荐必看，方便理解）：

(1)野指针：野指针则是指向不确定内存区域的指针。这通常是因为指针变量未被初始化就开始使用，或是指向的内存已被释放或重新用于其他用途后，该指针未被立即清零或重新赋值。野指针的危害在于它可能指向任何位置，对其的任何操作都可能导致不可预测的结果，如程序崩溃或数据损坏。

(2)悬空指针：悬空指针是指向一块已经被释放的内存的指针。这通常发生在对象或内存块被删除后，指向该内存的指针没有被及时设为 nullptr。这种指针的危险在于，如果继续使用它，可能会操作一些已被操作系统回收或重新分配给其他程序的内存，从而导致数据损坏或程序异常。

代码示例（老秦建议：自己动手实践一下）

```

#include <iostream>
int main() {
    // 野指针示例
    int* wildPtr; // 未初始化的指针
    // 未定义行为：尝试使用未初始化的指针
    // std::cout << *wildPtr << std::endl; // 可能导致崩溃
    // 悬空指针示例
    int* danglingPtr = new int(10); // 分配内存
    delete danglingPtr; // 释放内存
    danglingPtr = nullptr; // 推荐做法：设置为 nullptr
    // 未定义行为：尝试访问已释放的内存
    // std::cout << *danglingPtr << std::endl; // 可能导致崩溃
}

```

```
return 0;
}
```

43.内存对齐？

图示（方便记忆）：

表. 内存对齐

优化方面	描述
提高内存访问速度	对齐的数据允许 CPU 以最快的方式访问内存。数据对齐到特定的内存地址，使得 CPU 可以在单个或最少的内存访问周期内完成数据读写。
防止硬件错误	某些处理器（如 ARM、SPARC）对未对齐的数据访问可能引发硬件异常。内存对齐避免这类问题，确保程序的稳定运行。
遵守硬件和编译器约定	大多数现代编译器和 CPU 架构都有内存对齐的要求。遵循这些约定可以优化代码的执行，提高性能。
平台兼容性	在不同平台之间共享数据时，正确的内存对齐确保数据结构的一致性和正确解析，特别是在网络通信和文件格式标准中。
提高缓存利用率	正确对齐的数据结构更可能与缓存行边界对齐，减少缓存行加载次数，提高缓存利用率，从而减少 CPU 缓存未命中的情况。

总结（老秦推荐必看，方便理解）：

内存对齐：一种计算机系统中用于安排数据存储和访问的技术，以提高内存访问的效率。主要原因和优势包括：

提高性能：内存对齐使得数据存放在处理器预期的内存地址上，减少对内存的访问次数，从而提高处理速度。

防止错误：某些处理器要求数据严格对齐，否则可能导致运行时错误。

兼容性：正确的内存对齐保证了数据结构在不同计算机系统之间能够正确读取和写入，尤其是在涉及网络通信和文件存储的应用中。

代码示例（老秦建议：自己动手实践一下）

```
struct Aligned {
    int a;
    int b;
    int c;
}; // 自然对齐，通常按照最大成员（这里是 int，假设 4 字节）对齐
#pragma pack(push, 1)
struct Unaligned {
    char x; // 1 字节
    int a;  // 4 字节，应对齐至 4 字节边界
    char y; // 1 字节
    int b;  // 4 字节，应对齐至 4 字节边界
    char z; // 1 字节
    int c;  // 4 字节，应对齐至 4 字节边界
};
```

```
};
```

```
#pragma pack(pop)// 强制编译器不自动对齐，可能导致性能下降和硬件错误
```

char x	填充3个字节
int a	
char y	填充3个字节
int b	
char z	填充3个字节
int c	

44.extern 关键字？

图示（方便记忆）：

表. Extern 关键字

特征	描述
名称修饰解除 (Name Mangling)	C++为支持函数重载，会对函数名进行复杂的修饰。extern "C"告诉C++编译器按C语言的简单名称处理，避免这种修饰。这使得C++编译的程序可以链接到由C编译器编译的代码。 extern "C"提供了一种机制，使得由C编译器编译的代码可以被C++代码链接和使用，因为它防止了C++编译器对函数名称进行修改，从而避免了链接时找不到符号的问题。
链接兼容性	通过使用extern "C"，C++项目可以无需修改直接重用现有的C语言库，这极大地提高了项目的开发效率和代码的可重用性。
代码重用	在C和C++混合编程环境中，extern "C"提供了一种标准化的方法来声明接口。这确保了C和C++组件之间的无缝操作和更好的代码管理。
接口标准化	使用extern "C"可以防止C++中的名称与C库中的全局符号冲突，特别是在大型项目或多库环境中尤为重要。
避免冲突	对于那些仅用C语言编写的函数库，如操作系统的API或已经存在的成熟库，extern "C"简化了从C++代码中调用这些函数的过程。
简化调用	

45.C++编译流程？

图示（方便记忆）：

表. C++编译流程表格

步骤	描述
预处理	处理宏定义、头文件包含、条件编译指令等
编译	（词法分析、语法分析、语义分析、中间代码生成和优化），将预处理后的代码转换为汇编代码
汇编	将汇编代码转换为机器代码
链接	将多个目标文件和库文件链接生成可执行文件

46.编译器在底层如何实现重载？

总结（老秦推荐必看，方便理解）：

编译器通过 名称改编（Name Mangling）实现函数重载。每个函数在编译时，编译器会将其名称和参数类型组合成一个唯一的符号，以区别重载的不同函数。

代码示例（老秦建议：自己动手实践一下）s

```
void foo(int) {} // 可能变成 _Z3fooi
void foo(double) {} // 可能变成 _Z3food
```

47.C++函数调用的压栈过程？

图示（方便记忆）：

表. 函数调用的压栈过程表格

步骤	描述
参数压栈	从右到左将参数压入栈中
返回地址压栈	将函数返回地址压入栈中
跳转到函数代码	程序跳转到函数代码执行
函数体执行	在函数体中执行具体的代码
返回值处理	将返回值存储在寄存器或栈中
栈清理和返回	清理栈并返回到调用位置

总结（老秦推荐必看，方便理解）：

函数调用时从右到左将参数压入栈中，随后压入返回地址，跳转到函数代码执行，处理返回值并清理栈，最后返回到调用位置。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
void foo(int a, double b) {
```

```
std::cout << "a: " << a << ", b: " << b << std::endl;
}
int main() {
    foo(42, 3.14);
    return 0;
}
```

48.什么是回调函数，回调函数的使用场景？

图示（方便记忆）：

表. 回调函数使用场景

使用场景	描述	示例
事件驱动编程	在图形用户界面（GUI）编程中，回调函数用于处理按钮点击、鼠标移动等事件。例如，点击按钮时调用一个特定的函数来处理该点击事件。	GUI 编程中处理按钮点击、鼠标移动等事件回调。
异步编程	在网络编程中，回调函数用于处理异步 I/O 操作。例如，当数据从服务器返回时，回调函数被调用来处理这些数据。	网络编程中处理服务器返回的数据。
定时器	定时器到期时调用回调函数执行某些操作。例如，设置一个定时器，每隔一段时间调用一个函数。	定时任务，如每隔一段时间执行某个操作。
函数式编程	许多函数式编程语言或库中，回调函数用于高阶函数。例如，map、filter 等函数接受回调函数作为参数，以对集合中的每个元素进行操作。	使用 map、filter 等高阶函数对集合元素进行操作。
排序和搜索	在排序算法（如 qsort）中，回调函数用于比较元素。用户可以定义自己的比较函数，从而定制排序行为。	自定义比较函数用于排序算法，如 qsort。

总结（老秦推荐必看，方便理解）：

回调函数：

回调函数是一种委托机制，通过把函数的指针作为参数传递给另一个函数，从而在该函数内调用指针指向的函数。换句话说，回调函数是由其他代码在适当的时间点调用的函数。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
using namespace std;
// 定义回调函数类型
typedef void (*CallbackFunction)(int);
// 一个接受回调函数作为参数的函数
void performAction(int value, CallbackFunction callback) {
    // 在某些条件下调用回调函数
}
```



```
    if (value > 0) {
        callback(value);
    }
}
// 回调函数的具体实现
void myCallbackFunction(int num) {
    cout << "Callback called with value: " << num << endl;
}
int main() {
    int num = 5;
    // 将回调函数传递给 performAction 函数
    performAction(num, myCallbackFunction);
    return 0;
}
```

49.内联函数和宏定义的区别

图示（方便记忆）：

表. 内联函数和宏定义对比表格

特性	内联函数	宏定义
类型检查	进行类型检查	不进行类型检查
调试支持	支持调试，可以设置断点	调试困难，无法设置断点
安全性	具有类型安全性和作用域范围	缺乏类型安全性和作用域范围
代码替换	编译器在调用处内联展开	预处理器在编译时文本替换

总结（老秦推荐必看，方便理解）：

内联函数和宏定义在性能优化上有相似之处，但内联函数更安全、更灵活。内联函数支持类型检查和调试，适用于短小的函数。宏定义通过预处理器进行文本替换，缺乏类型检查，容易导致难以发现的错误。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#define SQUARE(x) ((x) * (x)) // 宏定义
inline int square(int x) { // 内联函数
    return x * x;
}
int main() {
    int a = 5;
    std::cout<< "Macro Square: " << SQUARE(a + 1) << std::endl; // 输出 11,
错误的结果,注意括号 SQUARE(x) ((x) * (x))
    std::cout << "Inline Function Square: " << square(a + 1) << std::endl; //
输出 36, 正确的结果
}
```

```
return 0;}
```

50.如何正确使用内联函数

图示（方便记忆）：

表. 内联函数使用建议表格

特性	建议使用情况
函数长度	短小的函数
调用频率	调用频繁的函数
类型检查	需要类型安全检查的场合
调试需求	需要调试和设置断点的函数

表. 内联函数编译器决定表格

特性	描述
编译器决定	最终是否内联由编译器决定
复杂度限制	复杂函数可能不会被内联
大小限制	过大的函数可能不会被内联
调用频率	高调用频率的函数更可能被内联
优化设置	编译器的优化选项影响内联决策

总结（老秦推荐必看，方便理解）：

内联函数适用于短小、调用频繁且需要类型检查和调试支持的场合。内联函数应尽量保持简短，以便编译器能够有效展开并优化代码。虽然 `inline` 关键字建议编译器将函数内联展开，但实际内联与否由编译器决定。编译器会考虑函数的复杂度、大小、调用频率及优化设置等因素。短小且高频调用的函数更可能被内联，而过于复杂或庞大的函数则不太可能被内联。

51.浅拷贝和深拷贝的区别？

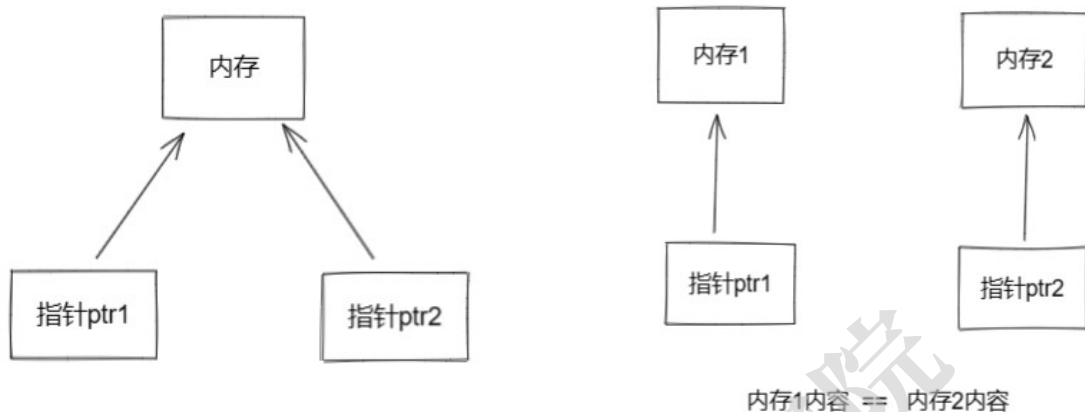
图示（方便记忆）：

表. 浅拷贝和深拷贝的区别

特性	浅拷贝	深拷贝
内存复制	只复制对象的指针，不复制指针指向的内存。	复制对象及其指针指向的所有内存内容。
数据共享	多个对象共享同一块内存资源。	每个对象拥有自己独立的内存副本。
副作用	一个对象的修改会影响到共享内存的所有对象。	一个对象的修改不会影响其他对象。

浅拷贝：

深拷贝：



代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class CopyExample {
private:
    int* data;
    bool isDeepCopy;
public:
    // 构造函数初始化数据
    CopyExample(int val, bool deepCopy = false) : isDeepCopy(deepCopy) {
        data = new int(val);
    }
    // 构造函数重载，实现浅拷贝和深拷贝
    CopyExample(const CopyExample& source) {
        if (source.isDeepCopy) {
            // 深拷贝：分配新的内存并复制数据
            data = new int(*source.data);
            isDeepCopy = true;
        }
        else {
            // 浅拷贝：复制指针
            data = source.data;
            isDeepCopy = false;
        }
    }
    ~CopyExample() {
        delete data; // 释放内存
    }
}
```

```

int getData() const {
    return *data;
}
void setData(int val) {
    *data = val;
}
};

int main() {
    CopyExample obj1(100, false); // 浅拷贝对象
    CopyExample obj2 = obj1;      // 浅拷贝发生
    CopyExample obj3(200, true);  // 深拷贝对象
    CopyExample obj4 = obj3;      // 深拷贝发生
    obj2.setData(300);           // 更改 obj2 将影响 obj1
    obj4.setData(400);           // 更改 obj4 不影响 obj3
    std::cout << "Obj1 data after obj2 change: " << obj1.getData() <<
std::endl; // 输出 300
    std::cout << "Obj2 data: " << obj2.getData() << std::endl;      //
输出 300
    std::cout << "Obj3 data after obj4 change: " << obj3.getData() <<
std::endl; // 输出 200
    std::cout << "Obj4 data: " << obj4.getData() << std::endl;      //
输出 400
    return 0;
}

```

52.隐式转换，如何消除隐式转换？

图示（方便记忆）：

表. 隐式转换和消除方法表格

特性	描述
隐式转换	编译器自动将一种类型转换为另一种类型
消除方法	使用 explicit 关键字防止隐式转换

总结（老秦推荐必看，方便理解）：

隐式转换是编译器自动将一种类型转换为另一种类型的过程。使用 **explicit** 关键字可以防止构造函数的隐式转换，确保只能显式调用构造函数。

53.说一下零拷贝？

图示（方便记忆）：

表. 零拷贝技术表格

特性	描述
定义	数据在处理过程中不进行内存拷贝，直接在不同模块间传递
优点	提高性能，减少 CPU 和内存带宽消耗
使用场景	高性能网络传输、文件 I/O 操作
常见实现技术	sendfile、mmap、splice、DMA

总结（老秦推荐必看，方便理解）：

零拷贝技术在数据传输过程中避免了内存拷贝，直接在不同模块（如用户空间和内核空间）之间传递数据，从而提高了系统性能，减少了 CPU 和内存带宽的消耗。常用于高性能网络传输和文件 I/O 操作，典型实现包括 sendfile、mmap、splice 和 DMA 技术。

(1)详细解释

1) 定义：零拷贝技术使得数据在处理过程中不需要从一个缓冲区拷贝到另一个缓冲区，而是通过指针或引用传递。这减少了内存带宽的消耗和 CPU 的负载。

优点：

- 2) 性能提升：由于减少了数据在内存中的拷贝操作，提高了数据传输的速度。
- 3) 降低 CPU 负载：避免了不必要的内存拷贝操作，减少了 CPU 的使用率。
- 4) 内存带宽利用率：减少了内存总线的使用，提高了系统整体性能。

(2)使用场景：

1) 高性能网络传输：例如服务器需要快速传输大量数据时，零拷贝技术可以显著提高传输效率。

2) 文件 I/O 操作：例如文件复制、文件发送等场景，使用零拷贝可以减少文件在用户空间和内核空间之间的拷贝。

(3)常见实现技术：

- 1) sendfile：在 Linux 中，将文件数据直接从文件描述符发送到网络套接字。
- 2) mmap：内存映射文件，允许文件的内容直接映射到进程的地址空间。
- 3) splice：将数据在两个文件描述符之间移动，而无需将数据拷贝到用户空间。
- 4) DMA (Direct Memory Access)：允许设备直接访问主内存，减少 CPU 参与数据传输。

54.strlen 和 sizeof 区别？

图示（方便记忆）：

表. strlen 和 sizeof 区别

特性	strlen	sizeof
定义	strlen 是一个函数，用于计算 C 风格字符串（以 null 结尾的字符数组）的长度，不包括结尾的 null 字符。	sizeof 是一个编译时运算符，用于获取变量、数据类型或数据结构占用的内存大小，以字节为单位。
返回类型	返回字符串的长度，类型为 size_t。	返回所查询对象或类型的大小，类型为 size_t。

参数	接受一个指向以 null 结尾的字符数组的指针。	可以是变量、数组、指针、类型或表达式。
计算时机	运行时计算字符串的实际长度，直到遇到第一个 null 字符。	编译时确定，不依赖于程序运行时的状态。

总结（老秦推荐必看，方便理解）：

strlen 和 sizeof 虽然都用于测量大小，但它们的用途截然不同。

Strlen：Strlen（）专门用于计算 C 风格字符串的长度，即字符数组直到第一个 null 字符的距离。它在运行时执行，因此对于动态内容很有用。

Sizeof：Sizeof（）是一个编译时运算符，用于确定数据类型或结构的内存占用量，非常适合在编译时需要确定数据布局大小的场景。在编程中正确选择和使用这两者对于优化内存使用和保证程序的正确性至关重要。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <cstring>
int main() {
    char str[] = "Hello";
    std::cout << "strlen(str): " << strlen(str) << std::endl; // 输出: 5
    std::cout << "sizeof(str): " << sizeof(str) << std::endl; // 输出: 6 (包括末尾的 null 字符)
    int arr[] = { 1, 2, 3, 4, 5 };
    std::cout << "sizeof(arr): " << sizeof(arr) << std::endl; // 输出: 数组总大小, 取决于 int 类型大小
    std::cout << "sizeof(arr) / sizeof(arr[0]): " << sizeof(arr) / sizeof(arr[0]) << std::endl; // 输出: 5 (数组元素个数)
    return 0;
}
```

55.头文件如何避免出现重定义的问题？

总结（老秦推荐必看，方便理解）：

使用头文件保护符（头文件包围）来避免头文件的重复包含。常用的方法是使用预处理指令 #ifndef、#define 和 #endif。

56.静态多态与动态多态？

图示（方便记忆）：

表. 静态多态与动态多态

特性	静态多态（编译时多态）	动态多态（运行时多态）
实现方式	通过函数重载和模板（泛型编程）实现。	通过虚函数和继承实现。
解析时间	在编译时进行。这意味着编译器已经确定	在运行时进行。这允许在运行时决定调用

	了函数的哪个版本将被调用。	哪个方法。
性能	通常速度更快，因为函数调用已经在编译时解析。	速度可能稍慢，因为需要在运行时查找适当的函数。

57.重载和重写的区别

图示（方便记忆）：

表. 重载和重写对比表格

特性	重载	重写
定义层次	同一作用域内	派生类与基类
函数签名	不同参数列表	参数列表相同
关键词	无需特殊关键词	使用 virtual 关键词
目的	提供同名函数的不同实现	提供基类函数的不同实现

总结（老秦推荐必看，方便理解）：

重载是在同一作用域内定义多个参数列表不同的同名函数，而重写是派生类重新定义基类中使用 virtual 声明的函数。重载用于提供同名函数的不同实现，重写用于提供基类函数的不同实现。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
class Base {
public:
    virtual void display() const {
        std::cout << "Base display" << std::endl;
    }
};
class Derived : public Base {
public:
    void display() const override {
        std::cout << "Derived display" << std::endl;
    }
    void display(int x) const { // 重载
        std::cout << "Derived display with int: " << x << std::endl;
    }
};
int main() {
    Base* basePtr = new Derived();
    basePtr->display(); // 调用重写的函数
    Derived derived;
    derived.display();
    derived.display(10); // 调用重载的函数
}
```



```
delete basePtr;
return 0;
}
```

58.临时对象在什么时候会产生？

图示（方便记忆）：

表. 临时对象产生情况表格

产生情况	描述
返回值优化 (RVO)	当函数返回一个对象时，编译器通常会生成一个临时对象，然后通过优化直接将这个临时对象转移给接收对象，从而避免不必要的拷贝和移动操作。
隐式类型转换	当函数参数或返回值需要进行类型转换时，编译器会创建一个临时对象来存放转换后的值。
运算符重载	在重载运算符时，尤其是涉及到操作结果返回的情况，临时对象会产生。
按值传递和返回	当函数参数按值传递或返回值按值返回时，临时对象会在拷贝或移动过程中产生。
临时对象的延长生命周期	当一个临时对象被绑定到一个 const 引用时，其生命周期会被延长。

Ps:在以下情况下，临时对象的生命周期会被延长

- 1.临时对象被绑定到一个 const 引用，且这个引用的作用域是局部的。
- 2.临时对象被绑定到一个 const 引用，且这个引用是一个函数的参数或返回值。

59.C++的返回值优化？

总结（老秦推荐必看，方便理解）：

(1)返回值优化 (RVO) ： 适用于函数直接返回一个临时对象的情况。

(2)命名返回值优化 (NRVO) ： 适用于函数返回一个命名的局部对象的情况。

代码示例（老秦建议：自己动手实践一下）

返回值优化 (RVO)

在以下示例中，编译器可以通过 RVO 避免创建和销毁临时对象：

```
#include <iostream>
class MyClass {
public:
    MyClass() { std::cout << "Constructor" << std::endl; }
    MyClass(const MyClass&) { std::cout << "Copy Constructor" <<
std::endl; }
    ~MyClass() { std::cout << "Destructor" << std::endl; }
};
```



```

MyClass createObject() {
    return MyClass(); // 这里可以应用 RVO
}

int main() {
    MyClass obj = createObject();
    return 0;
}

```

命名返回值优化 (NRVO)

在以下示例中，编译器可以通过 NRVO 优化返回命名的局部对象：

```

#include <iostream>
class MyClass {
public:
    MyClass() { std::cout << "Constructor" << std::endl; }
    MyClass(const MyClass&) { std::cout << "Copy Constructor" <<
std::endl; }
    ~MyClass() { std::cout << "Destructor" << std::endl; }
};

MyClass createObject() {
    MyClass obj;
    return obj; // 这里可以应用 NRVO
}

int main() {
    MyClass obj = createObject();
    return 0;
}

```

60. 局部 Static 变量为什么线程安全？

总结（老秦推荐必看，方便理解）：

(1) 初始化

Static 变量在程序启动时（全局 static 变量）或第一次使用时（局部 static 变量）初始化。初始化过程由编译器和运行时环境控制，确保在多线程环境下只进行一次初始化。

(2) 固定内存位置

Static 变量存储在全局/静态存储区，该区域在程序的整个生命周期内存在，且位置固定。这意味着不同线程访问 static 变量时，访问的是同一个内存位置。

(3) 单例模式

Static 变量的单次实例化特性类似于单例模式，在多线程环境下，通过一次性实例化避免竞争条件。编译器和运行时确保 static 变量只实例化一次。

(4) 具体原因和机制

C++11 标准规定：根据 C++11 标准，第 6.7.4 节中规定，局部静态变量的初始化

将在第一个进入该变量所在的作用域的线程完成，并且是线程安全的。这意味着在多个线程同时访问一个局部静态变量时，只有一个线程会进行初始化，而其他线程会被阻塞，直到初始化完成。

编译器的实现：编译器在生成代码时，会在局部静态变量的初始化代码周围插入同步机制，确保只有一个线程能执行初始化代码，而其他线程会等待该初始化过程完成。

实现方式：

加锁 (Mutex)：在变量初始化代码周围加锁，确保同一时刻只有一个线程能执行初始化。

原子操作：在某些平台上，使用原子操作来确保变量只初始化一次。

双重检查锁定 (Double-Checked Locking)：减少不必要的锁开销，先检查变量是否已经初始化，如果没有，再加锁进行初始化。

特别注意：全局静态变量和命名空间静态变量和类的静态成员变量都是没有内置的线程安全保证

61.memcpy 和 strcpy 什么区别？

总结 (老秦推荐必看，方便理解)：

memcpy：用于内存块拷贝，可以拷贝任意类型的数据，需指定拷贝的字节数，适用于二进制数据。

strcpy：用于拷贝字符串，拷贝直到遇到 \0 结束符，适用于字符串拷贝。

62. 在成员函数中调用 delete this 会出现什么问题？

在类对象的内存空间中，只有数据成员和虚函数表指针，并没有成员函数代码内容。它们放在代码段中，当调用成员函数时会隐含传递一个 **this** 指针，让成员函数知道当前是哪个对象在调用它。如果调用 **delete this**，类对象内存空间释放，只要不涉及 **this** 指针的任何函数调用都能够正常运行。否则会报错。”

第二章 c++11 常用新特性

1.自动类型推导 (auto)

auto 关键字在 C++11 中引入，用于让编译器自动推导变量的类型。

基本用法：

auto 可以用于变量的声明，让编译器根据初始化表达式来推导变量的类型。

```
#include <iostream>
```

```
#include <vector>
```

```
int main() {
```

```
    auto i = 42;    // i 被推导为 int
```

```
    auto d = 3.14;  // d 被推导为 double
```

```

    auto s = "Hello"; // s 被推导为 const char*
    std::cout << "i = " << i << ", d = " << d << ", s = " << s << std::endl;
    return 0;
}

```

用于复杂类型

```

#include <iostream>
#include <vector>
#include <map>
int main() {
    std::vector<int> vec = { 1, 2, 3, 4, 5 };
    std::map<int, std::string> myMap = { {1, "one"}, {2, "two"}, {3, "three"} };
    auto it = vec.begin(); // it 被推导为 std::vector<int>::iterator
    auto mit = myMap.begin(); // mit 被推导为 std::map<int, std::string>::iterator
    std::cout << "First element in vec: " << *it << std::endl;
    std::cout << "First element in myMap: " << mit->first << " => " << mit->second << std::endl;
    return 0;
}

```

用于返回类型

```

#include <iostream>
auto add(int a, int b) -> decltype(a + b) {
    return a + b;
}
int main() {
    auto result = add(3, 4); // result 被推导为 int
    std::cout << "Result: " << result << std::endl;
    return 0;
}

```

2.范围 for 循环

C++11 引入了范围 for 循环，用于遍历容器。

基本用法：

范围 for 循环使用语法 **for (Declaration: Expression)**，其中

Declaration:是一个变量声明；

Expression:是一个表示容器的表达式；

```

#include <iostream>
#include <vector>
int main() {
    std::vector<int> vec = { 1, 2, 3, 4, 5 };
    // 范围 for 循环遍历 vec
    for (int value : vec) {
        std::cout << value << " ";
    }
}

```

```

    }
    std::cout << std::endl;
    return 0;
}

```

引用类型，如果容器中的元素较大，或我们需要修改元素，可以使用引用类型：

```

#include <iostream>
#include <vector>
int main() {
    std::vector<int> vec = { 1, 2, 3, 4, 5 };
    // 使用引用类型
    for (int& value : vec) {
        value *= 2; // 修改 vec 中的每个元素
    }
    for (const int& value : vec) {
        std::cout << value << " "; // 输出修改后的 vec
    }
    std::cout << std::endl;
    return 0;
}

```

与 **auto** 结合使用：

```

#include <iostream>
#include <vector>
#include <map>
int main() {
    std::vector<int> vec = { 1, 2, 3, 4, 5 };
    std::map<int, std::string> myMap = { {1, "one"}, {2, "two"}, {3, "three"} };
    // 使用 auto
    for (auto& value : vec) {
        value += 1; // 修改 vec 中的每个元素
    }
    for (const auto& value : vec) {
        std::cout << value << " "; // 输出修改后的 vec
    }
    std::cout << std::endl;
    for (const auto& pair : myMap) {
        std::cout << pair.first << " ==> " << pair.second << std::endl; // 遍历 myMap
    }
    return 0;
}

```

3. Lambda 表达式

总结（老秦推荐必看，方便理解）：

Lambda 表达式在 C++11 中引入，为匿名函数提供了简洁的语法。Lambda 表达

式可以捕获外部变量，并且可以作为参数传递给函数，尤其在处理 STL 算法时非常有用
基本语法：

```
[capture](parameters) -> return_type {  
// function body  
}  
[捕获列表] (参数列表) -> 返回类型，可以省略  
函数体  
}
```

capture (捕获列表)：用于捕获外部变量。

parameters (参数列表)：函数的参数列表，类似于普通函数的参数列表。

return_type (返回类型)：函数的返回类型，可以省略，由编译器推导。

function body (函数体)：实际执行的代码。

简单示例：

```
auto add = [](int a, int b) -> int {  
    return a + b;  
};  
std::cout << add(3, 4) << std::endl; // 输出 7  
//捕获外部变量：  
int x = 10;  
auto print_x = [x]() {  
    std::cout << "x = " << x << std::endl;  
};  
print_x(); // 输出 x = 10  
//按引用捕获：  
int y = 20;  
auto increment_y = [&y]() {  
    y++;  
};  
increment_y();  
std::cout << "y = " << y << std::endl; // 输出 y = 21  
//捕获所有外部变量：  
    //按值捕获：[=]  
int a = 1, b = 2;  
auto add_ab = [=]() {  
    return a + b;  
};  
std::cout << "a + b = " << add_ab() << std::endl; // 输出 a + b = 3  
    //按引用捕获：[&]  
int c = 3, d = 4;  
auto multiply_cd = [&]() {  
    c *= 2;  
    d *= 2;  
};  
multiply_cd();
```

```
std::cout << "c = " << c << ", d = " << d << std::endl; // 输出 c = 6, d = 8
//带有默认参数的 Lambda 表达式 :
auto greet = [](std::string name = "World") {
    std::cout << "Hello, " << name << "!" << std::endl;
};
greet(); // 输出 Hello, World!
greet("Alice"); // 输出 Hello, Alice!
```

4. 类型别名 (using)

基本语法:

using alias = existing_type;

//简单类型别名 :

```
using Integer = int;
```

```
Integer x = 10;
```

```
std::cout << x << std::endl; // 输出 10
```

//指针类型别名 :

```
using IntPtr = int*;
```

```
int a = 5;
```

```
IntPtr p = &a;
```

```
std::cout << *p << std::endl; // 输出 5
```

//模板类型别名 :

```
template<typename T>
```

```
using Vec = std::vector<T>;
```

```
Vec<int> numbers = { 1, 2, 3, 4, 5 };
```

```
for (const auto& num : numbers) {
```

```
    std::cout << num << " ";
```

```
}
```

```
std::cout << std::endl; // 输出 1 2 3 4 5
```

//复杂类型别名 :

```
using StringMap = std::map<std::string, std::string>;
```

```
StringMap myMap = { {"name", "Alice"}, {"age", "30"} };
```

```
for (const auto& [key, value] : myMap)
```

```
{
```

```
    std::cout << key << ": " << value << std::endl;
```

```
}
```

5. 原子操作 (std::atomic)

C++11 引入了 `std::atomic` 类，提供了一种无锁机制来实现线程安全的操作。`std::atomic` 通过原子操作避免了数据竞争；

基本语法:

原子变量类型声明 :

```
std::atomic<int> counter(0);
```

原子操作代码示例：

```
#include <atomic>
#include <iostream>
#include <thread>
#include <vector>
std::atomic<int> counter(0);
void increment(int n) {
    for (int i = 0; i < n; ++i) {
        ++counter;
    }
}
int main() {
    const int num_threads = 10;
    const int increments_per_thread = 1000;
    std::vector<std::thread> threads;
    for (int i = 0; i < num_threads; ++i) {
        threads.emplace_back(increment, increments_per_thread);
    }
    for (auto& t : threads) {
        t.join();
    }
    std::cout << "Final counter value: " << counter << std::endl;
    // 输出 10000
    return 0;
}
```

原子操作方法：

fetch_add：增加并获取旧值。

fetch_sub：减少并获取旧值。

load：读取原子变量的值。

store：设置原子变量的值。

compare_exchange_strong 和 **compare_exchange_weak**：比较并交换，用于实现复杂的同步机制。

```
std::atomic<int> value(0);
int old_value = value.fetch_add(1); // 增加 1 并获取旧值
std::cout << "Old value: " << old_value << std::endl;
std::cout << "New value: " << value.load() << std::endl; // 输出增加后的值
value.store(10); // 设置为 10
std::cout << "Stored value: " << value.load() << std::endl; // 输出 10
```

原子操作的好处

无锁编程：原子操作是**无锁的**，不会引起死锁。

高效：原子操作**通常比使用互斥锁更高效**。

线程安全：原子操作保证了对变量的操作是原子的，即操作不可分割，不会被中断。

6.nullptr

nullptr 是 C++11 引入的一个关键字，用于表示空指针。它解决了传统 **NULL** 常量在某些情况下的歧义问题，因为 **NULL** 通常被定义为整数 **0**，在重载和模板场景中可能会引发问题。

7.移动语义和 std::move 和右值引用 (&&)

移动语义和右值引用是 C++11 引入的两个紧密相关的特性，用于优化对象的转移操作，从而避免不必要的拷贝，提高程序性能。std::move 是一个标准库函数，用于将左值转换为右值引用，以便启用移动语义。

(1)移动语义

移动语义通过**将资源从一个对象“移动”到另一个对象，而不是复制资源**，从而提高了程序的效率。移动语义在对象的移动构造函数和移动赋值运算符中实现。

(2)右值引用

右值引用 (rvalue references) 是 C++11 引入的新类型，表示一个只能绑定到右值 (临时对象) 的引用。使用**右值引用可以避免不必要的拷贝操作，从而提高程序的性能**。

(3)std::move

std::move 是一个标准库函数，用于将左值显式转换为右值引用，从而启用移动语义。

8.std::thread

std::thread 是 C++11 引入的标准库类，用于**创建和管理线程**。它使得多线程编程变得更加方便和安全。

基本用法：

创建线程：

```
#include <iostream>
#include <thread>
void threadFunction() {
    std::cout << "Hello from thread!" << std::endl;
}
int main() {
    std::thread t(threadFunction); // 创建并启动线程
    t.join(); // 等待线程完成
    return 0;
}
```

传递参数：

```
#include <iostream>
#include <thread>
void printNumber(int n) {
    std::cout << "Number: " << n << std::endl;
}
```



```

}
int main() {
    std::thread t(printNumber, 42); // 传递参数给线程函数
    t.join();
    return 0;
}

```

使用 **Lambda** 表达式：

```

#include <iostream>
#include <thread>
int main() {
    std::thread t([]() {
        std::cout << "Hello from lambda thread!" << std::endl;
    });
    t.join();
    return 0;
}

```

9.enum class

enum class 是 C++11 引入的一种强类型枚举，解决了传统枚举类型的若干问题，如命名冲突和隐式转换。enum class 使枚举类型更安全、可读性更强。

基本语法：

```

enum class EnumName {
    Value1,
    Value2,
    Value3
};

```

定义和使用 enum class：

```

enum class Color {
    Red,
    Green,
    Blue
};

void printColor(Color color) {
    switch (color) {
        case Color::Red:
            std::cout << "Red" << std::endl;
            break;
        case Color::Green:
            std::cout << "Green" << std::endl;
            break;
        case Color::Blue:
            std::cout << "Blue" << std::endl;
            break;
    }
}

```

```

}
int main() {
    Color color = Color::Red;
    printColor(color); // 输出 "Red"
    return 0;
}

    避免命名冲突：
enum class Color {
    Red,
    Green,
    Blue
};
enum class TrafficLight {
    Red,
    Yellow,
    Green
};
int main() {
    Color color = Color::Red;
    TrafficLight light = TrafficLight::Red;
    // 不会产生命名冲突
    return 0;
}

```

10.std::chrono

std::chrono 是 C++11 引入的时间库，用于处理时间点、时间段和时钟。std::chrono 提供了类型安全和高精度的时间处理。

基本组件

时间点 (std::chrono::time_point) : 表示某个时钟上的特定时间。

时间段 (std::chrono::duration) : 表示时间间隔。

时钟(std::chrono::steady_clock, std::chrono::system_clock,

std::chrono::high_resolution_clock) : 提供当前时间点。

基本语法：

获取当前时间：

```

#include <iostream>
#include <chrono>
int main() {
    auto now = std::chrono::system_clock::now();
    std::time_t now_c = std::chrono::system_clock::to_time_t(now);
    std::cout << "Current time: " << std::ctime(&now_c);
    return 0;
}

```

测量代码执行时间：

```
void longRunningTask() {
    std::this_thread::sleep_for(std::chrono::seconds(2));
}

int main() {
    auto start = std::chrono::high_resolution_clock::now();
    longRunningTask();
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> duration = end - start;
    std::cout << "Task took " << duration.count() << " seconds" << std::endl;
    return 0;
}
```

时间段的加减法：

```
#include <iostream>
#include <chrono>

int main() {
    std::chrono::seconds sec1(10);
    std::chrono::seconds sec2(5);
    auto result = sec1 + sec2;
    std::cout << "Result: " << result.count() << " seconds" << std::endl;
    return 0;
}
```

11. 默认和删除的函数 (default 和 delete)

C++11 引入了 default 和 delete 关键字，允许显式声明默认构造函数、析构函数、拷贝构造函数和赋值运算符，也可以禁止特定函数的使用。

default 用于显式指定编译器生成默认的函数实现。

delete 用于显式禁止某个函数。

基本语法：

显式默认构造函数：

```
class MyClass {
public:
    MyClass() = default; // 使用编译器生成的默认构造函数
    ~MyClass() = default; // 使用编译器生成的默认析构函数
};
```

显式禁止函数：

```
class MyClass {
public:
    MyClass() = default;
    MyClass(const MyClass&) = delete; // 禁用拷贝构造函数
    MyClass& operator=(const MyClass&) = delete; // 禁用赋值运算符
};

int main() {
    MyClass obj1;
```

```

// MyClass obj2 = obj1; // 错误，拷贝构造函数被删除
// MyClass obj3;
// obj3 = obj1; // 错误，赋值运算符被删除
return 0;
}

```

12.列表初始化

C++11 引入了统一的初始化语法，使得初始化更加一致和灵活。列表初始化可以用于初始化数组、结构体和类对象。

基本语法：

初始化数组：

```
int arr[] = { 1, 2, 3, 4, 5 };
```

初始化 `std::vector`：

```

#include <iostream>
#include <vector>
int main() {
    std::vector<int> vec = { 1, 2, 3, 4, 5 };
    for (const auto& elem : vec) {
        std::cout << elem << " ";
    }
    std::cout << std::endl;
    return 0;
}

```

初始化自定义类：

```

class MyClass {
public:
    int x;
    double y;
    MyClass(int x, double y) : x(x), y(y) {}
};
int main() {
    MyClass obj{ 10, 3.14 }; // 列表初始化
    std::cout << "x = " << obj.x << ", y = " << obj.y << std::endl;
    return 0;
}

```

结构体初始化：

```

struct Point {
    int x;
    int y;
};
int main() {
    Point p = { 10, 20 }; // 列表初始化
    std::cout << "x = " << p.x << ", y = " << p.y << std::endl;
    return 0;
}

```

}

13.shared_ptr 底层实现，是否线程安全？

总结（老秦推荐必看，方便理解）：

底层实现

std::shared_ptr 是 C++11 引入的智能指针，用于管理动态分配的对象的生命周期。shared_ptr 的底层实现依赖于引用计数来跟踪共享对象的使用情况。它使用一个控制块（control block）来存储引用计数和其他元数据。控制块包含两个引用计数：

(1) .共享计数（shared count）：表示有多少个 shared_ptr 实例共享同一个对象。

(2) .弱计数（weak count）：表示有多少个 weak_ptr 实例引用该对象。

内部结构

shared_ptr 的内部结构通常如下：

指向对象的指针：指向被管理的对象。

控制块的指针：指向包含引用计数和其他元数据的控制块。

控制块的典型结构如下：

```
struct ControlBlock {  
    std::atomic<long> shared_count; // shared_ptr 的引用计数  
    std::atomic<long> weak_count;  // weak_ptr 的引用计数  
    // 其他元数据  
};
```

当创建 shared_ptr 时，会初始化控制块并将 shared_count 设为 1。当 shared_ptr 被复制或赋值时，shared_count 增加；当 shared_ptr 被销毁时，shared_count 减少。如果 shared_count 减至 0，被管理的对象会被销毁。弱计数（weak_count）的目的是跟踪 weak_ptr 的数量，当 weak_count 和 shared_count 都为 0 时，控制块自身也会被销毁。

线程安全性

std::shared_ptr 的引用计数操作是线程安全的，这意味着多个线程可以安全地拷贝和销毁 shared_ptr 对象，而不需要额外的同步机制。这是通过使用 std::atomic 类型来实现的，保证了对共享计数和弱计数的原子操作。

然而，需要注意以下几点：

(1) .线程安全的引用计数：shared_ptr 的引用计数操作是线程安全的，即引用计数的增加和减少是原子的，不会发生竞争条件。

(2) .非线程安全的对象访问：shared_ptr 所管理的对象的访问并不是线程安全的。如果多个线程需要同时访问共享对象，仍然需要额外的同步机制（如互斥锁）来保证线程安全。

14.shared_ptr 赋值给 weak_ptr 时内部做了什

么？

总结（老秦推荐必看，方便理解）：

当将 `shared_ptr` 赋值给 `weak_ptr` 时，会进行以下操作：

- (1) 引用控制块：`weak_ptr` 会指向与 `shared_ptr` 相同的控制块。
- (2) 增加弱计数：控制块中的 `weak_count` 会增加 1，以表明有一个新的 `weak_ptr` 引用了该对象。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <memory>
int main() {
    std::shared_ptr<int> sp = std::make_shared<int>(42);
    std::weak_ptr<int> wp(sp); // `shared_ptr` 赋值给 `weak_ptr`
    // 打印引用计数
    std::cout << "Shared count: " << sp.use_count() << std::endl; // 输出 1
    std::cout << "Weak count: " << wp.use_count() << std::endl; // 输出 1
    return 0;
}
```

在上面的例子中，`std::weak_ptr<int> wp(sp);` 语句执行时，`weak_ptr` 的构造函数会增加控制块的 `weak_count`，表示有一个新的 `weak_ptr` 引用了对象。

赋值操作的内部过程

- (1) 获取控制块：从 `shared_ptr` 中获取控制块指针。
- (2) 增加弱计数：对控制块的 `weak_count` 进行原子增加操作。
- (3) 初始化 `weak_ptr`：将 `weak_ptr` 的内部指针指向该控制块。

第三章 STL 容器基础

1.STL 容器是线程安全的吗？你怎么样在多线程的环境下使用容器？

总结（老秦推荐必看，方便理解）：

STL 容器的线程安全性：大多数 **STL 容器本身不是线程安全的**。这意味着在多个线程同时访问同一个 STL 容器时，如果没有适当的同步机制，就可能会出现数据竞争和未定义行为。

读操作与写操作：通常，多个线程同时读取同一个 STL 容器是安全的，但**同时进行写操作或一个线程写另一个线程读则不安全**。

代码示例（老秦建议：自己动手实践一下）

```
#include <vector>
#include <thread>
```

```
#include <mutex>
std::vector<int> vec;
std::mutex vec_mutex;
void add_to_vector(int value) {
    std::lock_guard<std::mutex> lock(vec_mutex);
    vec.push_back(value);
}
int main() {
    std::thread t1(add_to_vector, 1);
    std::thread t2(add_to_vector, 2);
    t1.join();
    t2.join();
    return 0;
}
```

2.c++ 的 map 和 unordered_map 有什么区别？

图示（方便记忆）：

表. map 和 unordered_map 对比表格

特性	map	unordered_map
底层实现	红黑树（平衡二叉搜索树）	哈希表（哈希映射）
元素有序性	按键值排序存储	元素无序存储
查找/插入/删除复杂度	$O(\log n)$	平均 $O(1)$ ，最坏 $O(n)$
迭代器有效性	在插入和删除操作后仍然有效	插入和删除操作可能使迭代器失效
内存使用	较高（需要额外的树结构）	较低（主要用于哈希桶）

总结（老秦推荐必看，方便理解）：

map 和 unordered_map 都是用于存储键值对的容器，但 map 采用红黑树实现，元素有序，查找、插入、删除的时间复杂度为 $O(\log n)$ ；而 unordered_map 采用哈希表实现，元素无序，查找、插入、删除的平均时间复杂度为 $O(1)$ ，最坏情况下为 $O(n)$ 。根据具体应用需求选择适合的容器。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <map>
#include <unordered_map>
int main() {
    std::map<int, std::string> orderedMap;
    std::unordered_map<int, std::string> unorderedMap;
```

```

orderedMap[1] = "one";
orderedMap[2] = "two";
orderedMap[3] = "three";
unorderedMap[1] = "one";
unorderedMap[2] = "two";
unorderedMap[3] = "three";
std::cout << "Ordered Map:" << std::endl;
for (const auto& pair : orderedMap) {
    std::cout << pair.first << ": " << pair.second << std::endl;
}
std::cout << "Unordered Map:" << std::endl;
for (const auto& pair : unorderedMap) {
    std::cout << pair.first << ": " << pair.second << std::endl;
}
return 0;
}

```

3.红黑树和哈希表的复杂度呢？

图示（方便记忆）：

表. 红黑树和哈希表复杂度表格

操作	红黑树 (map)	哈希表 (unordered_map)
查找	$O(\log n)$	平均 $O(1)$ ，最坏 $O(n)$
插入	$O(\log n)$	平均 $O(1)$ ，最坏 $O(n)$
删除	$O(\log n)$	平均 $O(1)$ ，最坏 $O(n)$
内存使用	较高（需要存储树结构）	较低（主要用于哈希桶）

总结（老秦推荐必看，方便理解）：

红黑树用于有序数据存储，提供 $O(\log n)$ 的查找、插入和删除复杂度，但内存开销较大。哈希表提供平均 $O(1)$ 的查找、插入和删除复杂度，内存开销较低，但在哈希冲突严重时，最坏情况下复杂度为 $O(n)$ 。

4.哈希表的底层实现，哈希平衡算法？

总结（老秦推荐必看，方便理解）：

(1)哈希表的底层实现：

1) 数据结构：哈希表由一个数组和一个哈希函数组成。数组用于存储数据，哈希函数将键（Key）映射到数组的索引位置。

2) 哈希函数：哈希函数是一个映射函数，将输入键转换为数组的索引。理想的哈希函数应该均匀分布键，避免冲突。

3) 冲突处理：当多个键被映射到相同的索引时，会发生冲突。常见的冲突处理方法有：

链地址法 (Separate Chaining) : 每个数组索引维护一个链表 (或其他容器) , 所有哈希到相同索引的元素存储在链表中。

开放地址法 (Open Addressing) : 在数组内寻找下一个空闲位置来存储冲突的元素。

常见的探测方式有 :

1. **线性探测 (Linear Probing)** : 线性地检查下一个位置。
2. **二次探测 (Quadratic Probing)** : 按二次函数探测下一个位置。
3. **双重哈希 (Double Hashing)** : 使用第二个哈希函数计算探测步长。

(2) 哈希平衡算法 :

1) **负载因子 (Load Factor)** : 负载因子是元素数量与数组大小的比值。高负载因子意味着数组较满, 增加冲突概率; 低负载因子则浪费内存。

2) **动态扩容** : 当负载因子超过阈值时, 哈希表会进行扩容。扩容时通常将数组大小增加到原来的两倍, 然后重新计算所有元素的哈希值, 重新插入新数组中。

3) **动态缩容** : 当负载因子低于某个阈值时, 哈希表可能进行缩容, 以节省内存。缩容同样需要重新计算所有元素的哈希值并重新插入。

代码示例 (老秦建议 : 自己动手实践一下)

```
#include <iostream>
#include <vector>
#include <list>
#include <iterator>
class HashTable {
public:
    HashTable(size_t size) : table(size), num_elements(0),
load_factor_threshold(0.75) {}
    void insert(int key, int value) {
        if ((double)num_elements / table.size() >= load_factor_threshold) {
            resize(table.size() * 2);
        }
        size_t index = hashFunction(key);
        for (auto& kv : table[index]) {
            if (kv.first == key) {
                kv.second = value;
                return;
            }
        }
        table[index].emplace_back(key, value);
        num_elements++;
    }
    bool find(int key, int& value) {
        size_t index = hashFunction(key);
        for (const auto& kv : table[index]) {
```

```

        if (kv.first == key) {
            value = kv.second;
            return true;
        }
    }
    return false;
}

void remove(int key) {
    size_t index = hashFunction(key);
    auto& cell = table[index];
    for (auto it = cell.begin(); it != cell.end(); ++it) {
        if (it->first == key) {
            cell.erase(it);
            num_elements--;
            return;
        }
    }
}

private:
    size_t hashFunction(int key) const {
        return key % table.size();
    }

    void resize(size_t new_size) {
        std::vector<std::list<std::pair<int, int>>> new_table(new_size);
        for (const auto& cell : table) {
            for (const auto& kv : cell) {
                size_t new_index = kv.first % new_size;
                new_table[new_index].emplace_back(kv.first, kv.second);
            }
        }
        table = std::move(new_table);
    }

    std::vector<std::list<std::pair<int, int>>> table;
    size_t num_elements;
    double load_factor_threshold;
};

int main() {
    HashTable ht(10);
    ht.insert(1, 100);
    ht.insert(11, 200);
    ht.insert(21, 300);
    int value;
    if (ht.find(11, value)) {

```

```
std::cout << "Key 11 has value " << value << std::endl;
}
else {
    std::cout << "Key 11 not found" << std::endl;
}
ht.remove(11);
if (ht.find(11, value)) {
    std::cout << "Key 11 has value " << value << std::endl;
}
else {
    std::cout << "Key 11 not found" << std::endl;
}
return 0;
}
```

5.红黑树有什么好处？

图示（方便记忆）：

表. 红黑树优点表格

特性	描述
自平衡	保持二叉树的平衡性，避免退化成链表
动态操作效率高	插入、删除和查找的时间复杂度为 $O(\log n)$
多用途	适用于实现有序容器，如 map 和 set
保持顺序	保证元素的有序性

总结（老秦推荐必看，方便理解）：

红黑树是一种自平衡二叉搜索树，保证了插入、删除和查找操作的时间复杂度为 $O(\log n)$ ，有效避免了退化成链表的情况。红黑树广泛用于实现有序容器，如 map 和 set，能够保持元素的有序性。

6.linux 的线程调度底层是不是红黑树？

红黑树在 Linux 的线程调度，主要是完全公平调度器（CFS）中的应用，主要用于高效地管理和查找就绪队列中的进程。

1. 虚拟运行时间（vruntime）

每个进程有一个虚拟运行时间（vruntime），表示该进程在 CPU 上的运行时间。CFS 通过 vruntime 来决定哪个进程应当运行。
vruntime 是通过标准运行时间调整的，目的是使不同优先级的进程得到公平的 CPU 时间。

2. 红黑树存储就绪进程

就绪队列中的所有进程按照它们的 vruntime 存储在红黑树中。

红黑树是一种自平衡二叉搜索树，能够在 $O(\log n)$ 时间内进行插入、删除和查找操作。

3. 查找最小 `vruntime` 的进程

调度器需要快速找到 `vruntime` 最小的进程，即红黑树中的最左边节点。

红黑树确保了快速的查找、插入和删除操作，使得调度器能在常数时间内找到下一个要运行的进程。

4. 平衡操作

红黑树的平衡操作确保在插入和删除进程时，树的高度保持在 $O(\log n)$ ，从而保证了调度操作的高效性。

具体操作步骤：

1. 进程插入：

当一个新进程进入就绪队列时，调度器将其根据 `vruntime` 插入到红黑树中。插入操作是 $O(\log n)$ 时间复杂度。

2. 进程删除：

当一个进程被调度器选中运行并从就绪队列移除时，调度器从红黑树中删除该进程。删除操作也是 $O(\log n)$ 时间复杂度。

3. 进程选择：

调度器通过查找红黑树中 `vruntime` 最小的节点来选择下一个要运行的进程。这可以在 $O(\log n)$ 时间内完成。

7. Vector 底层实现？

图示（方便记忆）：

表. Vector 底层实现

特性	描述
数据结构	动态数组
内存管理	通过动态分配和扩展机制管理内存
元素存储	连续的内存块
访问效率	随机访问高效，插入删除在末尾高效，中间较慢

总结（老秦推荐必看，方便理解）：

`std::vector` 是基于动态数组实现的。它通过动态分配内存和扩展机制来管理内存，存储元素在连续的内存块中。`vector` 提供高效的随机访问性能，插入和删除操作在末尾最为高效，但在中间进行插入删除操作的效率相对较低。

代码示例（老秦建议：自己动手实践一下）

```
#include <vector>
#include <iostream>
int main() {
    std::vector<int> vec = { 1, 2, 3, 4, 5 };
    for (int val : vec) {
```

```

        std::cout << val << " ";
    }
    std::cout << std::endl;
    return 0;
}

```

8.Vector 默认大小？

总结（老秦推荐必看，方便理解）：

std::vector 在默认构造时是空的，大小为 **0**，不会分配任何内存。

代码示例（老秦建议：自己动手实践一下）

```

#include <vector>
#include <iostream>
int main() {
    std::vector<int> vec;
    std::cout << "Size: " << vec.size() << std::endl; // 输出: Size: 0
    return 0;
}

```

9.Vector 的扩容机制？

总结（老秦推荐必看，方便理解）：

std::vector 的扩容机制是在容量不足时，通常以 **1.5 倍或 2 倍** 的增长率扩展容量。
扩容时会分配新内存，复制旧数据到新内存，释放旧内存。

代码示例（老秦建议：自己动手实践一下）

```

#include <vector>
#include <iostream>
int main() {
    std::vector<int> vec;
    for (int i = 0; i < 10; ++i) {
        vec.push_back(i);
        std::cout << "Size: " << vec.size() << ", Capacity: " <<
vec.capacity() << std::endl;
    }
    return 0;
}

```

10.迭代器的++it、it++哪个好？

图示（方便记忆）：

表. 迭代器的++it 和 it++对比

操作	描述
++it	前置递增，效率更高
it++	后置递增，需要临时对象，效率较低

总结（老秦推荐必看，方便理解）：

++it 前置递增效率更高，因为它直接对迭代器进行递增操作，不需要创建临时对象；而 it++ 后置递增需要创建临时对象，效率较低。一般建议使用 ++it。

代码示例（老秦建议：自己动手实践一下）

```
#include <vector>
#include <iostream>
int main() {
    std::vector<int> vec = { 1, 2, 3, 4, 5 };
    for (auto it = vec.begin(); it != vec.end(); ++it) { // 使用++it
        std::cout << *it << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

11.Vector 与 List 的区别？

图示（方便记忆）：

表. Vector 与 List 的区别

特性	vector	list
数据结构	动态数组	双向链表
内存布局	连续的内存块	分散的内存块
访问效率	随机访问高效	随机访问低效，需遍历
插入/删除效率	末尾高效，中间低效	中间高效，末尾和开头较高效

总结（老秦推荐必看，方便理解）：

std::vector 是基于动态数组实现的，提供高效的随机访问性能，插入和删除操作在末尾最为高效，但在中间进行插入删除操作的效率相对较低。而 std::list 是基于双向链表实现的，随机访问性能较低，但在任意位置的插入和删除操作都很高效。

12.STL 中 list 与 duque 之间的区别？

图示（方便记忆）：

表. Vector 释放空间

特性	list	duque
数据结构	双向链表	双端队列，分段连续内存
内存布局	分散的内存块	分段连续的内存块
访问效率	随机访问低效，需遍历	随机访问高效
插入/删除效率	中间高效，末尾和开头较高效	末尾和开头高效，中间较低效

总结（老秦推荐必看，方便理解）：

std::list 是基于双向链表实现的，适用于需要频繁在任意位置插入或删除元素的场景。而 std::deque 是双端队列，分段连续内存布局，提供高效的随机访问和在两端进行插入和删除的高效操作。

13.Vector 如何释放空间？

图示（方便记忆）：

表. Vector 释放空间

操作	描述
清空内容	clear 清空内容，但不释放容量
释放多余容量	Shrink_to_fit 释放多余容量，减少内存使用
删除元素	删除元素后，内存不会自动释放，需要手动调用 Shrink_to_fit

总结（老秦推荐必看，方便理解）：

std::vector 在删除元素后不会自动释放内存，需要调用 shrink_to_fit 以减少内存使用。clear 可以清空内容但不释放容量。

代码示例（老秦建议：自己动手实践一下）

```
#include <vector>
#include <iostream>
int main() {
    std::vector<int> vec = { 1, 2, 3, 4, 5 };
    vec.clear(); // 清空内容
    std::cout << "Size after clear: " << vec.size() << ", Capacity: " <<
vec.capacity() << std::endl;
    vec.shrink_to_fit(); // 释放多余容量
    std::cout << "Size after shrink_to_fit: " << vec.size() << ", Capacity: " <<
vec.capacity() << std::endl;
    return 0;
}
```

14.map 插入方式有哪几种？区别是什么？

图示（方便记忆）：

表. map 插入方式

插入方式	描述
insert	插入元素，返回 <code>pair<iterator, bool></code> ，表示插入结果
operator[]	使用 <code>[]</code> 操作符插入或修改元素
emplace	直接构造元素，避免不必要的拷贝和移动操作

总结（老秦推荐必看，方便理解）：

std::map 提供多种插入方式：

- insert**：返回插入结果的指示器和布尔值，表示是否插入成功；
- operator[]**：插入或修改元素；
- emplace**：直接构造元素，避免不必要的拷贝和移动操作，提高效率。

代码示例（老秦建议：自己动手实践一下）

```
#include <map>
#include <iostream>
int main() {
    std::map<int, std::string> m;
    // insert
    m.insert({ 1, "one" });
    // operator[]
    m[2] = "two";
    // emplace
    m.emplace(3, "three");
    for (const auto& [key, value] : m) {
        std::cout << key << ": " << value << std::endl;
    }
    return 0;
}
```

15.迭代器失效情况？

图示（方便记忆）：

表. 迭代器失效表格

容器类型	迭代器失效的情况
std::vector	插入、删除、扩容操作后迭代器失效

<code>std::list</code>	删除元素后迭代器指向被删除的元素时失效
<code>std::map</code>	删除元素后迭代器指向被删除的元素时失效
<code>std::deque</code>	插入、删除、扩容操作后迭代器失效

总结（老秦推荐必看，方便理解）：

迭代器失效是指在容器进行插入、删除或扩容操作后，原有的迭代器可能变得无效。不同容器的迭代器失效情况不同，`std::vector` 在插入、删除、扩容操作后迭代器失效；`std::list` 和 `std::map` 在删除元素后，指向被删除元素的迭代器失效；`std::deque` 在插入、删除、扩容操作后迭代器失效。

16.vector 容器避免 push_back 时扩展容器开销的办法。

总结（老秦推荐必看，方便理解）：

使用 `reserve` 方法预分配足够的容量，可以避免在使用 `push_back` 插入元素时多次扩展容器的开销，从而提高性能。

代码示例（老秦建议：自己动手实践一下）

```
#include <vector>
#include <iostream>
int main() {
    std::vector<int> vec;
    vec.reserve(100); // 预分配 100 个元素的容量
    for (int i = 0; i < 100; ++i) {
        vec.push_back(i);
    }
    std::cout << "Size: " << vec.size() << ", Capacity: " << vec.capacity()
    << std::endl;
    return 0;
}
```

17.哈希冲突 stl 怎么解决的？

图示（方便记忆）：

哈希冲突解决方法对比表格

哈希冲突解决方法	优点	缺点
拉链法	1. 弹性扩展：链表理论上	1. 不一致的查找时间：查找效率取决于

	可以无限长，只受限于可用内存，能处理大量冲突。 2. 实现简单：链表的数据结构简单，易于实现和维护。	链表的长度，在最坏情况下可能变成线性时间，即 $O(n)$ 。 2. 内存使用不均：部分槽位可能对应长链表，而其他槽位则可能空闲。
开放寻址法	1. 内存利用率高：不需要额外空间存储指针或链表。 2. 缓存友好：数据连续存储，有利于缓存优化。	1. 填充因子敏感：填充因子高时，性能显著下降。 2. 删除操作复杂：删除元素后，不能简单地将槽位标记为空，否则会破坏探测序列。通常标记为“已删除”，并在新插入时重用这些槽位。

干货(C++的哈希表底层知识，面试加分)

阈值和负载因子 (Load Factor)

负载因子：哈希表的负载因子定义为当前存储在哈希表中的元素数量与哈希表桶 (bucket) 数量的比例。负载因子是衡量哈希表满的程度的一个指标。公式为：

$$\text{Load Factor} = \frac{\text{Number of elements in the hash table}}{\text{Number of buckets}}$$

阈值 (Threshold)：是控制哈希表何时扩容的负载因子的最大值。当**哈希表的负载因子超过这个阈值时**，哈希表将**自动增加桶的数量 (rehashing)**，以分散现有的元素并减少碰撞，维持操作的效率。

扩容 (Rehashing)：当**元素的插入导致负载因子超过预定的阈值时**，哈希表需要进行扩容。这个过程包括创建一个更大的哈希表并重新计算每个元素的哈希值，将它们插入到新的哈希表中。这是一个计算成本较高的操作，但是它有助于维持哈希表的性能，防止由于桶过度填满而导致的效率下降。

配置和影响

设置和获取阈值：在 C++ 中，你可以通过 `load_factor()` 函数来获取当前的负载因子，通过 `max_load_factor()` 函数来获取或设置允许的最大负载因子。例如，如果你认为哈希表的扩容过于频繁，可能会选择增加最大负载因子的值。

性能考量：选择合适的最大负载因子是重要的，较低的负载因子可以减少哈希碰撞，提高查询效率，但会使用更多的内存。较高的负载因子虽然内存使用率更高效，但可能会增加哈希碰撞的频率，降低操作效率。

哈希表的应用：

快速查找：哈希表提供平均时间复杂度为 $O(1)$ 的查找效率，极大提高数据处理速度。

数据去重：使用哈希集合（如 `std::unordered_set`）来快速判断元素是否已经存在于集合中，从而实现去重。

数据索引：哈希表能够快速定位数据，常用于构建数据库和缓存系统的索引。

18.vector 的 `resize()` 和 `reserve()` 区别？

图示（方便记忆）：

表. `resize()`和 `reserve()`的区别

方法	描述
<code>resize</code>	改变 <code>vector</code> 的大小，并初始化新元素
<code>reserve</code>	预分配内存，但不改变 <code>vector</code> 的大小

总结（老秦推荐必看，方便理解）：

`resize` 用于改变 `vector` 的大小，并且如果新大小大于原大小，会初始化新元素。`reserve` 用于预分配内存，但不改变 `vector` 的大小，仅改变容量，以减少多次分配内存的开销。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <vector>
int main() {
    std::vector<int> vec;
    vec.resize(5); // 改变大小为 5，初始化新元素
    std::cout << "Size after resize: " << vec.size() << ", Capacity: " <<
vec.capacity() << std::endl;
    vec.reserve(10); // 预分配容量为 10，但不改变大小
    std::cout << "Size after reserve: " << vec.size() << ", Capacity: " <<
vec.capacity() << std::endl;
    return 0;
}
```

19.Vector 的 `emplace_back()` 和 `push_back()`区别?

图示（方便记忆）：

表. `emplace_back()`和 `push_back()`的区别

方法	描述
<code>Emplace_back()</code>	在原地构造元素，避免不必要的拷贝和移动操作
<code>Push_back()</code>	复制或移动元素到 <code>vector</code> 的末尾

总结（老秦推荐必看，方便理解）：

`Emplace_back()`用于在原地构造元素，避免了不必要的拷贝和移动操作，适用于需要高效插入复杂对象的情况。`push_back()` 则是将已有的元素复制或移动到 `vector` 的末尾。

20.描述 priority_queue 的底层原理及使用场景？

总结（老秦推荐必看，方便理解）：

底层原理：std::priority_queue 是基于堆实现的，通常使用 std::vector 作为底层容器，并通过堆算法（如 std::make_heap、std::push_heap 和 std::pop_heap）来维护堆结构。

使用场景：适用于需要快速访问最大或最小元素的场景，如任务调度、事件驱动的模拟、路径搜索算法等。

代码示例（老秦建议：自己动手实践一下）

```
#include <queue>
#include <vector>
#include <functional>
#include <iostream>
int main() {
    std::priority_queue<int> pq; // 最大堆
    pq.push(10);
    pq.push(30);
    pq.push(20);
    while (!pq.empty()) {
        std::cout << pq.top() << " ";
        pq.pop();
    }
    return 0;
}
```

21.何时应使用 forward_list ？

总结（老秦推荐必看，方便理解）：

forward_list 的特点：std::forward_list 是**单向链表**，内存占用更少，因为它只存储一个指向下一个元素的指针。

使用场景：适用于**内存有限且只需要单向遍历的场景**，例如读取大量数据并进行线性处理的操作。

代码示例（老秦建议：自己动手实践一下）

```
#include <forward_list>
#include <iostream>
int main() {
    std::forward_list<int> fl = { 1, 2, 3, 4, 5 };
    for (const auto& num : fl) {
        std::cout << num << " ";
    }
}
```

```
}  
return 0;  
}
```

22. 什么时候会使用 `std::stack`，它的底层容器是什么？

总结（老秦推荐必看，方便理解）：

用途：`std::stack` 是后进先出（LIFO）的数据结构，适用于需要这种访问顺序的场景，如函数调用栈、撤销操作等。

底层容器：默认使用 `std::deque` 作为底层容器，但可以使用其他容器如 `std::vector` 或 `std::list`。

代码示例（老秦建议：自己动手实践一下）

```
#include <stack>  
#include <iostream>  
int main() {  
    std::stack<int> stack;  
    stack.push(1);  
    stack.push(2);  
    stack.push(3);  
    while (!stack.empty()) {  
        std::cout << stack.top() << " ";  
        stack.pop();  
    }  
    return 0;  
}
```

23. 如何优化大量数据的插入处理，特别是在 `std::vector` 中？

总结（老秦推荐必看，方便理解）：

reserve 方法：使用 `reserve` 预先分配足够的内存，避免多次重新分配和数据移动。

批量插入：使用 `insert` 进行批量插入，而不是逐个插入，减少内存重新分配次数。

代码示例（老秦建议：自己动手实践一下）

```
#include <vector>  
#include <iostream>  
int main() {  
    std::vector<int> vec;
```

```

    vec.reserve(100); // 预分配 100 个元素的空间
    for (int i = 0; i < 100; ++i) {
        vec.push_back(i);
    }
    std::cout << "Size: " << vec.size() << ", Capacity: " << vec.capacity()
<< std::endl;
    return 0;
}

```

24.std::vector::at()

与

std::vector::operator[]的区别是什么？

总结（老秦推荐必看，方便理解）：

at()：提供边界检查，如果访问越界会抛出 `std::out_of_range` 异常。

operator[]：不提供边界检查，访问越界会导致未定义行为。

代码示例（老秦建议：自己动手实践一下）

```

#include <vector>
#include <iostream>
int main() {
    std::vector<int> vec = { 1, 2, 3 };
    try {
        std::cout << vec.at(2) << std::endl; // 安全访问
        std::cout << vec.at(5) << std::endl; // 会抛出异常
    }
    catch (const std::out_of_range& e) {
        std::cerr << e.what() << std::endl;
    }
    std::cout << vec[2] << std::endl; // 不安全访问，可能会导致未定义行为
    return 0;
}

```

25.STL 中排序算法如何工作？它们如何与容器互动？

总结（老秦推荐必看，方便理解）：

std::sort：通常使用快速排序（quicksort）、堆排序（heapsort）和插入排序（insertionsort）的混合算法，适用于随机访问迭代器，如 `vector` 和 `deque`。

std::stable_sort：使用归并排序，保持相同元素的相对顺序，适用于所有类型的迭代器。

代码示例（老秦建议：自己动手实践一下）

```
#include <vector>
#include <algorithm>
#include <iostream>
int main() {
    std::vector<int> vec = { 5, 2, 9, 1, 5, 6 };
    std::sort(vec.begin(), vec.end());
    for (const auto& num : vec) {
        std::cout << num << " ";
    }
    return 0;
}
```

26.STL 迭代器和指针有什么区别？

总结（老秦推荐必看，方便理解）：

迭代器：抽象化容器的访问，不一定是指针。提供统一的接口用于访问不同类型的容器。

指针：实际的内存地址表示，适用于数组等简单结构。

图示（方便记忆）：

特性	指针	迭代器
定义	基本数据类型，直接指向内存地址。	抽象到容器元素的访问器。
支持操作	支持算术运算，如加减、比较等。	运算能力依赖于迭代器的类型，不一定支持算术运算。
类型安全	类型安全性较低，易出错。	更类型安全，通常由容器直接提供。
用途	适用于任何类型的内存操作和数组。	主要用于 STL 容器中，管理容器元素的遍历。
管理难度	需要手动管理指针的生命周期和边界。	容器通常提供方法管理迭代器的起始和结束。

27.deque 的实现？

总结（老秦推荐必看，方便理解）：

(1) 实现细节

- 内部结构：由多个固定大小的数组和一个中控结构组成，中控结构含有指向这些数组的指针。
- 操作实现：

- o 随机访问：通过计算索引快速定位数组和位置。
- o 插入和删除：
 - 头部：若头部数组满，新建数组。
 - 尾部：若尾部数组满，同样新建数组。

(2) 应用场景

- 实时应用：适用于需要两端快速操作的系统，如消息队列。
- 滑动窗口：适用于需要频繁添加和删除的算法。

(3) 优势和缺点

- 优势：两端操作快速，灵活管理内存。
- 缺点：随机访问中间元素可能比连续内存容器慢。

28.Vector 扩容后只使用小部分，怎么释放后面的空间？

总结（老秦推荐必看，方便理解）：

在 C++ 中，如果 **vector** 扩容后只使用了少部分空间，可以通过下面的方法释放未使用的空间：

(1) Swap 技巧：

- 创建一个新的 vector 对象，使用当前 vector 的内容初始化。
- 与原 vector 交换内容。
- 由于新 vector 的容量仅足够存储其元素，这一操作将减小原 vector 的容量。

```
std::vector<int> v;  
// 假设对 v 进行了一系列操作  
  
std::vector<int>(v).swap(v);
```

(2) C++11 及以上版本的 shrink_to_fit()：

- 调用 shrink_to_fit() 成员函数尝试请求容器减少其容量以匹配其大小。

29.常用 STL 底层实现？

图示（方便记忆）：

STL 容器	底层实现结构	特点
vector	动态数组	支持快速随机访问，动态扩容。
deque	多个固定大小数组	两端可扩展，随机访问。
list	双向链表	支持快速插入和删除。
set	平衡二叉树（红黑树）	元素唯一，自动排序。
map	平衡二叉树（红黑树）	基于键值对，自动根据键排序。
unordered_set	哈希表	元素唯一，快速访问。
unordered_map	哈希表	基于键值对，快速访问。
stack	通常使用 deque 或 list	LIFO（后进先出）操作。
queue	通常使用 deque	FIFO（先进先出）操作。
priority_queue	通常使用 vector	堆结构，支持快速最大或最小元素访问。

30.map 和 set 插入删除有啥区别？

总结（老秦推荐必看，方便理解）：

在 C++ STL 中，map 和 set 都使用红黑树（一种自平衡二叉搜索树）作为底层数据结构，但它们处理插入和删除操作的细节有所不同：

- **map**：存储键值对（pair<const Key, Value>）。插入操作会检查键是否已存在，存在则插入失败，保持原值不变；不存在则创建新节点。删除操作通过键找到节点并移除。
- **set**：只存储键，不存值。插入操作同样检查元素是否存在，存在则失败；不存在则添加新元素。删除操作直接通过元素值找到并移除节点。

两者操作的时间复杂度均为 $O(\log n)$ ，因为红黑树保证树的高度始终保持对数级别。

31.vector clear() 是清空所有元素还是清空内存

存？

总结（老秦推荐必看，方便理解）：

在 C++ STL 的 vector 中，clear() 操作**销毁容器中的所有元素**，但**并不释放已分配的内存**。此函数通过调用每个元素的析构函数来销毁元素，然后将 vector 的 size 设置为 0。然而，vector 的 capacity() 值保持不变，表示内存空间依旧被保留，以避免重新分配内存所需的开销。

第四章 计算机网络

一．基础知识

1. OSI 七层模型和 TCP/IP 模型的区别？

图示（方便记忆）：

表. OSI 七层模型和 TCP/IP 模型的区别

模型	OSI 七层模型	TCP/IP 模型
层级数	7 层	4 层
层级划分	物理层、数据链路层、网络层、传输层、会话层、表示层、应用层	网络接口层、网络层、传输层、应用层

表. 层级作用对比

OSI 层级	TCP/IP 层级	作用
物理层	网络接口层	传输原始比特流，通过物理媒介传输数据。
数据链路层		节点之间的可靠数据传输，数据帧的发送与接收。
网络层	网络层	数据包的路由和转发，实现网络间的通信。
传输层	传输层	提供端到端的通信服务，确保数据的完整传输 (TCP、UDP)。
会话层	应用层	管理会话和连接。
表示层		数据格式转换、加密解密、数据压缩。
应用层		提供网络服务和应用程序接口 (HTTP、FTP、SMTP 等)。

2. TCP 和 UDP 的区别？

图示（方便记忆）：

表. TCP 和 UDP 的区别

特性	TCP	UDP
连接类型	面向连接（需要建立连接）	无连接（无需建立连接）
传输可靠性	可靠传输，提供确认、重传和流量控制	不可靠传输，不保证数据到达
数据顺序	保证数据按序到达	数据可能乱序到达
速度	较慢	较快
适用场景	文件传输、HTTP、邮件	视频流、VoIP、DNS

3. IP 地址的分类及其作用？

图示（方便记忆）：

表. IP 地址的分类及其作用

类别	地址范围	网络数量	每个网络的主机数	适用场景
A类	1.0.0.0 - 126.0.0.0	128	16,777,214	大型网络
B类	128.0.0.0 - 191.255.0.0	16,384	65,534	中型网络
C类	192.0.0.0 - 223.255.255.0	2,097,152	254	小型网络
D类	224.0.0.0 - 239.255.255.255	-	-	多播
E类	240.0.0.0 - 255.255.255.255	-	-	实验用途

4. 什么是子网掩码？如何进行子网划分？

图示（方便记忆）：

表. 子网掩码及示例

类别	默认子网掩码
A类	255.0.0.0
B类	255.255.0.0
C类	255.255.255.0

子网划分过程

假设需要将一个 B 类网络 172.16.0.0 划分成 16 个子网：

- 1) 计算子网位数：需要 4 个子网位，因为 2 的 4 次方=16。
- 2) 新子网掩码：默认 B 类子网掩码是 255.255.0.0，增加 4 个子网位后，变为 255.255.240.0。
- 3) 新网络地址范围：每个子网包含 $2^{(32-20)} - 2 = 4096 - 2 = 4094$ 个主机。

5. MAC 地址和 IP 地址的区别？

图示（方便记忆）：

表. MAC 地址和 IP 地址的区别

特性	MAC 地址	IP 地址
层级	数据链路层	网络层
长度	48 位	32 位 (IPv4) /128 位 (IPv6)
唯一性	全球唯一	网络内唯一
固定性	硬件固定，不易更改	可配置和更改
特性	MAC 地址	IP 地址

6.解释网络带宽的概念及其优化方法？

总结（老秦推荐必看，方便理解）：

(1)网络带宽

网络带宽是指网络能够传输数据的最大速率，通常以**比特每秒（bps）**为单位表示。带宽越大，网络传输能力越强。

(2)优化方法

- 1) 增加带宽：升级网络设备，如交换机、路由器等。购买**更高带宽**的网络服务。
- 2) 数据压缩：使用压缩技术**减少传输的数据量**，从而提高有效带宽利用率。
- 3) 负载均衡：使用**负载均衡技术**将流量分配到多个服务器上，避免单点过载。
- 4) 网络缓存：使用缓存技术**减少重复传输**的内容，提高传输效率。
- 5) 流量控制：实施**流量控制策略**，防止网络拥塞，确保重要数据优先传输。

7.什么是丢包和延迟？如何处理？

总结（老秦推荐必看，方便理解）：

丢包：丢包是指数据包在传输过程中丢失，导致接收方未能收到数据。

延迟：延迟是指数据从发送方到接收方的时间延迟。

处理方法

(1)丢包处理：

- 1) 增加带宽：通过**升级网络设备和**服务增加带宽，减少丢包率。
- 2) 优化网络设备：使用**高性能路由器、交换机**等设备，确保数据包顺利传输。
- 3) 使用可靠传输协议：如 **TCP 协议**，通过确认和重传机制保证数据传输的可靠性。
- 4) 重传机制：在检测到丢包时，**重新发送数据包**。

(2)延迟处理：

- 1) 优化路由和网络路径：选择**最优路径**传输数据，**减少中间节点和跳数**。
- 2) 使用内容分发网络（CDN）：在**接近用户的位置**缓存内容，**减少传输距离和时间**。
- 3) 减少中间节点：通过**直连方式**减少数据传输中的中间节点，降低延迟。

8.端口有效范围是多少到多少？

图示（方便记忆）：

表. 端口范围

范围	描述
0-1023	系统保留端口（知名服务）
1024-49151	用户端口（注册服务和应用）
49152-65535	动态端口（临时连接）

总结（老秦推荐必看，方便理解）：

(1)0-1023（系统端口）：

这些端口号是为一些知名的服务和协议保留的，如 HTTP（80）、HTTPS（443）、FTP（21）、SSH（22）等。

(2)1024-49151（用户端口）：

这些端口号可用于用户注册的服务和应用程序。需要注册以防冲突。

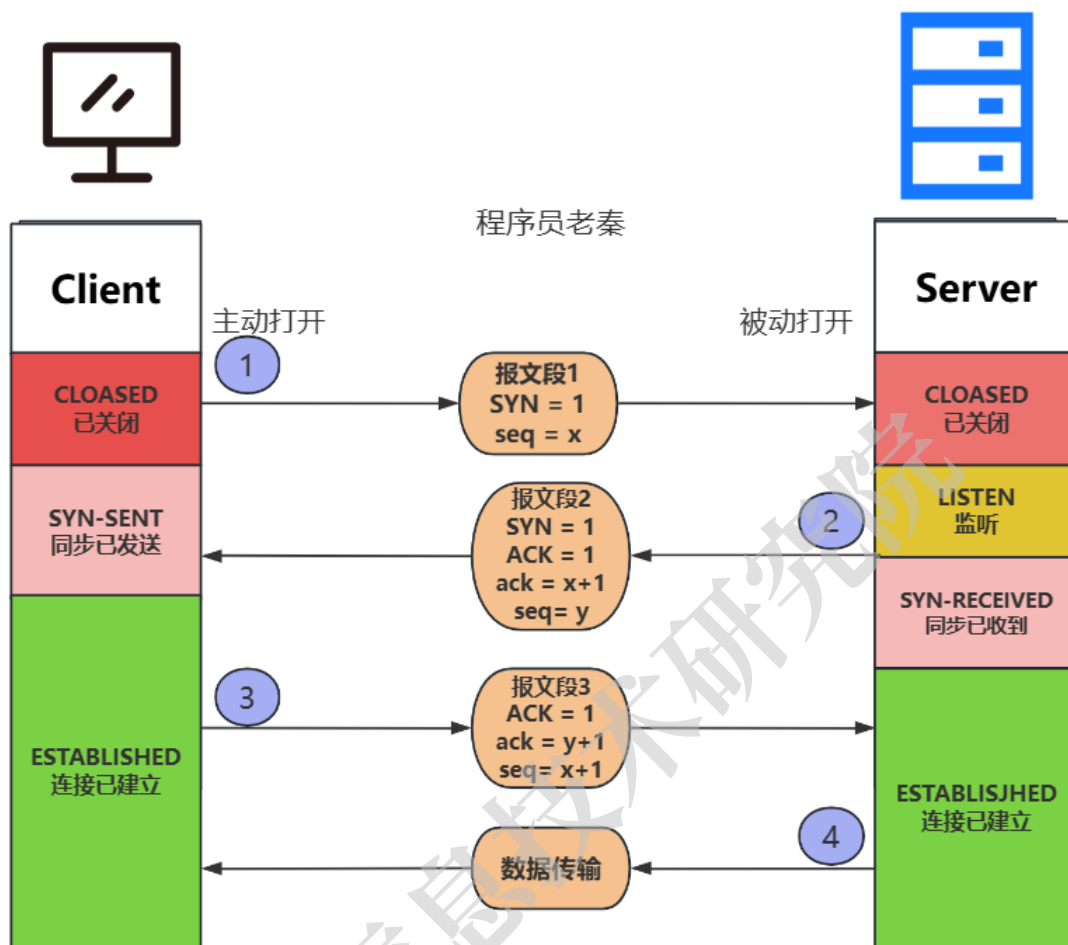
(3)49152-65535（动态端口）：

这些端口号通常用于临时连接，也称为私有端口或临时端口。由操作系统动态分配，用于客户端连接。

二. TCP/IP 协议

9.TCP 三次握手？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)第一次握手（SYN）：

1) 客户端发送 **SYN**：客户端选择一个初始序列号 $seq=x$ ，发送 **SYN** 包给服务器。这个包仅包含 SYN 标志位，并且表示客户端请求建立连接。

2) 客户端状态：客户端进入 **SYN-SENT** 状态，等待服务器的响应。

(2)第二次握手（SYN-ACK）：

1) 服务器收到并响应 **SYN**：服务器收到客户端的 SYN 包后，选择一个初始序列号 $seq=y$ ，并发送一个 **SYN-ACK** 包给客户端。SYN-ACK 包中包含服务器的序列号 $seq=y$ 以及对客户端 SYN 的确认 $ack=x+1$ 。

2) 服务器状态：服务器进入 **SYN-RECEIVED** 状态，等待客户端的确认。

(3)第三次握手（ACK）：

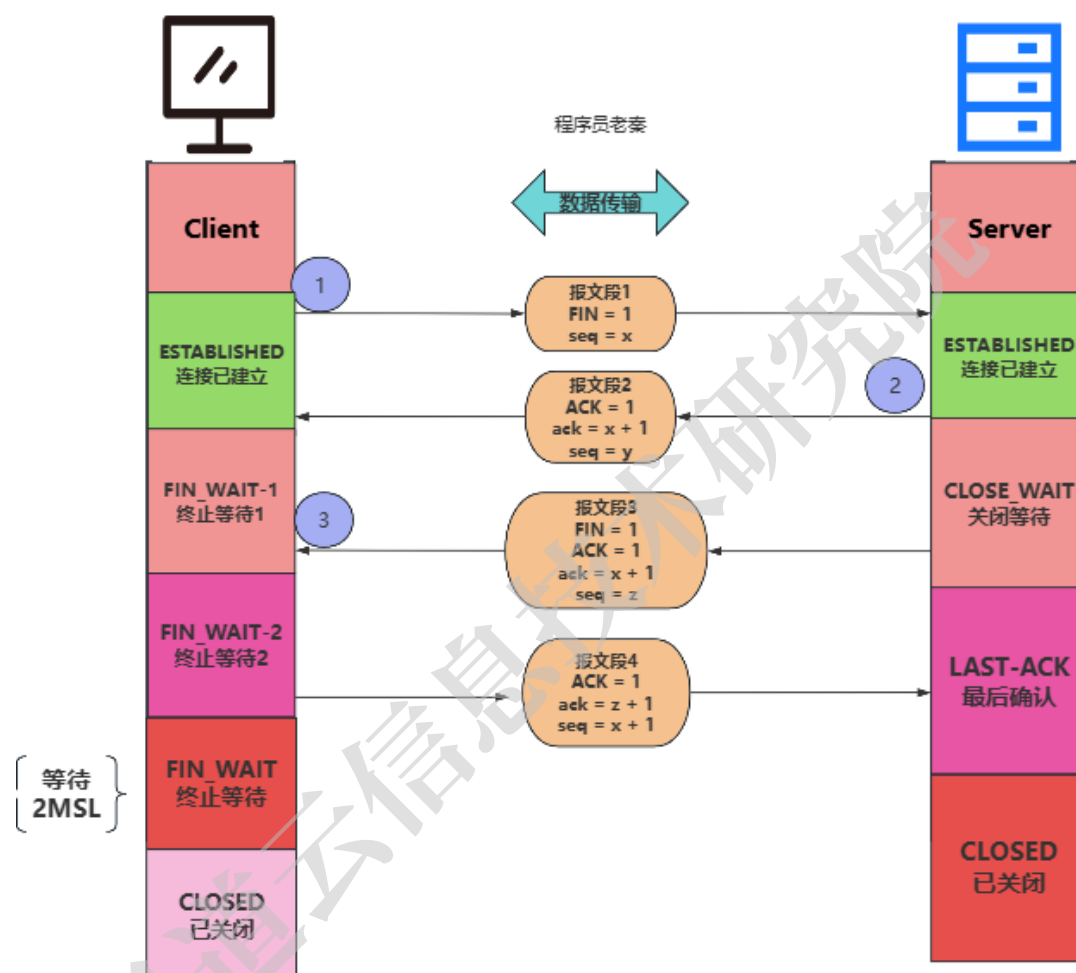
1) 客户端确认连接：客户端收到服务器的 SYN-ACK 包后，发送一个 **ACK** 包给服务器。ACK 包中包含对服务器序列号的确认 $ack=y+1$ ，表示客户端已经收到了服务器的 SYN 包。

2) 双方状态：客户端和服务器都进入 **ESTABLISHED** 状态，连接成功建立，接下来

可以进行数据传输。

10. TCP 的四次挥手？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)第一次挥手（FIN）：

1) 客户端发送 **FIN**：客户端选择一个序列号 $seq=x$ ，发送 **FIN** 包给服务器。这个包表示客户端已经没有数据要发送，准备关闭连接。

2) 客户端状态：客户端进入 **FIN-WAIT-1** 状态，等待服务器的响应。

(2)第二次挥手（ACK）：

1) 服务器发送 **ACK**：服务器接收到 **FIN** 包后，发送一个 **ACK** 包 ($ack=x+1$) 给客户端，表示已经收到客户端的关闭请求。

2) 服务器状态：服务器进入 **CLOSE-WAIT** 状态，表示服务器仍然可能有数据要发送。

3) 客户端状态：客户端进入 FIN-WAIT-2 状态，等待服务器的关闭请求。

(3)第三次挥手 (FIN)：

1) 服务器发送 FIN：服务器完成数据发送后，选择一个序列号 $seq=y$ ，发送 FIN 包给客户端，表示服务器也准备关闭连接。

2) 服务器状态：服务器进入 LAST-ACK 状态，等待客户端的确认。

(4)第四次挥手 (ACK)：

1) 客户端发送 ACK：客户端收到服务器的 FIN 包后，发送一个 ACK 包 ($ack=y+1$) 给服务器，表示确认关闭连接。

2) 客户端状态：客户端进入 TIME-WAIT 状态，等待 2MSL (最大报文生存时间) 后关闭连接，确保连接彻底断开。

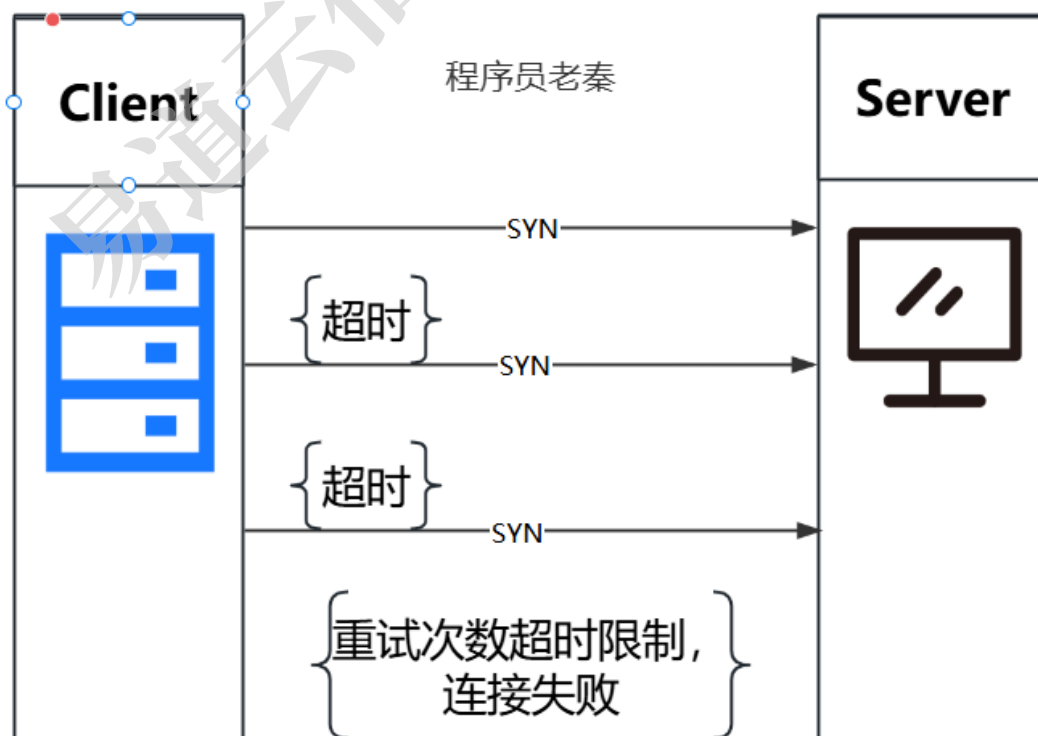
3) 服务器状态：服务器收到 ACK 包后，进入 CLOSED 状态，连接断开。

注意事项：

TIME-WAIT 状态：客户端在进入 TIME-WAIT 状态后，需等待 2MSL (Maximum Segment Lifetime, 最大报文生存时间) 以确保所有报文在网络中被正确处理，避免旧的重复数据包干扰新连接。

11. 三次握手中如果数据发送失败，让你设计，你会怎么处理？

图示 (方便记忆)：



总结（老秦推荐必看，方便理解）：

(1)重试机制：

在客户端发送 SYN 包后，如果在设定的超时时间内未收到服务器的 SYN-ACK 包，客户端将重发 SYN 包。

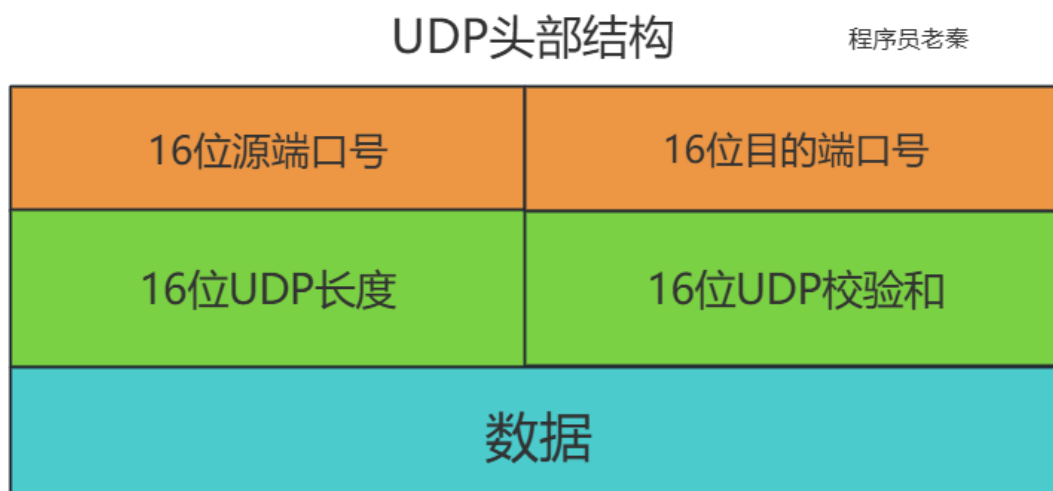
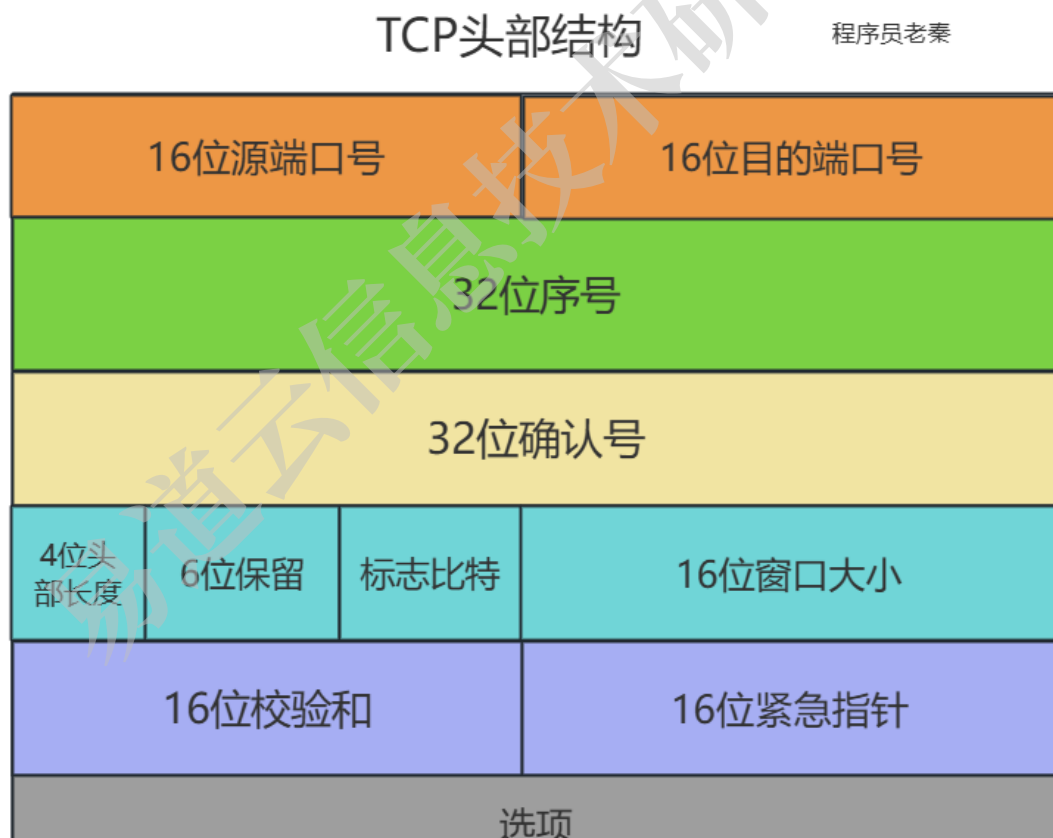
重试次数有一个上限，例如 5 次，如果重试次数达到上限，客户端将放弃连接请求。

(2)超时机制：

设定一个合理的超时时间，例如 2 秒，客户端在发送 SYN 包后等待这个时间，如果未收到 SYN-ACK 包，则进行重试。

12. TCP 和 UDP 的头部结构？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)TCP 头部：

- 1) 源端口 (**Source Port**)：发送方的端口号。
- 2) 目的端口 (**Destination Port**)：接收方的端口号。
- 3) 序列号 (**Sequence Number**)：用于标识发送的数据字节流的顺序。
- 4) 确认号 (**Acknowledgment Number**)：用于确认接收到的数据字节的下一个序列号。
- 5) 数据偏移 (**Data Offset**)：表示 TCP 头部的长度，以 32 位字为单位。
- 6) 保留 (**Reserved**)：保留位，未使用。
- 7) 控制标志 (**Control Flags**)：包括 URG、ACK、PSH、RST、SYN、FIN 六个控制位。
- 8) 窗口大小 (**Window Size**)：接收窗口的大小，用于流量控制。
- 9) 校验和 (**Checksum**)：用于校验头部和数据部分的完整性。
- 10) 紧急指针 (**Urgent Pointer**)：指示紧急数据的结束位置。
- 11) 选项 (**Options**)：可选字段，长度可变。

(2)UDP 头部：

- 1) 源端口 (**Source Port**)：发送方的端口号。
- 2) 目的端口 (**Destination Port**)：接收方的端口号。
- 3) 长度 (**Length**)：UDP 头部和数据的总长度，以字节为单位。
- 4) 校验和 (**Checksum**)：用于校验头部和数据部分的完整性。

13. TCP 流量控制？

总结（老秦推荐必看，方便理解）：

TCP 流量控制是通过滑动窗口机制实现的，确保发送方不会淹没接收方的处理能力。

(1)滑动窗口机制：

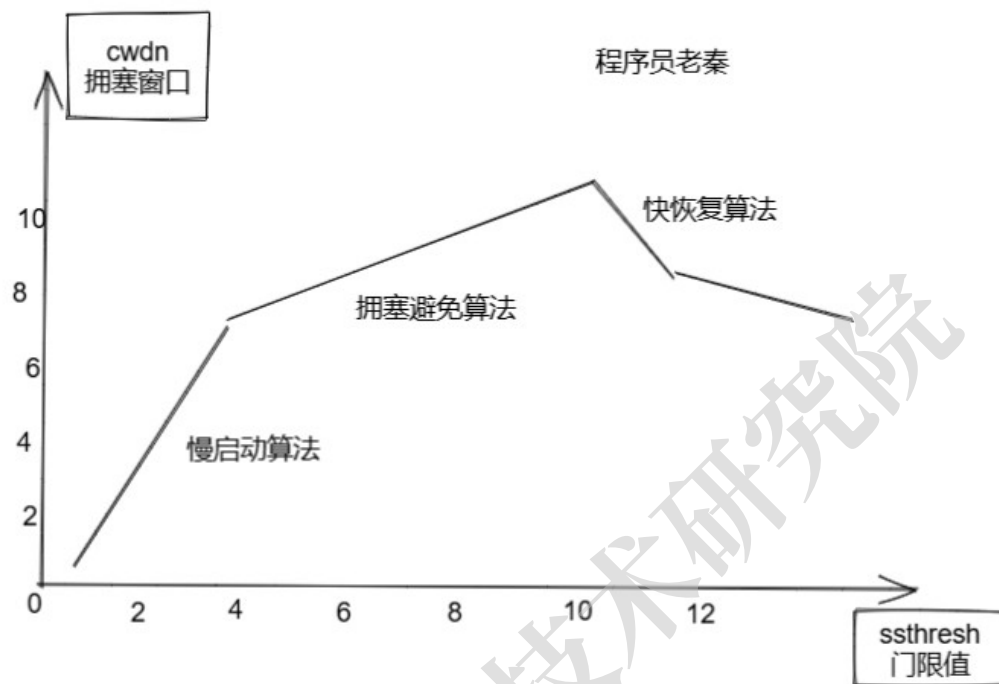
- 1) 发送窗口：发送方维护一个发送窗口，用于控制可以发送但未确认的数据量。发送窗口由已发送但未确认的数据和可以发送的最大数据量组成。
- 2) 接收窗口：接收方维护一个接收窗口，用于控制可以接收但未处理的数据量。接收窗口由接收到但未处理的数据和可以接收的最大数据量组成。

(2)窗口大小的控制：

- 1) 发送方根据接收窗口大小发送数据：发送方在发送数据时，会根据接收方的接收窗口大小来控制发送的数据量，确保不会超出接收方的处理能力。
- 2) 接收方通知发送方当前窗口大小：接收方在接收到数据后，会通过 ACK 包通知发送方当前的接收窗口大小，表示接收方可以继续接收的数据量。
- 3) 零窗口的处理：当接收方的接收窗口为 0 时，发送方会停止发送数据，直到接收到非零窗口通知。发送方可以周期性地发送窗口探测包，询问接收方的接收窗口大小。

14. TCP 拥塞控制机制？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

TCP 拥塞控制机制通过慢启动、拥塞避免、快重传和快恢复来防止网络拥塞。

(1)慢启动 (Slow Start)：

初始阶段：当连接刚建立时，拥塞窗口 (cwnd) 从 1 开始，慢启动阈值 (sssthresh) 通常设为 64。

指数增长：每次收到一个 ACK，拥塞窗口 (cwnd) 加倍，直到达到慢启动阈值 (sssthresh)。

(2)拥塞避免 (Congestion Avoidance)：

达到阈值：当拥塞窗口 (cwnd) 达到慢启动阈值 (sssthresh) 后，进入拥塞避免阶段。

线性增长：每次收到一个 ACK，拥塞窗口 (cwnd) 增加 1，避免网络拥塞。

(3)快重传 (Fast Retransmit)：

检测重复 ACK：如果发送方收到三个重复的 ACK，立即重传丢失的数据包。

快速恢复：避免等待超时重新传输，减少数据传输的延迟。

(4)快恢复 (Fast Recovery)：

恢复拥塞窗口：重传丢失的数据包后，拥塞窗口 (cwnd) 恢复到慢启动阈值

(ssthresh) 的一半，避免拥塞窗口重置为 1。

15. 说一下 KCP ?

总结（老秦推荐必看，方便理解）：

(1)KCP 协议概述：

1) 基于 UDP：KCP 是在 UDP 协议之上实现的，利用 UDP 的简单和快速传输特点，同时引入了 TCP 的一些可靠传输机制。

2) 目标：通过减少延迟和提高传输效率，为实时应用（如在线游戏(王者荣耀)，实时视频等）提供更好的传输性能。

(2)主要功能：

1) 可靠传输：通过确认机制和重传机制，确保数据包按顺序、完整地到达接收方。

2) 流量控制：KCP 使用滑动窗口和流量控制算法，动态调整发送速率，避免网络拥塞。

3) 拥塞控制：KCP 实现了一种自适应的拥塞控制算法，根据网络状况调整发送窗口大小，优化网络资源利用。

4) 快速重传：检测到数据包丢失时，KCP 可以快速重传丢失的数据包，减少传输延迟。

(3)KCP 与 TCP 的区别：

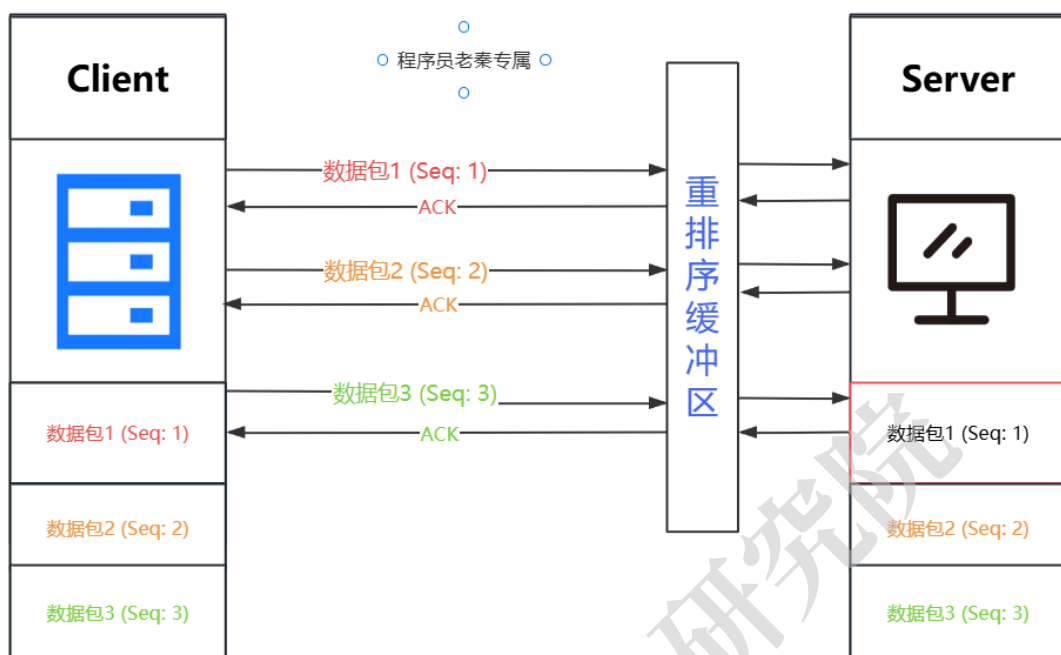
1) 传输层协议：TCP 是内置于操作系统的传输层协议，而 KCP 是基于 UDP 实现的用户态协议。

2) 传输效率：KCP 通过减少握手过程和拥塞控制的优化，具有更高的传输效率和更低的延迟。

3) 实现复杂度：TCP 由操作系统管理，而 KCP 需要在应用层手动管理和实现，但提供了更灵活的控制和优化空间。

16. 你怎么保证 UDP 下帧与帧间的有序性？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1) 序列号机制：

1) 发送方：在发送数据包时，为每个数据包添加一个唯一的序列号。序列号从 0 开始，每发送一个数据包序列号加 1。

2) 接收方：接收到数据包后，检查序列号，以确定数据包的顺序。如果数据包按序到达，则将其交给应用层处理；如果数据包乱序到达，则将其存入重排序缓冲区，等待缺失的数据包到达后再按序处理。

(2) 重排序缓冲区：

接收方：维护一个重排序缓冲区，用于缓存乱序到达的数据包。接收方会检查数据包的序列号，将其插入到正确的位置。如果发现有缺失的数据包，则等待缺失的数据包到达后再按序处理。

(3) ACK 确认机制：

接收方：在接收到数据包后，发送一个 ACK 确认包给发送方，包含已成功接收的最高序列号。发送方根据 ACK 确认包判断哪些数据包已被接收，哪些数据包需要重传。

(4) 丢包重传机制：

发送方：如果在一定时间内没有收到接收方的 ACK 确认包，则认为数据包可能丢失，发送方会重传未确认的数据包，以确保接收方能够接收到所有的数据包。

17. 为什么连接的时候是三次握手，关闭的时候却

是四次挥手？

总结（老秦推荐必看，方便理解）：

三次握手是为了建立连接，确保双方都有能力进行数据传输，防止历史的重复连接请求。

四次挥手是为了可靠地关闭连接，确保双方都能完成各自的数据传输和处理，并释放资源。

(1)三次握手的原因：

1) 防止重复连接：通过三次握手，双方都能确认对方已经准备好通信，防止历史的重复连接请求影响当前连接。

2) 确保双方都能接收和发送：三次握手确保双方都能接收和发送数据，建立可靠的全双工连接。

(2)四次挥手的原因：

1) 半关闭状态：TCP 连接是全双工的，双方都可以独立关闭自己的发送和接收通道。四次挥手可以实现这种半关闭状态，确保双方都能完成各自的数据传输。

2) 可靠释放资源：通过四次挥手，确保双方都能可靠地释放连接资源，避免资源泄露和重复释放的问题。

18. 为什么要三次握手，不能两次吗？

总结（老秦推荐必看，方便理解）：

三次握手确保客户端和服务端都知道连接的存在，并且确认彼此的接收和发送能力。两次握手无法确保双方都确认了连接的存在和彼此的状态，可能导致旧的连接请求被误认为是新的连接。

(1)三次握手的必要性：

1) 第一次握手：客户端发送一个 SYN 包到服务器，表示希望建立连接，并告诉服务器客户端的初始序列号。

2) 第二次握手：服务器收到 SYN 包后，回复一个 SYN+ACK 包，表示同意建立连接，并告诉客户端服务器的初始序列号和确认客户端的序列号。

3) 第三次握手：客户端收到 SYN+ACK 包后，再发送一个 ACK 包给服务器，表示确认收到服务器的 SYN 包，连接建立。

(2)两次握手的问题：

1) 旧连接请求问题：如果使用两次握手，旧的连接请求（可能因为网络延迟而滞留的 SYN 包）可能被误认为是新的连接请求，从而导致连接状态的不一致。

2) 确认机制不足：两次握手无法确保双方都确认了连接的存在。例如，客户端发送 SYN 包后崩溃重启，服务器可能仍然认为连接存在，而客户端已经不再维护这个连接。

(3)三次握手的优点：

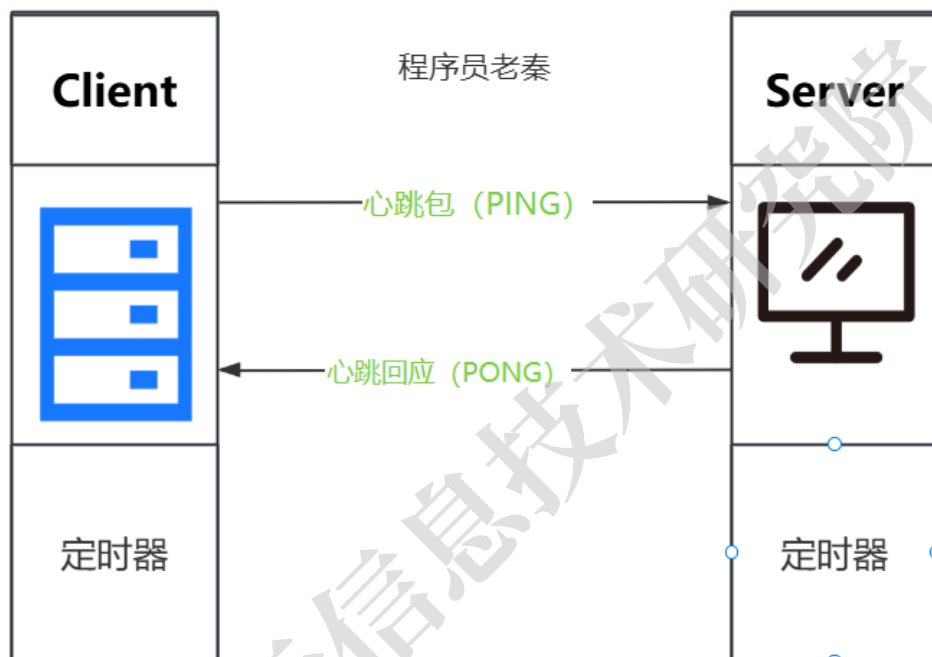
1) 确认双方状态：三次握手能够确保双方都确认了彼此的接收和发送能力，确保连

接的可靠性。

2) 避免旧连接误判：通过三次握手，可以避免旧的连接请求被误认为是新的连接。

19. 长连接是如何维持的，心跳机制，如果一直想要连接着怎么办？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1) 保持长连接的方法：

- 1) 定期发送心跳包：设置定时器，定期发送心跳包，保持连接活跃。
- 2) 连接超时检测：设置合理的超时时间，检测连接状态，及时进行重连操作。
- 3) 资源管理：确保系统资源合理分配，避免长时间保持大量连接导致资源耗尽。

(2) 长连接的维持：

- 1) 应用场景：长连接广泛应用于需要频繁通信的场景，如即时通讯、在线游戏、实时数据传输等。
- 2) 保持连接：通过长连接，可以减少连接建立和关闭的开销，提高通信效率。
- 3) 资源管理：在保持长连接的同时，需要合理管理系统资源，避免长时间保持大量连接导致资源耗尽。

(3)心跳机制：

1) 心跳包 (PING)：客户端和服务端之间定期发送心跳包，以检测连接状态，确保连接在空闲时依然保持活跃。

2) 心跳回应 (PONG)：接收方收到心跳包后，立即发送心跳回应，通知发送方连接状态正常。

3) 超时检测：如果发送方未能在规定时间内接收到心跳回应，则认为连接已断开，进行重连操作。

20. 第一次握手可以传输数据吗？

总结（老秦推荐必看，方便理解）：

(1)第一次握手不能传输数据的原因：

1) 连接未完全建立：在第一次握手时，**连接尚未建立**，服务器还未确认客户端的连接请求。

2) 缺乏确认机制：在第一次握手时，**客户端无法确定服务器是否已收到 SYN 包**，无法保证数据的可靠传输。

3) 安全性考虑：如果**在第一次握手时传输数据**，可能导致旧的连接请求（例如，网络延迟导致的滞留 SYN 包）被误认为是新的连接，造成数据传输混乱。

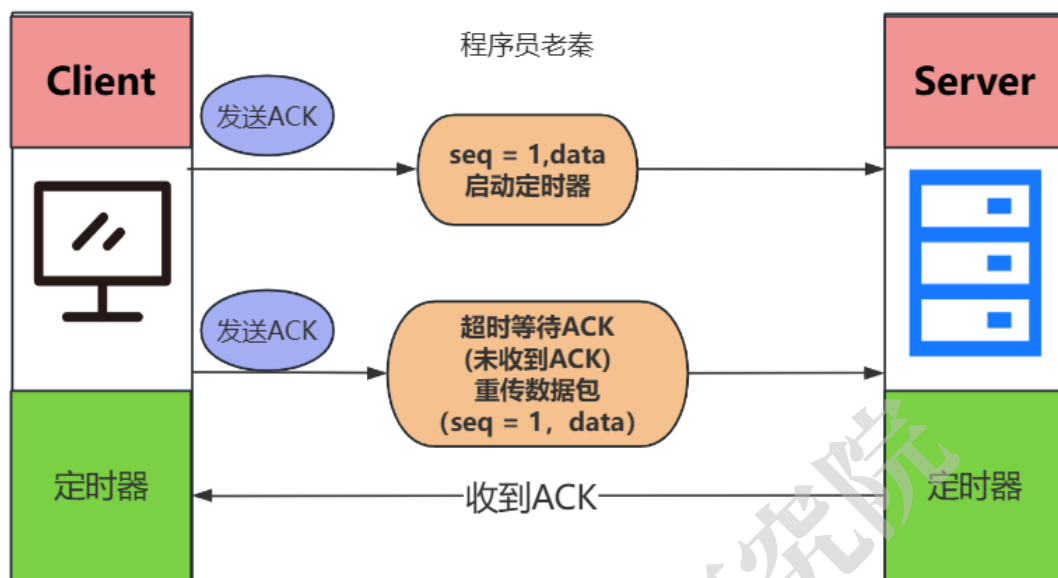
(2)数据传输的时机：

1) 第三次握手之后：**数据传输通常在第三次握手之后进行**，因为只有此时连接才被**完全建立**，双方都确认了彼此的接收和发送能力。

2) 连接建立：在三次握手完成后，客户端和服务端之间的连接被正式建立，数据可以在此基础上进行可靠传输。

21. 超时重传如何实现？超时重传时间怎么确定？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

1) 超时重传：在 TCP 协议中，如果发送方在预定的时间内没有收到接收方的确认 (ACK)，则会认为数据包可能丢失，并重传该数据包。

2) 超时重传时间 (RTO)：重传超时时间是根据往返时间 (RTT) 的测量和估计来确定的。通过动态调整 RTO，TCP 能够适应网络状况的变化，确保可靠传输。

(1) RTT 测量和 RTO 计算：

- 1) RTT 测量：在发送数据包并接收到 ACK 后，测量往返时间 (RTT)。
- 2) SRTT 计算：使用加权平均算法计算平滑 RTT 估计值 (SRTT)。
- 3) RTT 方差计算：计算 RTT 的变化幅度 (RTT_variance)。
- 4) RTO 计算：根据 SRTT 和 RTT_variance 计算重传超时时间 (RTO)。

(2) 超时重传的实现：

- 1) 发送数据包：发送数据包并启动计时器。
- 2) 等待 ACK：在预定时间内等待 ACK 确认。
- 3) 超时重传：如果未收到 ACK，在 RTO 超时时间到达后重传数据包，并重新启动计时器。

(3) 动态调整 RTO：

- 1) RTT 变化：随着网络状况变化，RTT 和 RTT_variance 会动态变化。
- 2) 调整 RTO：根据最新的 RTT 测量和估计，动态调整 RTO，以适应当前的网络状况。

代码示例（老秦建议：自己动手实践一下）

```

#include <iostream>
#include <chrono>
#include <thread>
#include <cstdlib>
#include <ctime>
// 模拟 RTT 测量和 RTO 计算
class TCPConnection {
public:
    TCPConnection() : srtt(0), rttvar(0), rto(1000) {}
    void sendPacket(int seq) {
        std::cout << "发送数据包 (seq=" << seq << ")" << std::endl;
        auto start = std::chrono::high_resolution_clock::now();
        // 模拟网络延迟
        std::this_thread::sleep_for(std::chrono::milliseconds(rand() % 100 +
50));
        if (simulatePacketLoss()) {
            std::cout << "数据包丢失 (seq=" << seq << ")" << std::endl;
        }
        else {
            auto end = std::chrono::high_resolution_clock::now();
            std::chrono::duration<double, std::milli> elapsed = end - start;
            double rtt = elapsed.count();
            std::cout << "收到 ACK (seq=" << seq << ", RTT=" << rtt <<
"ms)" << std::endl;
            updateRTO(rtt);
        }
    }
    void retransmit(int seq) {
        std::cout << "超时重传 (seq=" << seq << ", RTO=" << rto << "ms)"
<< std::endl;
        sendPacket(seq);
    }
    void updateRTO(double rtt) {
        const double alpha = 0.125;
        const double beta = 0.25;
        if (srtt == 0) {
            srtt = rtt;
            rttvar = rtt / 2;
        }
        else {
            rttvar = (1 - beta) * rttvar + beta * std::abs(srtt - rtt);
            srtt = (1 - alpha) * srtt + alpha * rtt;
        }
    }
};

```

```

        rto = srtt + 4 * rttvar;
        std::cout << "更新 RTO: " << rto << "ms" << std::endl;
    }
    int getRTO() const {
        return rto;
    }
private:
    bool simulatePacketLoss() const {
        return rand() % 5 == 0; // 20%概率丢包
    }
    double srtt; // 平滑 RTT 估计值
    double rttvar; // RTT 变化幅度
    int rto; // 重传超时时间
};
int main() {
    srand(static_cast<unsigned>(time(0)));
    TCPConnection connection;
    for (int i = 1; i <= 5; ++i) {
        connection.sendPacket(i);

        std::this_thread::sleep_for(std::chrono::milliseconds(connection.getRTO()));
        connection.retransmit(i);
    }
    return 0;
}

```

22. TCP CLOSE_WAIT 状态还能收到报文吗？

总结（老秦推荐必看，方便理解）：

CLOSE_WAIT 状态：表示一方已收到对方的 **FIN** 包（关闭请求），并发送了 **ACK** 包（确认关闭请求），等待自身的应用层决定何时关闭连接。在 **CLOSE_WAIT** 状态下，仍然可以接收数据。**TCP** 连接在接收到 **FIN** 包并回复 **ACK** 包后，仍然可以接收数据，直到应用层调用 **close()** 关闭连接。

(1) CLOSE_WAIT 状态：

当 TCP 连接中的一方（如客户端）发送 **FIN** 包请求关闭连接，另一方（如服务器）在接收到 **FIN** 包后，会发送 **ACK** 包确认收到关闭请求，并进入 **CLOSE_WAIT** 状态。在 **CLOSE_WAIT** 状态下，接收方的连接并未完全关闭，仍然可以接收来自发送方的数据包。

(2) 继续接收数据：

虽然接收方已进入 **CLOSE_WAIT** 状态，但连接的发送方向仍然可以发送数据包。接收方可以继续处理这些数据，直到应用层决定关闭连接。接收方在处理完所有数据并调用

close()关闭连接后，会发送 FIN 包请求关闭连接。

(3)连接完全关闭：

当接收方的应用层调用 close()关闭连接后，会发送 FIN 包给发送方，表示自己也准备关闭连接。发送方接收到 FIN 包后，发送 ACK 包确认关闭请求，连接完全关闭，双方进入 CLOSED 状态。

23. 两台服务器之间可以同时建立多条 TCP 链接吗？怎么实现的？

总结（老秦推荐必看，方便理解）：

可以：两台服务器之间可以同时建立多条 TCP 连接。

方法：通过使用不同的端口号或同一端口的多个套接字（socket）来实现。每个 TCP 连接都是由四元组（源 IP 地址、源端口号、目的 IP 地址、目的端口号）唯一标识的。

(1)TCP 连接的唯一性：

1) 每个 TCP 连接由四元组唯一标识，即源 IP 地址、源端口号、目的 IP 地址、目的端口号。只要这四个元素中有一个不同，就可以建立新的 TCP 连接。

2) 两台服务器之间可以通过使用不同的端口号来建立多条独立的 TCP 连接。

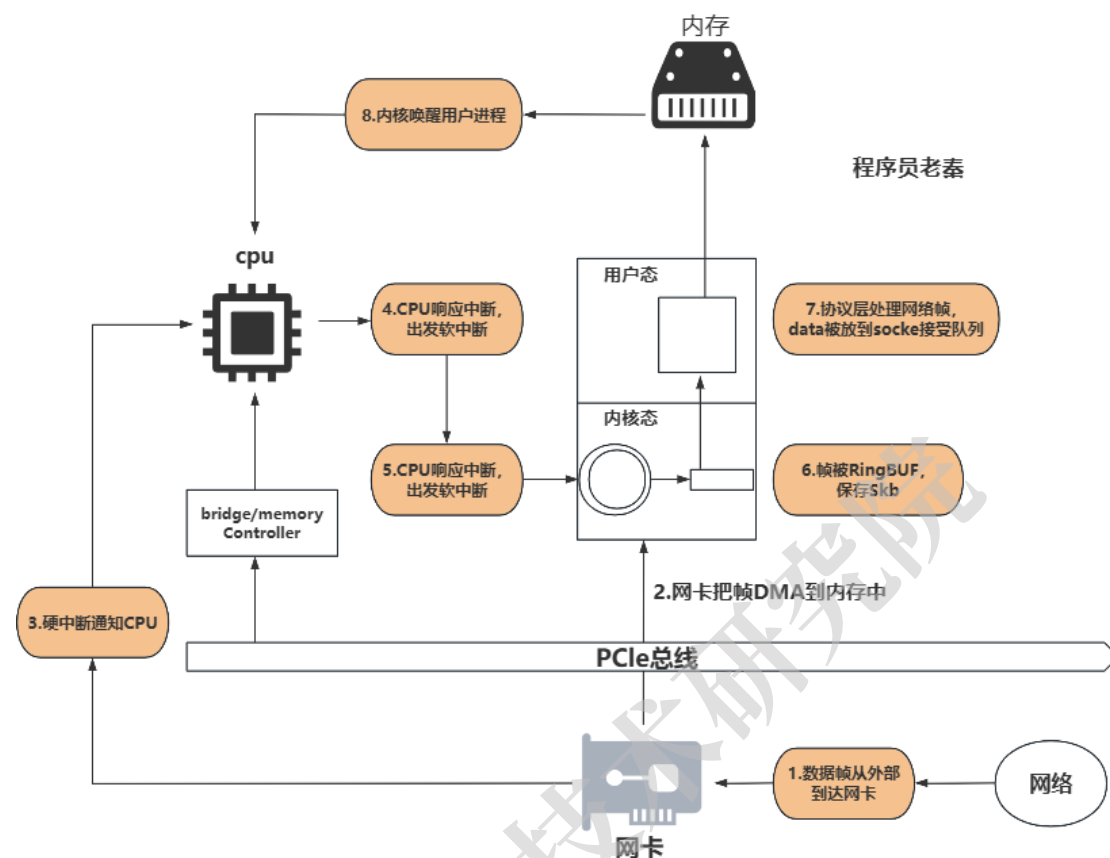
(2)多条 TCP 连接的实现方法：

1) 使用不同的端口号：可以在服务器 A 和服务器 B 上分别使用不同的端口号来建立多条连接。

2) 使用同一端口的多个套接字：可以在同一个端口上创建多个套接字，每个套接字对应一个独立的 TCP 连接。

24. linux 内核收包流程？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)网卡接收数据包：

步骤 1：网卡接收到从网络传来的数据包，并将其存储在网卡的接收缓冲区中。

(2)DMA 传输：

步骤 2：网卡通过 DMA（Direct Memory Access）将数据包直接传输到系统内存中，避免 CPU 参与数据传输，提高效率。

(3)中断通知：

步骤 3：网卡通过 PCIe 总线生成硬件中断，通知 CPU 有数据包到达，CPU 通过桥接/内存控制器接收中断信号。

(4)软中断处理：

步骤 4 和步骤 5：CPU 响应中断，执行硬件中断处理程序，硬件中断处理程序触发软中断（NET_RX_SOFTIRQ），软中断处理程序继续处理数据包。

(5)内核网络协议栈处理：

步骤 6：软中断处理程序将数据包从内存缓冲区（RingBUF）中取出，保存到 skb（socket buffer）中。

(6)数据包队列：

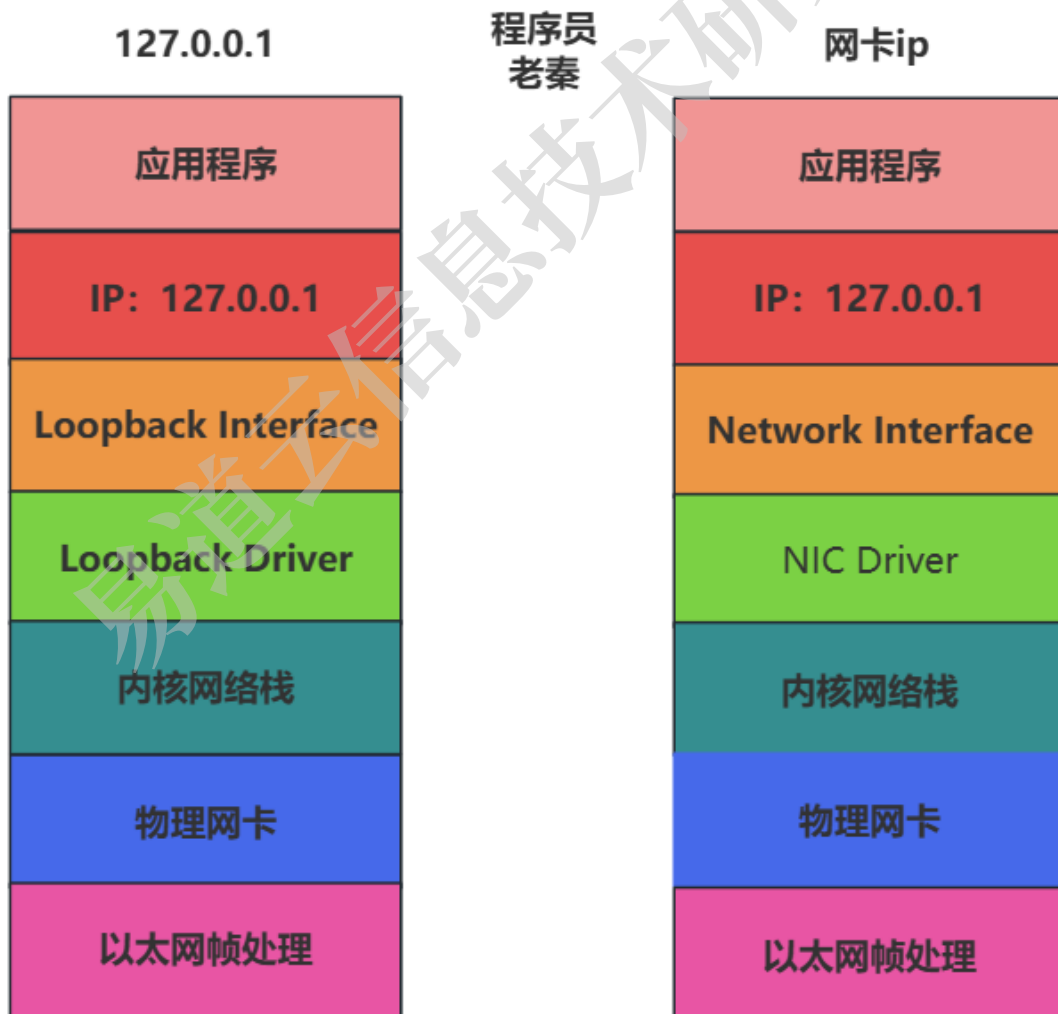
步骤 7：内核网络协议栈进一步处理数据包，将其放入对应 socket 的接收队列中（data 被放到 socket 接收队列）。

(7)唤醒用户进程：

步骤 8：内核唤醒等待数据的用户进程，用户进程从 socket 接收队列中读取数据进行处理。

25. 本机向本机发送请求，IP 填 127.0.0.1 和网卡 ip 地址有区别吗？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

127.0.0.1（本地回环地址）：数据包在内核网络栈中被处理，经过回环接口（Loopback Interface），不会通过物理网卡（NIC）发送。用于测试和本地通信。

网卡 IP 地址（本机的实际 IP 地址）：数据包通过内核网络栈被处理，并经过物理网卡（NIC）发送和接收，走完整的网络栈路径。用于正常网络通信。

(1) 127.0.0.1（本地回环地址）：

1) 回环接口（Loopback Interface）：当应用程序使用 127.0.0.1 发送请求时，数据包会被路由到回环接口。回环接口是在软件层面模拟的网络接口，专门用于本地通信。

2) 内核处理：数据包不会离开内核网络栈，而是直接在内核中被处理并回送给发送方。数据包不经过物理网卡（NIC），因此不消耗网络带宽。

3) 测试和本地通信：127.0.0.1 常用于网络应用的本地测试和调试，因为它不依赖实际的网络硬件。

(2) 网卡 IP 地址（本机的实际 IP 地址）：

1) 物理网卡（NIC）：当应用程序使用本机的实际 IP 地址发送请求时，数据包会被路由到物理网卡。网卡会根据 MAC 地址处理数据包，经过物理网络接口发送和接收。

2) 完整的网络栈路径：数据包会走完整的网络栈路径，包括物理层、数据链路层和网络层。数据包会真正通过物理网卡发送和接收，可能会在网络上产生负载。

3) 正常网络通信：使用本机的实际 IP 地址进行通信时，数据包的处理方式和与其他设备通信时相同。

26. tcp 粘包，如何解决？

总结（老秦推荐必看，方便理解）：

TCP 粘包：由于 TCP 是面向流的协议，发送的数据没有边界，接收方可能在一次读取操作中接收到多个发送方的数据包，导致数据粘在一起。

解决方法：

1) 定长消息：每个消息固定长度，接收方按固定长度读取数据。

2) 分隔符：在每个消息之间添加特殊字符作为分隔符，接收方按分隔符拆分数据。

3) 消息头标识长度：在每个消息前添加一个消息头，消息头包含该消息的长度，接收方根据长度读取完整消息。

27. 滑动窗口过大怎么样，过小怎么样？

总结（老秦推荐必看，方便理解）：

(1) 滑动窗口机制：

1) 发送窗口：发送方维护的一个窗口，表示可以发送但尚未确认的数据范围。

2) 接收窗口：接收方维护的一个窗口，表示可以接收的数据范围。

3) 窗口大小：发送方根据接收方的接收窗口大小调整发送窗口大小，确保数据在接

收方缓冲区内能够被正确接收。

(2)滑动窗口过大的情况：

1) 优点：在**高带宽、低延迟**的网络环境中，**较大的滑动窗口**可以允许发送方在**不等待 ACK 确认**的情况下发送更多数据，充分利用带宽，减少传输时间。

2) 缺点：在**高延迟或高丢包率**的网络环境中，**较大的滑动窗口**可能导致大量未确认数据堆积，一旦发生丢包或错误，需要重传的数据量大，浪费带宽，可能导致网络拥塞。

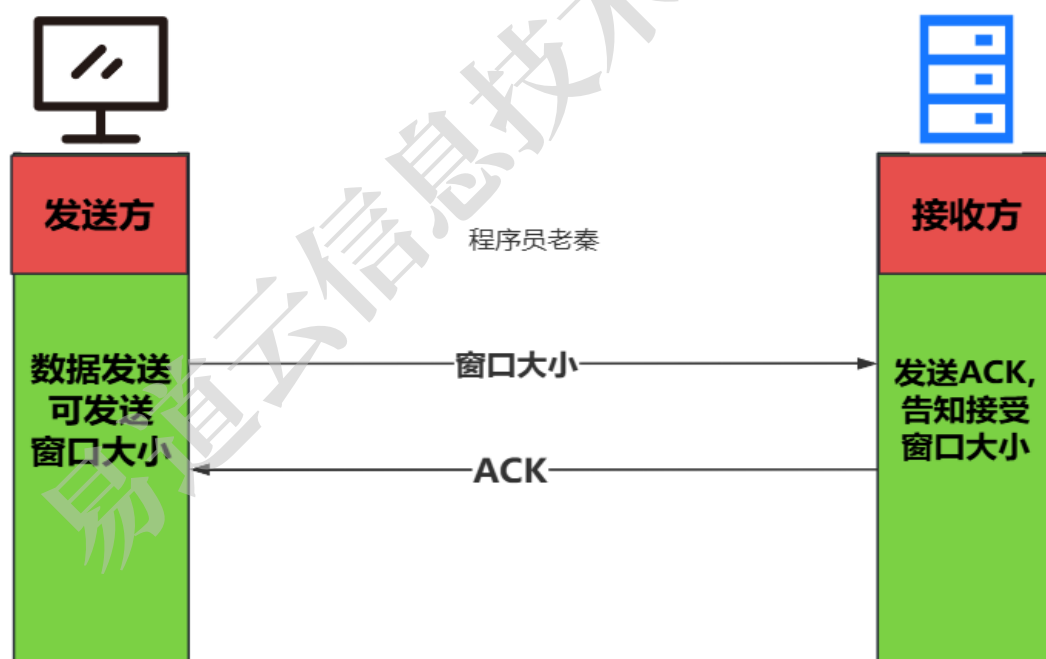
(3)滑动窗口过小的情况：

1) 优点：在**高延迟或高丢包率**的网络环境中，**较小的滑动窗口**可以限制未确认数据的数量，减少重传的数据量，降低网络拥塞的风险。

2) 缺点：在**高带宽、低延迟**的网络环境中，**较小的滑动窗口**限制了发送方的数据传输速率，频繁等待 ACK 确认，导致带宽利用率低，传输效率下降。

28. 什么是流控制？如何在 TCP 中实现？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)流控制的目的：

1) 防止溢出：防止发送方发送数据过快，导致接收方的缓冲区溢出。

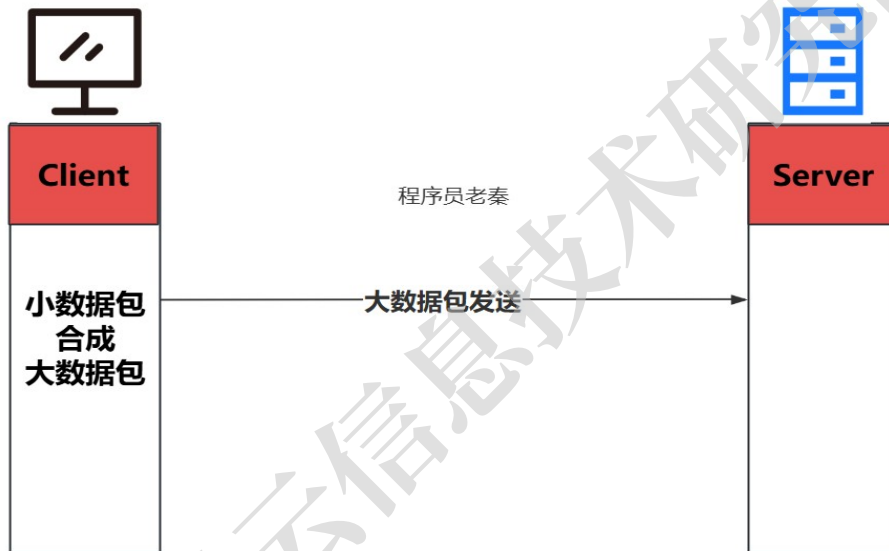
2) 确保稳定：确保网络传输的稳定性，提高传输效率。

(2)TCP 中的流控制：

- 1) 接收窗口：接收方维护一个接收窗口，表示当前可以接收的数据量大小。
- 2) 窗口大小通告：接收方在发送 ACK 报文时，通知发送方当前的接收窗口大小 (Window Size)。
- 3) 发送方调整发送速度：发送方根据接收方的窗口大小，调整数据发送速度，避免发送数据超过接收方的处理能力。

29. TCP 的 nagle 算法，nagle 会带来什么坏处？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)Nagle 算法的目的：

- 1) 减少小包：通过将小数据包合并成大包发送，减少网络中小包的数量。
- 2) 提高效率：减少每个包的协议开销，提高传输效率，减少网络拥塞。

(2)Nagle 算法的工作原理：

发送策略：当发送方有小数据包需要发送时，如果前一个数据包的 ACK 未收到，新的小数据包会被缓冲，等待前一个包的 ACK 到达或缓冲区积累到足够大的数据量再发送。

(3)Nagle 算法的坏处：

- 1) 增加延迟：在实时性要求高的应用中，如在线游戏、即时通讯等，Nagle 算法的延迟会影响用户体验。

2) 应用场景选择：根据应用需求，可以选择关闭 Nagle 算法，通过设置 TCP_NODELAY 选项来禁用。

30. UDP 有可靠性保证吗？UDP 如何可靠？

总结（老秦推荐必看，方便理解）：

(1)UDP 的特性：

- 1) 无连接：UDP 是无连接的，不需要建立连接即可发送数据。
- 2) 不可靠：UDP 不保证数据包的顺序和完整性，可能会发生丢包、重复包和乱序。

(2)实现可靠性的方法（应用层协议）：

- 1) ACK 确认：接收方接收到数据包后，发送 ACK 确认，发送方根据 ACK 判断是否需要重传。
- 2) 超时重传：发送方在发送数据后启动计时器，如果在规定时间内未收到 ACK，则进行重传。
- 3) 序列号：为每个数据包添加序列号，接收方根据序列号判断数据包的顺序，进行重排序。
- 4) FEC（前向纠错）：冗余数据，在数据包中增加冗余数据，通过冗余信息进行错误检测和纠正，确保数据完整性。

31. UDP 常见的应用场景？

图示（方便记忆）：

表. UDP 应用场景

应用场景	举例
视频流媒体传输	例如 IPTV、视频会议等，要求低延迟，高容错
实时语音通信	例如 VoIP，语音通话对实时性要求高，丢包少量可接受
在线游戏	例如多人在线游戏，对低延迟要求高，偶尔丢包影响不大。
简单请求-响应：	简单请求-响应：例如 DNS 查询，响应时间比可靠性更重要。
网络广播：	网络广播：例如 DHCP 等，需要向多个设备发送数据。

总结（老秦推荐必看，方便理解）：

- (1)视频流媒体传输：UDP 适合视频流媒体传输，因为丢失少量数据包不会显著影响视频质量，而低延迟是关键。
- (2)实时语音通信：语音通话要求低延迟和实时性，UDP 可以满足这些要求，即使有少量数据包丢失也不影响通话质量。
- (3)在线游戏：多人在线游戏需要低延迟和快速响应，UDP 的无连接特性使其适合此类应用。

(4)简单请求-响应：如 DNS 查询，使用 UDP 进行简单的请求和响应操作，速度快且效率高。

(5)网络广播：如 DHCP，使用 UDP 可以方便地向多个设备发送广播消息。

32. 为什么客户端的 TIME-WAIT 状态必须等待 2MSL ？

总结（老秦推荐必看，方便理解）：

在 TCP 协议中，当一方主动关闭连接时，它会进入 TIME-WAIT 状态。TIME-WAIT 状态的持续时间是 2MSL (Maximum Segment Lifetime) 的两倍。MSL 是一个 TCP 报文段在网络中可以存在的最长时间。2MSL 的等待时间主要有以下原因：

(1)确保被动关闭方收到 ACK：

1) 当一方（主动关闭方）发送最后一个 ACK 报文时，这个 ACK 报文段可能会丢失。如果被动关闭方没有收到这个 ACK，它会重新发送 FIN 报文段。

2) 主动关闭方必须等待一段时间，以确保它能够接收到这个可能重发的 FIN 报文段，并再次发送 ACK 确认。

3) 如果没有 2MSL 的等待时间，主动关闭方可能会在重新发送的 FIN 报文段到达时，无法处理，从而导致连接不正常关闭。

(2)确保旧的报文段在网络中消失：

1) 在 2MSL 时间内，网络中的所有旧的、重复的报文段都会超时并被丢弃。这确保了下次新的连接不会收到前一个连接的旧报文段，从而避免数据混乱和连接错误。

2) 如果不等待 2MSL，新的连接可能会收到前一个连接遗留的报文段，这会导致数据混乱和协议错误。

33. 什么是 MTU？如何处理 MTU 的问题？

总结（老秦推荐必看，方便理解）：

什么是 MTU？

MTU（最大传输单元）：指网络层协议数据包在数据链路层的最大传输字节数。典型值为 1500 字节（以太网）。

为什么 MTU 很重要？

MTU 的大小直接影响网络的效率和性能。如果数据包大于 MTU，必须进行分片传输，分片会增加额外的开销，并可能导致重组的负担。同时，过小的 MTU 会导致数据包数量增加，增加了网络负担。

处理 MTU 问题的方法

(1)路径 MTU 发现 (Path MTU Discovery, PMTUD)：

1) 描述：路径 MTU 发现是一个动态确定路径上最小 MTU 的过程，通过发现路径上的所有路由器的 MTU 大小，确定最小的 MTU 以避免分片。

2) 工作原理：发送带有 DF (Don't Fragment) 标志的数据包，如果某个路由器无法转发这个数据包，它会丢弃该数据包并返回一个 ICMP “需要分片但设置了 DF 标志”的消息。源主机根据此消息调整 MTU 大小，直到找到合适的 MTU。

3) 优点：动态调整 MTU，避免不必要的分片，提高网络效率。

4) 缺点：依赖于 ICMP 消息，某些网络环境下 ICMP 消息可能被过滤或丢弃，导致路径 MTU 发现失败。

(2)手动调整 MTU：

1) 描述：手动设置网络接口的 MTU 值以匹配网络环境。

2) 适用场景：在已知的网络环境中，可以手动配置适当的 MTU 值，避免分片问题。

3) 优点：简单直接，可以在网络设备上直接配置。

4) 缺点：需要对网络环境有明确的了解，不灵活。

(3)MSS (Maximum Segment Size) 调整：

1) 描述：TCP 协议中使用 MSS 来指定传输层能够发送的最大数据段大小，MSS 通常设置为 MTU 减去 TCP/IP 头部的大小。

2) 工作原理：在建立 TCP 连接时，通过 TCP 握手过程协商 MSS，确保数据段不会超过接收端的 MTU，避免分片。

3) 优点：在 TCP 层面进行控制，适用于 TCP 连接。

4) 缺点：仅适用于 TCP 连接，无法解决其他协议的 MTU 问题。

(4)使用较小的 MTU 值：

1) 描述：为网络接口配置较小的 MTU 值，以确保所有路径上的 MTU 都大于等于该值，避免分片。

2) 适用场景：在不确定路径 MTU 的情况下，可以设置保守的 MTU 值，以确保数据包不会被分片。

3) 优点：简单易行，能够避免分片。

4) 缺点：可能降低网络传输效率，增加数据包数量。

34. IPv4 和 IPv6 的区别？

图示（方便记忆）：

表.IPv4 和 IPv6 的区别

IPv4	IPv6
32 位地址	128 位地址
地址耗尽	地址丰富
无内置安全性	内置 IPsec
需 NAT 支持	无需 NAT

总结（老秦推荐必看，方便理解）：

(1)地址空间：

IPv4：32 位地址，约 43 亿个地址，面临地址耗尽问题。

IPv6：128 位地址，支持约 340 万亿亿个地址，解决地址耗尽问题。

(2)安全性：

IPv4：无内置安全性，依赖外部协议（如 IPsec）提供安全性。

IPv6：内置 IPsec 协议，提供更好的安全性。

(3)NAT 支持：

IPv4：由于地址耗尽问题，广泛使用 NAT（网络地址转换）。

IPv6：地址丰富，无需 NAT，支持端到端通信。

(4)其他特性：

IPv6：支持自动配置、多播、任播，更高效的路由和报头格式。

35. 什么是 NAT？它的作用是什么？

总结（老秦推荐必看，方便理解）：

网络地址转换（NAT）：在路由器或防火墙上，将私有网络地址转换为公有网络地址，允许多个设备共享一个公有 IP 地址。

NAT 的作用：

1) **解决 IPv4 地址耗尽问题**：通过地址转换，多个设备共享一个公有 IP 地址，缓解 IPv4 地址短缺。

2) **提高网络安全性**：隐藏内部网络结构，防止外部攻击。

36. IP 分片是什么？为什么需要 IP 分片？

总结（老秦推荐必看，方便理解）：

(1)IP 分片的概念：

IP 分片：将大于 MTU 的数据包分割成多个小包传输，在接收方重新组装成完整的数据包。

(2)需要 IP 分片的原因：

1) **适应不同 MTU**：不同网络设备可能有不同的 MTU，分片确保数据包能够通过所有网络设备。

2) **避免数据包丢弃**：如果数据包过大而无法通过某个网络设备，分片可以避免数据包被丢弃。

37. 什么是 ICMP 协议？它的作用是什么？

图示（方便记忆）：

表.ICMP 消息类型

类型	名称	描述
0	回显应答 (Echo Reply)	用于响应 Ping 请求的应答消息
3	目标不可达	当数据包无法到达目标时，发送此消息
4	源抑制 (Source Quench)	用于请求发送方降低发送速率（已废弃）
5	重定向 (Redirect)	建议源主机改用更好的路径
8	回显请求 (Echo Request)	用于 Ping 请求消息
11	时间超时 (Time Exceeded)	当数据包的 TTL 减到 0 时，发送此消息
12	参数问题 (Parameter Problem)	当发现数据包头部错误时，发送此消息

总结（老秦推荐必看，方便理解）：

ICMP (Internet Control Message Protocol，互联网控制消息协议) 是一种网络层协议，用于在 IP 主机和路由器之间传递控制消息和错误报告。ICMP 是 IP 协议的一部分，主要用于报告网络连接中的错误和进行网络诊断。

ICMP 协议的作用

(1) 错误报告

1) 网络不可达 (Destination Unreachable)：当路由器或主机无法到达目标网络或主机时，会发送此类型的 ICMP 消息，以通知源主机。

2) 时间超时 (Time Exceeded)：当数据包在网络中传输的时间超过了生存时间 (TTL) 字段规定的最大值时，会发送此类型的 ICMP 消息，以通知源主机。

3) 参数问题 (Parameter Problem)：当路由器或主机在处理数据包时发现问题（如 IP 头部字段错误），会发送此类型的 ICMP 消息，以通知源主机。

4) 重定向 (Redirect)：当路由器发现有更好的路径到达目标主机时，会发送此类型的 ICMP 消息，以建议源主机改用更好的路径。

(2) 诊断工具

1) Ping：Ping 命令使用 ICMP 的回显请求 (Echo Request) 和回显应答 (Echo Reply) 消息来测试主机之间的连通性。源主机发送回显请求消息，目标主机收到后发送回显应答消息，源主机通过计算往返时间来评估网络连通性和延迟。

2) Traceroute：Traceroute 命令使用 ICMP 时间超时消息来跟踪从源主机到目标主机的路径。源主机发送 TTL 逐渐递增的 IP 数据包，每经过一个路由器，TTL 减 1，当 TTL 减到 0 时，路由器会返回时间超时的 ICMP 消息。源主机根据返回的 ICMP 消息确定路径上的每个路由器。

38. 如何防止 ARP 攻击？

总结（老秦推荐必看，方便理解）：

(1) 静态 ARP 表：

手动设置映射：在网络设备上手动设置固定的 IP 地址和 MAC 地址映射，防止 ARP 欺骗。

(2)ARP 防护工具：

检测和阻止：使用 ARP 防护工具，如 arpswatch，实时监控网络中的 ARP 流量，检测和阻止 ARP 攻击。

(3)交换机端口安全：

限制 MAC 地址：启用交换机的端口安全功能，限制每个端口可以学习的 MAC 地址数量，防止 ARP 欺骗。

(4)网络入侵检测系统 (NIDS)：

监控网络流量：使用 NIDS 实时监控网络流量，检测和防止网络中的 ARP 攻击。

39. 什么是 TCP 的序列号机制？

总结（老秦推荐必看，方便理解）：

(1)TCP 序列号机制的概念：

1) 序列号 (Sequence Number)：每个 TCP 数据包都包含一个序列号，用于标识数据包的顺序，确保数据包按正确顺序组装。

2) 确认号 (Acknowledgment Number)：每个 TCP 数据包都包含一个确认号，表示接收方期望接收到的下一个数据包的序列号，用于确认已接收到的数据。

(2)序列号机制的作用：

1) 数据包排序：通过序列号，接收方可以按正确顺序组装数据包，即使数据包顺序错乱。

2) 丢包检测：通过确认号，发送方可以检测到数据包是否丢失，并进行重传，确保数据包的可靠传输。

三．网络编程

40. Socket 编程的基本流程？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)客户端：

1)socket()：使用 socket()函数创建一个套接字，指定协议族、套接字类型和协议。

2)connect()：使用 connect()函数连接到服务器的指定 IP 地址和端口。

3)send()/recv()：使用 send()函数发送数据，使用 recv()函数接收数据。

4)close()：使用 close()函数关闭套接字连接。

(2)服务器：

1)socket()：使用 socket()函数创建一个套接字。

2)bind()：使用 bind()函数将套接字绑定到本地地址和端口。

3)listen()：使用 listen()函数使套接字进入监听状态，等待客户端连接。

4)accept()：使用 accept()函数接受客户端连接，返回新的套接字用于通信。

5)send()/recv()：使用 send()函数发送数据，使用 recv()函数接收数据。

6)close()：使用 close()函数关闭套接字连接。

41. 说一下 Linux 内核网络的连接队列？

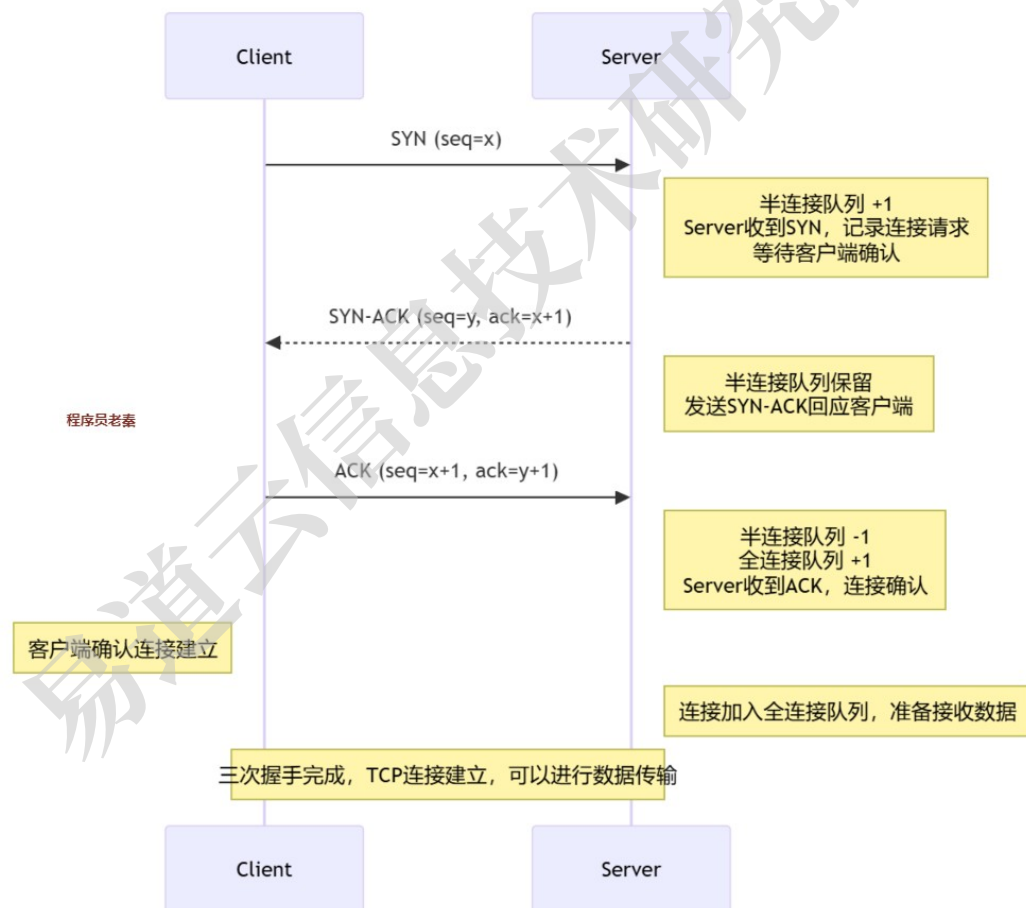
总结（老秦推荐必看，方便理解）：

TCP 三次握手的过程中，**Linux 内核会维护两个队列**，分别是：

1) 全连接队列 (**Accept Queue 链表**)：全连接队列通过 `rskq_accept_head` 和 `rskq_accept_tail` 以链表的形式来管理。

2) 半连接队列 (**SYN Queue 哈希表**)：半连接队列 `struct listen_sock`，真正半连接用 `request_sock` 对象是在握手过程中分配，计算完 Hash 值后挂到这个 Hash 表上。

(1) TCP 握手图解：



(2) 正常的 TCP 三次握手过程：

1) Client 端向 Server 端发送 SYN 发起握手，Client 端进入 `SYN_SENT` 状态。

2) Server 端收到 Client 端的 SYN 请求后, Server 端进入 SYN_RECV 状态, 此时内核会将连接存储到半连接队列(**SYN Queue**), 并向 Client 端回复 SYN+ACK。

3) Client 端收到 Server 端的 SYN+ACK 后, Client 端回复 ACK 并进入 ESTABLISHED 状态。

4) Server 端收到 Client 端的 ACK 后, 内核将连接从半连接队列(**SYN Queue**)中取出, 添加到全连接队列(**Accept Queue**), Server 端进入 ESTABLISHED 状态;

5) Server 端应用进程调用 accept 函数时, 将连接从全连接队列(**Accept Queue**)中取出;

(3) 实际验证

```
1 # -n 不解析服务名称
2 # -t 只显示 tcp sockets
3 # -l 显示正在监听(LISTEN)的 sockets
4 $ ss -lnt
5 State      Recv-Q Send-Q   Local Address:Port      Peer Address:Port
6 LISTEN     0      128     [::]:2380               [::]:*
7 LISTEN     0      128     [::]:80                  [::]:*
8 LISTEN     0      128     [::]:8080                [::]:*
9 LISTEN     0      128     [::]:8090                [::]:*
10 $ ss -nt
11 State      Recv-Q Send-Q   Local Address:Port      Peer Address:Port
12 ESTAB      0      0       [::ffff:33.9.95.134]:80 [::ffff:33.51.103.59]:47452
13 ESTAB      0      536     [::ffff:33.9.95.134]:80 [::ffff:33.43.108.144]:37656
14 ESTAB      0      0       [::ffff:33.9.95.134]:80 [::ffff:33.51.103.59]:38130
15 ESTAB      0      536     [::ffff:33.9.95.134]:80 [::ffff:33.51.103.59]:38280
16 ESTAB      0      0       [::ffff:33.9.95.134]:80 [::ffff:33.51.103.59]:38204
```

1) 查看全连接队列信息: 通过 ss 命令查看全连接队列的信息:

1.对于 LISTEN 状态的 socket:

- **Recv-Q**: 当前全连接队列的大小, 即已完成三次握手等待应用程序 accept() 的 TCP 链接。
- **Send-Q**: 全连接队列的最大长度, 即全连接队列的大小。

2.对于非 LISTEN 状态的 socket:

- **Recv-Q**: 已收到但未被应用程序读取的字节数。
- **Send-Q**: 已发送但未收到确认的字节数。

3.相关内核代码:

```
// https://github.com/torvalds/linux/blob/master/net/ipv4/tcp_diag.c
static void tcp_diag_get_info(struct sock* sk, struct inet_diag_msg* r,
    void* _info)
{
    struct tcp_info* info = _info;
    if (inet_sk_state_load(sk) == TCP_LISTEN) { // socket 状态是 LISTEN 时
```

```

    r->idiag_rqueue = READ_ONCE(sk->sk_ack_backlog); // 当前全连接队列大小
    r->idiag_wqueue = READ_ONCE(sk->sk_max_ack_backlog); // 全连接队列最大长度
}
else if (sk->sk_type == SOCK_STREAM) { // socket 状态不是 LISTEN 时
    const struct tcp_sock* tp = tcp_sk(sk);
    r->idiag_rqueue = max_t(int, READ_ONCE(tp->rcv_nxt) -
        READ_ONCE(tp->copied_seq), 0); // 已收到但未被应用程序读取的字节数
    r->idiag_wqueue = READ_ONCE(tp->write_seq) - tp->snd_una; // 已发送但未收到
    确认的字节数
}
if (info)
    tcp_get_info(sk, info);
}

```

2) 查看连接队列溢出信息：通过 **netstat -s** 命令查看半连接队列、全连接队列溢出情况；

```

1  $ netstat -s | grep -i "listen"
2      189088 times the listen queue of a socket overflowed
3      30140232 SYNs to LISTEN sockets dropped

```

- 上面输出的数值是累计值，分别表示有多少 TCP socket 链接因为全连接队列、半连接队列满了而被丢弃：
 - 代表有 **189088** 次全连接队列溢出。
 - 代表有 **30140232** 次半连接队列溢出。

(4) TCP 全连接队列详解

1) 全连接队列最大长度控制

全连接队列的最大长度由 **min(somaxconn, backlog)** 控制，其中：

- somaxconn 是 Linux 内核参数，由 `/proc/sys/net/core/somaxconn` 指定。
- backlog 是 TCP 协议中 listen 函数参数，即 `int listen(int sockfd, int backlog)` 函数中的 backlog 大小。
 - listen 的 backlog 参数使用的是 `/proc/sys/net/core/somaxconn` 文件中的值。
- 如果 backlog 参数大于 `/proc/sys/net/core/somaxconn` 的值，那么系统会将其限制在 `/proc/sys/net/core/somaxconn` 的值范围内。
 - 从 Linux 5.4 开始，`/proc/sys/net/core/somaxconn` 的默认值为 4096。
 - 在早期版本（例如 2.4.25 之前）中，该值是硬编码的，SOMAXCONN 值为 128。

2) 相关内核代码：

```
//file: net/socket.c
```

```

SYSCALL_DEFINE2(listen, int, fd, int, backlog)
{
    //根据 fd 查找 socket 内核对象
    sock = sockfd_lookup_light(fd, &err, &fput_needed);
    if (sock) {
        //获取内核参数 net.core.somaxconn
        somaxconn = sock_net(sock->sk)->core.sysctl_somaxconn;
        if ((unsigned int)backlog > somaxconn)
            backlog = somaxconn;
        //调用协议栈注册的 listen 函数
        err = sock->ops->listen(sock, backlog);
        // .....
    }
}

// https://github.com/torvalds/linux/blob/master/net/socket.c
/*
 * Perform a listen. Basically, we allow the protocol to do anything
 * necessary for a listen, and if that works, we mark the socket as
 * ready for listening.
 */
int __sys_listen(int fd, int backlog)
{
    struct socket* sock;
    int err, fput_needed;
    int somaxconn;
    sock = sockfd_lookup_light(fd, &err, &fput_needed);
    if (sock) {
        somaxconn = sock_net(sock->sk)->core.sysctl_somaxconn; //
/proc/sys/net/core/somaxconn
        if ((unsigned int)backlog > somaxconn)
            backlog = somaxconn; // TCP 全连接队列最大长度 min(somaxconn, backlog)
        err = security_socket_listen(sock, backlog);
        if (!err)
            err = sock->ops->listen(sock, backlog);
        fput_light(sock->file, fput_needed);
    }
    return err;
}

```

3) 全连接队列的实际行为：

- 进入全连接队列的 Socket 数量：能够进入全连接队列的 Socket 最大数量始终比配置的全连接队列最大长度多 1。
- 判断队列是否已满：内核在判断全连接队列是否已满时，使用的是 $>$ 而非 $\geq$ 。

4) 相关内核代码：

```
/* Note: If you think the test should be:
 * return READ_ONCE(sk->sk_ack_backlog) >= READ_ONCE(sk->sk_max_ack_backlog);
 * Then please take a look at commit 64a146513f8f ("[NET]: Revert incorrect accept
queue backlog changes.")
 */
static inline bool sk_acceptq_is_full(const struct sock* sk)
{
    return READ_ONCE(sk->sk_ack_backlog) > READ_ONCE(sk->sk_max_ack_backlog);
}
```

5) 全连接队列满的处理方式：当全连接队列已满时，Server 端的默认行为是丢弃请求。可以通过 `/proc/sys/net/ipv4/tcp_abort_on_overflow` 配置 Server 的处理方式：

1. `tcp_abort_on_overflow` 可选值：

- **0 (默认)**：如果全连接队列满了，Server 丢弃 (DROP) Client 的 ACK。
- **1**：如果全连接队列满了，Server 向 Client 发送 RST 报文，终止 TCP 连接。

(5) TCP 半连接队列详解

1) 半连接队列最大长度控制

- **早期**：在早期，`backlog` 参数通常指的是半连接队列的长度。半连接队列用于存放已经发送 SYN 报文但尚未完成三次握手的连接请求。因此，`backlog` 参数在早期主要用于控制半连接队列的长度，以限制服务器端同时处理的连接请求数量。可以防治 syn 泛洪（内核版本 2.2-2.6.20）
- **中期**：在中期，`backlog` 参数通常指的是半连接队列加上全连接队列的长度之和。全连接队列用于存放已经完成了三次握手但尚未被 `accept` 函数接受的连接请求。因此，`backlog` 参数在中期主要用于控制半连接队列和全连接队列的总长度，以限制服务器端同时处理的连接请求数量。
- **后期**：目前，`backlog` 参数通常指的是全连接队列的长度。随着 TCP 协议和操作系统的发展，服务器对于半连接队列的管理方式也发生了变化，`backlog` 参数逐渐专门用于控制全连接队列的长度，以限制服务器端同时处理的连接请求数量。这也是在较新的操作系统中，`listen` 函数的 `backlog` 参数通常只影响全连接队列的长度，有了防火墙等安全措施，不用考虑 dos 攻击以提升性能。

Ps : 很多博文中说半连接队列最大长度由

/proc/sys/net/ipv4/tcp_max_syn_backlog 参数指定，实际只有在 **linux** 内核版本小于 **2.6.20** 时，半连接队列才等于 **backlog** 的大小。

2) 半连接队列长度计算

1.半连接队列的长度是 $\min(\text{backlog}, \text{somaxconn}, \text{tcp_max_syn_backlog}) + 1$ 再上取整到 2 的幂次，但最小不能小于 16。半连接队列的长度由三个参数指定：

- 调用 listen 时，传入的 backlog
- /proc/sys/net/core/somaxconn 默认值为 128
- /proc/sys/net/ipv4/tcp_max_syn_backlog 默认值为 1024

2.在 Linux 2.6.32 内核版本，它们之间的关系，总体可以概况为：

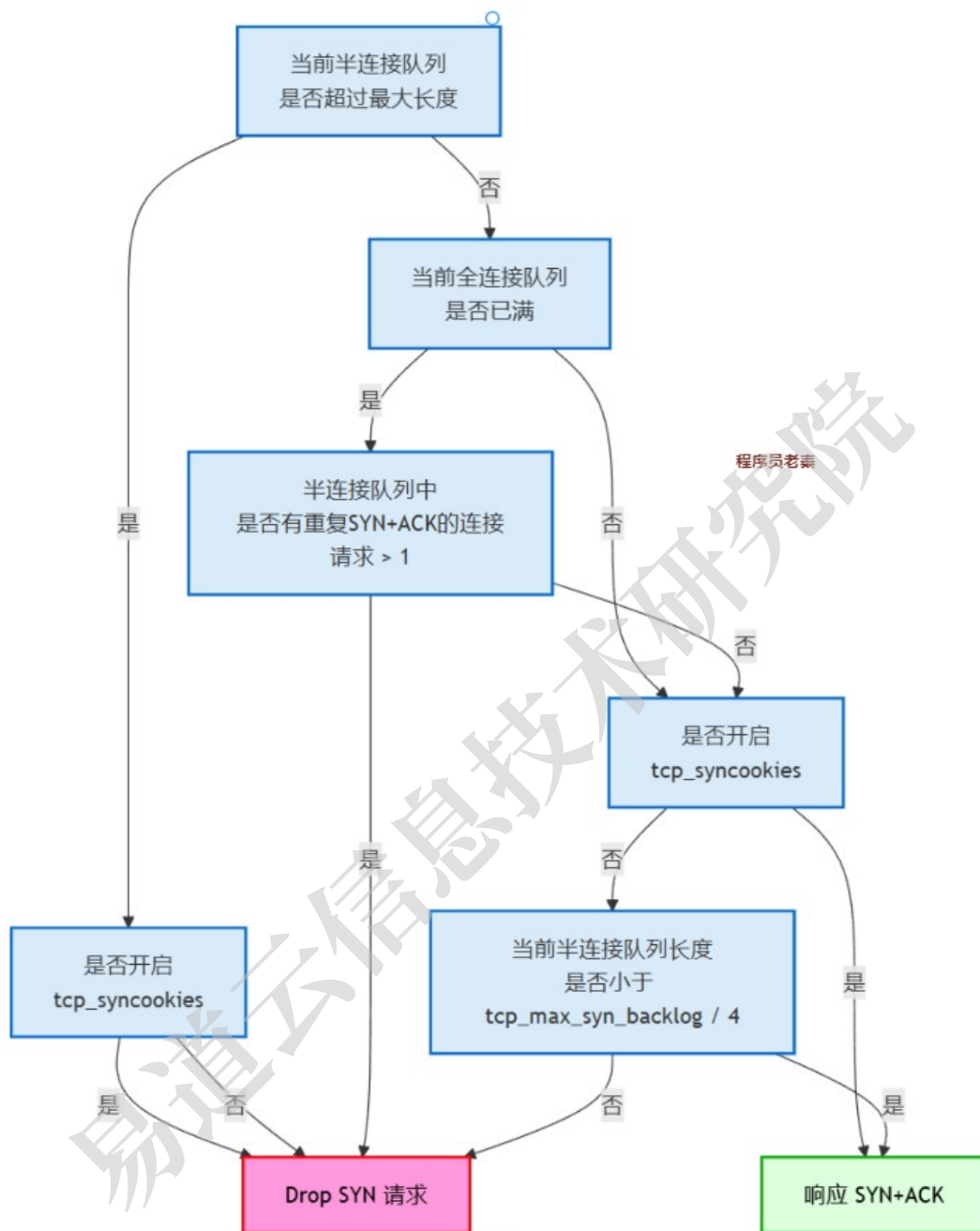
- 当 $\text{tcp_max_syn_backlog} > \min(\text{somaxconn}, \text{backlog})$ 时，半连接队列最大值 $\text{max_qlen_log} = \min(\text{somaxconn}, \text{backlog}) * 2$;
- 当 $\text{tcp_max_syn_backlog} < \min(\text{somaxconn}, \text{backlog})$ 时，半连接队列最大值 $\text{max_qlen_log} = \text{tcp_max_syn_backlog} * 2$;

3) 相关内核代码：

```
// backlog = min(somaxconn, backlog)
// file: net/core/request_sock.c
int reqsk_queue_alloc(struct request_sock_queue* queue,
    unsigned int nr_table_entries)
{
    // 计算半连接队列的长度
    nr_table_entries = min_t(u32, nr_table_entries, sysctl_max_syn_backlog);
    nr_table_entries = max_t(u32, nr_table_entries, 8);
    nr_table_entries = roundup_pow_of_two(nr_table_entries + 1);
    // 为了效率，不记录 nr_table_entries
    // 而是记录 2 的几次幂等于 nr_table_entries
    for (lopt->max_qlen_log = 3;
        (1 << lopt->max_qlen_log) < nr_table_entries;
        lopt->max_qlen_log++);
    // .....
}
```

(6) 判断是否 Drop SYN 请求

1) 详细图解



2) 详细解释

第一个判断条件「当前半连接队列是否已超过半连接队列最大长度」在不同内核版本中的判断不一样，Linux4.19.91 内核判断的是当前半连接队列长度是否 $\geq$ 全连接队列最大长度。

3) 相关内核代码：

```
static inline int inet_csk_reqsk_queue_is_full(const struct sock* sk)
{
```

```
return inet_csk_reqsk_queue_len(sk) >= sk->sk_max_ack_backlog;
}
```

4) Connection timeout 有两种可能情况：

- 半连接队列未满，全连接队列满，Client 端向 Server 端发起 SYN 被 DROP。
- 全连接队列未满，半连接队列大小超过全连接队列最大长度。问题的最快修复方式是将 net.core.somaxconn 调大，以及 net.ipv4.tcp_abort_on_overflow 设置为 1，net.ipv4.tcp_abort_on_overflow 设置为 1 是为了让 client fail fast。

42. 什么是多路复用？如何使 select/poll/epoll？

总结（老秦推荐必看，方便理解）：

多路复用（Multiplexing）是一种技术，允许一个单独的进程同时监控多个文件描述符，以查看是否有任何一个文件描述符准备好了进行 I/O 操作。这种技术在需要同时处理大量并发连接的网络服务器中非常有用，因为它可以有效地提高系统的资源利用率和响应速度。

常见的多路复用机制：select、poll、epoll

(1) select：

- 1) 创建一个文件描述符集合（fd_set）。
- 2) 将需要监控的文件描述符添加到集合中。
- 3) 调用 select 函数，并传递文件描述符集合和超时时间。
- 4) select 函数返回准备好进行 I/O 操作的文件描述符数量。

代码示例（老秦建议：自己动手实践一下）

```
#include <sys/select.h>
fd_set readfds;
FD_ZERO(&readfds);
FD_SET(sockfd, &readfds);
struct timeval tv;
tv.tv_sec = 5;
tv.tv_usec = 0;
int retval = select(sockfd + 1, &readfds, NULL, NULL, &tv);
if (retval == -1) {
    perror("select()");
}
else if (retval) {
    // 有数据可读
```



```
}  
else {  
    // 超时  
}
```

优点：简单易用，几乎在所有平台上都可用。

缺点：在处理大量文件描述符时性能较差，因为每次调用 `select` 都需要遍历所有文件描述符。

(2) Poll :

- 1) 创建一个 `pollfd` 结构数组，并将需要监控的文件描述符添加到数组中。
- 2) 调用 `poll` 函数，并传递文件描述符数组和超时时间。
- 3) `poll` 函数返回准备好进行 I/O 操作的文件描述符数量。

代码示例（老秦建议：自己动手实践一下）

```
#include <poll.h>  
struct pollfd fds[2];  
fds[0].fd = sockfd1;  
fds[0].events = POLLIN;  
fds[1].fd = sockfd2;  
fds[1].events = POLLIN;  
int retval = poll(fds, 2, 5000);  
if (retval == -1) {  
    perror("poll()");  
}  
else if (retval) {  
    // 有数据可读  
}  
else {  
    // 超时  
}
```

优点：没有文件描述符数量限制。

缺点：在处理大量文件描述符时性能仍然较差，因为每次调用 `poll` 都需要遍历整个数组

(3) Epoll :

- 1) 使用 `epoll_create` 创建一个 `epoll` 实例。
- 2) 使用 `epoll_ctl` 添加、修改或删除需要监控的文件描述符。
- 3) 使用 `epoll_wait` 等待事件，并获取准备好进行 I/O 操作的文件描述符。

代码示例（老秦建议：自己动手实践一下）

```
#include <poll.h>  
struct pollfd fds[2];  
fds[0].fd = sockfd1;
```

```
fds[0].events = POLLIN;
fds[1].fd = sockfd2;
fds[1].events = POLLIN;
int retval = poll(fds, 2, 5000);
if (retval == -1) {
    perror("poll()");
}
else if (retval) {
    // 有数据可读
}
else {
    // 超时
}
```

优点：性能优越，适用于大量并发连接的场景；不需要每次调用都遍历所有文件描述符。

缺点：仅在 Linux 系统上可用。

43. 如何实现长连接和短连接？

总结（老秦推荐必看，方便理解）：

(1)长连接：

1) 定义：在 TCP 连接建立后，保持连接状态，进行多次请求和响应，而不是每次请求后立即关闭连接。

2) 实现：服务器和客户端在第一次连接后保持连接，通过心跳包等机制维持连接状态。

3) 应用场景：适用于需要频繁通信的场景，如数据库连接、消息队列等。

(2)短连接：

1) 定义：每次请求和响应后，立即关闭连接，下一次请求时重新建立连接。

2) 实现：每次请求和响应完成后，客户端和服务器立即关闭连接。

3) 应用场景：适用于请求频率较低、每次请求独立的场景，如 HTTP 1.0 请求。

44. 什么是端口号？常见的端口号有哪些？

总结（老秦推荐必看，方便理解）：

端口号：在网络通信中，端口号用于标识特定的网络服务。每个端口号对应一个网络服务，范围从 0 到 65535。

常见端口号：

1) HTTP (80)：用于 Web 浏览的超文本传输协议。

2) HTTPS (443)：用于安全 Web 浏览的超文本传输协议。

3) FTP (21)：用于文件传输协议。

4) **SSH (22)** : 用于安全外壳协议。

5) **DNS (53)** : 用于域名系统。

45. 如何通过 Socket 实现可靠数据传输？

总结（老秦推荐必看，方便理解）：

(1) 序列号和确认号：

序列号：标识数据包的顺序，确保接收方按正确顺序组装数据。

确认号：接收方发送确认号，通知发送方已成功接收的数据。

(2) 超时重传：

超时机制：在指定时间内未收到确认，发送方重传数据包，确保数据到达。

(3) 流控制：

接收窗口：接收方通过接收窗口通知发送方当前可接收的数据量，避免缓冲区溢出。

(4) 拥塞控制：

拥塞窗口：发送方根据网络状况调整拥塞窗口大小，避免网络拥塞，提高传输效率。

四．网络安全

46. 常见的网络攻击方式有哪些？

总结（老秦推荐必看，方便理解）：

(1) **DDoS 攻击**：分布式拒绝服务攻击，通过大量请求使目标服务器无法正常服务。

(2) **中间人攻击**：攻击者截获并篡改通信双方的数据。

(3) **SQL 注入**：通过在输入字段中插入恶意 SQL 代码，攻击数据库。

(4) **跨站脚本攻击 (XSS)**：在网页中注入恶意脚本代码，盗取用户信息。

(5) **跨站请求伪造 (CSRF)**：伪造用户请求，执行未授权操作。

(6) **钓鱼攻击**：伪装成可信网站或邮件，诱骗用户输入敏感信息。

(7) **缓冲区溢出**：通过向应用程序输入超出其处理能力的数据，执行恶意代码。

47. 什么是防火墙？它的基本原理是什么？

总结（老秦推荐必看，方便理解）：

(1) **防火墙**：用于监控和控制进出网络流量的安全设备。

(2) **包过滤防火墙**：基于 IP 地址和端口的规则过滤数据包。

(3)状态检测防火墙：跟踪每个连接的状态，根据状态进行过滤。

(4)应用层防火墙：对应用层数据进行深度检测和过滤。

48. 什么是 VPN？它是如何工作的？

总结（老秦推荐必看，方便理解）：

VPN（Virtual Private Network，虚拟专用网络）是一种通过公共网络（如互联网）建立加密连接的技术，使用户能够在不安全的网络环境中安全地传输数据，就像他们直接连接到一个私人网络一样。VPN 主要用于保护数据传输的隐私和安全，同时可以绕过地理限制和防火墙访问受限资源。

(1)VPN 的工作原理

VPN 的工作原理可以简化为以下几个步骤：

- 1) 建立连接：用户启动 VPN 客户端，并选择连接到特定的 VPN 服务器。客户端和服务器之间通过互联网建立初始连接。
- 2) 隧道建立：客户端和服务器之间建立一个加密的隧道，这个隧道通常使用协议如 PPTP、L2TP/IPsec、OpenVPN、IKEv2 等。
- 3) 数据加密：用户的数据在发送到公共网络之前，通过加密算法进行加密（如 AES）。加密后的数据通过加密隧道传输到 VPN 服务器。
- 4) 数据传输和解密：VPN 服务器接收到加密的数据后进行解密，然后将数据发送到目标服务器。目标服务器返回的数据通过相同的路径反向传输，VPN 服务器加密数据并发送回 VPN 客户端，客户端解密数据后呈现给用户。

(2)VPN 的主要功能

- 1) 隐私保护：VPN 加密用户的数据，并隐藏其真实 IP 地址，保护用户的隐私。
- 2) 安全传输：通过加密隧道，确保数据在公共网络上传输时不会被窃听或篡改。
- 3) 绕过限制：通过连接到不同国家的 VPN 服务器，用户可以访问被地理限制的内容。
- 4) 远程访问：允许用户通过 VPN 安全地连接到企业内部网络，支持远程办公。

49. 什么是 HTTPS？与 HTTP 的区别？

图示（方便记忆）：

表. HTTPS 与 HTTP 的区别

特性	HTTP	HTTPS
协议	超文本传输协议	超文本传输协议安全版
端口	80	443
安全性	数据以明文形式传输，容易被窃听和篡改	数据通过 SSL/TLS 加密传输，确保数据的机密性和完整性

证书	不需要证书	需要 SSL/TLS 证书来验证服务器身份
开销	较低的处理开销	加密和解密过程增加了一定的处理开销
SEO	搜索引擎对 HTTP 站点的排名较低	搜索引擎更倾向于 HTTPS 站点，排名较高
应用场景	适用于一般的非敏感数据传输	适用于需要保护敏感数据的传输，如银行、电子商务等

总结（老秦推荐必看，方便理解）：

HTTPS (HyperText Transfer Protocol Secure, 超文本传输协议安全版) 是安全版的 HTTP。它通过 SSL/TLS 协议对数据进行加密，确保数据在客户端和服务端之间的传输过程中不会被窃听或篡改。

HTTPS 的工作原理

- 1) 建立连接：**客户端（浏览器）向服务器发送 HTTPS 请求。服务器返回 SSL/TLS 证书，包含公钥和服务器身份信息。
- 2) 身份验证：**客户端验证服务器的证书是否由可信的 CA（证书颁发机构）签发，并且证书是否有效。
- 3) 加密密钥协商：**客户端生成一个会话密钥，并使用服务器的公钥加密这个会话密钥，然后发送给服务器。服务器使用自己的私钥解密会话密钥，双方都拥有相同的会话密钥。
- 4) 加密通信：**双方使用会话密钥对数据进行加密和解密，确保传输的数据不会被窃听或篡改。

50. 什么是 SSL/TLS？它的基本工作流程？

总结（老秦推荐必看，方便理解）：

SSL (Secure Sockets Layer, 安全套接层) 和 TLS (Transport Layer Security, 传输层安全) 是用于在网络上建立安全连接的协议。TLS 是 SSL 的升级版，提供了更强的安全性。SSL/TLS 通过加密数据传输、认证服务器和客户端身份、并保证数据完整性，确保通信的安全性。

SSL/TLS 的基本工作流程

- 1) 客户端问候 (Client Hello)**
客户端向服务器发送一个 "Client Hello" 消息，消息中包含客户端支持的 SSL/TLS 协议版本、加密算法列表、压缩方法以及一个随机数 (Client Random)。
- 2) 服务器问候 (Server Hello)**
服务器接收到 "Client Hello" 消息后，选择一个协议版本、加密算法和压缩方法，并将这些选择通过 "Server Hello" 消息返回给客户端。同时，服务器也会生成一个随机数 (Server Random)。
- 3) 服务器证书 (Server Certificate)**
服务器向客户端发送其 SSL/TLS 证书，证书中包含服务器的公钥以及由 CA（证书颁发机构）签名的服务器身份信息。
- 4) 服务器密钥交换 (Server Key Exchange)**

如果使用的加密算法需要，服务器 还会发送一个 "Server Key Exchange" 消息，包含 Diffie-Hellman 参数或其他密钥交换信息。

5) 服务器问候完成 (Server Hello Done)

服务器 发送一个 "Server Hello Done" 消息，表示服务器端的问候阶段已完成。

6) 客户端密钥交换 (Client Key Exchange)

客户端 生成一个预主密钥 (Pre-Master Secret)，用服务器的公钥加密该预主密钥，然后发送给服务器。

7) 会话密钥生成 (Session Key Generation)

客户端和服务器 分别使用之前的 Client Random、Server Random 和预主密钥生成主密钥 (Master Secret)，并从主密钥中派生出会话密钥 (Session Keys)，这些密钥用于对后续的数据传输进行加密。

8) 客户端完成 (Client Finished)

客户端 发送一个 "Finished" 消息，包含所有握手消息的哈希值并用会话密钥加密，表示客户端的握手阶段已完成。

9) 服务器完成 (Server Finished)

服务器 接收到 "Client Finished" 消息后，也发送一个 "Finished" 消息，包含所有握手消息的哈希值并用会话密钥加密，表示服务器的握手阶段已完成。

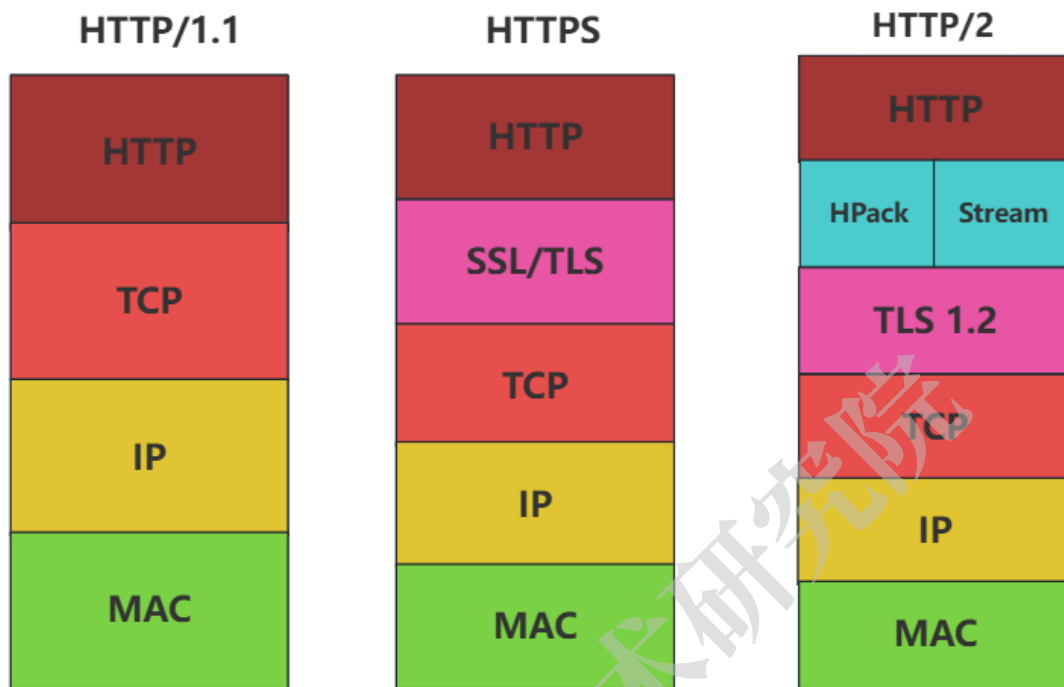
10) 加密数据传输 (Encrypted Data Transfer)

客户端和服务器 使用会话密钥对后续的数据进行加密和解密，确保数据传输的机密性和完整性。

五. 应用层协议

51. HTTP/1.1、HTTP/2.0 和 HTTP/3.0 的区别？

图示 (方便记忆)：



总结（老秦推荐必看，方便理解）：

(1)HTTP/1.1

1.持久连接：

描述：**HTTP/1.0** 每次请求需要建立一个新的 **TCP 连接**，而 **HTTP/1.1** 引入了持久连接，允许多个请求和响应复用 **一个 TCP 连接**，**减少了连接建立和关闭的开销**。

优点：减少了频繁建立和关闭连接的开销，提升了传输效率。

2.管道化：

描述：允许客户端在发送第一个请求之后，未等待其响应就发送第二个请求。虽然理论上可以提高效率，但在实际中由于中间节点和服务器对管道化支持不佳，头阻塞问题严重，因此并未广泛采用。

优点：可以提高请求的并发性，减少整体响应时间。

缺点：头阻塞问题严重，实际应用中支持不佳。

3.分块传输编码：

描述：服务器在响应体未知大小时可以**使用分块传输编码**，将数据分成若干块进行传输，每个块都有自己的大小标识，接收端根据块大小逐一接收并组装。

优点：**支持大文件和流式数据的传输**。

4.缓存机制：

描述：**提供了丰富的缓存控制头部字段**，如 Cache-Control、ETag、Expires 等，用于控制客户端和代理服务器的缓存行为。

优点：提升了资源的利用效率，**减少了重复请求**。

5.性能瓶颈：

- 1).请求/响应头部未经压缩，信息越多延迟越大。
- 2).每次发送相同的头部信息，导致了大量的冗余和浪费。
- 3).服务器按请求顺序响应，头阻塞问题严重。
- 4).无法控制请求的优先级，重要的请求不能优先处理。
- 5).请求只能由客户端发起，服务器只能被动响应。

(2)HTTP/2.0

1.二进制协议：

描述：相较于 HTTP/1.1 的文本协议，**HTTP/2.0 使用二进制格式传输**，解析更高效且减少了协议解析中的错误。

优点：提高了数据传输的效率和可靠性。

2.多路复用：

描述：在一个 **TCP 连接上同时发送多个请求和响应**，每个请求/响应对应一个流 (Stream)，通过流标识区分不同的请求和响应，避免了 HTTP/1.1 中的头阻塞问题。

优点：解决了头阻塞问题，提高了连接的利用率。

3.头部压缩：

描述：使用 HPACK 算法对头部进行压缩，显著减少了头部大小，尤其是对于频繁重复的头部字段，如 Cookie。

优点：减少了头部信息的传输开销，提高了传输速度。

4.服务器推送：

描述：服务器可以在客户端未明确请求时主动推送资源，提高页面加载速度。例如，服务器在响应 HTML 文档时，可以同时推送相关的 CSS、JS 文件。

优点：减少了客户端的请求次数，提升了用户体验。

5.流优先级：

描述：客户端可以为每个流设置优先级，告诉服务器哪些资源更重要，应该优先传输，提升关键资源的加载速度。

优点：提升了重要资源的传输速度。

6.性能瓶颈：

基于 TCP，仍然存在 TCP 层的头阻塞问题。

(3)HTTP/3.0

1.基于 QUIC：

描述：HTTP/3.0 使用基于 UDP 的 QUIC 协议而不是 TCP，QUIC 提供了类似 TCP 的可靠传输和连接管理，但具备更低的延迟和更好的抗丢包性能。QUIC 的握手结合了传输层和 TLS 的握手过程，使得连接建立速度更快。

优点：避免了 TCP 层的头阻塞问题，提高了连接建立速度和抗丢包性能。

2.内置加密：

描述：默认使用 TLS 1.3 加密，QUIC 协议本身包含加密机制，提供更高的安全性，简化了配置和部署。

优点：提供了更高的安全性和隐私保护。

3.无头阻塞：

描述：QUIC 的多路复用特性避免了 TCP 中的头阻塞问题，即使某个流的数据丢失，也不会阻塞其他流的数据传输。

优点：提高了整体数据传输的效率和可靠性。

4.更快的连接建立：

描述：QUIC 通过结合 TLS 和传输层握手，通常只需一次往返时间 (RTT) 就能完成连接建立，相比 TCP+TLS 通常需要两次往返时间，大幅减少了连接建立时间。

优点：显著减少了连接建立时间，提升了用户体验。

5.改进的错误恢复：

描述：QUIC 协议中，每个数据包都有独立的序列号，丢包时可以仅重传丢失的数据包，不像 TCP 需要重传整个窗口的数据，提高了网络效率。

优点：提高了数据传输的可靠性和效率。

(4)底层原理详细解析

1.持久连接和多路复用

1).HTTP/1.1 持久连接：使用 Connection: keep-alive 头部字段来保持连接，多个请求复用同一个 TCP 连接，减少了频繁建立和关闭连接的开销。

2).HTTP/2.0 多路复用：基于帧 (Frame) 的概念，每个 HTTP 消息被拆分成多个帧，每个帧都有流标识 (Stream Identifier)，这些帧可以在一个 TCP 连接中并行传输，接收端根据流标识重新组装。

2.头部压缩

1).HTTP/1.1：每个请求和响应的头部字段都是完整的文本格式，容易造成冗余，尤其是在多次请求相同资源时。

2).HTTP/2.0：使用 HPACK 算法，对头部字段进行动态表 (Dynamic Table) 和静态表 (Static Table) 压缩，大大减少了重复头部字段的传输开销。

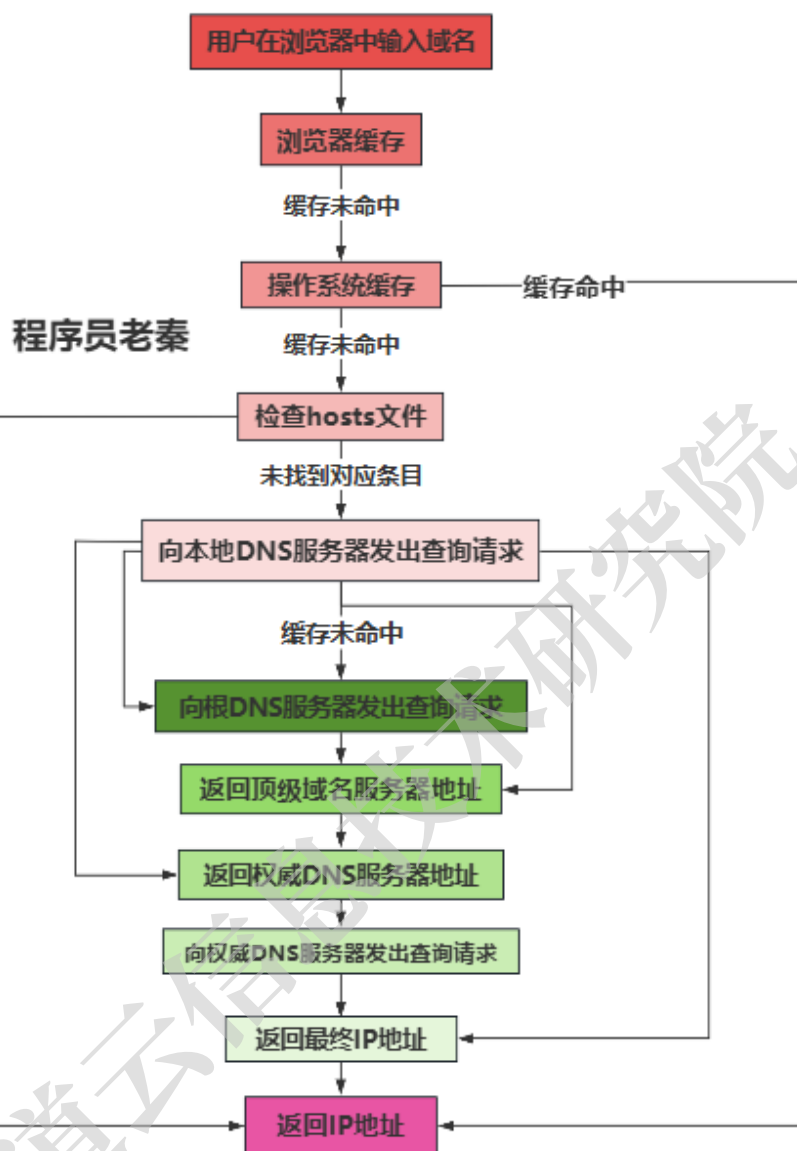
3.QUIC 和 TCP 的区别

1)TCP：基于连接的传输协议，提供可靠的数据传输，但建立连接需要三次握手，且丢包时整个连接受影响。

2)QUIC：基于 UDP，但提供类似 TCP 的可靠性和顺序保证，同时结合 TLS 加密，减少连接建立时间，具备更好的抗丢包性能，适用于高延迟和不稳定的网络环境。

52. DNS 解析的过程？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

DNS (Domain Name System) 解析是将域名转换为 IP 地址的过程。**DNS** 解析的过程可以分为以下几个步骤：

(1)客户端缓存:

当用户在浏览器中输入域名（例如：www.example.com）时，浏览器首先会检查本地 DNS 缓存，看看是否有对应的 IP 地址。如果有，则直接使用缓存中的 IP 地址，不再进行后续的查询。

(2)操作系统缓存:

如果浏览器缓存中没有找到对应的 IP 地址，操作系统会检查其缓存。

(3)本地 hosts 文件:

如果操作系统缓存也没有找到对应的 IP 地址，操作系统会检查本地的 hosts 文件是否有对应的条目。

(4)本地 DNS 服务器（递归解析器）：

如果以上步骤都未能找到对应的 IP 地址，浏览器会将查询请求发送到配置的本地 DNS 服务器（通常是 ISP 提供的 DNS 服务器或公司内部的 DNS 服务器）。

本地 DNS 服务器会先检查自身的缓存。如果缓存中有对应的 IP 地址，则返回给客户端。

(5)根 DNS 服务器:

如果本地 DNS 服务器的缓存中没有对应的 IP 地址，它会向根 DNS 服务器发出查询请求。根 DNS 服务器会告诉本地 DNS 服务器应该查询的顶级域名服务器（例如，.com、.net 等）。

(6)顶级域名服务器（TLD 服务器）：

本地 DNS 服务器接到根 DNS 服务器的响应后，会向相应的顶级域名服务器发出查询请求。顶级域名服务器会告诉本地 DNS 服务器应该查询的权威 DNS 服务器（例如，负责 example.com 域的 DNS 服务器）。

(8)权威 DNS 服务器:

本地 DNS 服务器接到顶级域名服务器的响应后，会向权威 DNS 服务器发出查询请求。权威 DNS 服务器会返回最终的 IP 地址给本地 DNS 服务器。

(9)返回结果给客户端:

本地 DNS 服务器接到权威 DNS 服务器的响应后，会将 IP 地址缓存起来，并返回给客户端（浏览器）。

(10)浏览器建立连接:

浏览器接到本地 DNS 服务器返回的 IP 地址后，会使用该 IP 地址与目标服务器建立连接，发送 HTTP 请求，获取网页内容。

53. 什么是负载均衡？常见的负载均衡算法？

总结（老秦推荐必看，方便理解）：

负载均衡（Load Balancing）是一种分布式计算技术，通过将工作负载和请求分配到多个服务器（或其他计算资源）上，来优化资源使用、提高响应速度、增加系统的可用性和稳定性。负载均衡器（Load Balancer）是执行这一任务的设备或软件，其主要目标是确保没有单一服务器被过度使用，从而避免性能瓶颈和系统故障。

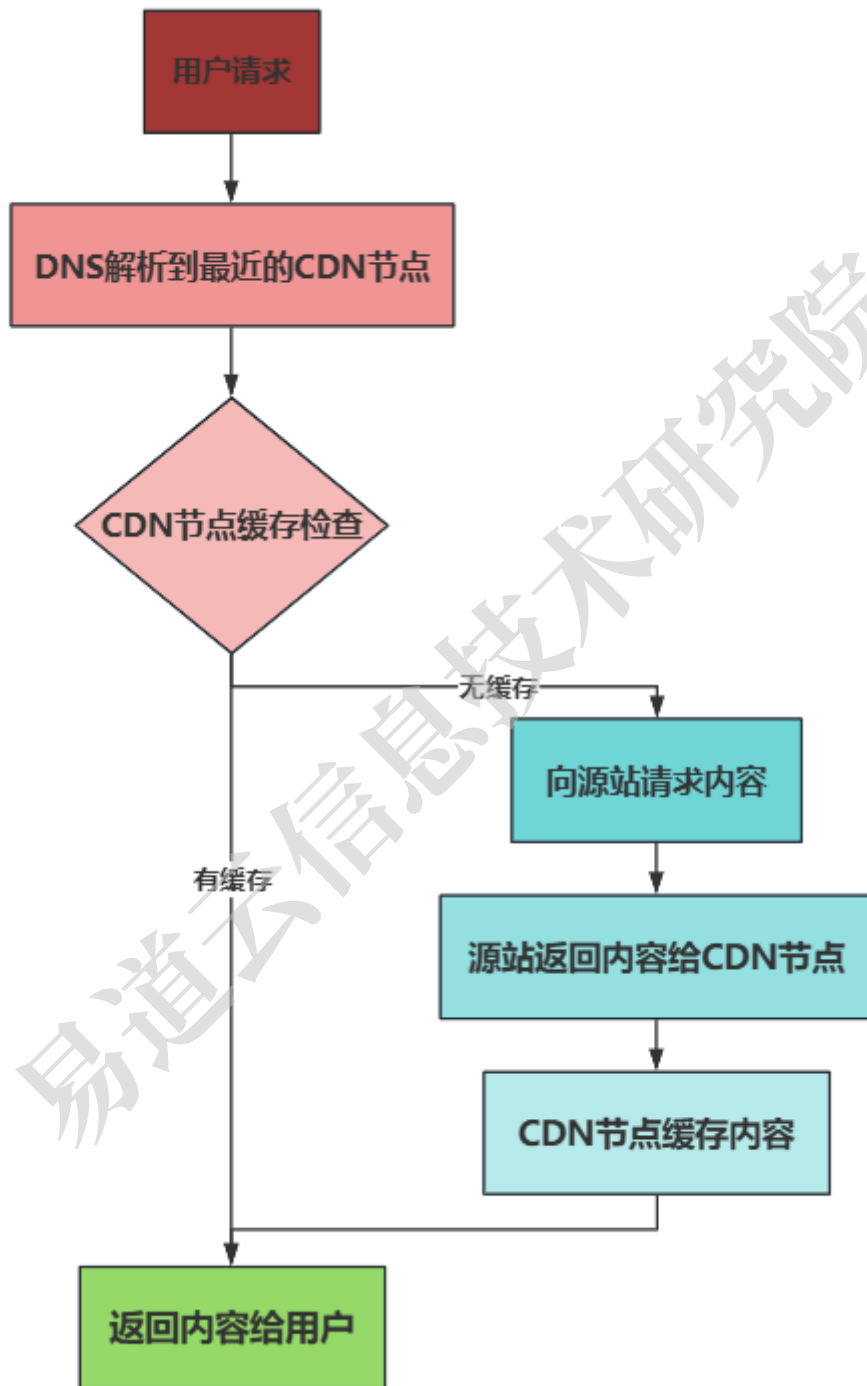
图示（方便记忆）：

算法	描述	优点	缺点
----	----	----	----

轮询	依次将请求分配给每台服务器	简单易实现，适用于负载均衡需求较低的场景	不考虑服务器的当前负载和处理能力，可能导致负载不均衡
加权轮询	为每台服务器分配权重，权重高的接收更多请求	考虑了服务器的处理能力，能更好地分配负载	权重设置和调整需要根据实际情况优化
最小连接数	将请求分配给当前活动连接数最少的服务器	动态调整，适合长连接的场景	需要持续跟踪服务器的连接数，计算量较大
加权最小连接数	在最小连接数的基础上引入权重概念	结合了权重和最小连接数的优点，能更精确地分配负载	需要实时跟踪服务器状态，增加了一定的复杂性
源地址哈希	根据客户端 IP 地址的哈希值将请求分配到固定的服务器	适用于需要会话保持的场景	服务器数量变化时，哈希算法可能导致请求重新分配
随机	随机选择一台服务器处理请求	简单易实现，适用于负载均衡需求较低的场景	不考虑服务器的当前负载和处理能力，可能导致负载不均衡
最小响应时间	将请求分配给平均响应时间最短的服务器	能动态适应服务器的负载情况，适用于对响应时间要求较高的场景	需要实时监控和统计服务器的响应时间，增加了实现复杂性

54. 什么是 CDN？它的工作原理是什么？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

CDN (Content Delivery Network, 内容分发网络) 是一种通过在全球范围内部署多个服务器节点, 帮助用户更快速、更稳定地访问网站内容的技术。CDN 的主要目标是提高内容传输的速度和可靠性, 减小服务器负载, 提升用户体验。

CDN 的优势

- 1) 加速内容传输: 通过将内容缓存到靠近用户的节点, 减少了传输延迟, 提高了访问速度。
- 2) 减轻源站负载: 大量请求由 CDN 节点处理, 减少了源站服务器的压力, 提高了网站的稳定性。
- 3) 提高可用性: CDN 节点分布在全球各地, 即使某些节点发生故障, 其他节点仍可以继续提供服务, 保证网站的高可用性。
- 4) 安全性增强: CDN 可以提供 DDoS 攻击防护、WAF (Web 应用防火墙) 等安全功能, 保护源站服务器免受恶意攻击。
- 5) 带宽优化: 通过缓存和内容优化, 减少了冗余数据传输, 节省了带宽资源。

55. 什么是 WebSocket ? 它与 HTTP 的区别 ?

总结 (老秦推荐必看, 方便理解) :

WebSocket 是一种在单个 TCP 连接上进行全双工通信的协议。WebSocket 允许服务器和客户端之间进行实时、双向的数据传输, 特别适用于需要频繁更新数据的应用, 如在线聊天、游戏、股票行情推送等。

WebSocket 的工作原理

- 1) 建立连接: WebSocket 连接从 HTTP 请求开始, 客户端发送一个带有特殊头部字段的 HTTP 请求到服务器, 表示希望升级到 WebSocket 协议。这个请求被称为“握手”请求。服务器响应握手请求, 确认可以切换协议。握手成功后, HTTP 连接升级为 WebSocket 连接。
- 2) 数据传输: 建立连接后, 客户端和服务器之间可以通过这个单一的 WebSocket 连接进行双向通信。WebSocket 使用帧 (frame) 来传输数据。帧可以包含文本数据或二进制数据, 且通信时无须每次都携带头部信息, 减少了传输开销。
- 3) 关闭连接: 连接可以由客户端或服务器任意一方主动关闭, 通过发送一个关闭帧。另一方收到关闭帧后, 会发送一个关闭帧以确认关闭, 然后连接被终止。

图示 (方便记忆) :

表. WebSocket 与 HTTP 的区别

特性	HTTP	WebSocket
通信模式	请求-响应模式, 单向通信	全双工通信, 双向通信
连接方式	每个请求/响应需要独立的 TCP 连接	通过单个长连接进行多次数据传输
头部信息	每个请求和响应都包含完整的头部信息	仅在握手阶段有头部信息, 之后的帧通信无需头部信息
实时性	由于每次请求都需要建立连	长连接方式, 实时性强

	接，实时性较差	
数据传输效率	头部开销大，每次请求都需要传输完整头部信息	帧的开销小，适合高频率、低延迟的数据传输
使用场景	适合普通的网页浏览和静态内容获取	适合实时数据传输和需要频繁更新数据的应用，例如聊天、游戏、股票行情等
连接状态保持	无状态协议，每次请求都是独立的	有状态协议，保持连接状态，支持状态管理

WebSocket 的优势

- 1) **实时通信**：WebSocket 支持双向通信，允许服务器和客户端实时互相发送数据，减少了延迟，提高了实时性。
- 2) **降低开销**：一旦建立连接，后续数据传输不需要重复传输头部信息，降低了带宽开销和处理开销。
- 3) **保持连接状态**：WebSocket 连接是持久的，适用于需要持续数据更新的应用场景。

56. 解释 FTP、SMTP、IMAP 和 POP3 的基本原理和应用场景？

总结（老秦推荐必看，方便理解）：

(1)FTP (File Transfer Protocol)

- 1) **基本原理**：**FTP 是一种用于在客户端和服务器之间传输文件的协议。**FTP 使用两个不同的连接：控制连接和数据连接。控制连接用于发送命令和接收响应，数据连接用于实际的数据传输。**FTP 支持两种模式：主动模式 (Active Mode) 和被动模式 (Passive Mode)。**
- 2) **主动模式**：客户端向服务器发送 PORT 命令，告诉服务器用于数据传输的端口，服务器使用这个端口连接客户端。
- 3) **被动模式**：客户端向服务器发送 PASV 命令，服务器打开一个随机端口并通知客户端，客户端连接这个端口进行数据传输。

(2)SMTP (Simple Mail Transfer Protocol)

基本原理：**SMTP 是一种用于在电子邮件服务器之间传输电子邮件的协议。**SMTP 工作在 TCP 的 25 端口，通过简单的文本命令和响应进行通信。电子邮件客户端使用 SMTP 将邮件发送到邮件服务器，邮件服务器之间也使用 SMTP 传输邮件。

邮件发送流程：

- 1) 客户端连接到 SMTP 服务器。
- 2) 客户端通过 EHLO 或 HELO 命令向服务器问候。
- 3) 客户端通过 MAIL FROM 命令指定发件人地址。
- 4) 客户端通过 RCPT TO 命令指定收件人地址。
- 5) 客户端通过 DATA 命令发送邮件内容。
- 6) 客户端通过 QUIT 命令结束会话。

(3)IMAP (Internet Message Access Protocol)

基本原理：**IMAP 是一种用于从邮件服务器读取电子邮件的协议。**IMAP 允许用户在多个设备上查看和管理他们的电子邮件，因为邮件存储在服务器上，并且客户端只需同步邮件的状态和内容。IMAP 工作在 TCP 的 143 端口 (IMAP4) 或 993 端口 (IMAPS，使用 SSL/TLS)。

邮件读取流程：

- 1) 客户端连接到 IMAP 服务器。
- 2) 客户端通过 LOGIN 命令进行身份验证。
- 3) 客户端通过 SELECT 命令选择邮箱。
- 4) 客户端通过 FETCH 命令获取邮件。
- 5) 客户端通过 STORE 命令修改邮件状态 (如标记已读)。
- 6) 客户端通过 LOGOUT 命令结束会话。

(4)POP3 (Post Office Protocol 3)

基本原理：**POP3 是一种用于从邮件服务器下载电子邮件的协议。**与 IMAP 不同，POP3 通常会将邮件从服务器下载到本地设备，并从服务器上删除。POP3 工作在 TCP 的 110 端口或 995 端口 (POPS，使用 SSL/TLS)。

邮件读取流程：

- 1) 客户端连接到 POP3 服务器。
- 2) 客户端通过 USER 和 PASS 命令进行身份验证。
- 3) 客户端通过 LIST 命令列出邮件。
- 4) 客户端通过 RETR 命令下载邮件。
- 5) 客户端通过 DELE 命令删除邮件。
- 6) 客户端通过 QUIT 命令结束会话。

图示 (方便记忆)：

表. 协议区别

协议	基本原理	应用场景
FTP	用于在客户端和服务器之间传输文件，使用控制连接和数据连接。	文件上传下载、文件共享、数据备份
SMTP	用于在电子邮件服务器之间传输电子邮件，使用简单文本命令和响应进行通信。	电子邮件发送、邮件中继
IMAP	用于从邮件服务器读取电子邮件，允许多设备同步访问，邮件存储在服务器上。	多设备同步、服务器端存储
POP3	用于从邮件服务器下载电子邮件，通常下载后会从服务器删除。	单设备访问、离线访问

六 . 网络性能和优化

57. 如何监控和分析网络性能？

总结（老秦推荐必看，方便理解）：

(1)使用网络监控工具

- 1) **Wireshark**：一个免费的开源网络协议分析工具，用于捕获和分析网络数据包。可以详细查看每个数据包的内容，帮助诊断网络问题。
- 2) **Nagios**：一个开源的网络监控工具，可以监控网络设备、服务器和服务的状态，提供警报和报告。
- 3) **SolarWinds Network Performance Monitor (NPM)**：一款商业网络监控工具，提供实时网络性能监控和故障排除功能。

(2)网络性能指标监控

- 1) **带宽使用率**：监控网络接口的带宽使用情况，识别是否有带宽不足或过度使用
- 2) **延迟 (Latency)**：监控网络设备和应用之间延迟，确保数据包在合理时间内传输。
- 3) **抖动 (Jitter)**：监控延迟的变化，尤其对于实时应用(如音视频)来说非常重要。
- 4) **丢包率 (Packet Loss)**：监控数据包的丢失情况，高丢包率会严重影响网络性能。
- 5) **网络利用率**：监控网络设备和链路的利用率，识别是否存在过度利用的情况。

(3)性能测试和基准测试

- 1) **iPerf**：一个网络性能测试工具，用于测量带宽和丢包率，支持 TCP 和 UDP 测试。
- 2) **Ping 和 Traceroute**：基本的网络诊断工具，用于测试网络连通性和路由路径。
- 3) **NetFlow/SFlow**：用于收集和分析网络流量数据，识别流量模式和潜在的瓶颈。

(4)使用网络分析工具

- 1) **NetFlow Analyzer**：用于分析 NetFlow 数据，监控和报告网络流量。
- 2) **PRTG Network Monitor**：一个综合性的网络监控工具，支持带宽监控、流量分析、SNMP、NetFlow 等多种功能。

(5)使用日志和事件管理工具

- 1) **Splunk**：一个日志管理和分析平台，可以收集和分析网络设备的日志，帮助识别和解决网络问题。
- 2) **ELK Stack (Elasticsearch, Logstash, Kibana)**：一个开源的日志管理和分析套件，用于实时收集、存储和分析网络日志数据。

58. 常见的网络性能瓶颈有哪些？

总结（老秦推荐必看，方便理解）：

(1)带宽不足

- 1) 描述：网络链路的**带宽不足以支持当前的流量需求**，导致拥塞和延迟增加。
- 2) 解决方案：升级带宽、优化流量管理、实施 QoS（服务质量）策略。

(2)高延迟

- 1) 描述：数据包在网络中**传输的时间过长**，影响应用程序的响应时间和用户体验。
- 2) 解决方案：优化路由、减少中间节点、使用 CDN 加速内容分发。

(3)丢包

- 1) 描述：网络中**数据包的丢失**会导致重传，增加延迟并降低传输效率。
- 2) 解决方案：检查和修复网络设备和链路故障、优化网络配置、实施流量控制策略。

(4)网络设备性能不足

- 1) 描述：路由器、交换机或防火墙等**网络设备的性能不足**，无法处理当前的流量负载。
- 2) 解决方案：升级网络设备、优化设备配置、分布流量负载。

(5)网络抖动

- 1) 描述：数据包**延迟的变化不稳定**，特别是对于实时应用（如语音、视频）影响较大。
- 2) 解决方案：优化网络路径、实施 QoS 策略、减少网络拥塞。

(6)DNS 解析慢

- 1) 描述：DNS 服务器的**响应时间过长**，导致域名解析延迟。
- 2) 解决方案：使用更快的 DNS 服务器、实施 DNS 负载均衡、优化 DNS 缓存。

59. 如何优化网络传输性能？

总结（老秦推荐必看，方便理解）：

(1)增加带宽

- 1) 升级网络带宽：根据需求升级带宽，以支持更高的数据传输速率，减少拥塞和延迟。
- 2) 链路聚合（Link Aggregation）：将多个网络接口绑定在一起，形成一个逻辑接口，提高带宽和冗余性。

(2)优化网络设备

- 1) 升级网络设备：使用性能更高的路由器、交换机、防火墙等设备，支持更高的吞吐量和并发连接数。

2) 合理配置设备：调整设备配置以支持 QoS（服务质量）、流量控制和负载均衡等功能，确保关键应用的性能。

(3)实施 QoS 策略

1) 定义优先级：通过 QoS 策略，为不同类型的流量（如语音、视频、数据）分配优先级，确保关键应用的带宽和响应时间。

2) 流量整形：限制低优先级流量的带宽使用，防止其影响高优先级流量。

(4)使用 CDN（内容分发网络）

1) 内容缓存：将静态内容（如图片、视频、文件等）缓存到离用户更近的 CDN 节点，减少传输延迟，提高访问速度。

2) 分布式负载：通过 CDN 将流量分布到多个节点，减轻源服务器的负载，提高整体性能。

(5)优化协议

1) HTTP/2 和 HTTP/3：使用更高效的 HTTP 协议版本（如 HTTP/2 和 HTTP/3），支持多路复用、头部压缩和服务器推送，减少延迟和开销。

2) TCP 优化：调整 TCP 窗口大小、使用 SACK (Selective Acknowledgement) 和 TFO (TCP Fast Open) 等技术，提高传输效率。

3) UDP 优化：对于实时应用（如语音、视频），使用 QUIC 等基于 UDP 的高效传输协议。

(6)减少延迟

1) 路径优化：选择最优的路由路径，减少中间节点和跳数，降低传输延迟。

2) DNS 优化：使用快速的 DNS 服务器、启用 DNS 缓存和负载均衡，减少域名解析时间。

(7)减少丢包率

1) 网络可靠性：检查和修复网络设备和链路，确保硬件的稳定性和可靠性，减少丢包。

2) 流量控制：避免网络过载，使用流量整形和拥塞控制技术，减少丢包率。

(8)使用压缩技术

1) 数据压缩：在传输数据前进行压缩，减少传输的数据量，提高传输效率。

2) 协议压缩：如 HTTP 压缩 (gzip、brotli)，压缩 HTTP 头部和内容，提高传输速度。

60. 什么是 QoS？如何实现 QoS？

总结（老秦推荐必看，方便理解）：

QoS (Quality of Service, 服务质量) 是一种技术，**用于管理和保证网络服务的性能、可靠性和可用性**。QoS 的主要目标是提供不同级别的服务质量，以满足不同应用和用户的需求。QoS 通过优先级调度、流量整形、带宽管理等机制，确保关键应用和服务的优先传输。

(1)分类和标记 (Classification and Marking)

1) 描述：根据数据包的类型、源地址、目的地址、端口号等信息对数据包进行分类，并打上标记（如 DSCP，Differentiated Services Code Point）。

2) 作用：为后续的流量管理和调度提供基础。

(2)流量整形 (Traffic Shaping)

1) 描述：控制流量的传输速率，使其符合预设的带宽限制，避免突发流量

2) 作用：平滑流量，防止链路过载。

(3)拥塞管理 (Congestion Management)

1) 描述：在网络设备中，通过队列管理和调度算法（如 FIFO、WFQ、CBWFQ）来控制和管理流量。

2) 作用：确保高优先级的流量得到优先处理，降低拥塞对关键业务的影响。

(4)拥塞避免 (Congestion Avoidance)

1) 描述：使用算法（如 RED，Random Early Detection）提前检测和丢弃部分数据包，以避免拥塞发生。

2) 作用：预防网络拥塞，保持网络流量的平稳。

(5)带宽管理和分配 (Bandwidth Management and Allocation)

1) 描述：为不同的应用和服务分配预定的带宽，确保关键应用有足够的带宽使用。

2) 作用：保证关键业务的带宽需求，防止低优先级流量占用过多带宽。

(6)优先级调度 (Priority Scheduling)

1) 描述：基于数据包的优先级（如高、中、低）进行调度，确保高优先级的数据包优先通过网络。

2) 作用：提高关键应用的响应速度和可靠性。

61. 如何实现一个高效的聊天应用？

总结（老秦推荐必看，方便理解）：

(1)架构设计

1) 选择合适的协议：WebSocket：用于实现实时双向通信，比传统的 HTTP 请求-响应模型更高效。MQTT：一种轻量级的发布/订阅消息传输协议，适用于需要低带宽和不稳定网络环境的场景。

2) 负载均衡：使用负载均衡器（如 Nginx、HAProxy）将客户端连接分发到多个服务器，防止单点故障，提高系统可用性。

3) 水平扩展：将聊天服务器进行水平扩展，通过集群方式处理大量并发连接。

4) 消息队列：使用消息队列（如 RabbitMQ、Kafka）管理和分发消息，确保消息的可靠传输和高效处理。

(2)数据库选择：

1) **持久化存储**：对于重要的聊天记录，可以使用持久化存储（如 SQL 数据库）进行存储，确保数据的持久性和可靠性。Redis 可以用于存储在线用户状态和最近的消息，以实现快速访问。

(3)性能优化

1) **缓存**：使用缓存技术（如 Redis、Memcached）缓存热点数据，减少数据库压力，提高响应速度。

2) **异步处理**：将耗时操作（如文件上传、消息持久化）放到后台异步处理，使用消息队列进行任务调度。

3) **压缩和优化数据传输**：压缩消息数据，减少带宽占用，优化网络传输。

62. 如何确保数据传输的安全性？

总结（老秦推荐必看，方便理解）：

确保数据传输的安全性需要采取多种安全措施，包括加密、身份验证和安全协议。以下是一些常见的方法：

(1)加密

1) **TLS/SSL**：使用 TLS/SSL 协议加密传输层的数据，确保数据在传输过程中不被窃听和篡改。

2) **端到端加密 (E2EE)**：在发送端对消息进行加密，在接收端进行解密，确保只有通信双方能够查看消息内容。

(2)身份验证

1) **OAuth**：使用 OAuth 协议进行身份验证和授权，确保只有经过授权的用户才能访问资源。

2) **JWT (JSON Web Token)**：使用 JWT 进行身份验证，确保请求的合法性和安全性。

(3)安全协议

1) **HTTPS**：使用 HTTPS 协议传输数据，确保传输层的安全性。

2) **IPsec**：使用 IPsec 协议在网络层对数据进行加密和认证，确保数据包的完整性和保密性。

(4)安全实践

1) **防火墙和入侵检测系统 (IDS)**：部署防火墙和 IDS，检测和防止网络攻击。

2) **安全审计和监控**：定期进行安全审计和监控，及时发现和修复安全漏洞。

63. 解释一下消息队列的工作原理及其在分布式系统中的应用？

总结（老秦推荐必看，方便理解）：

消息队列是一种用于异步通信的中间件，通过在生产者和消费者之间传递消息来实现解耦。

(1)工作原理：

- 1) 消息生产：生产者（发布者）将消息发送到消息队列。
- 2) 消息存储：消息队列负责存储消息，直到消费者（订阅者）读取消息。
- 3) 消息消费：消费者从消息队列中读取消息进行处理。
- 4) 消息确认：消费者处理完消息后，向消息队列发送确认，队列将该消息标记为已处理。
- 5) 消息删除：已处理的消息从队列中删除，未确认的消息在一段时间后可以重新投递。

(2)在分布式系统中的应用：

- 1) 异步处理：通过消息队列实现异步处理，将耗时任务放到后台执行，减轻主应用的负担，提高系统响应速度。
- 2) 解耦组件：使用消息队列将不同组件解耦，减少组件间的依赖性，增强系统的可扩展性和维护性。
- 3) 负载均衡：通过消息队列实现负载均衡，将任务分发到多个消费者，提升系统的并发处理能力。
- 4) 峰值削减：在并发场景下，通过消息队列缓冲请求，平滑处理负载，防止系统过载。
- 5) 数据同步：通过消息队列实现跨服务的数据同步，确保数据一致性。

64. 什么是零拷贝技术？

总结（老秦推荐必看，方便理解）：

零拷贝（Zero-Copy）技术是一种减少或消除数据在内存中拷贝次数的方法，以提高数据传输效率。其主要目标是在网络传输、文件操作等场景中，减少 CPU 负载和内存带宽的使用。

(1)零拷贝的实现方式

1) sendfile：

描述：将数据从一个文件直接发送到网络接口，而不需要经过用户空间。

应用：用于文件服务器、HTTP 服务器等需要高效传输大文件的场景。

2) mmap + write：

描述：使用 mmap 将文件映射到进程的虚拟地址空间，然后通过 write 将数据发送到网络接口。

应用：适用于需要频繁读取和写入文件的场景。

3) splice：

描述：将数据在两个文件描述符之间直接移动，而不需要经过用户空间。

应用：用于高效的数据管道操作，如网络通信、日志记录等。

(2)零拷贝的优点

- 1) 减少 CPU 负载：通过减少数据拷贝次数，降低了 CPU 的负载，提高了系统的整体性能。
- 2) 提高传输速度：由于减少了数据在内存中的移动，提高了数据传输的速度。
- 3) 降低内存带宽使用：减少了内存带宽的使用，避免了不必要的数据传输，提高了内存的利用率。

七. 网络与并发

65. Reactor 和 Proactor 模式的区别及应用场景。

66. 一台机器最多可以建立多少个 TCP 连接？

67. 网络通信中的阻塞和非阻塞 I/O 的区别。

68. Socket 通信中如何处理并发连接？

69. IO 多路复用的实现机制。

第五章 操作系统

一. 进程、线程、协程

1.进程、线程和协程的区别？

图示（方便记忆）：

特性	进程	线程	协程
定义	操作系统中资源分配的基本单位	进程中的一个执行单元	用户级别的轻量级线程
拥有资源	独立的内存空间和资源	共享进程的内存和资源	共享线程的内存和资源
创建开销	高，需分配独立的内存空间	较低，共享进程的资源	很低，基于函数调用和堆栈
切换开销	高，涉及内存页切换等	较低，但需进行上下文切换	非常低，仅切换堆栈指针
通信方式	通过 IPC（如管道、消息队列）	通过共享内存或信号量	通过直接函数调用和回调
并发性	可以并发执行	可以并发执行	不能并发，仅顺序执行
调度	由操作系统内核调度	由操作系统内核调度	由用户代码自行调度
适用场景	独立运行的程序，多任务处理	多线程并发操作，I/O 密集型	并发且轻量的任务处理
兼容性	独立于其他进程	与同一进程的其他线程紧密关联	与同一线程的其他协程紧密关联

总结（老秦推荐必看，方便理解）：

进程、线程和协程都是并发编程的重要概念。**进程**是资源分配的最小单位，**拥有独立的地址空间，适合独立的应用程序**。**线程**共享进程的资源，但能独立执行，**适合执行并行任务**。**协程**则是更轻量级的执行单元，**运行在线程之上，通过用户态的调度实现非阻塞的任务切换，适合处理大量的异步任务**。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <thread>
#include <coroutine> // C++20 开始支持协程
#include <future>
// 线程函数
void threadFunction() {
    std::cout << "Hello from Thread" << std::endl;
}
// 协程函数
std::future<void> coroutineFunction() {
    co_await std::suspend_always{};
    std::cout << "Hello from Coroutine" << std::endl;
}
int main() {
    // 创建进程的例子通常涉及创建完全独立的应用实例
    // 创建并启动线程
    std::thread t(threadFunction);
    if (t.joinable()) t.join();
    // 启动协程
    auto future = coroutineFunction();
    future.wait(); // 等待协程完成
    return 0;
}
```

2.进程和线程在上下文切换时切换了什么、共享了什么？

总结（老秦推荐必看，方便理解）：

(1)进程

1) 定义：进程是操作系统进行资源分配和调度的基本单位，是程序的一次执行过程，拥有独立的地址空间和系统资源。

2) 资源共享：独立性：每个进程拥有独立的内存空间和系统资源（如文件描述符、环境变量等）。通信方式：进程间通信（IPC）需要特定的机制，如管道、信号、消息队列、共享内存、套接字等。

3) 上下文切换：涉及内容：切换进程时，操作系统需要保存当前进程的状态（包括程序计数器、寄存器集、内存管理信息等）并加载新进程的状态。

4) 开销：进程切换涉及完整的硬件地址空间切换，包括页表、缓存清空等，因此开销较大。

(2)线程

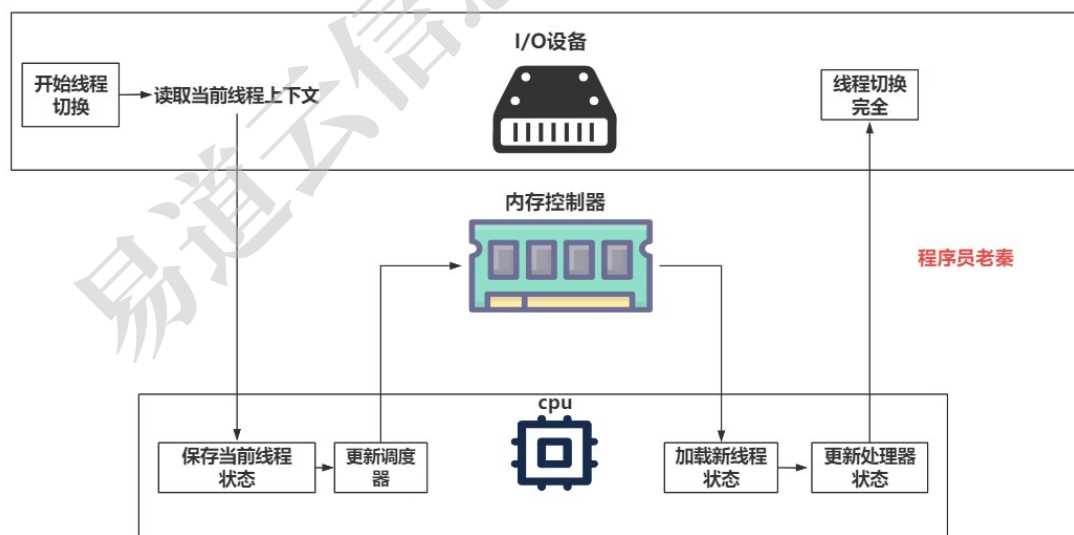
1) 定义：线程是进程中的实际运行单位，是系统进行运算调度的最小单位。它们共享其所属进程的地址空间和部分系统资源，但拥有独立的执行栈和程序计数器。

2) 资源共享：共享内容：线程共享进程的内存空间（代码段、数据段和打开的文件等），但各自持有独立的栈（用于存储执行历史）、寄存器状态和程序计数器。独立内容：线程栈、局部变量、寄存器和程序计数器等。

3) 上下文切换：涉及内容：切换线程时，主要保存和恢复寄存器状态、程序计数器和栈的指针。开销：相较于进程切换，线程切换的开销小，因为不需要切换内存地址空间。

3.线程独占什么资源？在切换时内核执行了哪些操作？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)线程独占的资源

线程作为进程的执行单元，虽然与其他线程共享大部分资源，但它们依然拥有一些独占资源，这些资源包括：

1) 线程栈 (Thread Stack) : 每个线程拥有自己的调用栈, 用于存储函数调用的局部变量、返回地址等。这是必需的, 因为每个线程可能在执行不同的任务或函数调用序列。

2) 程序计数器 (Program Counter, PC) : 程序计数器指示线程下一条执行指令的位置。每个线程必须独立维护自己的程序计数器, 以保证线程执行的正确性。

3) 线程特定数据 (Thread-Local Storage, TLS) : 线程可以拥有特定的数据, 即使在相同的进程中, 这些数据也不被其他线程共享。

4) 寄存器集 (Registers) : 每个线程都有自己的寄存器集, 这包括通用寄存器、索引寄存器、栈指针和状态寄存器等。这些寄存器在执行过程中保存线程特有的状态信息。

(2)线程切换时内核的行为

线程切换通常是由操作系统内核自动管理的, 当发生线程切换时, 内核需要执行以下操作来保证系统的稳定和线程的正确执行:

1) 保存线程状态: 内核首先保存当前线程的寄存器状态、程序计数器和其他关键信息到线程的控制块中 (如线程上下文)。这确保了线程可以在之后的某个时刻从相同的位置恢复执行。

2) 加载新线程状态: 内核从即将运行的线程的控制块中加载寄存器状态、程序计数器等信息, 准备这个线程的运行。

3) 更新调度结构: 内核可能需要更新内部的调度数据结构, 例如优先级队列或事件等待列表, 以反映当前活跃或等待的线程变化。

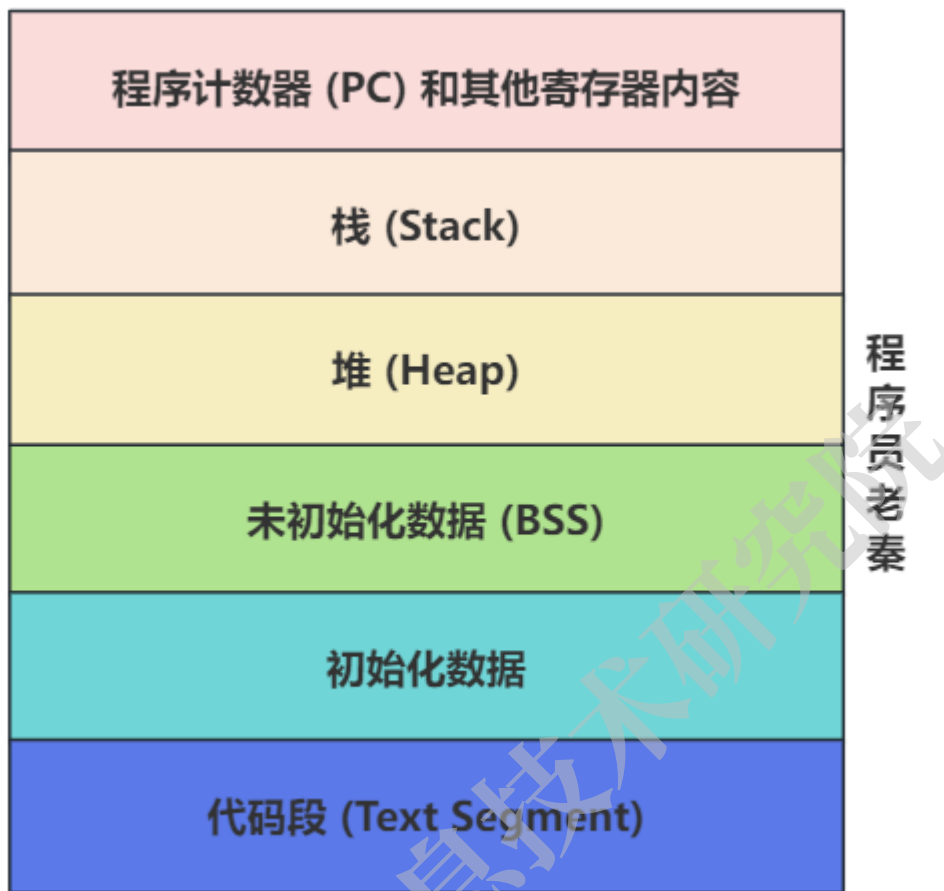
4) 栈切换: 线程的栈指针需要被更新, 以指向即将执行的线程的栈顶。

5) 处理器缓存: 为了效率, 有时候还需要清理或更新处理器缓存, 尤其是在多核处理器上, 以避免缓存一致性问题。

线程切换比进程切换开销小, 因为线程切换不涉及地址空间的变化, 但仍然涉及到显著的处理开销。因此, 在设计多线程程序时, 应尽量减少不必要的线程切换, 以优化性能。

4.进程的内存空间包括哪些部分?

图示 (方便记忆) :



总结（老秦推荐必看，方便理解）：

(1)程序计数器和其他寄存器：这部分通常不直接出现在内存布局图中，但对于理解程序如何控制流和管理状态是至关重要的。

(2)栈：位于内存布局的较高部分（**从高地址向低地址增长**），存放函数的局部变量、返回地址等。

(3)堆：位于栈和全局/静态变量之间，用于动态内存分配，其大小和位置在运行时可变。

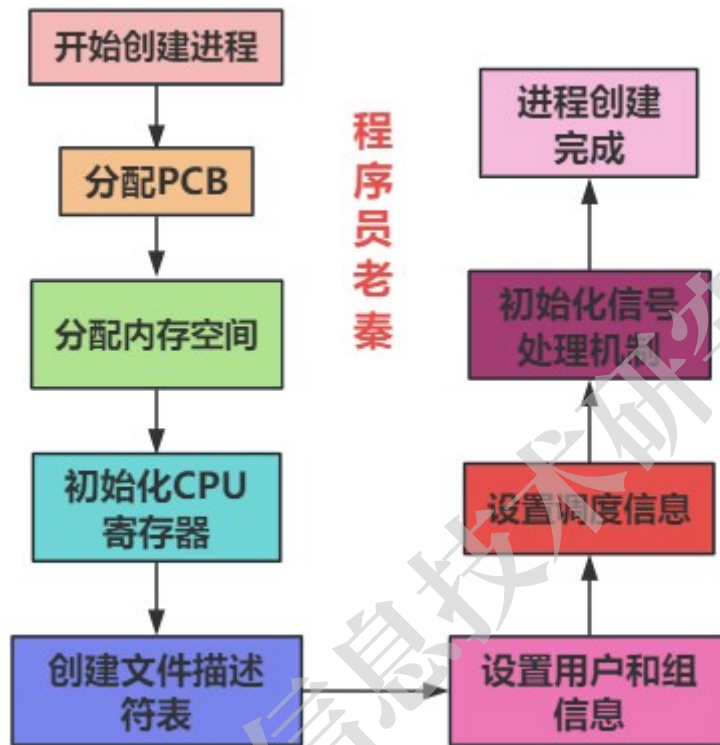
(4)未初始化数据（BSS）：存放**未初始化或初始化为零的全局变量和静态变量**，位于初始化数据段之下。

(5)初始化数据：包含**已初始化的全局变量和静态变量**。

(6)代码段：位于最底部，**存放程序的机器指令**，通常设置为只读以防止修改。

5.一个可执行文件如何变成进程？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)加载可执行文件

1) 启动过程：当你点击一个程序图标或在命令行中输入程序名称时，操作系统的 shell 或其他界面将用户的请求转化为启动程序的指令。

2) 读取文件：操作系统读取可执行文件的元数据（如 ELF 在 Linux，PE 在 Windows），这些元数据包括程序的入口点、所需的资源、依赖库等信息。

(2)创建进程控制块 (PCB)

1) 分配 PCB：操作系统为新进程创建一个进程控制块(PCB)，PCB 是操作系统用来维护进程状态、程序计数器、CPU 寄存器信息、内存管理信息和其他关键信息的数据结构。

(3)分配内存

1) 分配地址空间：操作系统为进程分配独立的虚拟地址空间。这通常包括为程序的代码段、数据段、堆和栈分配内存。

2) 设置内存保护：操作系统设置内存访问权限，例如代码段设置为只读。

(5)加载依赖库

1) 动态链接库：如果程序依赖于共享库（如 DLL 文件或 .so 文件），操作系统将这些库加载到内存中。如果这些库已经被加载，操作系统会重用已加载的库。

(6)初始化程序运行环境

1) 环境设置：操作系统设置程序运行的环境变量和输入输出设备。

2) 传递参数：命令行参数和环境变量被传递给进程。

(7)开始执行

1) 设置程序计数器：操作系统将程序计数器(PC)设置到可执行文件指定的入口点，开始执行程序的第一条指令。

(8)调度和执行

1) 调度进程：操作系统的调度器将进程加入到调度队列，根据调度策略（如轮询、优先级调度等），进程将被 CPU 执行。

6.什么时候该用多线程，什么时候该用多进程？

总结（老秦推荐必看，方便理解）：

(1)使用多线程的情况：

1) 共享内存和资源：当需要多个执行单元共享大量数据或状态时，线程是更好的选择。线程共享其所属进程的内存空间和资源，如文件句柄和数据结构等，这使得线程间的数据交换和通信更加高效。

2) 轻量级并发任务：线程的创建和销毁比进程更快，资源开销较小。如果应用程序需要频繁地创建和销毁执行单元，或者需要大量轻量级的并发操作，线程通常是更合适的选择。

3) 响应性要求高的应用程序：在需要快速响应用户输入或其他事件的应用程序中使用线程，可以通过并行执行提高应用程序的响应性。

(2)使用多进程的情况：

1) 隔离和安全性：进程提供了更好的隔离级别，每个进程拥有自己的内存空间和系统资源。这种隔离可以防止进程间的干扰，并提高系统的稳定性。在安全性和稳定性要求较高的应用中，如网银系统，多进程是更佳的选择。

2) 利用多核处理器：多进程可以更好地利用多核处理器的能力。操作系统可以将不同的进程调度到不同的 CPU 核心上，从而提高程序的执行效率和系统的整体性能。

3) 资源密集型应用：对于大量计算和资源需求很高的应用程序，使用多进程可以避免一个进程中的错误影响到其他进程，提高程序的健壮性。

(3)具体应用场景示例：

- 1) **多线程**：网络服务器和客户端应用程序，如 Web 服务器和现代 Web 浏览器。多媒体应用，如视频播放软件，需要在后台加载数据的同时保持用户界面的响应。
- 2) **多进程**：数据科学和大数据处理应用，每个进程执行不同的数据集分析。大型计算应用，如渲染软件，科学计算软件，它们需要大量计算资源而且稳定性要求极高。

7.僵尸进程、守护进程和孤儿进程的区别？

图示（方便记忆）：

类型	定义	特点	影响	示例/备注
僵尸进程	已终止但仍在进程表中占位置的进程。	不占用系统资源如内存和CPU，只保留在进程表中的一个位置。	耗尽进程表空间，可能阻止新进程的创建。需要父进程调用 wait() 来清理。	通常发生在父进程没有正确处理子进程终止状态的情况下。
守护进程	在后台运行的长生命周期进程，用于处理系统级任务。	没有控制终端，独立于用户交互操作，父进程通常是 init。	提供系统服务如日志记录、系统监控等，不直接与用户交互。	crond、syslogd，在Linux 系统启动时初始化并持续运行。
孤儿进程	父进程终止后仍在运行的子进程。	被 init 或其他系统级进程接管，继续运行直到完成或被显式终止。	由系统自动管理，通常不会导致资源泄露或性能问题。	如果一个服务进程在启动子任务后异常终止，子任务会成为孤儿进程。

总结（老秦推荐必看，方便理解）：

更多细节：

- (1) **僵尸进程**：僵尸进程通常是程序编写不当造成的，特别是在父进程未能正确管理子进程的终止。**这些进程已经完成执行但未被父进程回收，导致在系统的进程表中占据条目。**系统管理员通常需要定期检查并清理僵尸进程，以保持系统资源的有效利用。
- (2) **守护进程**：守护进程通常在系统引导时启动，以 root 权限或其他特定用户身份运行。**它们的设计目的是使其在没有用户登录时后台运行，执行如电子邮件服务、打印服务等任务。**守护进程的创建通常需要特定的编程技巧，如在 C 语言中通过 fork() 产生子进程然后结束父进程，使子进程成为 init 的子进程，这样就没有控制终端，可以在后台运行。
- (3) **孤儿进程**：**孤儿进程不会对系统性能造成负面影响，因为操作系统（如 Linux 的 init 进程）会自动接管这些进程并在适当的时候进行清理。**操作系统的 init 进程会周期性地调用 wait() 来回收任何已终止子进程的资源，防止它们变成僵尸进程。

8.什么情况适合用协程池，什么情况适合用线程池？什么情况使用进程池？

图示（方便记忆）：

区别表格			
特性	协程池	线程池	进程池
任务类型	高并发 I/O 密集型	CPU 或 I/O 中等规模任务	CPU 密集型、大任务隔离
资源消耗	内存占用少，切换成本低	独立栈空间，消耗中等	独立内存，消耗最高
多核利用	单线程，无法充分利用多核	多核并行	多核并行，隔离性强
任务隔离性	低	中	高
适用场景	网络服务器、爬虫	后台任务处理、中等并发	分布式计算、风险隔离任务

总结（老秦推荐必看，方便理解）：

(1) 协程池

适用场景：

1. 高并发 I/O 密集型任务：

- 网络通信、文件读取、数据库查询等任务中，线程可能因 I/O 操作被阻塞。协程切换成本低，适合应对大量并发连接。
- 示例：高性能 Web 服务器、异步 HTTP 客户端。

2. 需要大量短时间任务处理：

- 协程的上下文切换消耗极低，适合处理短时间、小粒度的任务。
- 示例：爬虫系统中抓取多个网页的任务。

3. 对资源消耗敏感的场景：

- 协程通过单线程运行多个任务，不需要为每个任务分配独立的栈空间，内存占用少。
- 示例：嵌入式系统的任务调度。

限制：

- 协程适合单线程运行，无法充分利用多核 CPU。
- 如果任务中存在长时间计算或阻塞操作，协程无法发挥优势。

(2) 线程池

适用场景：

1. CPU 密集型任务：

- 大量计算需要多核 CPU 并行处理。
- 示例：数据分析、图像处理。

2. 中小规模的 I/O 密集型任务：

- 线程池可以通过线程的阻塞等待处理 I/O 操作，适合中等规模的并发需求。
- 示例：文件处理服务、数据库操作。

3. 需要较多并行任务但又需控制线程数量：

- 线程池避免因频繁创建和销毁线程导致的系统开销。
- 示例：后台服务的定时任务处理。

限制：

- 线程池中每个线程有独立栈空间，资源消耗较协程大。
- 高并发任务中线程上下文切换开销较大。

(3) 进程池

适用场景：

1. 需要隔离任务的场景：

- 进程间内存完全隔离，适合需要高可靠性或运行高风险代码的任务。
- 示例：沙箱运行环境、分布式任务调度。

2. CPU 密集型任务且单任务计算量大：

- 每个进程运行在独立的 CPU 核心上，可以充分利用多核性能。
- 示例：机器学习模型训练、大数据计算。

3. 需要更高稳定性的场景：

- 任务崩溃不会影响其他进程，进程池可以单独回收出错进程。
- 示例：银行系统的独立任务处理。

限制：

- 进程间通信开销较大，不适合需要频繁数据交换的任务。

- 资源消耗高，启动和销毁进程的时间成本较大。

面试总结回答

协程池适合高并发 I/O 密集型任务，线程池适合中等规模并发任务或 CPU 密集型任务，进程池适合隔离性高、计算量大的任务。选择应根据任务特点、系统资源和扩展需求综合判断。

9.fork 与 system 调用的区别？

图示（方便记忆）：

特性	fork	system
基本功能	创建一个子进程，子进程是父进程的副本	执行指定的命令字符串，创建子进程运行外部命令
使用场景	多进程编程，手动管理子进程的执行	执行外部命令或脚本，简化命令执行过程
进程控制	提供细粒度的控制（如进程间通信、同步）	控制较少，主要用于简单的命令执行
资源继承	子进程继承父进程的地址空间、文件描述符等资源	子进程执行新的命令，资源继承取决于命令的执行方式
同步/异步执行	父进程和子进程可以并发执行，也可以通过 wait 同步	父进程等待子进程完成，类似于同步执行
典型应用	创建复杂的多进程应用，如服务器进程	运行简单的系统命令，如脚本执行或调用外部程序

总结（老秦推荐必看，方便理解）：

fork 和 system 是两个常用的系统调用，用于进程创建和管理，它们有不同的用途和行为：

fork：创建一个新的子进程，该子进程是调用进程的副本。子进程继承了父进程的地址空间、文件描述符等资源，但有独立的进程 ID。适用于多进程编程，需要手动管理进程间的通信和同步。

system：执行一个命令字符串，通常是一个外部程序。system 调用会创建一个子进程来运行指定的命令，父进程等待子进程完成。适用于需要在程序中执行外部命令或脚本的场景，简化了执行外部命令的过程，但缺乏对新进程的细粒度控制。

fork：用于创建子进程，是多进程编程的基础，提供了对子进程的精细控制。

system：用于执行系统命令，简单易用，但控制较少，适用于需要调用外部命令的场景。

10.CPU 是如何调度进程的？介绍几种调度算法。

总结（老秦推荐必看，方便理解）：

CPU 调度是操作系统根据某种策略选择下一个要运行的进程的过程。调度的基本单位可以是进程或线程，具体取决于操作系统的设计。CPU 调度的主要目标是提高系统的效率和资源利用率，提供公平的资源分配，并满足系统和用户的性能要求。

图示（方便记忆）：

调度算法	原理	优点	缺点	使用场景
先来先服务 (FCFS)	按照进程到达就绪队列的顺序进行调度。先到达的进程先得到 CPU 执行。	简单易于实现。没有复杂的优先级计算。	可能导致“饥饿”现象，平均等待时间较长。	适用于简单的批处理系统
短作业优先 (SJF)	优先调度执行时间最短的进程。分为非抢占式和抢占式 (SRTF)。	平均等待时间最短。	难以预知每个进程的执行时间，可能导致“饥饿”现象。	适用于可以准确预估作业时间的环境
轮转调度 (RR)	为每个进程分配一个固定的时间片，按顺序循环调度。	公平，所有进程都能得到执行。响应时间短，适用于交互式系统。	时间片设置过长或过短都会影响性能。	适用于交互式系统
优先级调度	为每个进程分配一个优先级，优先级高的进程先执行。	可以实现关键任务的优先处理。	可能导致低优先级进程“饥饿”。需要避免优先级反转。	适用于需要优先级区分的任务
多级反馈队列 (MLFQ)	将进程按优先级分成多个队列，每个队列有不同的调度策略。	结合多种调度算法的优点，灵活且适应性强。	实现复杂，需要合理设置队列数和调度策略。	适用于多任务混合型系统

11.进程如何实现访问隔离？

总结（老秦推荐必看，方便理解）：

(1)Linux 操作系统

1) 虚拟内存管理：Linux 使用硬件支持的虚拟内存，通过页表将每个进程的虚拟地址空间映射到物理内存。每个进程都有自己的页表，确保其不能访问到其他进程的内存空间。

2) 内核模式与用户模式：进程在用户模式下运行其代码，而系统资源的管理和访问控制则通过内核模式进行。系统调用是进程从用户模式切换到内核模式的桥梁。

3) Cgroups 和命名空间：Linux 提供了控制组 (Cgroups) 和命名空间技术，可以进一步隔离进程的资源使用（如 CPU、内存）和运行环境（如网络、文件系统）。

(2)Windows 操作系统

1) 虚拟内存管理：与 Linux 类似，Windows 为每个进程分配一个独立的虚拟地址

空间，通过硬件支持的页表来隔离不同进程的内存访问。

2) 访问令牌和安全描述符：Windows 使用访问令牌 (Access Tokens) 和安全描述符 (Security Descriptors) 来实施安全策略。每个进程或线程都有一个与之关联的访问令牌，其中包含了用户身份和权限信息，用于访问控制决策。

3) Integrity Levels：Windows 还引入了完整性级别 (Integrity Levels)，这是一个用于确定进程和其他对象之间交互权限的机制。例如，一个低完整性级别的进程不能写入一个高完整性级别的对象。

(3)实际应用

1) 多用户环境：在多用户操作系统中，例如在一个共享的服务器上，访问隔离确保了一个用户的活动不会影响到其他用户的进程，无论是数据安全还是系统稳定性。

2) 云计算平台：在云计算环境中，虚拟化技术（如 KVM、Xen、VMware）和容器化技术（如 Docker、Kubernetes）依赖于操作系统的隔离机制来确保不同客户的应用相互独立，保障数据隐私和应用安全。

12.进程可以忽视信号吗？

总结（老秦推荐必看，方便理解）：

(1)忽视信号的机制

进程可以通过设置信号处理函数来决定如何响应特定的信号。对于大多数信号，进程可以选择以下几种方式之一进行响应：

1) 默认行为：执行信号默认的操作，这通常包括终止进程、停止（暂停）进程、忽略信号等。

2) 捕获信号：指定一个函数来处理信号。当信号发生时，系统会调用这个函数。

3) 忽略信号：明确指示系统忽视该信号。对于可以被忽视的信号，这意味着当信号被递送时，系统不会对进程采取任何行动。

(2)可以忽视的信号

在 Unix 和类 Unix 系统中，大多数信号都可以被忽视，但有两个例外：

1) SIGKILL：此信号用于强制终止进程。进程不能捕获、忽视或更改此信号的行为。

2) SIGSTOP：此信号用于强制停止（暂停）进程的执行。与 SIGKILL 一样，它不能被捕获、忽视或更改。

(3)如何设置忽视信号

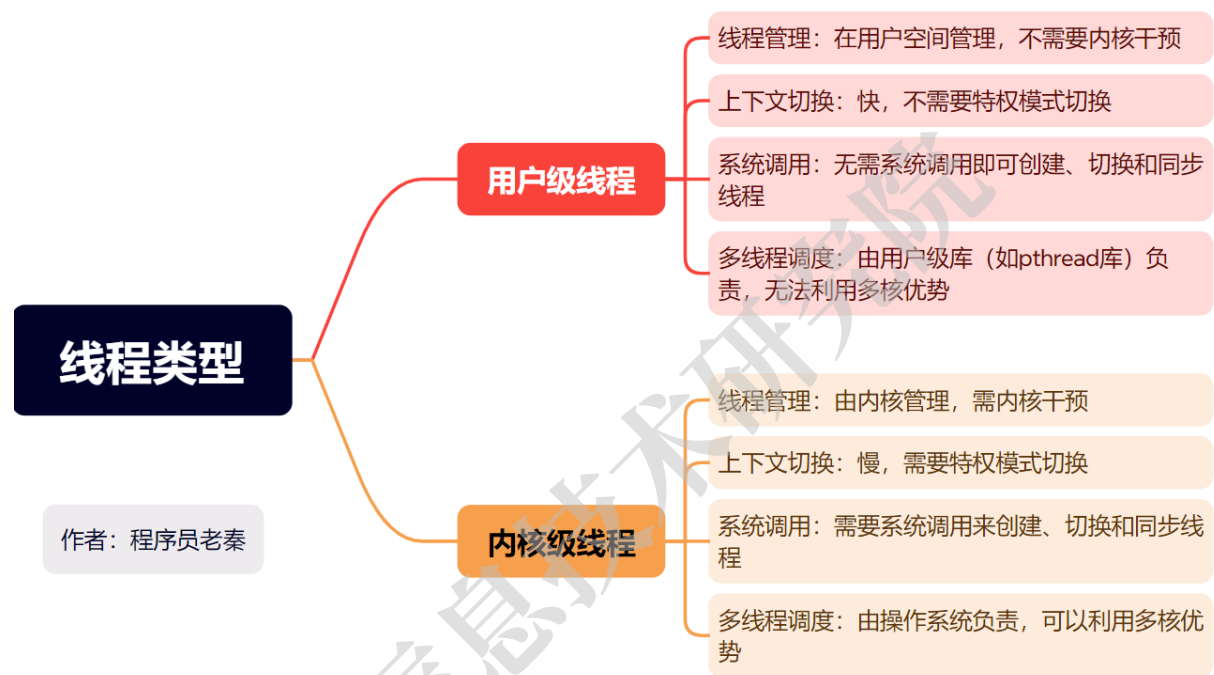
在 C 语言中，你可以使用 signal() 函数或更为复杂但功能更全面的 sigaction() 函数来设置信号处理方式。例如，忽视 SIGINT（通常由 Ctrl+C 触发）的一个简单示例：

```
#include <signal.h>
int main() {
    // 忽视 SIGINT 信号
    signal(SIGINT, SIG_IGN);
    // 无限循环，等待信号发生
```

```
while (1) {}  
return 0;  
}
```

13.用户级线程和内核级线程的区别？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)用户级线程 (User-Level Threads, ULT)

- 1) 线程管理：用户级线程的创建、管理和销毁都在用户空间进行，不需要内核的参与。这些线程通常由线程库（如 POSIX 线程库）管理。
- 2) 上下文切换：由于上下文切换在用户空间完成，不涉及内核态的切换，因此速度快，开销小。
- 3) 系统调用：用户级线程的操作无需系统调用，这使得线程操作更加高效，但在进行 I/O 操作时，整个进程会被阻塞。
- 4) 多线程调度：用户级线程由用户级库调度，无法利用多核处理器的优势，因为内核只看到单个进程，而不识别其内部的多个线程。

(2)内核级线程 (Kernel-Level Threads, KLT)

- 1) 线程管理：内核级线程的创建、管理和销毁由操作系统内核负责，需要通过系统调用进行。
- 2) 上下文切换：上下文切换需要从用户态切换到内核态，这使得切换速度较慢，开

销较大。

3) **系统调用**：内核级线程的操作需要系统调用，尽管开销较大，但可以利用操作系统的各种功能，如进程调度和 I/O 操作。

4) **多线程调度**：内核级线程由操作系统调度，可以充分利用多核处理器的优势，因为内核能够识别和管理每个线程。

14. 程序什么时候单线程效率高？

总结（老秦推荐必看，方便理解）：

(1) 任务不需要并行处理

如果任务本质上是顺序的，没有并行执行的需求或潜力，使用单线程可能更简单且高效。这类任务包括对数据进行线性处理，如读取文件然后处理数据，再写回文件。

(2) 避免上下文切换的开销

多线程程序涉及线程之间的上下文切换，尤其是在单核处理器上运行时，这会带来显著的性能开销。单线程程序没有这种额外负担，因此在资源受限的环境中可能表现得更好。

(3) 避免同步机制的开销

多线程程序需要同步机制（如互斥锁、信号量等）来管理对共享资源的访问，这不仅增加了编程的复杂性，还可能引入死锁和竞态条件等问题。这些同步操作本身就是一种开销，可能会降低程序的运行效率。单线程程序则无需考虑这些问题。

(4) I/O 密集型应用

在 I/O 密集型应用中，程序的性能瓶颈通常是磁盘 I/O 或网络 I/O，而不是 CPU。在这种情况下，使用多线程可能不会带来明显的性能提升，因为大部分时间都花在了等待 I/O 操作完成上。单线程模型在这种情况下简化了设计，且效率足够高。

(5) 内存使用优化

多线程程序每个线程都可能需要自己的堆栈等资源，这在内存使用上可能不如单线程程序高效。在内存资源受限的环境中，单线程程序能更好地利用有限的内存资源。

(6) 非共享资源的操作

如果程序中的任务完全独立，不需要共享任何资源，单线程执行可能更为直接和高效。这样可以避免多线程程序中资源锁定和同步的复杂性。

15. 进程可以创建多少线程，与什么有关？

总结（老秦推荐必看，方便理解）：

(1) 影响因素

系统资源限制：

1) **内存**：每个线程需要分配一定的栈空间，通常在几百 KB 到几 MB 之间。如果系统

内存有限，能够创建的线程数量也会受到限制。

2) 虚拟内存：即使有足够的物理内存，操作系统的虚拟内存地址空间也会对线程数量施加限制。

操作系统配置：

1) 线程限制：操作系统通常对每个进程的最大线程数量有默认的限制。可以通过系统配置文件或命令进行调整。例如，在 Linux 系统中，可以通过修改 /etc/security/limits.conf 文件或使用 ulimit 命令来改变线程限制。

2) 栈大小：可以通过编程语言或库的 API 调整每个线程的栈大小，从而间接影响可以创建的线程数量。例如，POSIX 线程（pthread）允许在创建线程时指定栈大小。

硬件资源：

3) CPU 核心数：多核处理器可以并行执行更多的线程，但线程数量不一定与核心数成正比。即使在单核系统上，也可以创建和管理多个线程，但它们需要时间分片。

4) 缓存和带宽：硬件资源的带宽和缓存大小也会影响线程调度和性能，从而间接影响可创建的线程数量。

(2)系统示例：Linux 中的线程限制

在 Linux 系统中，可以通过以下方法查看和调整线程限制：

```
/*
1.查看当前限制：
ulimit -u # 查看当前用户可以创建的最大进程（线程）数
2.修改限制：
ulimit -u 4096 # 将当前用户可以创建的最大进程（线程）数设置为 4096
3.永久修改：
编辑 / etc / security / limits.conf 文件，添加或修改以下行：
* soft nproc 4096
* hard nproc 4096
*/
```

16.外中断和异常有什么区别？

图示（方便记忆）：

特性	外中断 (External Interrupts)	异常 (Exceptions)
触发源	外部硬件设备	CPU 内部在执行指令时检测到的异常情况
触发时机	异步	同步
优先级和屏蔽	可以设置优先级，操作系统可以屏蔽某些中断	通常处理优先级较高，不能屏蔽
常见例子	键盘中断、定时器中断、网络中断	除零错误、页面错误、无效指令、系统调用
处理机制	通过中断向量表指向的中断处理程序处理	通过异常向量表指向的异常处理程序处理

总结（老秦推荐必看，方便理解）：

(1)外中断 (External Interrupts)

外中断是由外部硬件设备（如键盘、鼠标、网络卡、定时器等）触发的中断信号，目的是通知 CPU 有外部事件需要处理。

特点：

- 1) 异步性：外中断发生的时间是异步的，不依赖于 CPU 当前执行的指令。
- 2) 硬件触发：通常由硬件设备产生，通过中断控制器传递给 CPU。
- 3) 优先级和屏蔽：外中断可以有优先级，操作系统可以屏蔽某些中断以避免干扰关键操作。

常见外中断：

- 1) 键盘输入中断
- 2) 定时器中断
- 3) 网络包到达中断
- 4) 硬盘 I/O 完成中断

(2)异常 (Exceptions)

异常是由 CPU 在执行指令过程中检测到的异常情况（如非法指令、除零错误、内存访问违规等）引发的中断信号。

特点：

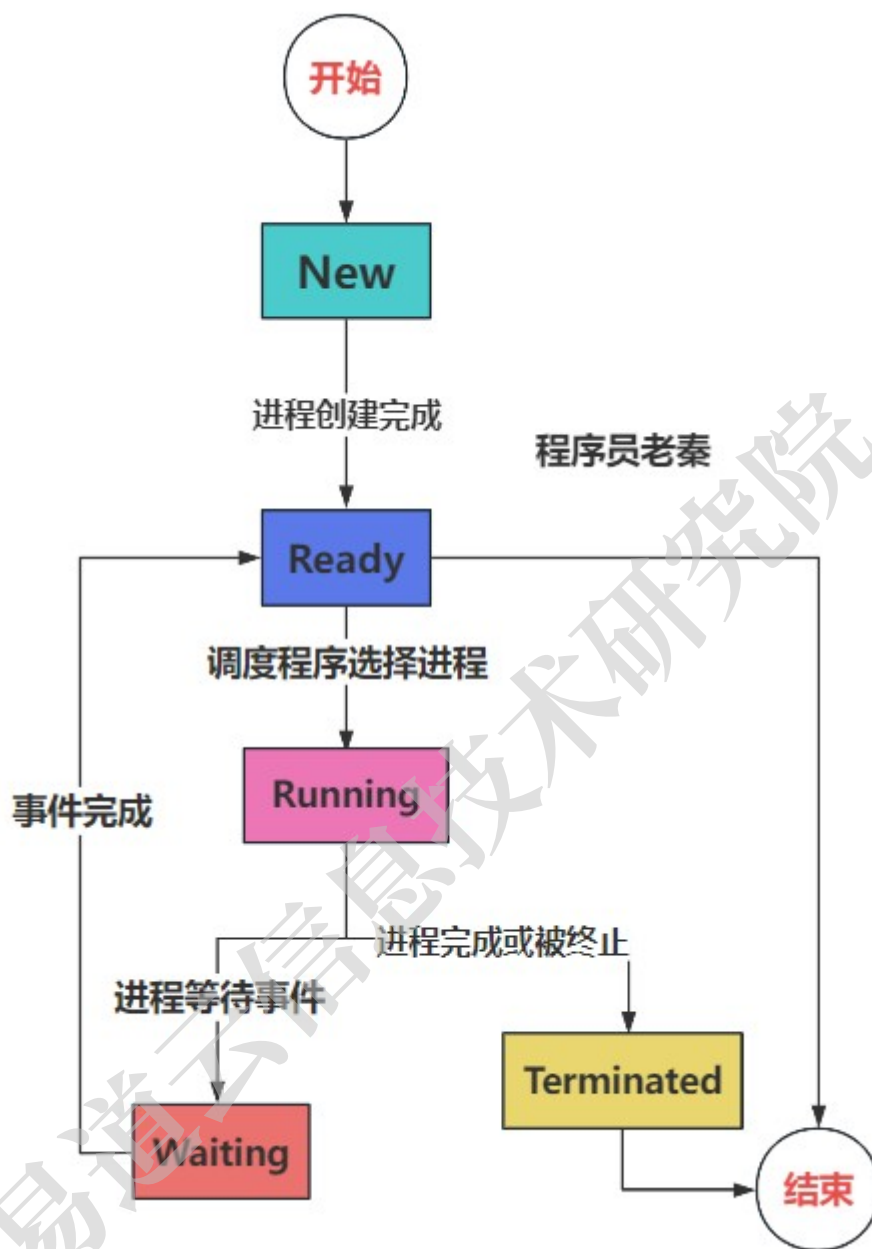
- 1) 同步性：异常发生的时间是同步的，直接与当前执行的指令相关。
- 2) 软件或硬件触发：由 CPU 在执行指令时检测到的异常情况触发。
- 3) 处理优先级：异常处理的优先级通常较高，因为它们可能指示程序错误或系统状态异常。

常见异常：

- 1) 除零错误
- 2) 页面错误 (Page Fault)
- 3) 无效指令错误
- 4) 系统调用 (通过软件中断指令触发)

17.进程的状态及其转换。

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

进程状态

- 1) 新建 (**New**)：进程正在被创建中，但尚未准备好运行。
- 2) 就绪 (**Ready**)：进程已准备好运行，等待 CPU 的分配。
- 3) 运行 (**Running**)：进程正在 CPU 上执行。
- 4) 等待 (**Waiting/Blocked**)：进程无法继续执行，等待某个事件（如 I/O 操作完成、信号到达等）。
- 5) 终止 (**Terminated**)：进程执行完成或因某种原因被终止。

18.进程的创建需要系统分配什么资源？

总结（老秦推荐必看，方便理解）：

(1)内存空间

1) 代码段：存储程序的可执行代码。每个进程都会分配一个代码段，用于存放程序指令。

2) 数据段：存储全局变量和静态变量。数据段在程序运行期间保持固定大小。

3) 堆区：用于动态内存分配。在程序运行期间，可以通过 malloc 或 new 动态分配和释放内存。

4) 栈区：用于存储局部变量和函数调用栈帧。每次函数调用都会在栈上分配新的栈帧。

(2)进程控制块 (PCB)

1) 进程状态：表示进程当前的状态，如运行、就绪、等待等。

2) 寄存器上下文：保存进程的 CPU 寄存器值，包括程序计数器、栈指针等，用于在上下文切换时恢复进程的执行状态。

3) 调度信息：包含进程优先级、调度队列等信息，用于操作系统调度进程。

(3)文件描述符

每个进程都有一个文件描述符表，存储进程打开的文件及其对应的文件描述符。

(4)进程号 (PID)

每个进程都有一个唯一的进程号 (PID)，用于标识进程。

(5)CPU 时间片

操作系统为每个进程分配 CPU 时间片，用于进程调度和执行。

(6)系统资源和权限

包括网络连接、设备访问权限等。操作系统在创建进程时会为其分配必要的系统资源和权限。

19.进程控制块 (PCB) 的作用是什么？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

- (1)唯一标识进程：通过进程标识符 (PID) 唯一标识每个进程。
- (2)存储进程状态：记录进程的当前状态，如运行、就绪、等待等。
- (3)保存寄存器上下文：保存进程的 CPU 寄存器值，如程序计数器、栈指针等，用于

上下文切换。

(4)管理内存分配：跟踪进程的内存使用情况，包括代码段、数据段、堆、栈等。

(5)文件描述符表：记录进程打开的文件及其对应的文件描述符。

(6)进程调度信息：包含进程的优先级、时间片等调度相关信息。

(7)进程间通信：管理进程间通信的相关信息，如管道、消息队列、共享内存等。

二．进程和线程管理

20.进程之间的通信方式有哪些？

总结（老秦推荐必看，方便理解）：

IPC 方式	原理描述	实现方式
管道 (Pipes)	单向数据流，基于内核缓冲区。	系统调用 pipe()创建，返回两个文件描述符用于读写。
信号 (Signals)	软件中断，用于通知事件。	操作系统处理信号分派，进程可以定义信号处理函数。
消息队列	独立于发送接收进程的消息存储。	使用系统调用如 msgget(), msgsnd(), msgrcv()创建和操作。
共享内存	允许多个进程访问同一内存区域。	shmget()创建内存段，shmat()进行内存映射。
信号量	同步工具，控制对共享资源的访问。	通过 semget()创建，semop()进行操作。
套接字 (Sockets)	网络通信的端点，支持不同主机间的数据交换。	使用 socket(), bind(), listen(), accept(), connect()等 API。
内存映射文件	使用文件映射实现内存共享。	通过 mmap()将文件映射至进程的地址空间。

21.线程之间的通信方式有哪些？

总结（老秦推荐必看，方便理解）：

通信方式	原理描述	实现方式
共享变量	线程访问共享内存中的变量。	直接访问，但需使用锁（如互斥锁）来同步访问。
条件变量	允许线程在条件未满足时等待，直到条件满足。	与互斥锁一起使用，使用 pthread_cond_wait()等函数。
读写锁	支持多读单写的同步机制。	通过 pthread_rwlock API 实现。
信号量	计数器，控制多线程对共享资源的访问。	使用 sem_init(), sem_wait(), sem_post()等函数。
屏障	同步一组线程的进度。	使用 pthread_barrier_init(),

		pthread_barrier_wait()等函数。
消息传递	线程通过消息交换数据。	可使用消息队列或其他高级同步机制如条件变量。

22.线程之间的数据竞争和数据同步？

总结（老秦推荐必看，方便理解）：

(1)数据竞争 (Race Condition)

数据竞争发生在两个或多个线程在没有适当的同步机制的情况下同时访问同一数据资源，并且至少有一个线程在进行写操作时。当多个线程以不可预测的顺序访问数据时，可能会导致程序的输出依赖于线程执行的具体时序，从而产生错误或不一致的结果。

示例：考虑两个线程同时更新同一个全局变量的简单示例。如果没有适当的同步，这两个线程可能会读取到变量的旧值，基于这个值做增加操作，然后将结果写回同一个变量，这导致某些更新丢失。

```
int counter = 0;
void increment() {
    int temp = counter; // 读取旧值
    temp = temp + 1;    // 增加
    counter = temp;     // 写回新值
}
```

如果两个线程几乎同时执行 increment()函数，它们可能都读取到 counter 为 0 的旧值，然后都将其增加到 1，并设置回 counter。最终结果是 counter 变为 1 而不是正确的 2。

(2)数据同步

数据同步是指在多线程环境中，通过各种机制确保对共享数据的访问以一种可控和一致的方式进行。数据同步的目的是防止数据竞争，确保数据的完整性和一致性。

同步机制：

1) 互斥锁 (Mutex)：互斥锁保证同一时间只有一个线程可以执行某段代码，以此来访问或修改共享数据。适用于保护关键区域，确保一次只有一个线程可以进入修改共享变量。

2) 读写锁 (Read-Write Locks)：允许多个读者同时访问共享数据，但写入者会独占访问权。适用于读多写少的情况，可以提高读取效率。

3) 条件变量 (Condition Variables)：用于在某些条件下阻塞一个或多个线程，直到收到另一线程的通知。常用于生产者-消费者问题。

4) 信号量 (Semaphores)：包含一个计数器，用于控制同时访问共享资源的线程数。适合控制有限资源的访问。

5) 原子操作 (Atomic Operations)：不可分割的操作单元，确保在读取、修改和更新值的过程中不会被其他线程中断。用于实现无锁的数据结构。

23.异步通信与同步通信的区别是什么？

图示（方便记忆）：

特性	同步通信	异步通信
阻塞性	阻塞调用，发送方等待响应以继续执行	非阻塞调用，发送方无需等待可继续执行
复杂性	相对简单，易于理解和实现	较高，需要处理回调和状态管理
并发性	低，发送方在等待期间不能执行其他任务	高，发送方可以同时进行其他任务
响应性	受限于响应时间，可能导致延迟	高响应性，减少等待时间
代码维护	通常更简单，逻辑直接清晰	复杂，可能引入回调地狱等问题
实时反馈	立即得到处理结果	不需要立即反馈，处理结果可以延后
适用场景	需要即时反馈的操作，如用户交互	并发应用，I/O 密集型任务

总结（老秦推荐必看，方便理解）：

(1)同步通信

在同步通信模式中，发送方发出通信请求后必须等待接收方的响应，才能继续执行后续操作。换句话说，调用操作是阻塞的，这意味着调用者在等待响应期间不能执行其他任务。

特点：

- 1) 阻塞操作：发送方在发送请求后，必须等待直到接收方处理完消息并响应。
- 2) 确定性：发送方总是知道何时接收方处理完毕，因为其必须接收到响应才能继续执行。
- 3) 易于理解和实现：同步通信模式逻辑简单直接，易于实现和理解。

适用场景：

对实时性要求较高的环境，如用户交互或需要即时反馈的操作。
简单的客户端-服务器交互，如 HTTP 请求（尽管 HTTP/2 支持异步操作）。

(2)异步通信

在异步通信模式中，发送方发出通信请求后不需要立即得到响应，它可以继续执行其他操作。接收方处理完消息后，通过状态、通知或回调等机制来通知发送方。

特点：

- 1) 非阻塞操作：发送方发送请求后可以继续进行其他任务，不需等待响应。
- 2) 并发性：提高了系统的并发性，因为发送方和接收方可以同时进行操作。
- 3) 复杂的控制流：管理异步操作可能导致代码更复杂，需要处理诸如回调地狱等问题。

24.进程同步方式有哪些？

总结（老秦推荐必看，方便理解）：

(1)互斥锁 (Mutex)

互斥锁是一种提供互斥访问共享资源的同步机制。当一个进程或线程需要访问共享资源时，它必须首先获取互斥锁。如果锁已被另一个进程持有，请求锁的进程将被阻塞，直到锁被释放。

1) **用途**：用于保护对共享数据的访问，确保在任意时刻只有一个进程可以操作共享资源。

2) **优点**：简单且有效地保护资源不发生冲突。

3) **缺点**：可能造成死锁，如果不正确管理锁的获取和释放。

(2)信号量 (Semaphores)

1) **工作原理**：信号量是一种更高级的同步机制，包含一个计数器，表示可用资源的数量。进程通过增加（释放资源）或减少（请求资源）信号量的值来进行同步，如果信号量的值为零，则进程阻塞，直到信号量值大于零。

2) **用途**：广泛用于控制有限数量的资源访问，如限制访问数据库的连接数。

3) **优点**：灵活，可以用于多种同步问题，包括互斥作为特殊情况（二进制信号量）。

4) **缺点**：如果不正确使用，可能导致死锁或饥饿。

(3)条件变量 (Condition Variables)

1) **工作原理**：条件变量用于阻塞一个或多个进程，直到满足某个特定条件。通常与互斥锁一起使用，以防止条件检查和条件发生之间的竞态条件。

2) **用途**：适用于生产者-消费者问题，其中消费者等待生产者生成数据。

3) **优点**：提供了一种等待特定条件的有效方式，而不是轮询。

4) **缺点**：使用复杂，需要小心编码以避免死锁和竞态条件。

(4)读写锁 (Read-Write Locks)

1) **工作原理**：读写锁允许并发的读取操作，但写入操作是互斥的。这意味着多个读操作可以同时进行，但写操作需要独占访问。

2) **用途**：适合读操作远多于写操作的情况，如缓存系统。

3) **优点**：提高了读操作的并发性，降低了写操作对读操作的阻塞。

4) **缺点**：管理复杂，写操作可能会由于频繁的读操作而延迟。

(5)屏障 (Barriers)

1) **工作原理**：屏障是同步点，所有进程必须到达屏障后才能继续执行。这确保所有进程在继续执行前达到一定的执行点。

2) **用途**：用于并行计算，确保所有并行处理的阶段同步完成。

3) **优点**：确保所有进程同步前进，无一个落后。

4) **缺点**：整体性能受到最慢参与者的影响。

25.生产者消费者模型是如何工作的？

总结（老秦推荐必看，方便理解）：

基本概念

互斥锁 (Mutex)：用于保证多个线程不会同时访问共享资源。

条件变量 (Condition Variable) : 用于在**某些条件尚未满足时阻塞线程**，直到另一个线程修改了条件并通知条件变量。

代码示例 (老秦建议：自己动手实践一下)

```
#include <iostream>
#include <thread>
#include <vector>
#include <mutex>
#include <condition_variable>
const int BUFFER_SIZE = 10;
std::vector<int> buffer;
std::mutex mtx;
std::condition_variable cond_var;
bool done = false; // 用于通知消费者结束
bool ready = false; // 用于通知消费者开始消费
void producer(int n) {
    for (int i = 0; i < n; ++i) {
        std::unique_lock<std::mutex> locker(mtx);
        cond_var.wait(locker, []() { return buffer.size() < BUFFER_SIZE; }); // 等待空间
        可用
        buffer.push_back(i);
        std::cout << "Produced: " << i << std::endl;
        ready = true;
        locker.unlock();
        cond_var.notify_one(); // 通知消费者
    }
    done = true;
    cond_var.notify_one(); // 通知消费者结束
}
void consumer() {
    while (!done) {
        std::unique_lock<std::mutex> locker(mtx);
        cond_var.wait(locker, []() { return ready || done; }); // 等待生产者生产数据或结
        束
        while (!buffer.empty()) {
            int val = buffer.back();
            buffer.pop_back();
            std::cout << "Consumed: " << val << std::endl;
        }
        ready = false;
        locker.unlock();
        cond_var.notify_one(); // 通知生产者缓冲区已空
    }
}
```



```
}
int main() {
    std::thread producer_thread(producer, 50);
    std::thread consumer_thread(consumer);
    producer_thread.join();
    consumer_thread.join();
    return 0;
}
```

26.信号量和自旋锁的区别是什么？

图示（方便记忆）：

特性	信号量	自旋锁
设计	用于控制有限数量的资源访问。	用于实现快速的互斥访问。
等待机制	线程在无法获取资源时进入睡眠状态，避免 CPU 资源浪费。	线程在等待锁释放时持续占用 CPU（忙等），消耗 CPU 资源。
使用场景	适用于资源数量有限的情况，以及需要复杂的同步逻辑。	适用于预期锁只会被短时间持有的情况。
性能开销	上下文切换（线程或进程阻塞和唤醒）。	CPU 时间（在获取锁之前线程一直占用 CPU）。
适用问题	生产者-消费者问题，读者-写者问题等。	适用于线程只需要短时间访问临界区的情况。

总结（老秦推荐必看，方便理解）：

(1)信号量 (Semaphores)

- 1) 定义与原理：
信号量是一个同步对象，用来控制对共享资源的访问。**信号量的值表示可用资源的数量。如果信号量的值大于零，表示可以访问资源；如果等于零，则表示资源已经被完全使用，后续的线程或进程需要等待。**
- 2) 类型：
二元信号量 (Binary Semaphore)：又称为互斥锁，其值只能是 0 或 1，用于实现互斥访问。
计数信号量 (Counting Semaphore)：可以取大于 1 的值，用于控制多个资源的访问。
- 3) 主要用途：
控制对一个或多个资源的访问。
用于实现生产者-消费者问题、读者-写者问题等复杂的同步问题。
- 4) 优缺点：
优点：灵活，可以控制多个相同的资源。
缺点：可能导致优先级反转，需要正确地初始化和使用，避免死锁。

(2)自旋锁 (Spinlocks)

1) 定义与原理：

自旋锁是一种忙等待锁，当线程尝试获取锁而锁已被其他线程持有时，**线程将在一个循环中不断检查锁的状态，这称为“自旋”**。

2) 主要用途：

通常用在等待时间非常短的场景中，因为自旋锁不会使线程进入睡眠状态，避免了上下文切换的开销。

3) 优缺点：

优点：对于持锁时间极短的情况效率非常高，因为它避免了线程睡眠和唤醒的开销。

缺点：如果持锁时间较长，会浪费大量的 CPU 资源，因为等待锁的线程仍在消耗 CPU 执行空循环。

27. 多线程中的栈之间如何传递数据？

总结（老秦推荐必看，方便理解）：

(1) 使用全局变量或共享内存

共享内存是最直接的方式，但需要注意同步问题，以避免数据竞争和不一致。

(2) 使用条件变量

条件变量用于线程间的同步和通信，允许一个线程等待某个条件满足后再继续执行。

(3) 使用线程安全队列

使用线程安全的数据结构（如队列）可以简化多线程之间的数据传递和管理。

(4) 使用消息传递机制

使用消息传递机制（如消息队列、管道等）在线程间传递数据，适用于复杂的多线程通信。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <thread>
#include <queue>
#include <mutex>
#include <condition_variable>
std::queue<int> dataQueue;
std::mutex mtx;
std::condition_variable cv;
void producer() {
    for (int i = 0; i < 10; ++i) {
        std::lock_guard<std::mutex> lock(mtx);
        dataQueue.push(i);
        cv.notify_one();
    }
}
void consumer() {
```

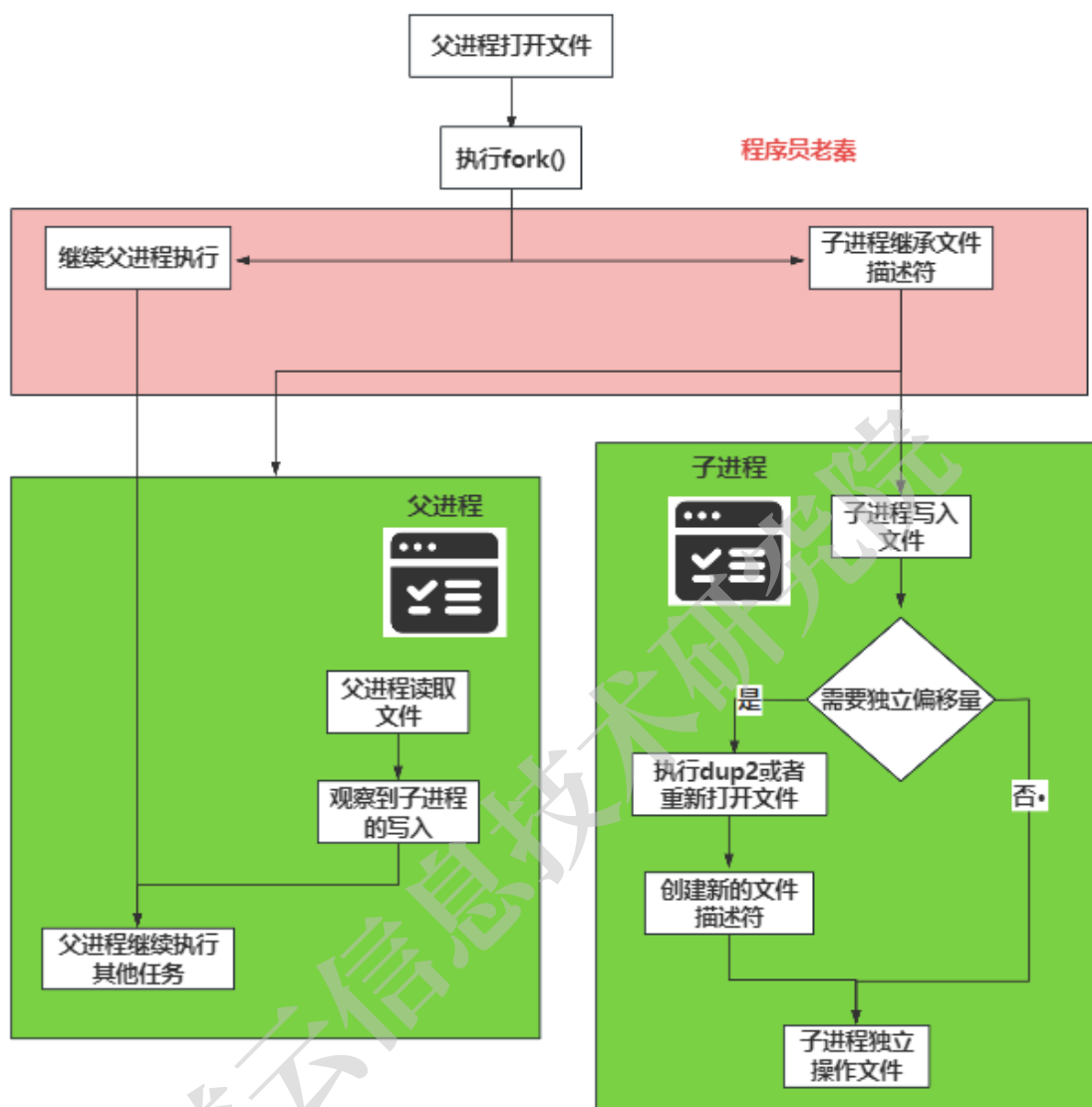
```

while (true) {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, [] { return !dataQueue.empty(); });
    int data = dataQueue.front();
    dataQueue.pop();
    lock.unlock();
    std::cout << "Consumer processed data: " << data << std::endl;
    if (data == 9) break;
}
}
int main() {
    std::thread t1(producer);
    std::thread t2(consumer);
    t1.join();
    t2.join();
    return 0;
}

```

28.Linux 文件标识符相关问题：父进程 fork 子进程后，子进程能否修改父进程的文件？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)文件描述符与 fork()系统调用

当一个父进程执行 fork()系统调用创建子进程时，子进程会继承父进程的文件描述符。这意味着父进程和子进程将共享相同的文件描述符集合。但是，虽然文件描述符的数字是相同的，父进程和子进程中的这些文件描述符都指向同一个打开文件表的条目。

(2)共享文件描述符的影响

由于子进程从父进程继承了文件描述符，这意味着它们实际上共享了对打开文件的访问。具体来说，子进程可以通过继承的文件描述符对文件进行读写操作。这些操作会影响到文件的当前偏移量，因为文件偏移量是在打开文件表条目中维护的，而不是在文件描述符或进程中。

(3)子进程对文件的修改

回答你的问题，子进程可以使用从父进程继承的文件描述符来修改文件。这些修改会影响到父进程打开的相同文件，因为它们共享同一文件偏移量。例如，如果子进程写入了文件，父进程读取同一个文件时也会看到这些变化。

(4)独立文件偏移量的处理

如果需要父进程和子进程独立地操作文件（即不共享文件偏移量），可以在 `fork()` 后立即对共享的文件描述符执行 `dup2()` 或重新打开文件。这将为每个进程创建单独的打开文件表条目，使得它们可以独立地管理文件偏移量。

29.管道会溢出吗？如何处理？

总结（老秦推荐必看，方便理解）：

(1)管道的工作原理

管道是基于内核缓冲区的，这意味着它们在内核中有一个有限的缓冲区来存储在读取之前发送的数据。对于匿名管道，通常用在具有亲缘关系的进程之间（如父子进程），而命名管道则可以在任何两个进程之间建立通信，即使它们没有直接的亲缘关系。

(2)管道溢出的可能性

管道是否会溢出取决于管道缓冲区的处理方式和管道的使用情况。管道的缓冲区大小在不同的系统和配置中可能有所不同，但一般来说，大小是有限的。

写入管道：当一个进程向管道写入数据时，数据被存储在内核缓冲区中。如果缓冲区已满，尝试写入的进程将会阻塞，直到有足够的空间可用来存放新数据。这意味着正常情况下，写进程不会继续执行，直到缓冲区中有足够的空间来存放它试图写入的数据。

读取管道：当另一个进程从管道读取数据时，它从缓冲区中取出数据。如果缓冲区为空，读进程将会阻塞，直到有新的数据写入管道。

30.操作系统最小单位是线程，那协程怎么实现？

总结（老秦推荐必看，方便理解）：

虽然线程是操作系统调度的最小单位，但协程（coroutine）提供了一种更轻量级的并发模型。协程的实现主要在用户态完成，不需要操作系统内核的直接支持。以下是协程的实现原理和一些代码示例，展示如何在 C++ 中实现和使用协程。

(1)协程的实现原理

1) 用户态调度：协程的调度完全在用户态进行，不依赖操作系统内核。用户代码显式地控制协程的切换。

2) 保存和恢复上下文：协程在切换时需要保存当前的执行上下文（如寄存器、栈指针）并在恢复时恢复这些上下文。

3) 栈管理：每个协程有自己的栈，存储协程的局部变量和函数调用信息。协程的栈通常比线程的栈更小。

(2)协程的实现方式

1) 使用语言支持的协程：许多现代编程语言提供了对协程的内置支持，例如 Python 的 `asyncio`，C++20 的协程，Go 语言的 `goroutine` 等。

2) 手动实现协程：可以通过手动保存和恢复上下文来实现协程，通常使用 `setjmp/longjmp`（C 语言）或汇编语言进行上下文切换。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <coroutine>
struct Task {
    struct promise_type {
        Task get_return_object() { return {}; }
        std::suspend_never initial_suspend() { return {}; }
        std::suspend_always final_suspend() noexcept { return {}; }
        void return_void() {}
        void unhandled_exception() { std::terminate(); }
    };
};
Task coroutine() {
    std::cout << "Coroutine started\n";
    co_await std::suspend_always{};
    std::cout << "Coroutine resumed\n";
}
int main() {
    auto handle = coroutine();
    std::cout << "Main function\n";
    handle.resume();
    return 0;
}
```

31.如何设计和管理多线程中的多协程？

总结（老秦推荐必看，方便理解）：

设计多线程中的多协程

- 1) **线程池管理**：使用线程池来管理和复用线程，避免频繁创建和销毁线程带来的开销。
- 2) **协程调度器**：每个线程运行一个协程调度器，负责在该线程上调度和执行协程。协程调度器可以基于事件驱动或时间片轮转的方式调度协程。
- 3) **任务队列**：使用任务队列来存储待执行的协程任务。线程从任务队列中获取任务并执行相应的协程。
- 4) **协程同步和通信**：使用协程安全的同步机制（如信号量、通道）来管理协程之间的同步和通信。避免使用线程级的锁机制，以防止阻塞线程，影响协程的调度。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <coroutine>
#include <vector>
#include <thread>
```

```

#include <queue>
#include <mutex>
#include <condition_variable>
#include <functional>
// 任务包装器，用于存储协程任务
struct Task {
    struct promise_type;
    using handle_type = std::coroutine_handle<promise_type>;
    handle_type coro;
    Task(handle_type h) : coro(h) {}
    ~Task() {
        if (coro) coro.destroy();
    }
    struct promise_type {
        auto get_return_object() { return
Task{ handle_type::from_promise(*this) }; }
        auto initial_suspend() { return std::suspend_always{}; }
        auto final_suspend() noexcept { return std::suspend_always{}; }
        void return_void() {}
        void unhandled_exception() { std::terminate(); }
    };
};

// 线程池类，管理一组线程并调度任务
class ThreadPool {
public:
    ThreadPool(size_t numThreads); // 构造函数，创建指定数量的线程
    ~ThreadPool(); // 析构函数，销毁所有线程
    void enqueue(Task::handle_type task); // 向任务队列中添加一个任务
private:
    std::vector<std::thread> workers; // 工作线程集合
    std::queue<Task::handle_type> tasks; // 任务队列
    std::mutex queueMutex; // 保护任务队列的互斥锁
    std::condition_variable condition; // 用于任务调度的条件变量
    bool stop; // 标志线程池是否停止
    void worker(); // 工作线程的执行函数
};

ThreadPool::ThreadPool(size_t numThreads) : stop(false) {
    for (size_t i = 0; i < numThreads; ++i) {
        workers.emplace_back(&ThreadPool::worker, this);
    }
}

ThreadPool::~ThreadPool() {
    {

```

```

        std::unique_lock<std::mutex> lock(queueMutex);
        stop = true;
    }
    condition.notify_all();
    for (std::thread& worker : workers) {
        worker.join();
    }
}

void ThreadPool::enqueue(Task::handle_type task) {
    {
        std::unique_lock<std::mutex> lock(queueMutex);
        tasks.push(task);
    }
    condition.notify_one();
}

void ThreadPool::worker() {
    while (true) {
        Task::handle_type task;
        {
            std::unique_lock<std::mutex> lock(queueMutex);
            condition.wait(lock, [this] { return stop || !tasks.empty(); });
            if (stop && tasks.empty()) return;
            task = tasks.front();
            tasks.pop();
        }
        task.resume();
    }
}

// 协程任务函数
Task coroutineTask(ThreadPool& pool) {
    std::cout << "Coroutine started in thread " << std::this_thread::get_id()
    << std::endl;
    co_await std::suspend_always{}; // 暂停协程
    std::cout << "Coroutine resumed in thread " << std::this_thread::get_id()
    << std::endl;
}

int main() {
    ThreadPool pool(4); // 创建一个包含 4 个线程的线程池
    // 向线程池中添加 8 个协程任务
    for (int i = 0; i < 8; ++i) {
        pool.enqueue(coroutineTask(pool).coro);
    }
    // 主线程休眠一段时间，以确保所有协程任务完成
}

```

```
std::this_thread::sleep_for(std::chrono::seconds(1));  
return 0;  
}
```

详细说明

任务包装器 (Task) :

Task 结构包含了协程句柄，用于管理协程的生命周期。

promise_type 内部类定义了协程的挂起和恢复行为。

线程池 (ThreadPool) :

ThreadPool 类管理一组工作线程和任务队列。

enqueue 方法将任务添加到队列中，并通知一个工作线程。

worker 方法是工作线程的主循环，等待任务并执行它们。

协程任务 (coroutineTask) :

coroutineTask 是一个协程函数，展示了协程的启动和暂停。

co_await std::suspend_always{} 用于暂停协程的执行。

主函数 (main) :

在 main 函数中，创建一个包含 4 个线程的线程池。

向线程池中添加 8 个协程任务，并让主线程休眠一段时间以确保所有任务完成。

32. 不同进程间是否可以共享内存吗？包括父子进程。

总结（老秦推荐必看，方便理解）：

不同进程间可以共享内存，这在多进程应用程序中是一种常见的通信方式。共享内存允许多个进程访问同一块物理内存区域，是一种非常快速的数据交换方法。在 Unix-like 系统中，可以使用 shmget, shmat, shmctl 等系统调用来实现共享内存。在 Windows 系统中，则可以使用 CreateFileMapping 和 MapViewOfFile 函数。

代码示例（老秦建议：自己动手实践一下）

(1) Unix-like 系统示例代码

```
#include <sys/ipc.h>  
#include <sys/shm.h>  
#include <stdio.h>  
int main() {  
    key_t key = ftok("shmfile", 65); // 生成唯一键  
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT); // 创建共享内存  
    char* str = (char*)shmat(shmid, (void*)0, 0); // 附加到共享内存  
    printf("Data read from memory: %s\n", str); // 读取数据  
    // 分离共享内存  
    shmdt(str);  
    // 销毁共享内存  
    shmctl(shmid, IPC_RMID, NULL);  
}
```



```
    return 0;
}
```

(2) Windows 系统示例代码

```
#include <windows.h>
#include <iostream>
int main() {
    HANDLE hMapFile;
    LPCTSTR pBuf;
    hMapFile = CreateFileMapping(
        INVALID_HANDLE_VALUE, // 使用系统分页文件
        NULL,                 // 默认安全性
        PAGE_READWRITE,       // 可读写访问
        0,                     // 最大对象大小
        256,                   // 缓冲区大小
        TEXT("Local\\MyFileMappingObject")); // 名称
    pBuf = (LPTSTR)MapViewOfFile(
        hMapFile,              // 映射对象句柄
        FILE_MAP_ALL_ACCESS,   // 可读写访问
        0,
        0,
        256);
    if (pBuf != NULL) {
        std::cout << "Data read from memory: " << pBuf << std::endl;
        UnmapViewOfFile(pBuf);
    }
    CloseHandle(hMapFile);
    return 0;
}
```

33. 如何使用互斥锁实现读写锁，特别是写者优先策略？

总结（老秦推荐必看，方便理解）：

写者优先的读写锁（**Reader-Writer lock**）可以确保写操作不会因频繁的读操作而饿死。在多线程编程中，这种策略有助于平衡读者和写者之间的公平性，特别是在写操作较少但需要高优先级处理时。

代码示例（老秦建议：自己动手实践一下）

```
#include <mutex>
#include <condition_variable>
#include <iostream>
#include <thread>
```

```

class ReadWriteLock {
    std::mutex mtx; // 互斥锁，用于同步对以下变量的访问
    std::condition_variable reader_ok; // 条件变量，让读者等待直到可以安全地读取
    std::condition_variable writer_ok; // 条件变量，让写者等待直到可以安全地写入
    int active_readers = 0; // 当前活跃的读者数量
    int waiting_writers = 0; // 正在等待的写者数量
    int active_writers = 0; // 当前活跃的写者数量 (0 或 1)
public:
    // 获取读锁
    void read_lock() {
        std::unique_lock<std::mutex> lock(mtx); // 锁定互斥锁
        // 等待，直到没有写者活跃或等待
        while (waiting_writers > 0 || active_writers > 0) {
            reader_ok.wait(lock);
        }
        active_readers++; // 增加活跃读者的数量
    }
    // 释放读锁
    void read_unlock() {
        std::unique_lock<std::mutex> lock(mtx); // 再次锁定互斥锁
        active_readers--; // 减少活跃读者的数量
        // 如果没有活跃的读者并且有等待的写者，通知一个写者
        if (active_readers == 0 && waiting_writers > 0) {
            writer_ok.notify_one();
        }
    }
    // 获取写锁
    void write_lock() {
        std::unique_lock<std::mutex> lock(mtx);
        waiting_writers++; // 增加等待写者的数量
        // 等待，直到没有活跃的读者或写者
        while (active_readers > 0 || active_writers > 0) {
            writer_ok.wait(lock);
        }
        waiting_writers--; // 开始写操作后减少等待写者的数量
        active_writers = 1; // 设置当前有一个活跃的写者
    }
    // 释放写锁
    void write_unlock() {
        std::unique_lock<std::mutex> lock(mtx);
        active_writers = 0; // 重置活跃写者的数量
        // 通知所有等待的读者和一个等待的写者
    }
}

```

```

        reader_ok.notify_all(); // 可能有多个读者等待
        writer_ok.notify_one(); // 只有一个写者可以进入
    }
};

ReadWriteLock rw_lock;
void reader_function(int id) {
    rw_lock.read_lock();
    std::cout << "Reader " << id << " is reading." << std::endl;
    std::this_thread::sleep_for(std::chrono::milliseconds(100)); // 模拟读操作耗时
    std::cout << "Reader " << id << " has finished reading." << std::endl;
    rw_lock.read_unlock();
}

void writer_function(int id) {
    rw_lock.write_lock();
    std::cout << "Writer " << id << " is writing." << std::endl;
    std::this_thread::sleep_for(std::chrono::milliseconds(100)); // 模拟写操作耗时
    std::cout << "Writer " << id << " has finished writing." << std::endl;
    rw_lock.write_unlock();
}

int main() {
    std::thread readers[5], writers[2];
    for (int i = 0; i < 2; ++i) {
        writers[i] = std::thread(writer_function, i + 1);
    }
    for (int i = 0; i < 5; ++i) {
        readers[i] = std::thread(reader_function, i + 1);
    }
    for (auto& writer : writers) {
        writer.join();
    }
    for (auto& reader : readers) {
        reader.join();
    }
    return 0;
}

```

34.什么是死锁以及其条件？如何解决死锁问题？

总结（老秦推荐必看，方便理解）：

(1)什么是死锁？

死锁是多线程或多进程编程中的一种情况，其中**一组进程或线程每个都在等待其他进程释放资源，或者等待其他进程执行操作，而这些进程或线程又在等待第一个进程释放资源**。结果是，没有一个进程能够向前推进，整个系统都陷入停滞。

(2)死锁的必要条件

死锁的发生通常依赖于以下四个条件，这些条件被称为死锁的四个必要条件：

1) 互斥条件 (Mutual Exclusion)：至少有一个资源必须处于非共享模式，即一次只有一个进程可以使用该资源。如果另一个进程也需要该资源，则必须等待直到资源被释放。

2) 持有和等待条件 (Hold and Wait)：一个进程必须已经持有至少一个资源，并且正在等待获取额外的被当前其他进程持有的资源。

3) 非抢占条件 (No Preemption)：资源不能被抢占，即资源只能由持有它的进程在使用完毕后自愿释放。

4) 循环等待条件 (Circular Wait)：必须有一个进程—资源的闭环链，其中每个进程至少持有一个资源，并等待获取另一个链中下一个进程所持有的资源。

(3)死锁的解决方法

解决死锁问题通常可以通过预防、避免或检测和恢复的方法来处理。

(4)预防死锁

预防死锁涉及到破坏死锁的四个必要条件中的至少一个：

1) 破坏互斥条件：这是很难做到的，因为某些资源天生是不可共享的。

2) 破坏持有和等待条件：可以要求进程在开始执行前一次性请求所有所需的资源。

3) 破坏非抢占条件：如果发现运行过程中会导致死锁，系统可以强制抢占已分配的资源。

4) 破坏循环等待条件：对所有资源类型进行排序，并强制每个进程按顺序请求资源，这样就不会形成环形等待链。

(5)避免死锁

避免死锁涉及到使用算法动态地检查资源分配状态以确保永远不会进入不安全的状态：

银行家算法：系统在**分配资源之前先检查这次分配是否可能导致系统进入不安全状态**，如果可能导致，就不分配这个资源。

(6)检测和恢复死锁

死锁检测：定期使用资源分配图或其他算法来检测系统是否进入死锁状态。

死锁恢复：一旦检测到死锁，系统可以采取解决措施，比如终止一个或多个进程，回滚操作或强制释放资源。

35.内存屏障分几种？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

内存屏障，也称为内存栅栏，是用于控制 CPU 和编译器对内存操作顺序的一种机制。
内存屏障可以确保特定的内存操作在程序执行的顺序上不会被重排序，从而保证多线程程序中内存访问的正确性。

(1)全局内存屏障 (Full Memory Barrier)

全局内存屏障确保在屏障前的所有内存操作在屏障后的内存操作之前完成。它会阻止所有类型的内存操作（读取和写入）重排序。

(2)读内存屏障 (Read Memory Barrier)

读内存屏障确保在屏障前的所有读操作在屏障后的读操作之前完成。它会阻止读操作被重排序。

(3)写内存屏障 (Write Memory Barrier)

写内存屏障确保在屏障前的所有写操作在屏障后的写操作之前完成。它会阻止写操作被重排序。

(4)依赖性屏障 (Dependency Barrier)

依赖性屏障确保依赖关系的顺序不会被重排序。它保证依赖于先前操作的内存访问不会被重排序到先前操作之前。这种屏障主要用于保护数据依赖。

36. 条件变量的缺点及为何需要与互斥锁 (mutex) 配合使用？

总结（老秦推荐必看，方便理解）：

(1) 条件变量的缺点

1) **复杂性**：条件变量的使用比简单的互斥锁复杂，需要正确的同步和逻辑来避免竞争条件和死锁。

2) **虚假唤醒 (Spurious Wakeups)**：条件变量可能会在没有明确通知的情况下唤醒等待线程，即假唤醒。需要额外的逻辑来处理这种情况，通常是使用循环来反复检查条件。

3) **资源开销**：条件变量的实现可能涉及系统调用，导致额外的资源开销和性能影响。

4) **等待条件的复杂管理**：条件变量要求等待条件在某些情况下能够正确地重新检查，这通常通过循环来实现，但增加了代码的复杂性。

(2) 需要与互斥锁 (mutex) 配合使用

1) **保护共享资源的访问**：条件变量用于线程之间的同步，需要与互斥锁一起使用来保护共享资源的访问。互斥锁确保在检查和修改共享资源时，不会有其他线程并发访问这些资源。

2) **防止竞争条件**：在等待条件变量的线程在被唤醒后，需要重新检查条件是否成立。互斥锁可以防止其他线程在条件检查和等待之间修改共享资源，避免竞争条件。

3) **确保原子操作**：等待和唤醒操作通常需要原子性。互斥锁可以确保线程在等待条件变量时，不会被其他线程打扰，从而实现原子操作。

4) **避免假唤醒问题**：条件变量可能会产生假唤醒，通过互斥锁可以在被唤醒后重新检查条件是否满足，确保线程不会在不适当的时候继续执行。

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
std::mutex mtx;
std::condition_variable cv;
bool ready = false;
void print_id(int id) {
    std::unique_lock<std::mutex> lck(mtx);
    while (!ready) { // 使用 while 循环防止虚假唤醒
        cv.wait(lck);
    }
    // 被唤醒后执行的代码
}
```

```

        std::cout << "Thread " << id << " is running\n";
    }
    void set_ready() {
        std::this_thread::sleep_for(std::chrono::seconds(1));
        {
            std::lock_guard<std::mutex> lck(mtx);
            ready = true;
        }
        cv.notify_all();
    }
}
int main() {
    std::thread threads[10];
    for (int i = 0; i < 10; ++i) {
        threads[i] = std::thread(print_id, i);
    }
    set_ready();
    for (auto& th : threads) {
        th.join();
    }
    return 0;
}

```

37.如何实现操作系统中的原子操作？

总结（老秦推荐必看，方便理解）：

在操作系统和计算机体系结构中，原子操作确保并发环境中的操作在执行时不被中断。这些操作通过以下两种主要方式实现：

硬件支持：大多数现代处理器提供特殊的原子指令，如 Compare-and-Swap (CAS)、Test-and-Set (TAS)、Fetch-and-Add。这些指令在单个指令周期内完成检查和修改值的操作，确保了操作的原子性。

操作系统支持：操作系统通过禁用中断创建临界区，防止上下文切换，确保代码段在执行过程中不被中断。主要用于单处理器系统或控制特定硬件交互。

代码示例（老秦建议：自己动手实践一下）

```

#include <atomic>
#include <iostream>
#include <thread>
#include <vector>
void increment(std::atomic<int>& counter) {
    for (int i = 0; i < 1000; ++i) {
        counter.fetch_add(1, std::memory_order_relaxed); // 原子加 1
    }
}

```

```

int main() {
    std::atomic<int> counter(0);
    std::vector<std::thread> threads;
    // 创建 10 个线程进行并发自增
    for (int i = 0; i < 10; ++i) {
        threads.push_back(std::thread(increment, std::ref(counter)));
    }
    // 等待所有线程完成
    for (auto& thread : threads) {
        thread.join();
    }
    std::cout << "Final counter value: " << counter << std::endl;
    return 0;
}

```

38. 线程从任务队列中取任务时，有办法不使用锁？

总结（老秦推荐必看，方便理解）：

无锁编程利用原子操作来保证数据结构的一致性，避免了锁的使用，减少了线程阻塞和上下文切换的开销。**无锁队列特别适用于生产者-消费者模型**，其中多个生产者和消费者线程频繁地入队和出队操作，而不会彼此干扰。

代码示例（老秦建议：自己动手实践一下）

```

#include <atomic>
#include <iostream>
#include <thread>
// 定义一个泛型无锁队列
template <typename T>
class LockFreeQueue {
private:
    // 节点结构
    struct Node {
        T data; // 存储的数据
        std::atomic<Node*> next{ nullptr }; // 指向下一个节点的原子指针
        explicit Node(T val) : data(val) {} // 节点构造函数
    };
    std::atomic<Node*> head; // 队列头部
    std::atomic<Node*> tail; // 队列尾部
public:
    // 构造函数
    LockFreeQueue() {
        Node* dummy = new Node(T()); // 创建一个哑元节点
    }
};

```



```

        head.store(dummy, std::memory_order_relaxed); // 初始化头部
        tail.store(dummy, std::memory_order_relaxed); // 初始化尾部
    }
    // 析构函数
    ~LockFreeQueue() {
        Node* node;
        while (node = head.load(std::memory_order_relaxed)) {
            head.store(node->next, std::memory_order_relaxed);
            delete node; // 删除节点释放内存
        }
    }
    // 入队操作
    void push(const T& value) {
        Node* newNode = new Node(value); // 创建新节点
        Node* oldTail = tail.load(std::memory_order_relaxed);
        while (!tail.compare_exchange_weak(oldTail, newNode,
std::memory_order_release, std::memory_order_relaxed)) {
            oldTail = tail.load(std::memory_order_relaxed); // 重新加载尾部
        }
        oldTail->next.store(newNode, std::memory_order_release); // 设置新节点为尾部节点的下一个节点
    }
    // 出队操作
    bool pop(T& value) {
        Node* oldHead = head.load(std::memory_order_consume);
        Node* tail = this->tail.load(std::memory_order_relaxed);
        Node* next = oldHead->next.load(std::memory_order_relaxed);
        if (oldHead == tail) {
            return false; // 队列为空
        }
        value = next->data; // 读取数据
        head.store(next, std::memory_order_release); // 移动头部指针到下一个节点
        delete oldHead; // 删除旧的头部节点
        return true;
    }
};

int main() {
    LockFreeQueue<int> queue;
    std::thread producer([&]() {
        for (int i = 0; i < 100; ++i) {
            queue.push(i); // 生产者线程向队列中推送数据
        }
    });
}

```

```

    });
    std::thread consumer([&]() {
        int value;
        for (int i = 0; i < 100; ++i) {
            while (!queue.pop(value)) { // 消费者线程从队列中获取数据
                std::this_thread::sleep_for(std::chrono::milliseconds(1)); // 如果队列为空，则稍等片刻
            }
            std::cout << "Consumed: " << value << std::endl; // 输出消费的数据
        }
    });
    producer.join(); // 等待生产者线程结束
    consumer.join(); // 等待消费者线程结束
    return 0;
}

```

39.介绍 CAS 中的 ABA 问题及其解决方案？

总结（老秦推荐必看，方便理解）：

(1)什么是 ABA 问题？

ABA 问题是在使用原子操作比如 **Compare-and-Swap (CAS)**时遇到的一个问题。在并发编程中，CAS 操作检查一个位置的值如果与预期相同，则将其更新为新值。ABA 问题发生在一个位置的值原本是 A，被一个线程改为 B，然后又改回 A。另一个线程使用 CAS 操作检查该值（期望它仍然是 A），然后进行更新。尽管此位置的值看起来未变，但其实它在此期间可能已经被改变过。

(2)ABA 问题的影响

这种情况在多线程程序中可能会导致严重的错误，因为虽然“当前值”与“期望值”相同，线程操作的上下文可能已经完全改变。例如，**如果这个值是一个指针，它可能已经被释放并重新分配，再次使用它就可能导致未定义行为或安全漏洞。**

(3)解决 ABA 问题的策略

1) 使用版本号：一种常用的解决方法是将**版本号与数据结合**。**每次变量被修改时，版本号增加。这样，CAS 操作不仅比较数据值还比较版本号**。如果一个变量从 A 变为 B 再变回 A，版本号将反映出这一变化，使得 CAS 操作能够检测到中间的变化。

2) 使用双字 (Double-Word) CAS：双字 CAS 可以同时比较和交换两个连续的字（通常是 64 位系统中的 128 位）。这使得开发者可以将一个额外的计数器或版本号与数据一起存储和检查。这种方法在现代处理器中已经得到了支持，比如在 Intel 的处理器中可以使用 CMPXCHG16B 指令

40.解释并发和并行的区别？

总结（老秦推荐必看，方便理解）：

(1)并发 (Concurrency)：

- 1) 多个任务在同一时间段内交替进行。
- 2) 任务切换提高响应性和资源利用率。
- 3) 可以在单核处理器上实现。
- 4) 任务之间是交替执行的，给人一种同时进行的错觉。

(2)并行 (Parallelism)：

- 1) 多个任务在同一时刻同时进行。
- 2) 需要多核处理器或多个处理器。
- 3) 提高计算速度和处理能力。
- 4) 任务之间是真正的同时执行。

(3)比喻

并发：想象一个人在做多件事，他**快速地在不同的事情之间切换**。虽然看起来他在**同时处理多个任务**，但实际上他在**每个时间点只处理一个任务**。

并行：想象**多个人同时做不同的事**，每个人都在独立地完成自己的任务，**所有任务在同一时间进行**。

(4)应用场景

并发：适用于 **I/O 密集型任务**（如文件读取、网络通信），可以提高系统的响应性。

并行：适用于 **计算密集型任务**（如矩阵乘法、大数据处理），可以显著缩短计算时间。

4.1.描述不同类型的锁及其用途。

总结（老秦推荐必看，方便理解）：

(1)互斥锁 (Mutex)

作用：确保同时只有一个线程可以执行某段代码或访问某个资源。

使用场景：适用于保护共享数据，如全局变量或数据结构，防止并发修改导致的数据不一致。

特点：简单易用，但可能导致死锁和优先级反转等问题。

(2)递归锁 (Recursive Mutex)

作用：允许同一个线程多次获得同一把锁。

使用场景：适用于递归函数或可以被同一个线程多次调用的代码块中，需要重复锁定。

特点：避免了因重入而死锁的问题，但使用不当可能会导致锁保持时间过长。

(3)读写锁 (Read-Write Lock)

作用：允许多个读者同时访问资源，但写者访问时独占资源。

使用场景：适用于读多写少的数据结构，可以显著提高读操作的并发性。

特点：提高了读操作的性能，但实现相对复杂，可能引入写者饥饿问题。

(4)自旋锁 (Spinlock)

作用：通过忙等（自旋）而不是休眠来等待锁的释放。

使用场景：适用于锁持有时间极短的场景，以减少线程上下文切换的开销。

特点：高效于处理器密集型环境，但在单处理器系统或锁持有时间较长的情况下效率低下。

42.线程属性值如何修改？

总结（老秦推荐必看，方便理解）：

在使用 POSIX 线程（pthread）编程时，可以通过 `pthread_attr_t` 结构来设置和修改线程的属性值。常见的线程属性包括栈大小、栈地址、调度策略和优先级、继承调度属性、分离状态等。

步骤概述

- 1) 初始化线程属性对象。
- 2) 设置或修改线程属性。
- 3) 创建线程时应用这些属性。
- 4) 销毁线程属性对象（可选）。

代码示例（老秦建议：自己动手实践一下）

```
#include <pthread.h>
#include <iostream>
#include <cstdlib>
void* threadFunc(void* arg) {
    std::cout << "Thread is running with arg: " << *static_cast<int*>(arg) <<
std::endl;
    return nullptr;
}
int main() {
    pthread_t thread;
    pthread_attr_t attr;
    int arg = 42;
    size_t stacksize;
    void* stackaddr;
    // 初始化线程属性对象
    pthread_attr_init(&attr);
    // 修改栈大小
    stacksize = 2 * 1024 * 1024; // 2 MB
    pthread_attr_setstacksize(&attr, stacksize);
    // 获取默认栈大小
    pthread_attr_getstacksize(&attr, &stacksize);
```

```
std::cout << "Stack size set to: " << stacksize << " bytes" << std::endl;
// 设置分离状态
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
// 创建线程并应用属性
if (pthread_create(&thread, &attr, threadFunc, &arg) != 0) {
    std::cerr << "Failed to create thread" << std::endl;
    return 1;
}
// 由于线程是分离状态，无法使用 pthread_join 等待线程结束
// 销毁线程属性对象
pthread_attr_destroy(&attr);
// 休眠一段时间以确保线程执行完毕
sleep(1);
std::cout << "Main thread completed" << std::endl;
return 0;
}
```

三 . 内存管理与虚拟内存

43.什么是虚拟内存和物理内存？区别和联系是什么？

图示（方便记忆）：

特性	虚拟内存 (Virtual Memory)	物理内存 (Physical Memory)
定义	操作系统创建的抽象内存空间，通过硬件和软件结合实现	实际存在的硬件内存（如 RAM）
容量	可以超过物理内存大小，通过使用硬盘上的交换空间（swap space）扩展	由物理内存硬件的大小决定
访问方式	通过页表和地址映射机制转换为物理地址，再被 CPU 访问	直接被 CPU 访问
隔离性	每个进程有独立的虚拟地址空间，增强了进程之间的隔离	多个进程共享物理内存，需要显式管理访问权限
性能优化	利用物理内存的缓存特性，提高内存访问速度	通过硬件高速访问
安全性和稳定性	提供每个进程独立的地址空间，避免进程之间的内存干扰，提高系统安全性和稳定性	需要额外管理，避免进程间冲突

代码示例（老秦建议：自己动手实践一下）

```
#include <iostream>
void physicalMemoryExample() {
    char buffer[1024]; // 分配在物理内存中的缓冲区
}
```

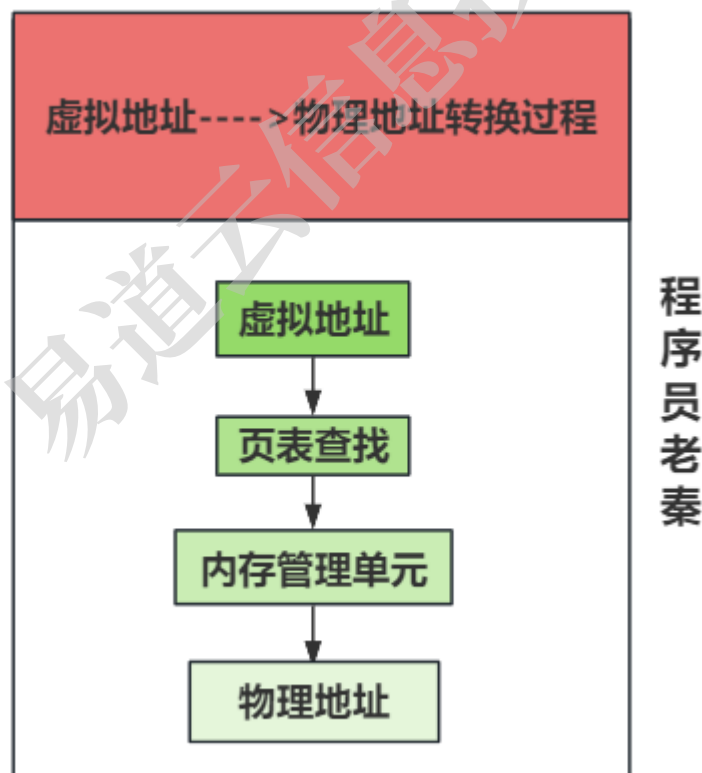
```

    buffer[0] = 'A';
    std::cout << "Buffer[0]: " << buffer[0] << std::endl;
}
void virtualMemoryExample() {
    char* dynamicBuffer = new char[1024]; // 动态分配在虚拟内存中的缓冲区
    dynamicBuffer[0] = 'B';
    std::cout << "DynamicBuffer[0]: " << dynamicBuffer[0] << std::endl;
    delete[] dynamicBuffer; // 释放虚拟内存
}
int main() {
    physicalMemoryExample();
    virtualMemoryExample();
    return 0;
}

```

44. 如何将虚拟地址转换为物理地址？页表和内存管理单元（MMU）的角色是什么？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)过程总结：

- 1) 获取页目录表基地址：从 CR3 寄存器中获取页目录表的基地址。
- 2) 计算页目录索引：从虚拟地址中提取页目录索引。
- 3) 查找页目录表：使用页目录索引从页目录表中查找页表的基地址。
- 4) 计算页表索引：从虚拟地址中提取页表索引。
- 5) 查找页表：使用页表索引从页表中查找物理页框号。
- 6) 计算物理地址：将物理页框号与页内偏移组合，生成物理地址。

(2)内存管理单元（MMU）：

- 1) 角色：MMU 是硬件组件，负责将 CPU 生成的虚拟地址转换为物理地址。
- 2) 工作原理：当 CPU 访问内存时，MMU 截获虚拟地址，根据页表进行查找，将其转换为物理地址，然后访问相应的内存位置。

45.操作系统如何分配和管理内存？解释 malloc 和 new 的内存分配机制。

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)操作系统内存管理

1) 虚拟内存：

虚拟内存概念：虚拟内存是操作系统提供的内存管理机制，通过将物理内存与虚拟地址空间分离，使每个进程拥有独立的地址空间，提升内存使用的效率和安全性。

虚拟地址转换：虚拟地址通过页表转换为物理地址，具体转换过程由内存管理单元（MMU）硬件实现。

2) 分页机制：

分页：将内存划分为固定大小的块，称为页（Page），**虚拟内存和物理内存都按页进行管理**。每个**页通常为 4KB**。

页表：操作系统使用**页表记录虚拟地址到物理地址的映射关系**。每个**进程有一个独立的页表**。

页表查找：通过虚拟地址的页号在**页表中查找对应的物理页框号**，再加上**页内偏移得到物理地址**。

3) 分段机制：

分段：内存被划分为不同的段（Segment），**每个段有不同的尺寸和权限**，用于表示代码段、数据段、堆栈段等。

段表：操作系统使用段表记录段的基地址和大小，通过段选择子和段偏移计算物理地址。

(2) 用户态内存分配

1) malloc 内存分配机制：

基本原理：malloc 是 C 语言中的动态内存分配函数，用于在堆上分配指定大小的内存。

实现机制：

小内存分配：对于小块内存，**malloc 从维护的空闲内存块链表中查找合适的块进行分配**。

大内存分配：对于大块内存，**malloc 通过 brk 或 mmap 系统调用向操作系统申请新的内存区域**。

释放内存：通过 free 函数释放分配的内存，将其归还到空闲链表。

2) new 内存分配机制：

基本原理：**new 是 C++ 中的运算符**，用于在堆上分配内存并调用构造函数初始化对象。

实现机制：

调用 **malloc**：new 运算符**底层通常调用 malloc 分配内存**。

调用**构造函数**：在分配的内存上调用对象的构造函数进行初始化。

释放内存：通过 delete 运算符释放内存并调用析构函数。

46. 虚拟内存系统中的内存分配策略有哪些？连续与非连续内存管理的区别。

图示（方便记忆）：

特性	连续内存管理	非连续内存管理
内存分配方式	连续分配，内存块必须是连续的	非连续分配，内存块可以分散在不同区域
内存利用效率	内存碎片多，利用率较低	内存碎片少，利用率较高
实现复杂度	简单，实现相对容易	复杂，需要页表或段表支持

常见策略	固定分区、可变分区	分页、分段、段页结合
地址转换	直接通过基地址和偏移计算	通过页表或段表查找物理地址
扩展性	不灵活，难以动态调整内存大小	灵活，可以动态分配和释放内存
保护和隔离	隔离较差，共享内存较难	隔离较好，易于实现内存保护和共享

总结（老秦推荐必看，方便理解）：

(1)连续内存管理：

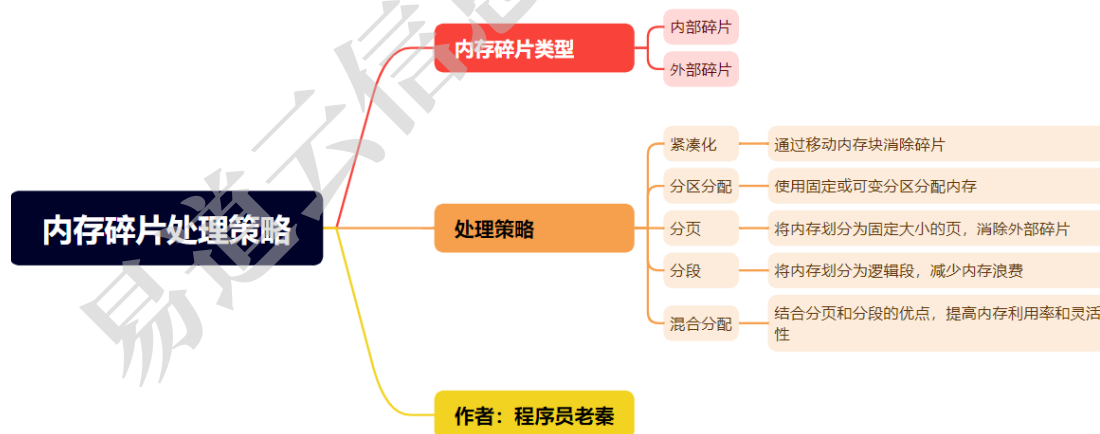
- 1) 优点：实现简单，直接通过基地址和偏移计算物理地址。
- 2) 缺点：容易产生内存碎片，内存利用率较低，难以动态调整内存大小。

(2)非连续内存管理：

- 1) 优点：内存利用率高，减少内存碎片，易于实现内存保护和共享，支持动态分配和释放。
- 2) 缺点：实现复杂，需要页表或段表支持，地址转换开销较大。

47.解释内存碎片的处理策略。为什么操作系统设计允许内存碎片的产生？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)为什么操作系统设计允许内存碎片的产生？

- 1) 内存利用效率：完全避免内存碎片会导致内存管理策略过于复杂，可能降低内存利用效率。在一些应用场景下，适当的内存碎片是可以接受的，以换取内存管理的简单性和高效性。
- 2) 性能考虑：一些高效的内存分配策略（如分页）虽然可能产生内部碎片，但能够

显著提高内存分配和回收的速度，减少操作系统的开销。紧凑化内存碎片虽然可以消除碎片，但需要频繁移动内存块，会导致性能下降。

3) 灵活性和扩展性：非连续内存管理策略（如分页和分段）虽然可能产生内存碎片，但提供了更大的灵活性和扩展性。分页和分段允许进程使用不连续的内存块，便于实现动态内存分配和扩展，提高内存利用率。

48.内存池如何保证不产生内存碎片？

总结（老秦推荐必看，方便理解）：

内存池是一种内存管理技术，通过预先分配一大块内存，并将其划分为多个固定大小的小块，以满足频繁的内存分配和释放需求。内存池管理器负责维护这些小块的分配和释放，减少内存碎片的产生，提高内存利用率和分配效率。

内存池如何减少内存碎片？

(1)固定大小块分配：

1) 基本原理：内存池将内存划分为大小相同的块，每次分配和释放的内存都是固定大小的块。

2) 优点：由于每次分配和释放的内存块大小相同，不会出现大小不一的内存块混合在一起，从而避免了外部碎片。

3) 示例：假设内存池中每个块大小为 64 字节，那么无论何时分配或释放内存，都是以 64 字节为单位进行。

(2)内存块复用：

1) 基本原理：内存块被释放后，不会立即归还给操作系统，而是保留在内存池中，以便下次分配时复用。

2) 优点：减少了频繁分配和释放内存导致的内存碎片，同时也减少了系统调用的开销。

3) 示例：当一个内存块被释放后，该块会被标记为可用，下次分配内存时，优先使用这些可用块。

(3)分区管理：

1) 基本原理：根据不同大小的内存需求，将内存池划分为多个区域，每个区域管理不同大小的内存块。

2) 优点：针对不同大小的内存分配需求，避免内存块大小不一导致的碎片问题，提高内存利用率。

3) 示例：内存池可以有多个子池，一个子池分配 64 字节块，另一个子池分配 128 字节块，根据需求选择合适的子池进行分配。

49.物理页面大小选择的依据是什么？为什么常选择 4KB 大小的页面？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

-物理页面大小选择的依据

(1)平衡内存管理和性能：

1) 内存管理：页面大小影响内存管理的效率。**较大的页面可以减少页表的大小和管理开销**，但可能导致**更大的内存碎片**。**较小的页面可以减少内存碎片**，但增加页表的大小和管理开销。

2) 性能：页面大小对内存访问性能有直接影响。**较大的页面可以减少页面故障的频率**，提高内存访问性能。**较小的页面可以提高内存的使用效率**，但可能**增加页面故障的频率**。

(2)内存碎片和页表大小：

1) 内存碎片：内存碎片是内存管理中的一个重要问题。**较大的页面可能导致更多的内部碎片**，而较小的页面可以减少这种碎片。

2) 页表大小：页表记录虚拟地址到物理地址的映射关系。**较大的页面可以减少页表条目的数量**，从而减少页表的大小和管理开销。

(3)硬件和架构支持：

1) 硬件支持：不同的硬件架构支持不同的页面大小。选择页面大小时需要考虑硬件的支持情况。

2) 架构特性：某些架构可能对特定大小的页面有优化，选择合适的页面大小可以充分利用这些优化特性。

-为什么常选择 4KB 大小的页面？

(1)适中大小：

1) **平衡性**：4KB 页面在内存碎片和页表大小之间取得了良好的平衡。它足够小，可以减少内存碎片，同时也足够大，可以保持页表的合理大小。

2) **广泛适用**：4KB 页面大小适用于大多数应用场景，能够满足大多数操作系统和应用程序的内存需求。

(2)硬件支持：

1) **普遍支持**：多数现代硬件架构（如 x86、ARM 等）对 4KB 页面有良好的支持。这种广泛的硬件支持使得 4KB 页面成为默认选择。

2) **性能优化**：许多硬件架构对 4KB 页面进行了特定的性能优化，使用 4KB 页面可以充分利用这些优化特性。

(3)操作系统和应用兼容性：

1) **操作系统标准**：许多操作系统（如 Linux、Windows）使用 4KB 作为标准页面大小，确保了广泛的兼容性。

2) **应用程序兼容**：许多应用程序设计和优化时考虑到了 4KB 页面大小，选择 4KB 页面可以确保这些应用程序的正常运行和最佳性能。

(4)现实内存需求：

1) **内存需求**：4KB 页面大小能够满足大多数应用程序的内存需求。对于需要大量小块内存的应用程序，4KB 页面可以有效减少内存碎片，提高内存利用率。

2) **灵活性**：4KB 页面大小提供了良好的灵活性，能够适应不同的内存分配需求。

50.程序从堆中动态分配内存时，虚拟内存上如何操作的？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

- (1)程序请求分配内存：使用动态内存分配函数（如 malloc、new）。
- (2)动态内存分配库：处理内存请求，可能会调用系统调用（如 brk、sbrk、mmap）。
- (3)操作系统请求内存页：如果需要，操作系统为进程分配新的物理内存页。
- (4)页表更新：操作系统更新页表，映射新的虚拟内存页到物理内存页。
- (5)内存映射：虚拟地址与物理地址建立映射关系。
- (6)返回指针：动态内存分配函数返回指向分配内存的指针。

51.交换空间与虚拟内存的关系是什么？

总结（老秦推荐必看，方便理解）：

(1)交换空间（Swap Space）：

1) 内存换出（Swap Out）：将不常用的内存页从物理内存移到交换空间，以释放物理内存。

2) 内存换入（Swap In）：当需要访问已换出的内存页时，将其从交换空间移回物理内存。

(2)交换空间在虚拟内存中的作用

1) 扩展内存容量：通过交换空间，操作系统可以将物理内存和磁盘上的交换空间结合使用，提供比实际物理内存更大的可用内存容量。

2) 提高内存利用率：将不常用的内存页移到交换空间，释放物理内存给更需要的进程使用，提高内存利用率。同时操作系统动态调整内存换出和换入的频率，保持内存的高效利用。

3) 减少内存不足引起的错误：当物理内存不足时，通过交换空间提供额外的存储，避免程序因内存溢出而崩溃。

52.常见内存分配方式有哪些？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)静态内存分配：在编译时分配内存，大小固定，生命周期与程序相同。

(2)栈内存分配：在函数调用时分配内存，自动管理，速度快，生命周期短。

- (3)堆内存分配：动态内存分配，大小灵活，需要手动管理，速度相对较慢。
- (4)内存池分配：预先分配固定大小的内存块，提高内存分配和释放的效率，减少碎片。

53.常见内存分配错误有哪些？

图示（方便记忆）：

错误类型	定义	原因	解决方法
内存泄漏	分配的内存没有被释放，导致内存不可用	动态分配内存后忘记释放，提前返回或异常未释放	使用智能指针，确保在所有路径中释放内存
双重释放	同一块内存被释放多次，导致程序崩溃或不稳定	误操作导致重复调用 delete 或 free	在释放内存后将指针置为 nullptr，使用智能指针
野指针	指针指向的内存已经被释放或未初始化，可能导致错误	使用已释放的内存，未初始化的指针被使用	在释放内存后将指针置为 nullptr，确保指针初始化
越界访问	访问超出分配内存范围的地址，可能导致程序崩溃	数组索引超出范围，指针运算错误	确保数组访问在合法范围内，使用标准库容器的 at 方法
使用未初始化的指针	指针在初始化前被使用，可能指向随机地址	忘记初始化指针	始终初始化指针，使用智能指针
空指针解引用	解引用空指针，导致程序崩溃	忘记检查指针是否为空	在解引用指针前检查指针是否为空，使用智能指针
内存碎片	动态内存分配和释放频繁，导致内存碎片	动态内存分配和释放频繁	使用内存池减少内存碎片，减少频繁的内存分配和释放

54.页表的构成和作用是什么？

图示（方便记忆）：

页表



程序员老秦

总结（老秦推荐必看，方便理解）：

(1) 页表的构成

1) 页表条目 (PTE, Page Table Entry)：

定义：页表条目是页表的基本单元，每个条目对应一个虚拟页。

内容：包含物理页框号 (PFN, Physical Frame Number) 和控制位（如有效位、读写位、用户/内核模式位等）。

2) 页目录表 (PD, Page Directory)：

定义：页目录表包含多个页表条目的集合。

内容：每个条目指向一个页表，页表条目进一步指向实际的物理内存页。

3) 页目录指针表 (PDP, Page Directory Pointer)：

定义：页目录指针表包含多个页目录表的指针。

内容：每个条目指向一个页目录表，页目录表包含页表的指针。

4) 页表目录 (PML4, Page Map Level 4)：

定义：页表目录是最高层的页表结构，包含多个页目录指针表的指针。

内容：每个条目指向一个页目录指针表，页目录指针表包含页目录表的指针。

(2) 页表的作用

1) 虚拟地址到物理地址映射：

作用：页表的主要作用是将虚拟地址映射到物理地址，使得操作系统能够使用虚拟内存。

过程：CPU 生成虚拟地址，通过 MMU（内存管理单元）查找页表。页表各级查找，最终找到对应的物理地址。

2) 内存保护：

作用：页表通过控制位实现对内存访问的权限控制，提供内存保护。

有效位 (Present bit)：指示该页是否在内存中。

读写位 (Read/Write bit)：控制该页的读写权限。

用户/内核模式位 (User/Supervisor bit)：控制该页的访问模式（用户模式或内核模式）。

3) 支持虚拟内存管理：

作用：页表支持虚拟内存管理，使得进程可以使用比物理内存更大的地址空间。

换页机制：当内存不足时，操作系统可以将不常用的页换出到磁盘上的交换空间，释放物理内存。当需要访问已换出的页时，操作系统将其从交换空间换入到物理内存。

55. 分页有什么好处，为什么需要分页？

图示（方便记忆）：



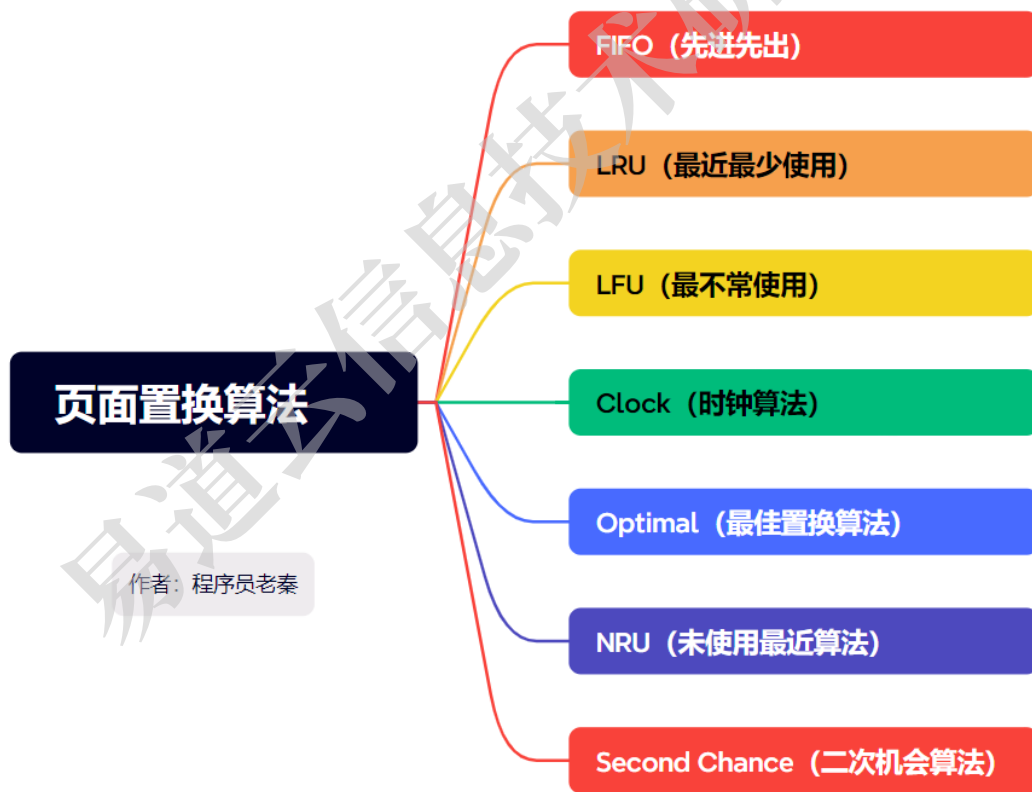
总结（老秦推荐必看，方便理解）：

(1) 简化内存管理：分页将内存划分为固定大小的页和页框，简化了内存的分配和回收。

- (2)提供内存保护**：分页支持页面级的权限控制，防止进程非法访问其他进程的内存。
- (3)支持虚拟内存**：通过分页可以实现虚拟内存，使得进程可以使用比物理内存更大的地址空间。
- (4)减少内存碎片**：分页避免了内存分配中的外部碎片问题，提高了内存利用率。
- (5)提高内存利用率**：分页支持按需加载和交换，优化了内存使用。
- (6)提供地址空间隔离**：每个进程拥有独立的页表，确保了地址空间的隔离，提高了系统的稳定性和安全性。

56. 页面置换算法有哪些？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)FIFO (先进先出)

- 1) 定义：按照页面进入内存的顺序进行置换，最先进入的页面最先被置换。

2) 过程：每次页面进入内存时，记录进入顺序。当需要置换时，选择最早进入内存的页面进行置换。

3) 优点：实现简单，易于理解。

4) 缺点：性能不佳，可能会置换掉仍然需要的页面（Belady's Anomaly）。

(2)LRU（最近最少使用）

1) 定义：置换最近最少使用的页面。

2) 过程：每次页面被访问时，更新其使用时间。当需要置换时，选择最近最少使用的页面进行置换。

3) 优点：性能好，适用于大多数场景。

4) 缺点：实现复杂，需要维护页面使用时间的数据结构。

(3)LFU（最不常使用）

1) 定义：置换访问频率最低的页面。

2) 过程：每次页面被访问时，更新其访问频率。当需要置换时，选择访问频率最低的页面进行置换。

3) 优点：适用于访问频率有规律的场景。

4) 缺点：容易产生“长期不变”现象，老页面可能一直占用内存。

(4)Clock（时钟算法）

1) 定义：FIFO 的改进版，提供“第二次机会”机制。

2) 过程：每个页面有一个“访问位”。当需要置换时，检查页面的“访问位”：如果为 0，置换该页面。如果为 1，将其置为 0，并检查下一个页面。

3) 优点：性能较好，避免置换掉仍然需要的页面。

4) 缺点：实现稍复杂，需要维护一个时钟指针。

(5)Optimal（最佳置换算法）

1) 定义：理论最优算法，选择将来最久不使用的页面进行置换。

2) 过程：预测每个页面的未来使用情况。当需要置换时，选择将来最久不使用的页面进行置换。

3) 优点：提供最优性能。

4) 缺点：实际不可实现，仅用于评估其他算法。

(6)NRU（未使用最近算法）

1) 定义：根据访问位和修改位进行分类，随机选择置换。

2) 过程：页面有两个位：访问位和修改位。每个时钟周期清除访问位。

分为四类：

访问位=0，修改位=0

访问位=0，修改位=1

访问位=1，修改位=0

访问位=1，修改位=1 随机选择最低类进行置换。

3) 优点：实现简单，性能较好。

4) 缺点：可能不如 LRU 和 Clock 算法高效。

(7)Second Chance (二次机会算法)

- 1) 定义：FIFO 的改进版，给页面第二次机会避免被置换。
- 2) 过程：每个页面有一个“访问位”。当需要置换时，检查页面的“访问位”：如果为 0，置换该页面。如果为 1，将其置为 0，并检查下一个页面。
- 3) 优点：性能较好，避免置换掉仍然需要的页面。
- 4) 缺点：实现稍复杂，需要维护一个时钟指针。

四 . CPU 与系统性能

57.CPU 缓存如何影响计算性能？解释缓存的概念。

图示（方便记忆）：

缓存层级	位置	大小	速度
L1 缓存	内置于 CPU 核心内部	通常为几十 KB（如 32KB、64KB）	最快的缓存，延迟极低
L2 缓存	内置于 CPU 核心内部或共享于多个核心之间	通常为几百 KB 到几 MB（如 256KB、1MB）	比 L1 缓存稍慢，但比主存快得多
L3 缓存	通常共享于多个 CPU 核心之间	通常为几 MB 到几十 MB（如 8MB、16MB）	比 L2 缓存稍慢，但仍比主存快得多

总结（老秦推荐必看，方便理解）：

(1)缓存的概念

定义：缓存（Cache）是一种临时存储数据的高效存储器，位于 CPU 和主存之间，用于存储经常访问的数据。

目的：缓存的主要目的是加速数据访问速度，减少从主存获取数据的延迟，提高 CPU 的处理性能。

(2)缓存的优点

加速数据访问：

原理：缓存存储的是 CPU 经常访问的数据，通过减少访问主存的次数，提高了数据访问速度。

效果：提高了 CPU 的处理性能，减少了处理等待时间。

减少内存延迟：

原理：从缓存中获取数据的速度远高于从主存中获取数据的速度。

效果：减少了数据访问的延迟，提高了程序的执行效率。

58.什么是超线程技术，它如何影响计算性能？

总结（老秦推荐必看，方便理解）：

(1)超线程技术的概念

1) 定义：超线程技术（Hyper-Threading Technology，HTT）是一种同时多线程（SMT）技术，使一个物理处理器核心能够模拟两个或多个逻辑处理器核心，从而提高并行处理能力。

2) 实现：在物理处理器核心上增加一些额外的硬件资源，如寄存器、队列等，以便能够在同一时间处理多个线程的指令。

(2)超线程技术的目标

1) 提高资源利用率：通过让单个物理核心同时处理多个线程，最大化利用 CPU 的计算资源，减少资源闲置时间。

2) 提高并行处理能力：在多线程应用程序中，超线程技术可以显著提高并行处理能力，从而提升整体性能。

(3)超线程技术的优点

1) 提高系统性能：

并行处理：在多线程环境下，超线程技术能够让更多的线程并行执行，提高任务处理的速度。

响应速度：在处理多个任务时，超线程技术可以提高系统的响应速度，减少等待时间。

2) 提升资源利用率：

硬件资源共享：超线程技术通过共享处理器核心的硬件资源，提高了资源的利用效率，减少了资源的闲置。

降低延迟：通过同时处理多个线程，超线程技术可以有效降低处理任务的延迟。

五．文件系统与 I/O 操作

59.文件系统在创建文件时进行了哪些底层操作？

图示（方便记忆）：



程序员老秦

总结（老秦推荐必看，方便理解）：

(1)分配 inode：

1) 定义：inode 是文件系统中用来存储文件元数据的结构，包含文件大小、权限、时间戳等信息。

2) 过程：文件系统在创建文件时，为文件分配一个唯一的 inode 编号，并在 inode 表中创建相应的 inode 条目。

(2)分配数据块：

1) 定义：数据块是文件系统中用于存储文件实际数据的基本单位。

2) 过程：文件系统根据文件大小和需要，为文件分配一个或多个数据块，并将这些数据块的地址记录在 inode 中。

(3)更新目录项：

1) 定义：目录项是目录中记录文件信息的条目，包含文件名和对应的 inode 编号。

2) 过程：在文件所在的目录中，创建一个新的目录项，记录新文件的文件名和分配给它的 inode 编号。

(4)更新超级块：

1) 定义：超级块是文件系统的控制结构，包含文件系统的总体信息，如总的 inode 数量、总的数据块数量、可用的 inode 和数据块数量等。

2) 过程：文件系统更新超级块中的信息，如已分配的 inode 数量和数据块数量，反映文件系统的当前状态。

(5)日志记录（可选）：

1) 定义：日志记录是文件系统为保持一致性和快速恢复而记录的操作日志。

2) 过程：在支持日志的文件系统中（如 ext4），文件创建操作会被记录在日志中，以便在系统崩溃后进行恢复。

60.非阻塞 I/O 与异步 I/O 的区别及其实实现机制。

总结（老秦推荐必看，方便理解）：

(1)非阻塞 I/O

非阻塞 I/O 是指 I/O 操作不会阻塞进程，I/O 操作立即返回，不等待操作完成。应用程序需要主动检查操作的完成状态。

实现机制：

1) 设置非阻塞标志：通过系统调用设置文件描述符的非阻塞标志（如 `O_NONBLOCK`）

2) 轮询：应用程序通过轮询方式反复检查 I/O 操作的状态。

3) 使用 `select/poll/epoll`：利用这些系统调用来监视多个文件描述符，检测哪些文件描述符可以进行 I/O 操作。

(2)异步 I/O

异步 I/O 是指 I/O 操作在后台进行，操作完成后通过回调函数、信号或事件驱动机制通知应用程序。

实现机制：

1) **POSIX AIO**：使用 `aio_read`、`aio_write` 等异步 I/O 系统调用，操作完成后通过信号或回调函数通知。

2) **Windows IOCP**：使用 I/O 完成端口（I/O Completion Ports）机制，实现高效的异步 I/O 操作。

3) **事件驱动**：利用事件循环机制，在操作完成时触发事件通知应用程序。

61. 直接 I/O （ Direct I/O ） 和 页面缓存（PageCache）的作用是什么？

总结（老秦推荐必看，方便理解）：

(1)直接 I/O (Direct I/O)

直接 I/O 是一种绕过操作系统的页面缓存，直接将数据从用户空间读写到磁盘的方法。

作用：

- 1) 减少内存拷贝：直接 I/O 绕过页面缓存，避免了从内存到缓存再到磁盘的多次拷贝，直接从用户空间读写数据到磁盘。
- 2) 降低 CPU 消耗：减少了由于内存拷贝和缓存管理带来的 CPU 开销，提高了系统效率。
- 3) 提高数据一致性：由于数据直接写入磁盘，避免了页面缓存中的数据与磁盘数据不一致的问题，适用于对数据一致性要求高的应用，如数据库系统。

(2)页面缓存 (PageCache)

页面缓存是操作系统在内存中为文件数据保留的缓冲区，用于加速文件读取和写入操作。

作用：

- 1) 提高文件读取速度：通过将文件数据缓存在内存中，可以快速访问文件数据，减少从磁盘读取数据的延迟。
- 2) 减少磁盘 I/O 操作：在内存中缓存文件数据，减少了对磁盘的频繁访问，延长了磁盘的使用寿命。
- 3) 提高系统性能：页面缓存通过减少磁盘 I/O 操作，提高了文件系统的整体性能，适用于读写频繁的应用。

62.文件系统在 Linux 和 Windows 下的差异。

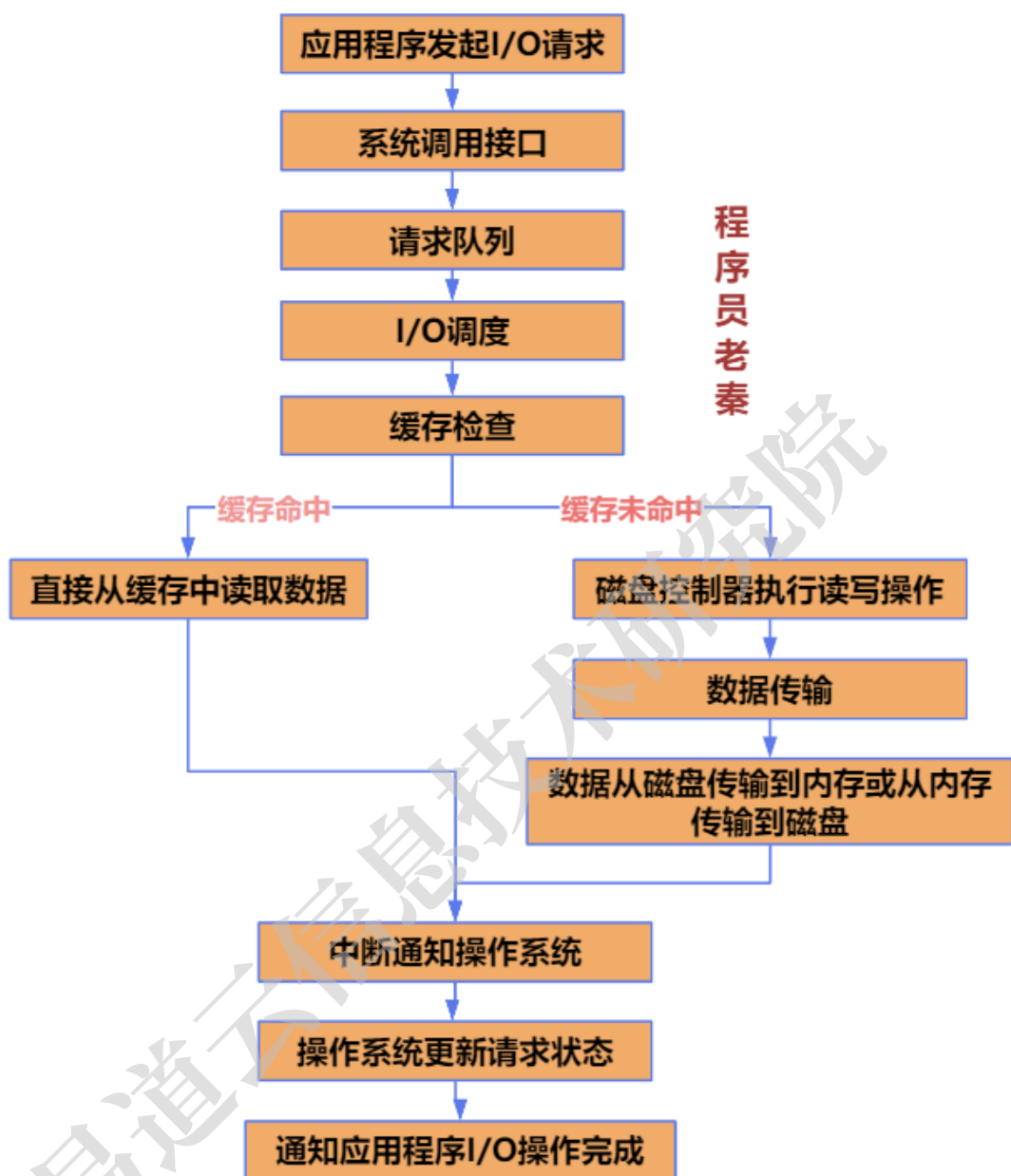
图示（方便记忆）：

特性	Linux	Windows
支持的文件系统类型	ext4、xfs、btrfs 等	NTFS、FAT32、exFAT 等
文件权限和安全机制	POSIX 权限模型和 ACL	NTFS 权限和 ACL
文件命名规则和路径格式	路径使用正斜杠 (/)，文件名区分大小写	路径使用反斜杠 (\)，文件名不区分大小写
文件锁机制	POSIX 文件锁和 fcntl 锁	Win32 文件锁
符号链接和硬链接	支持符号链接 (symbolic link) 和硬链接 (hard link)	仅支持符号链接，硬链接功能有限
文件系统检查和修复工具	fsck 工具用于检查和修复文件系统	chkdsk 工具用于检查和修复文件系统
日志文件系统	支持 (如 ext4、xfs)	支持 (如 NTFS)
文件系统扩展	支持在线扩展 (如 ext4、xfs)	一般需要离线扩展或使用特殊工具 (如 NTFS)
文件系统压缩和加密	部分文件系统支持压缩和加密 (如 btrfs)	NTFS 原生支持文件压缩和加密

文件恢复和快照	部分文件系统支持快照（如 btrfs）	通过第三方工具支持快照和恢复
存储设备管理	LVM、RAID、mdadm 等工具支持灵活的存储管理	通过磁盘管理工具支持基本的存储管理
磁盘配额管理	支持磁盘配额（quota）管理	NTFS 支持磁盘配额管理
文件系统的最大文件大小	ext4 : 16TB , xfs : 8EB	NTFS : 16EB , FAT32 : 4GB , exFAT : 16EB
最大文件系统大小	ext4 : 1EB , xfs : 8EB	NTFS : 256TB , FAT32 : 16TB , exFAT : 64ZB
文件系统性能	xfs 适合大文件和并发场景，ext4 性能稳定	NTFS 性能良好，兼容性强
备份和恢复工具	dump、restore、rsync 等	Windows Backup、第三方备份工具
一致性和恢复	日志文件系统提供一致性保证，快速恢复	NTFS 提供日志功能，确保数据一致性

63.磁盘 I/O 的工作原理是什么？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1) I/O 请求生成

1) 应用程序发起 I/O 请求：当应用程序需要读取或写入数据时，会通过系统调用（如 read、write 等）发起 I/O 请求。

2) 系统调用接口：应用程序使用操作系统提供的系统调用接口发起 I/O 请求，系统调用将请求传递给内核。

(2) 请求队列

1) 请求队列：操作系统内核将收到的 I/O 请求放入请求队列中，等待处理。请求队列可以是一个简单的 FIFO 队列，也可以是更复杂的结构，用于优化处理顺序。

2) I/O 请求类型：请求队列中可以包含不同类型的 I/O 请求，如读请求、写请求、同步请求、异步请求等。

(3) I/O 调度

1) I/O 调度算法：操作系统使用 I/O 调度算法优化请求的处理顺序，减少磁盘寻道时间和旋转延迟。常见的调度算法包括电梯算法（Elevator Algorithm）、最短寻道时间优先算法（SSTF）、循环扫描算法（C-SCAN）等。

2) 调度过程：调度算法根据优化策略选择下一个要处理的请求，并将其发送给磁盘控制器。

(4) 缓存

1) 页面缓存（Page Cache）：操作系统在内存中维护一个页面缓存，用于缓存最近访问的文件数据。页面缓存可以减少磁盘 I/O 操作，提高系统性能。

2) 缓存命中：当 I/O 请求的数据在页面缓存中时，直接从缓存中读取数据，避免了磁盘访问。

3) 缓存未命中：当 I/O 请求的数据不在缓存中时，需要从磁盘读取数据，并将数据加载到缓存中。

(5) 磁盘控制器执行读写操作

1) 磁盘控制器：磁盘控制器是计算机硬件的一部分，负责执行实际的读写操作。控制器根据操作系统的调度命令，控制磁盘的读写头定位和数据传输。

2) 数据传输：磁盘控制器将数据从磁盘传输到内存，或从内存传输到磁盘。

(6) 中断通知

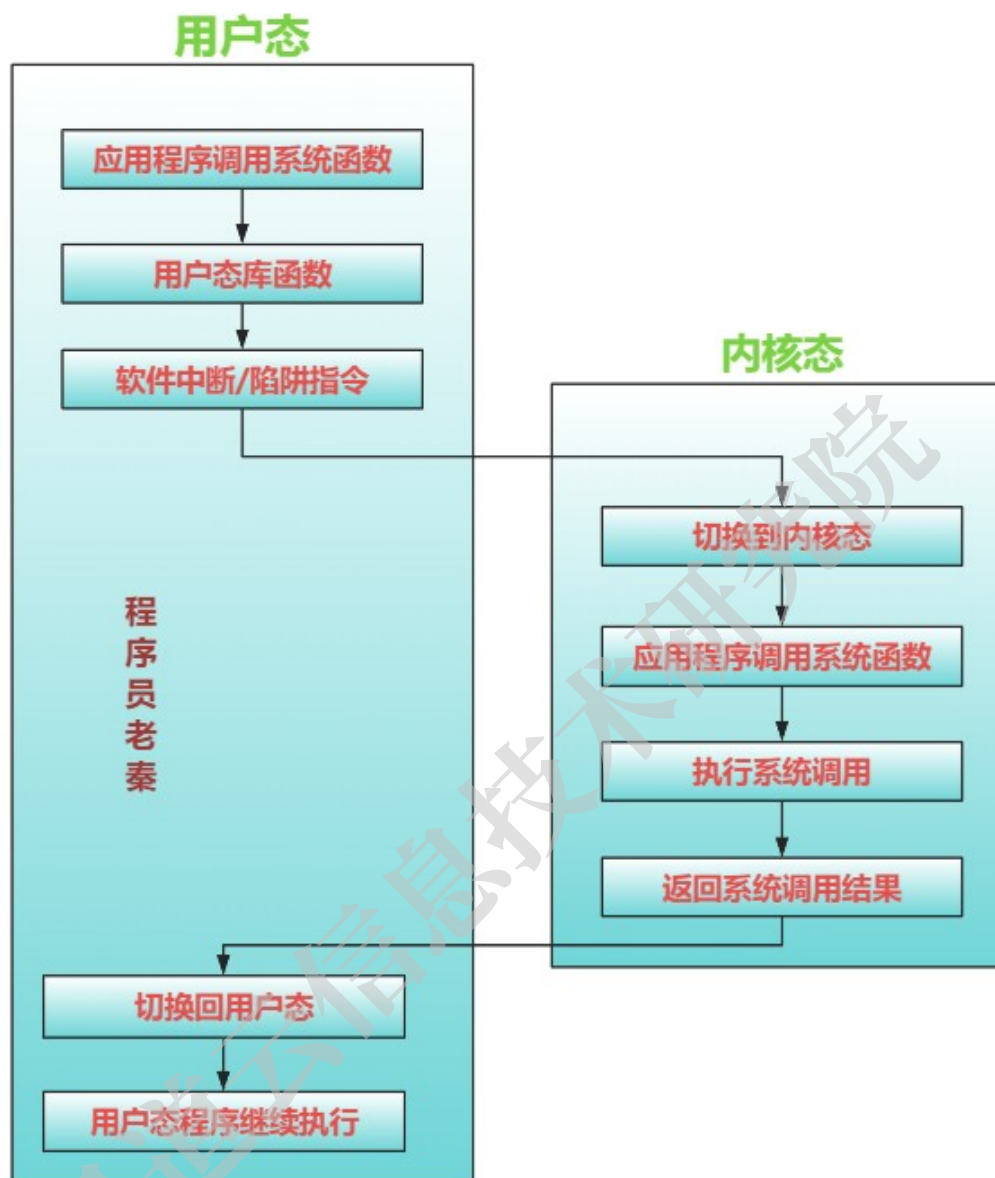
1) 中断机制：当磁盘控制器完成一个 I/O 操作后，会发送一个中断信号给操作系统内核，通知 I/O 操作已完成。

2) 中断处理程序：操作系统内核中的中断处理程序处理中断信号，更新请求状态，并通知相应的应用程序 I/O 操作已完成。

六．中断和系统调用

64. 系统调用是如何工作的？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

用户态与内核态：

用户态是应用程序运行的模式，具有较低的权限。

内核态是操作系统核心运行的模式，具有最高权限。

触发系统调用：

应用程序通过软件中断或陷阱指令触发系统调用。

CPU 切换到内核态，跳转到系统调用入口。

内核处理：

内核找到对应的系统调用处理程序并执行。

系统调用处理程序执行具体的系统操作，并返回结果。

返回用户态：

内核恢复用户态上下文，并切换回用户态。

应用程序接收到系统调用结果后继续执行。

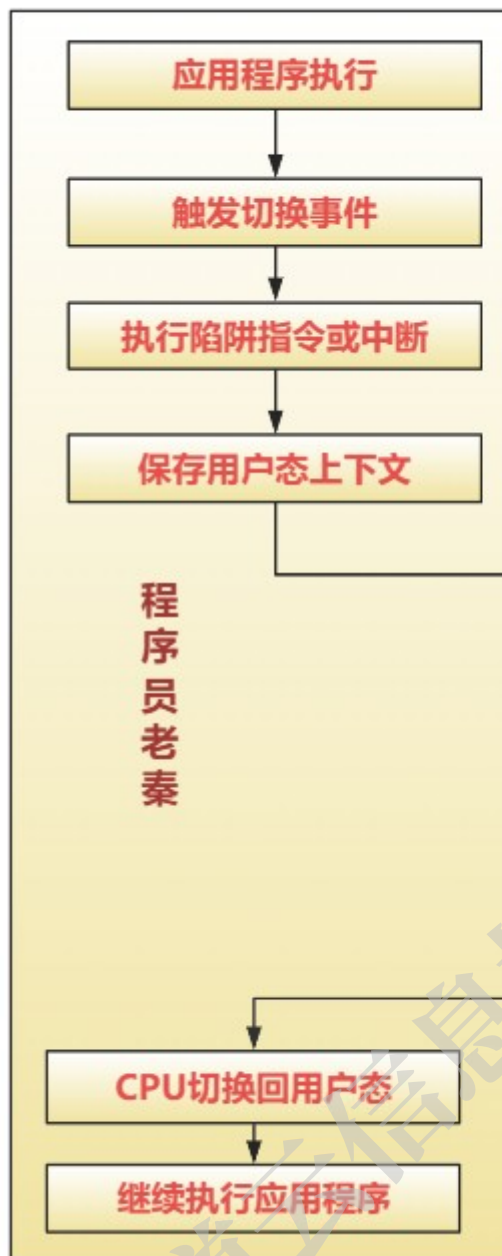
65.用户态与内核态之间如何切换？

图示（方便记忆）：

PS：用户态上下文切换在用户态中进行，保存包括：

- **通用寄存器**：如 RAX, RBX, RCX, RDX 等（x86 架构）或其他架构的类似寄存器。
- **程序计数器（PC）**：保存中断发生时下一条指令的地址。
- **栈指针（SP）**：指向当前栈的顶部，栈中可能保存有更多的局部变量和返回地址。
- **状态寄存器**：如 EFLAGS 或 RFLAGS 寄存器，包含控制和状态信息。
- **浮点寄存器**：如果用户程序使用浮点计算，这些也需要保存。

用户态



内核态



PS：恢复用户态上下文是在内核态进行；

总结（老秦推荐必看，方便理解）：

- (1)切换事件：切换事件可以是系统调用、硬件中断或异常等。
- (2)陷阱指令/中断：通过执行陷阱指令或响应中断信号，CPU 切换到内核态。
- (3)保存上下文：CPU 保存当前用户态的上下文，包括寄存器、程序计数器等。
- (4)查找处理程序：内核查找并执行相应的处理程序。
- (5)恢复上下文：处理程序执行完毕后，恢复之前保存的用户态上下文。
- (6)返回用户态：CPU 切换回用户态，继续执行应用程序。

66.中断的原理及其实现？

总结（老秦推荐必看，方便理解）：

中断的工作原理

(1)设备发出中断信号：

- 1) 过程：硬件设备在需要 CPU 处理时，通过中断控制器发出中断信号。
- 2) 目的：通知 CPU 有事件需要处理。

(2)中断控制器接收中断：

- 1) 定义：中断控制器是负责管理中断信号的硬件组件。
- 2) 过程：中断控制器接收来自多个设备的中断信号，并将其传递给 CPU。

(3)CPU 响应中断信号：

- 1) 过程：CPU 在接收到中断信号后，停止当前程序的执行，并保存当前上下文。
- 2) 作用：确保中断处理程序执行完毕后，能够继续执行中断前的任务。

(4)切换到内核态：

- 1) 过程：CPU 更新程序状态字（PSW），切换到内核态。
- 2) 作用：允许访问内核资源，并执行中断处理程序。

(5)查找中断处理程序：

- 1) 过程：内核根据中断向量表查找对应的中断处理程序。
- 2) 作用：确定中断处理逻辑，并执行相应的操作。

(6)执行中断处理程序：

- 1) 过程：中断处理程序执行具体的处理操作，如读取设备数据、处理异常等。
- 2) 返回结果：处理程序执行完毕后，将结果返回给内核。

(7)恢复上下文：

- 1) 过程：中断处理程序完成后，内核恢复之前保存的上下文。
- 2) 作用：确保 CPU 能够继续执行中断前的任务。

(8)返回之前的任务：

- 1) 过程：CPU 切换回用户态，继续执行中断前的任务。
- 2) 目的：确保系统正常运行，不受中断处理的影响。

67.硬中断和软中断的区别？

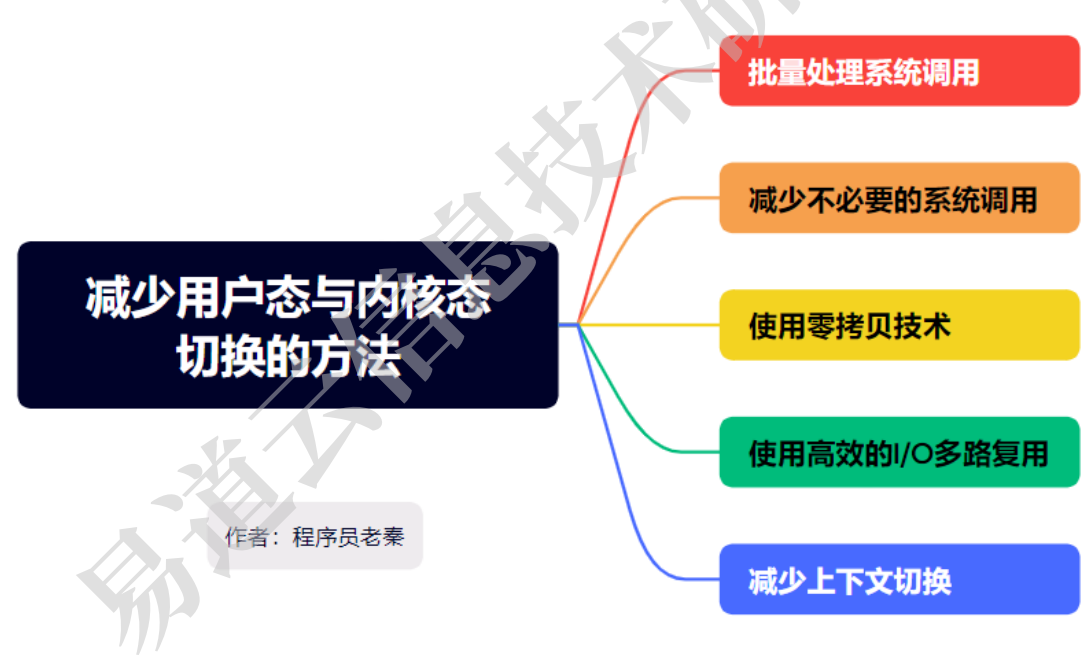
图示（方便记忆）：

特性	硬中断	软中断
触发源	由硬件设备触发（如键盘、网卡等）	由软件或程序触发（如系统调用、异常处理等）
优先级	高优先级	低优先级
实时性	要求高，需快速响应	要求相对低，可延迟处理

处理对象	处理外部设备事件（如输入输出操作）	处理系统内部事件（如系统调用、异常）
响应时间	严格要求，需尽可能短的响应时间	响应时间要求宽松，可稍后处理
处理程序	硬件驱动程序	操作系统内核
示例	键盘按键、网卡数据包、磁盘读写完成	系统调用、除零错误、非法内存访问
中断控制器	需要通过中断控制器传递	通过软件中断机制实现
处理中断过程	由中断控制器中断 CPU 并处理	由操作系统调度并处理
处理逻辑	硬件事件的即时处理	系统事件的调度处理
中断上下文	保存当前执行上下文	不保存上下文
嵌套处理	可以嵌套多个硬中断	不支持嵌套处理

68.如何减少用户态与内核态的切换？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

(1)批量处理系统调用：合并多个系统调用为一个，减少切换次数。例如，使用 sendmsg 和 recvmsg 一次性发送和接收多个消息。

(2)减少不必要的系统调用：优化代码，避免不必要的系统调用。例如，减少文件操作中的 open 和 close 次数。

(3)使用零拷贝技术：减少数据在用户态和内核态之间的复制。例如，使用

sendfile 函数直接发送文件内容。

(4)使用高效的 I/O 多路复用：使用 epoll 等高效的 I/O 多路复用技术，减少阻塞 I/O 带来的切换。例如，在并发网络编程中使用 epoll。

(5)减少上下文切换：优化调度和锁机制，减少任务之间的切换次数。例如，使用锁的粒度减小，并发控制策略优化。

七．调试与错误处理

69.如何使用 GDB 工具调试？



图示（方便记忆）：

总结（老秦推荐必看，方便理解）：

(1)启动和退出：

- 1) 启动：gdb <程序>
- 2) 退出：quit

(2)断点设置：

- 1) 设置断点：break <位置>
- 2) 设置观察点：watch <表达式>
- 3) 清除断点：clear <位置>

(3)程序控制：

- 1) 运行程序：run
- 2) 继续执行：continue
- 3) 单步执行：next（不进入函数）、step（进入函数）
- 4) 执行至函数返回：finish

(4)查看信息：

- 1) 打印变量：print <变量>
- 2) 查看调用栈：backtrace
- 3) 查看局部变量：info locals
- 4) 查看所有线程：info threads

(5)调试线程：

- 1) 切换线程：thread <线程号>
- 2) 所有线程执行命令：thread apply all <命令>
- 3) 查看线程信息：info threads

70.如何检测死锁？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

分析系统日志：检查系统日志中的死锁相关信息。

使用调试工具：如 **GDB** 和 **Valgrind** 来检测死锁。

添加超时机制：在获取锁时添加超时机制，超时后放弃锁定操作。

资源分配图：绘制资源分配图，分析死锁情况。

死锁检测算法：实现死锁检测算法，如银行家算法。

71.Coredump 文件是如何生成的？如何利用它进行错误分析？

总结（老秦推荐必看，方便理解）：

Coredump 文件是当程序发生崩溃时，操作系统生成的内存快照文件。它包含了程序崩溃时的内存内容、寄存器状态和其它调试信息。通过分析 **Coredump** 文件，可以了解程序崩溃的原因，找到并修复代码中的错误。

使用 **GDB** 加载 **Coredump** 文件

72.内存溢出与内存泄漏有什么不同？

图示（方便记忆）：

特性	内存溢出 (Out of Memory)	内存泄漏 (Memory Leak)
定义	程序试图使用超过系统可用内存的情况	程序未能释放已分配的内存，导致可用内存减少
常见原因	数据量过大、无限递归、内存碎片	未调用 free/delete、循环引用、错误管理内存
检测方法	系统监控工具（如 top、htop）、日志记录	内存检测工具（如 Valgrind、AddressSanitizer）
处理方法	优化代码、增加内存、设置内存限制	确保内存释放、使用智能指针、代码审查
结果	程序崩溃或无法继续运行	内存使用量不断增加，最终导致内存耗尽
例子	分配大量内存但系统内存不足	动态分配内存但未释放
监控工具	系统监控工具，如 top、htop	内存泄漏检测工具，如 Valgrind、AddressSanitizer
优化工具和方法	代码优化、增加物理内存或虚拟内存、设置内存限制	使用智能指针、代码审查、确保每次分配内存都能释放
产生问题的后果	程序无法继续运行，导致崩溃	系统资源被耗尽，导致程序性能下降或崩溃
解决办法的难易程度	通过优化和增加资源可以解决	需要详细检查和修改代码，可能需要重构

73.调试方法有哪些？说出优缺点？

图示（方便记忆）：

调试方法对比表格

调试方法	优点	缺点
打印调试	简单易用，无特殊工具要求	信息过多，影响性能，侵入性强
使用调试器	强大的调试功能，非侵入性，实时调试	学习成本高，复杂性高，依赖环境
日志调试	持久化记录，灵活性高，低侵入性	影响性能，日志管理复杂，信息过多
单元测试	自动化测试，提高代码质量，与持续集成结合	额外工作量大，覆盖范围有限，依赖测试环境和框架
静态分析	早期发现问题，自动化工具提高效率，提升代码质量	误报率高，覆盖面有限，依赖静态分析工具
动态分析	发现运行时错误，实时分析，工具支持丰富	性能开销大，复杂性高，误报和漏报
代码审查	多人检查，知识共享，提高代码质量	时间成本高，依赖团队成员，主观性强

74.如何监控服务器 CPU 信息、内存使用率？

图示（方便记忆）：

监控服务器方式表格

命令	优点	缺点
top	实时显示系统资源使用情况，易于使用	信息较多，难以快速找到关键指标，功能简单，缺乏高级分析
htop	友好界面，支持鼠标操作，显示进程树，更多筛选和排序选项	需要安装额外软件，缺乏高级分析和历史数据记录
vmstat	提供详细的系统性能统计信息，支持实时监控	输出信息复杂，需较强的分析能力，不提供图形化界面
iostat	专注于 I/O 设备监控，详细的磁盘 I/O 性能统计	输出信息复杂，需较强的分析能力，不提供图形化界面
free	简单易用，快速查看内存使用情况	功能简单，只提供内存使用信息，不提供实时监控和历史数据记录
sar	全面系统性能统计，支持历史数据记录和分析，灵活输出格式	需要安装额外软件，输出信息复杂，需较强的分析能力
nmon	友好界面，全面性能监控，支持保存监控数据	需要安装额外软件，界面复杂，需一定学习成本
glances	友好界面，全面性能监控，支持多种输出格式和远程监控	需要安装额外软件，界面复杂，需一定学习成本
collectl	全面性能监控，支持历史数据记录和分析，灵活输出格式	需要安装额外软件，输出信息复杂，需较强的分析能力

八．编程实践和工具使用

75.动态库与静态库的区别？

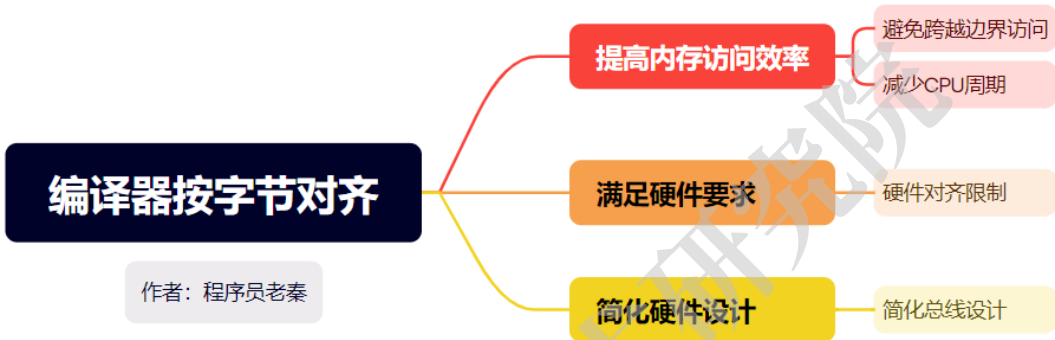
图示（方便记忆）：

特性	静态库 (Static Library)	动态库 (Dynamic Library)
概念	在编译时将库代码复制到可执行文件中	在运行时加载库文件
链接时间	编译链接时	运行时
文件扩展名	.lib (Windows), .a (Linux)	.dll (Windows), .so (Linux)
优点	独立性强，运行时不需要依赖库文件	可执行文件较小，库更新方便
缺点	可执行文件较大，更新库需要重新编译	运行时需要库文件，加载库有性能开销
使用场景	适用于不频繁更新的库，独立性要求高的应用	适用于库更新频繁的应用，需要节省磁盘空间
性能	无需运行时加载，性能较高	需要运行时加载，性能较低

依赖性	无运行时依赖	运行时依赖库文件
示例命令	gcc main.c -L. -lfoo -o main	gcc main.c -L. -lfoo -o main
环境变量	无需设置	需要设置 LD_LIBRARY_PATH 或 PATH 环境变量

76.编译器按字节对齐的原因是什么？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

编译器在生成机器代码时，通常会按照一定的字节对齐规则来安排数据在内存中的存储位置。字节对齐是一种将数据按照特定字节边界存放的方式，如 4 字节、8 字节或 16 字节边界。以下是编译器按字节对齐的主要原因和优势：

1. 提高内存访问效率

1.1 避免跨越边界访问

当数据存储在内存中时，如果数据跨越了多个内存单元的边界（如一个 32 位整数跨越了两个 4 字节的内存单元），则 CPU 需要进行多次内存访问才能读取或写入该数据。这种情况下，内存访问效率会显著降低。通过字节对齐，数据可以存放在单一的内存单元中，避免跨越边界访问，从而提高内存访问效率。

1.2 减少 CPU 周期

对齐的数据访问通常只需一个 CPU 周期，而未对齐的数据访问可能需要多个 CPU 周期。未对齐的访问需要额外的处理步骤，如拆分和合并数据，这会增加 CPU 的工作负担。通过字节对齐，可以简化内存访问操作，减少 CPU 周期，提高程序运行效率。

2. 满足硬件要求

2.1 硬件对齐限制

某些硬件平台（如 ARM、MIPS）要求数据必须按特定的字节边界对齐，否则会引发访问错误或性能下降。这是因为这些硬件在设计上假定了对齐的内存访问，从而简化了内存访问的电路设计。如果数据未对齐，硬件需要额外的逻辑来处理未对齐的数据访问，增加了硬件设计的复杂性和成本。

3. 简化硬件设计

3.1 简化总线设计

对齐的数据更易于硬件设计，尤其是在内存总线的设计上。内存总线通常按照特定的字节宽度设计，如 32 位或 64 位。如果数据未对齐，总线需要额外的逻辑来处理跨越多个总线周期的数据传输，这会增加总线设计的复杂性和成本。通过字节对齐，可以简化总线设计，减少硬件设计的复杂性和成本。

77. 字节序及其对系统的影响？

总结（老秦推荐必看，方便理解）：

字节序（Endianness）是指计算机系统中多字节数据在内存中的存储顺序。它决定了多字节数据（如整数、浮点数）在内存中的排列方式。字节序主要分为大端字节序和小端字节序两种。

1. 字节序类型

1.1 大端字节序（Big-endian）

在大端字节序中，多字节数据的最高字节存储在内存的最低地址。换句话说，高位字节排在低位字节的前面。

1.2 示例

例如，32 位整数 **0x12345678** 在大端字节序中的存储顺序为：

地址 | **0x00** | **0x01** | **0x02** | **0x03** |

值 | **0x12** | **0x34** | **0x56** | **0x78** |

1.3 小端字节序（Little-endian）

在小端字节序中，多字节数据的最低字节存储在内存的最低地址。换句话说，低位字节排在高位字节的前面。

1.4 示例

例如，32 位整数 **0x12345678** 在小端字节序中的存储顺序为：

地址 | **0x00** | **0x01** | **0x02** | **0x03** |

值 | **0x78** | **0x56** | **0x34** | **0x12** |

2. 字节序的影响

2.1 数据交换

字节序在跨平台数据传输中起着重要作用。不同平台可能使用不同的字节序，如果不进行适当的字节序转换，接收方可能无法正确解释发送方的数据。

例如，在网络通信中，网络字节序通常采用大端字节序，因此在传输数据时需要进行字节序转换：

2.1.1 示例

```
#include <arpa/inet.h>

// 主机字节序转换为网络字节序（大端）
uint32_t htonl(uint32_t hostlong);

// 网络字节序转换为主机字节序（小端）
uint32_t ntohl(uint32_t netlong);
```

2.2 性能

字节序也会影响系统性能。在小端字节序系统中，由于低位字节存储在低地址，处理器可以直接访问低地址开始的字节序列，这在某些情况下可以提高性能。而在大端字节序系统中，处理器可能需要额外的操作来访问高位字节，导致性能下降。

2.3 编程复杂性

字节序的差异会增加编程的复杂性。开发人员需要考虑字节序对数据处理的影响，尤其是在跨平台开发时。需要额外处理字节序的转换，以确保数据在不同平台之间的正确传输和解释。

78. 源码到可执行文件的过程是什么？

图示（方便记忆）：



总结（老秦推荐必看，方便理解）：

1. 预处理 (Preprocessing)

预处理是编译的第一个阶段，主要任务是处理以 # 开头的预处理指令。预处理器根据这些指令对源代码进行转换，生成预处理后的代码。

1.1 处理头文件

预处理器会将所有的 #include 指令替换为相应头文件的内容。这使得头文件中的声明和定义能够在源文件中使用。

1.2 宏替换

预处理器会将所有的宏定义 (#define) 替换为相应的宏值。这包括简单的常量替换和复杂的宏函数替换。

1.3 条件编译

预处理器会根据条件编译指令 (如 #ifdef、#ifndef、#if、#endif) 选择性地编译代码。这使得程序可以根据不同的条件编译不同的代码。

2. 编译 (Compilation)

编译是将预处理后的源代码转换为中间代码或目标代码的过程。编译阶段主要包括语法分析、语义分析和生成中间代码。

2.1 语法分析

编译器会检查代码的语法是否正确，确保代码符合编程语言的语法规则。语法分析的结果是语法树 (Syntax Tree)。

2.2 语义分析

编译器会检查代码的语义是否正确，确保变量和函数的使用符合其定义。语义分析的结果是抽象语法树 (Abstract Syntax Tree, AST)。

2.3 生成中间代码

编译器会将抽象语法树转换为中间代码 (Intermediate Code)，中间代码是独立于具体机器的代码形式。常见的中间代码形式有三地址代码 (Three-Address Code) 和中间表示 (Intermediate Representation, IR)。

3. 汇编 (Assembly)

汇编是将中间代码转换为机器代码的过程。汇编器会将中间代码指令逐条翻译为机器指令，生成目标文件 (Object File)。

4. 链接 (Linking)

链接是将多个目标文件和库文件合并为一个可执行文件的过程。链接器会将所有的目标文件和库文件中的符号表 (Symbol Table) 合并，解决符号引用，并生成最终的可执行文件。

4.1 合并文件

链接器会将多个目标文件和库文件中的代码和数据合并为一个文件，并解决所有的外部符号引用。

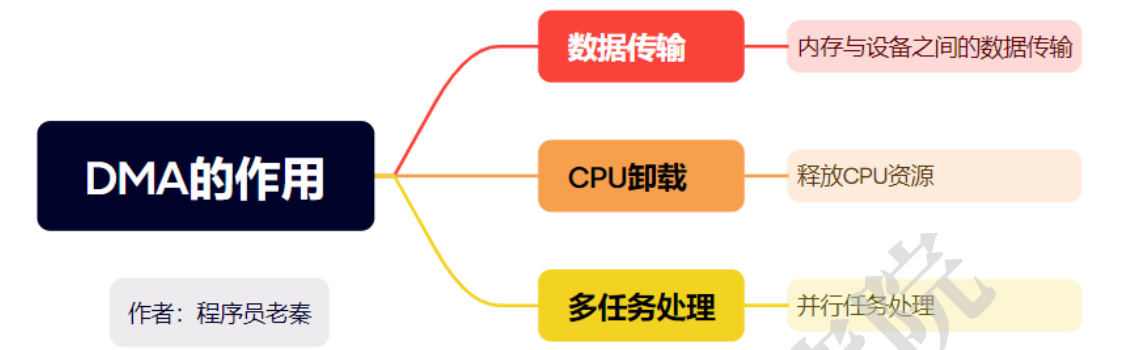
4.2 生成可执行文件

链接器会生成最终的可执行文件，并将所有的符号表、代码段和数据段合并到一个文件中。

九．架构和硬件特性

79.DMA（直接内存访问）的作用是什么？

图示（方便记忆）：



表格展示

DMA 的作用对比表格

作用	描述	优点	应用场景
数据传输	在内存和外设之间进行数据传输，绕过 CPU	提高数据传输效率，减少 CPU 干预	存储设备、网络设备等高效数据传输
CPU 卸载	将数据传输任务交给 DMA 控制器处理，释放 CPU 资源	CPU 可以处理其他任务，提高整体系统效率	多任务处理、高性能计算
多任务处理	允许数据传输和计算任务同时进行，提升系统并行处理能力	提高系统吞吐量和响应速度	实时音频视频处理、网络数据包处理

总结（老秦推荐必看，方便理解）：

(1)DMA 的工作原理

DMA 控制器是实现直接内存访问的核心组件。它负责管理外设和内存之间的数据传输。当外设需要传输数据时，DMA 控制器接管总线控制权，直接访问内存。DMA 控制器在内存和外设之间传输数据，而不需要 CPU 的干预。

(2)DMA 的基本步骤

- 1) 初始化：CPU 设置 DMA 控制器的传输参数，包括源地址、目的地址和传输大小。
- 2) 启动传输：CPU 启动 DMA 控制器。
- 3) 数据传输：DMA 控制器接管总线控制权，直接在内存和外设之间传输数据。
- 4) 传输完成：DMA 控制器发送中断信号通知 CPU 数据传输完成。

80.64 位和 32 位系统的区别是什么？

图示（方便记忆）：

特性	64 位系统	32 位系统
地址空间	支持高达 16 exabytes 的物理和虚拟内存地址空间	最大支持 4GB 内存
数据处理	处理 64 位数据类型，支持更大和更复杂的数据	处理 32 位数据类型，适合较小的数据
性能	在需要大量内存和数据处理的应用中性能更高	性能相对较低，特别是在处理大数据和复杂计算时
兼容性	向下兼容 32 位应用程序	不兼容 64 位应用程序
内存管理	支持更大的物理和虚拟内存，提升内存管理效率	内存管理能力有限，容易达到内存上限
指令集	使用 64 位指令集，处理大数据和地址	使用 32 位指令集，处理较小数据和地址
应用场景	高性能计算、大数据处理、虚拟化、数据库管理	轻量级应用、旧版软件、嵌入式系统
实例	64 位的 Windows、Linux、macOS	32 位的旧版 Windows、某些嵌入式 Linux 系统

第六章 数据结构与算法

链表专题文档阅读指南

背景与目的：

老秦通过深入研究编程平台如牛客、力扣和回顾学生面试经验，了解链表在编程面试中的关键地位。为了助力面试准备，本专题精选了一系列从简单到中等难度的链表题目，并深入探讨了链表的核心理论知识，以实用全面的内容帮助大家系统性学习和实践。

内容亮点：

核心理论：深入讲解链表的结构和类型，强调其在数据处理中的优势。

基础操作：详细指导如何进行链表的创建、遍历、插入和删除操作，确保打下坚实的操作基础。

高级技巧：探索链表反转、排序、合并和环检测等高级操作，涵盖面试中常见的高频题目。

学习策略：

基础先行：从理解链表的基本概念和操作开始，逐步深入。

实践相结合：通过大量练习来巩固理论知识，提升解题能力。

灵活解题：学习不同的解题方法，培养适应多变问题的能力。

专题目的：老秦希望这个专题能成为同学们面试准备的宝贵资源，无论是初学者还是有经验的开发者都能在这里获得全面提升。

老秦将根据最新的面试趋势和题目更新，确保内容和方法始终保持前沿。



链表知识大全

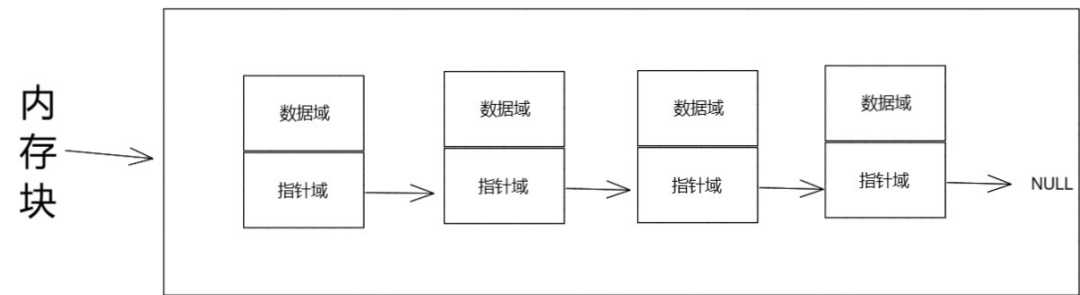
链表的基本概念

链表是一种线性数据结构，由一系列节点组成，每个节点包含数据部分和指向下一个节点的指针。不同于数组的连续存储，链表的元素在内存中可以分散存储。

图示

程序员老秦

节点 = 数据域 + 指针域



代码展示

```

struct ListNode
{
    int val;//数据域

    struct ListNode* next;//指针域
};

```

链表与数组对比的区别

Ps:老秦经验之谈——面试中常询问数组与链表的区别，即 C++ 中的 **vector** 与 **list**

存储方式：数组需要连续的内存空间，而链表的元素可以分散存储。

大小可变：数组的大小固定(vector 是动态数组)，链表的大小可动态调整。

访问时间：数组支持随机访问，时间复杂度为 $O(1)$ ；链表访问任何一个元素需要从头开始遍历，时间复杂度为 $O(n)$ 。

插入和删除：数组插入或删除元素可能需要移动大量元素，时间复杂度为 $O(n)$ ；链表插入或删除操作更高效，时间复杂度为 $O(1)$ ，只需修改指针。



图示

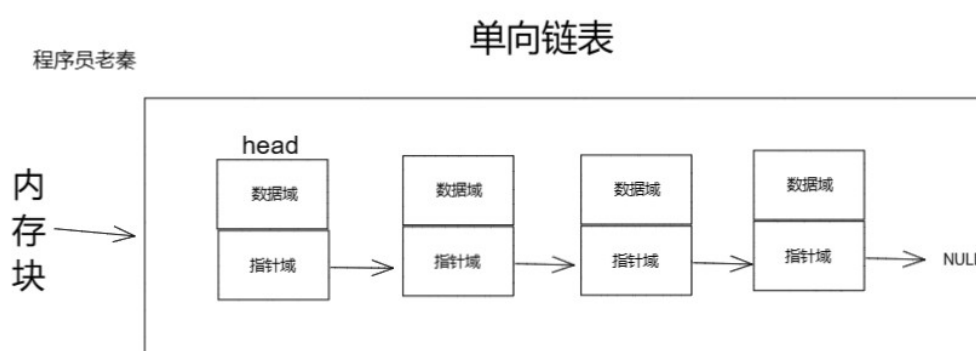
链表的类型

单向链表：每个节点只有指向下一个节点的指针。

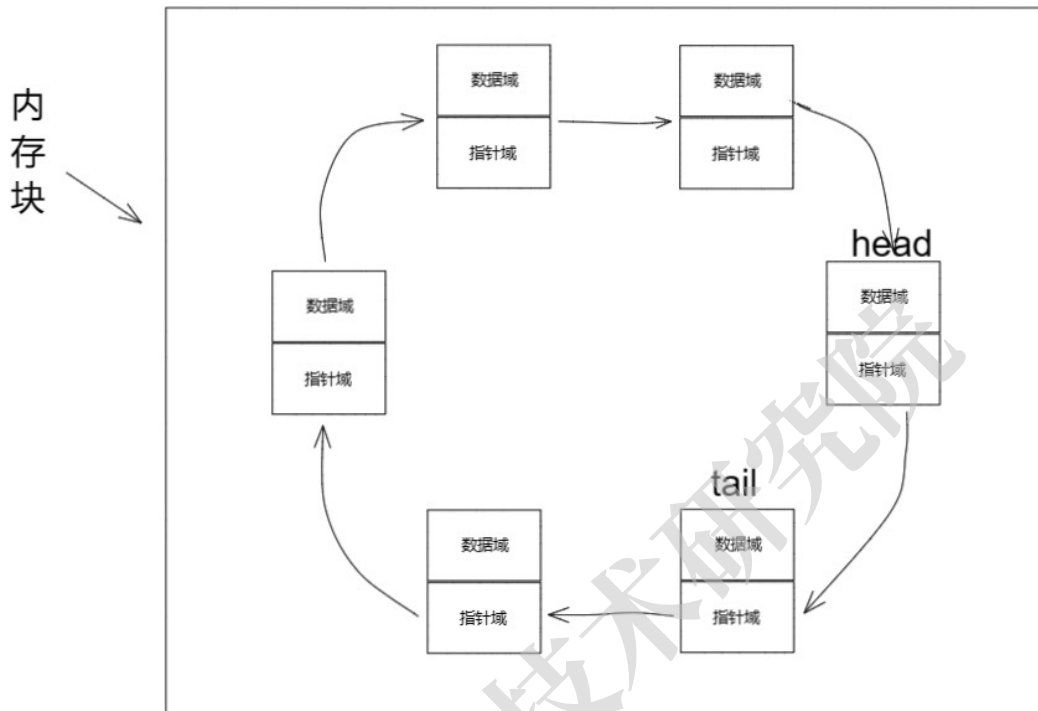
双向链表：每个节点有指向前一个和下一个节点的指针。

循环链表：链表的最后一个节点指向头节点，形成环。

图示



环形链表



代码展示

1. 单向链表

```
struct ListNode {
    int data; // 数据域

    struct ListNode* next; // 指针域，指向下一个节点
};

// 创建单向链表节点的示例函数
struct ListNode* createNode(int data) {
    struct ListNode* newNode = (struct ListNode*)malloc(sizeof(struct
        ListNode));
    if (newNode == NULL) {
        exit(-1); // 内存分配失败
    }
    newNode->data = data;
    newNode->next = NULL;
```

```
    return newNode;
}
```

2.双向链表

```
struct DoublyListNode {
    int data;           // 数据域

    struct DoublyListNode* prev; // 指向前一个节点的指针

    struct DoublyListNode* next; // 指向下一个节点的指针
};

// 创建双向链表节点的示例函数
struct DoublyListNode* createDoublyNode(int data) {
    struct DoublyListNode* newNode = (struct
        DoublyListNode*)malloc(sizeof(struct DoublyListNode));
    if (newNode == NULL) {
        exit(-1); // 内存分配失败
    }
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}
```

3.环形链表

循环链表可以是单向的也可以是双向的。以下是单向循环链表的一个简单示例：

```
struct CircularListNode {
    int data;           // 数据域

    struct CircularListNode* next; // 指向下一个节点的指针
};

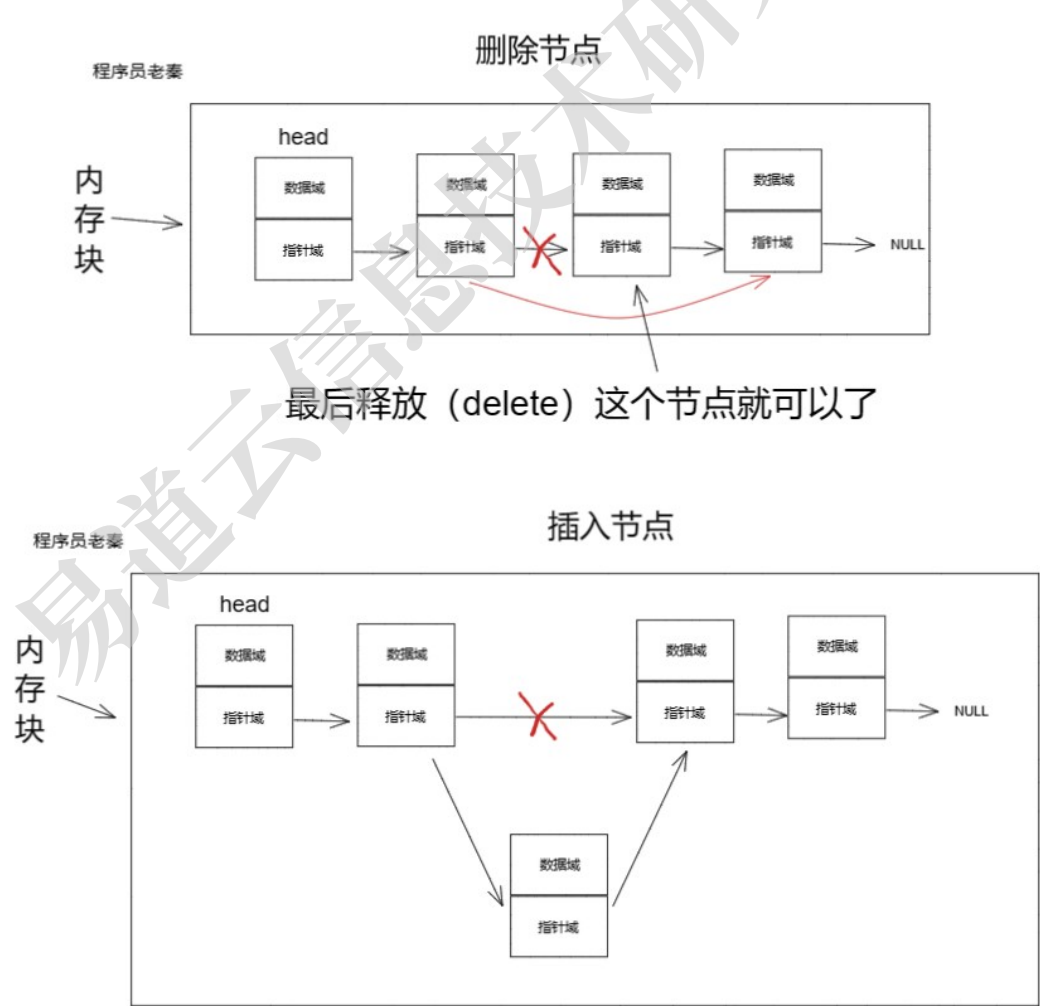
// 创建单向循环链表节点的示例函数
struct CircularListNode* createCircularNode(int data) {
    struct CircularListNode* newNode = (struct
        CircularListNode*)malloc(sizeof(struct CircularListNode));
    if (newNode == NULL) {
```

```
exit(-1); // 内存分配失败
}
newNode->data = data;
newNode->next = newNode; // 初始化时，节点指向自己，形成环
return newNode;
}
```

链表的基本操作

- 遍历链表：按顺序访问链表的每个元素。
- 插入节点：在链表的指定位置插入一个新节点。
- 删除节点：从链表中删除指定的节点。

图示



代码展示

```
#include <iostream>
using namespace std;

// 定义链表节点结构体
struct ListNode {
    int val; // 节点的值

    ListNode* next; // 指向下一个节点的指针

    ListNode(int x) : val(x), next(nullptr) {} // 构造函数
};

// 定义链表类
class LinkedList {
public:
    ListNode* head; // 链表的头节点

    LinkedList() : head(nullptr) {} // 构造函数

    // 遍历链表
    void traverse() {
        ListNode* current = head;
        while (current != nullptr) {
            cout << current->val << " -> ";
            current = current->next;
        }
        cout << "NULL" << endl;
    }

    // 在链表的指定位置插入一个新节点
    void insert(int position, int value) {
        ListNode* newNode = new ListNode(value);

        if (position == 0) { // 在头部插入
            newNode->next = head;
            head = newNode;
        }
        else {
            ListNode* temp = head;
```

```

        for (int i = 0; temp != nullptr && i < position - 1; i++) {
            temp = temp->next;
        }
        if (temp != nullptr) {
            newNode->next = temp->next;
            temp->next = newNode;
        }
    }
}

// 从链表中删除指定位置的节点
void remove(int position) {
    if (head == nullptr) return; // 空链表，直接返回
    ListNode* temp = head;
    if (position == 0) { // 删除头节点
        head = temp->next;
        delete temp;
    }
    else {
        for (int i = 0; temp != nullptr && i < position - 1; i++) {
            temp = temp->next;
        }
        if (temp == nullptr || temp->next == nullptr) return;
        ListNode* next = temp->next->next;
        delete temp->next;
        temp->next = next;
    }
}
};

int main() {
    LinkedList list;

    // 插入节点
    list.insert(0, 1);
    list.insert(1, 2);
    list.insert(2, 3);
    cout << "After insertion: ";
    list.traverse();

    // 删除节点
    list.remove(1);
}

```

```
cout << "After deletion: ";  
list.traverse();  
return 0;  
}
```

1.合并两个有序链表

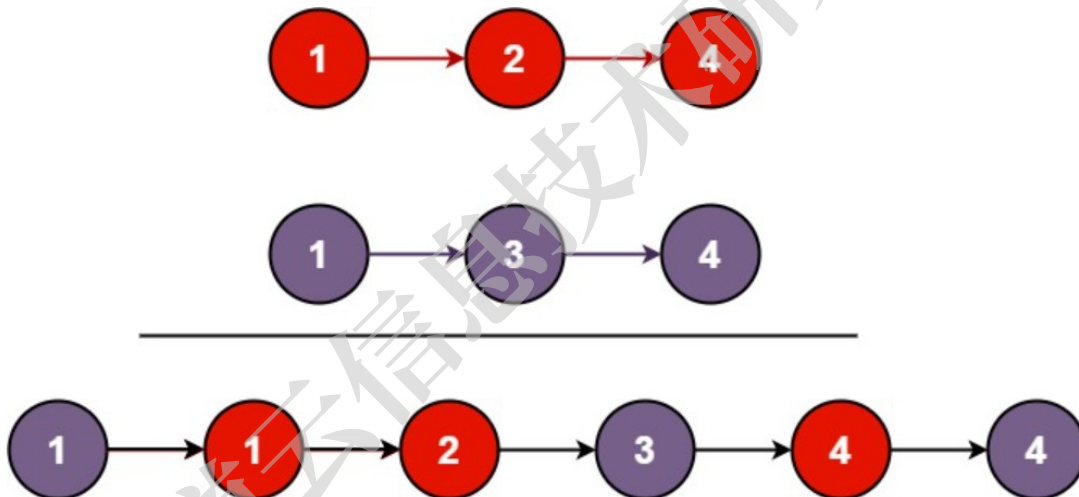
题目描述：

将两个升序链表合并为一个新的升序链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

力扣链接(此链接 ctrl+鼠标左键快速跳转)：

<https://leetcode.cn/problems/merge-two-sorted-lists/>

图示：



示例：

输入：l1 = [1,2,4], l2 = [1,3,4]

输出：[1,1,2,3,4,4]

Ps:在查看解答之前，先独立尝试解题是老秦非常推荐的做法。通过自己的努力解决问题，再对照题解进行学习，能显著促进个人技能的提升和深化理解。进一步提高解题效率和能力。

解题思路：

(1).初始化哨兵节点：

创建一个哨兵节点（也叫哑节点或虚拟头节点）。

Ps:哨兵节点的作用后文会讲解，点击跳转 **ctrl+鼠标左键(哨兵节点)**

(2).遍历两个链表：

同时遍历两个链表，每次比较两个链表当前节点的值，将较小值的节点接入到当前合并链表的末尾，并移动对应链表的指针和合并链表的末尾指针。

(3).处理剩余节点：

当一个链表被完全遍历后，将另一个链表的剩余部分直接接到合并链表的末尾。这一步骤是有效的，因为两个链表都是升序的，所以未遍历完的链表剩余部分也是已经排序的。

(4).返回合并后的链表：

合并过程使用了一个哨兵节点，其下一个节点指向合并后链表的头节点。最后返回哨兵节点的下一个节点作为合并后链表的头节点，并释放哨兵节点所占用的内存。

代码示例：

Ps:为了帮助同学们更好地应对算法笔试和面试，老秦这里准备两种风格的代码示例。一方面，**LeetCode** 风格的代码帮助理解和学习算法的基本思想和解题策略；另一方面，**ACM/公司笔试模式**的代码练习则更贴近实际的编程和解题环境，包括如何处理输入输出、如何组织代码结构、以及考虑代码的完整性和独立性。**特别注意：大部分公司笔试算法是 ACM 模式。**

(1).Leetcode 示例代码风格

```
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        // 创建一个哨兵节点，用于简化插入操作和返回结果。

        // 哨兵节点不存储任何数据，它的下一个节点将是合并后链表的头节点。
        ListNode* sentry_node = new ListNode(0);
        // 使用一个迭代节点指向当前合并链表的末尾。
        ListNode* iteration_node = sentry_node;
        // 当两个链表都非空时，比较当前节点的值，将较小的节点接入合并链表。

        while (list1 && list2) {
            if (list1->val < list2->val) {
                iteration_node->next = list1; // 链接 list1 的当前节点

                list1 = list1->next; // 移动 list1 的指针
            }
            else {
```

```

        iteration_node->next = list2; // 链接 list2 的当前节点

        list2 = list2->next; // 移动 list2 的指针
    }

    iteration_node = iteration_node->next; // 移动迭代节点到合并
链表的末尾
}

// 处理剩余节点，如果 list1 非空，则直接链接 list1，否则链接 list2。

// 此时，至少一个链表已经完全遍历完毕。
iteration_node->next = list1 ? list1 : list2;
// 哨兵节点的下一个节点是合并后的头节点。
ListNode* mergedHead = sentry_node->next;
delete sentry_node; // 释放哨兵节点以避免内存泄露
return mergedHead;
}
};

```

(2).ACM 示例代码风格

```

#include <iostream>
using namespace std;

// 定义链表节点结构
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

// 合并链表的函数
ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
    ListNode* sentry = new ListNode(0); // 哨兵节点
    ListNode* cur = sentry;
    while (list1 && list2) {
        if (list1->val < list2->val) {
            cur->next = list1;

```

```

        list1 = list1->next;
    }
    else {
        cur->next = list2;
        list2 = list2->next;
    }
    cur = cur->next;
}
cur->next = list1 ? list1 : list2;
ListNode* head = sentry->next;
delete sentry; // 释放哨兵节点
return head;
}
int main() {
    // 示例：构建两个链表并调用 mergeTwoLists 函数
    // 这部分通常根据具体问题和数据进行构建
    // 注意：这里仅为示例，实际使用时应根据具体情况构建链表
    // 输出合并后的链表结果
    // 注意：实际使用时，应遍历链表输出每个节点的值，这里未展示完整的遍
    历代码
    cout << "Merged list: ...";
    return 0;
}

```



哨兵节点讲解：

(1).简化代码逻辑：在链表的头部添加哨兵节点后，对链表的插入、删除等操作不需要单

独处理头节点的情况，因为头节点前有一个哨兵节点。这样，操作逻辑对于链表中的所有节点都是一致的，简化了代码的编写和理解。

(2).减少错误：由于哨兵节点的使用减少了对头节点特殊情况的判断，因此在编码过程中出错的可能性也相应降低。特别是在复杂的链表操作中，如反转链表、合并链表等，减少特殊情况的判断可以显著降低逻辑错误的发生。

(3).提高效率：使用哨兵节点后，某些链表操作（如插入和删除）不再需要前置节点的参与，因为即使是对头节点的操作，也有哨兵节点存在，使得操作过程中不需要频繁地检查是否为头节点，从而提高了操作的效率。

(4).统一返回结果：在某些操作中，如在本题的合并链表操作中，使用哨兵节点作为起始节点，无论合并过程如何，最终返回的链表头节点始终是哨兵节点的下一个节点。这样做避免了在返回结果时需要进行额外判断。

(5).保护原始头节点：在进行链表操作时，哨兵节点还可以作为一个额外的层次，保护原始链表的头节点不被修改，尤其是在需要频繁修改链表结构的场景中。

2. 反转链表

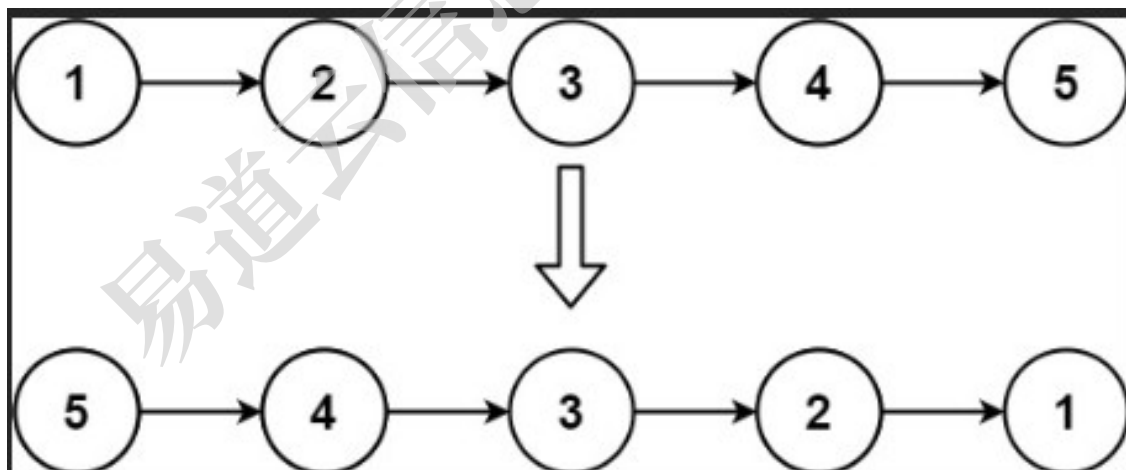
题目描述：

给定单链表的头节点 head，请反转链表，并返回反转后的链表的头节点。

力扣链接(此链接 **ctrl+鼠标左键**快速跳转)：

<https://leetcode.cn/problems/UHnkqH/description/>

图示：



示例：

输入：head = [1,2,3,4,5]

输出：[5,4,3,2,1]

Ps:在查看解答之前，先独立尝试解题是老秦非常推荐的做法。通过自己的努力解决问题，再对照题解进行学习，能显著促进个人技能的提升和深化理解。进一步提高解题效率和能力。

解题思路：

(1) .初始化三个指针;

prev_node : 用于追踪当前节点的前一个节点。在反转过程开始时, 没有节点在当前节点之前, 所以 **prev_node** 初始化为 **nullptr**。

cur_node : 当前操作的节点, 初始时指向链表的头节点。

next_node : 存储 **cur_node** 的下一个节点, 以便在修改 **cur_node** 指向 **prev_node** 后, 还能继续遍历链表。

(2) . 遍历链表:

对链表进行遍历, 直到 **cur_node** 为空。在每次迭代中, 执行节点指向的反转。

(3) . 反转节点指向:

在遍历的每一步中, 首先保存 **cur_node** 的下一个节点到 **next_node**, 以防在反转指向后丢失对链表剩余部分的访问。然后将 **cur_node** 的 **next** 指向 **prev_node**, 完成对当前节点指向的反转。此步骤完成后, **prev_node** 和 **cur_node** 都向前移动一位, 准备下一次迭代。

(4). 更新指针:

将 **prev_node** 更新为 **cur_node**, 即前进一位。

将 **cur_node** 更新为 **next_node**, 即继续遍历链表的下一个节点。

(5). 完成反转:

当 **cur_node** 遍历至链表末尾 (**nullptr**) 时, **prev_node** 会指向原链表的最后一个节点, 即反转后链表的新头节点。返回 **prev_node** 作为新链表的头节点, 完成整个链表的反转。

代码示例 :

Ps:为了帮助同学们更好地应对算法笔试和面试, 老秦这里准备两种风格的代码示例。一方面, **LeetCode** 风格的代码帮助理解和学习算法的基本思想和解题策略; 另一方面, **ACM/公司笔试模式**的代码练习则更贴近实际的编程和解题环境, 包括如何处理输入输出、如何组织代码结构、以及考虑代码的完整性和独立性。特别注意: 大部分公司笔试算法是 **ACM** 模式。

(1).Leetcode 示例代码风格

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        // 初始化前一个节点为 nullptr, 因为反转后的链表头节点将指向
        nullptr.
        ListNode* prev_node = nullptr;
```



```

// 当前节点开始于头节点。
ListNode* cur_node = head;
// 遍历链表直到当前节点为空，这表示已经处理完所有节点。
while (cur_node) {
    // 保存当前节点的下一个节点，这样在反转指针之后仍能继续遍历。
    ListNode* next_node = cur_node->next;
    // 反转当前节点的指针，使其指向前一个节点。
    cur_node->next = prev_node;
    // 前一个节点和当前节点都前进一步。
    prev_node = cur_node;
    cur_node = next_node;
}
// 在链表完全反转后，prev_node 将指向新的头节点。
return prev_node;
};

```

(2).ACM 示例代码风格

```

#include <iostream>
using namespace std;
// 定义链表节点结构体
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};
// 反转链表函数
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr, * cur = head, * next = nullptr;
    while (cur != nullptr) {
        next = cur->next; // 保存当前节点的下一个节点
        cur->next = prev; // 反转当前节点的指向
        prev = cur; // 更新 prev 为当前节点
        cur = next; // 移动到原来链表的下一个节点
    }
    return prev; // 返回新的头节点
}

```

// 创建链表 (简化示例, 实际应用中可能需要更复杂的输入处理)

```
ListNode* createList() {  
    ListNode* head = new ListNode(1);  
    head->next = new ListNode(2);  
    head->next->next = new ListNode(3);  
    return head;}  
// 打印链表
```

```
void printList(ListNode* head) {  
    while (head != nullptr) {  
        cout << head->val << " ";  
        head = head->next;  
    }  
    cout << endl;}  
int main() {
```

```
    ListNode* head = createList(); // 创建链表  
    cout << "Original List: ";  
    printList(head); // 打印原始链表  
    head = reverseList(head); // 反转链表  
    cout << "Reversed List: ";  
    printList(head); // 打印反转后的链表  
    return 0;}
```

3.LRU 算法(高频, 重点理解代码设计)

Ps:这题常考, 老秦推荐这题必须掌握。

题目描述 :

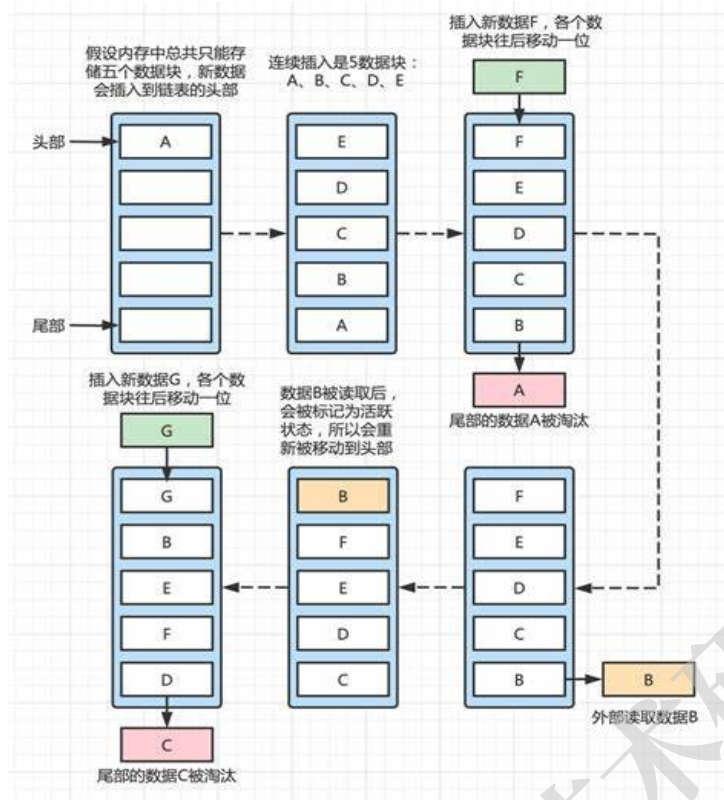
- (1). 请你设计并实现一个满足 **LRU (最近最少使用)** 缓存约束的数据结构。
- (2). `LRUCache(int capacity)` 以正整数作为容量 **capacity** 初始化 LRU 缓存。
- (3). `int get(int key)` 如果关键字 **key** 存在于缓存中, 则返回关键字的值, 否则返回 -1。
- (4). `void put(int key, int value)` 如果关键字 **key** 已经存在, 则变更其数据值 **value**; 如果不存在, 则向缓存中插入该组 **key-value**。如果插入操作导致关键字数量超过 **capacity**, 则应该逐出最久未使用的关键字。
- (5). 函数 **get** 和 **put** 必须以 $O(1)$ 的平均时间复杂度运行。

Ps:依据老秦的经验, 一般算法要求 $O(1)$ 平均时间复杂度, 大概率会用到哈希表。

力扣链接(此链接 **ctrl+鼠标左键** 快速跳转) :

<https://leetcode.cn/problems/lru-cache/description/>

图示:



输入：

["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]

输出：

[null, null, null, 1, null, -1, null, -1, 3, 4]

解释：

```
LRUCache lRUCache = new LRUCache(2);
lRUCache.put(1, 1); // 缓存是 {1=1}
lRUCache.put(2, 2); // 缓存是 {1=1, 2=2}
lRUCache.get(1);    // 返回 1
lRUCache.put(3, 3); // 该操作会使得关键字 2 作废，缓存是 {1=1, 3=3}
lRUCache.get(2);    // 返回 -1 (未找到)
lRUCache.put(4, 4); // 该操作会使得关键字 1 作废，缓存是 {4=4, 3=3}
lRUCache.get(1);    // 返回 -1 (未找到)
lRUCache.get(3);    // 返回 3
```

```
LRUCache.get(4); // 返回 4
```

Ps:在查看解答之前，先独立尝试解题是老秦非常推荐的做法。通过自己的努力解决问题，再对照题解进行学习，能显著促进个人技能的提升和深化理解。进一步提高解题效率和能力。

解题思路：

(1). 定义数据结构：

双向链表节点 (DLinkedNode)：每个节点存储键 (key)、值 (value) 和指向前后节点的指针 (prev、next)。双向链表用于维护缓存项的顺序，从最近使用到最久未使用。

哈希表：映射键 (key) 到其在双向链表中的节点 (DLinkedNode)，实现 **O(1)** 时间复杂度的查找。

(2). 实现 LRUCache 类：

构造函数(LRUCache(int capacity))：

初始化 LRU 缓存的最大容量和当前大小，创建双向链表的伪头部和伪尾部节点，伪头尾节点帮助简化边界条件的处理。

get 操作(int get(int key))：

1. 如果键不存在于哈希表中，返回-1。
2. 如果键存在，哈希表将直接定位到双向链表中的节点，然后将其移动到双向链表的头部，表示最近使用过，并返回节点的值。

put 操作(void put(int key, int value))：

1. 如果键已存在，更新其值，并将该节点移动到双向链表的头部。
2. 如果键不存在，创建一个新的节点，添加到双向链表的头部，并在哈希表中添加键和节点的映射。
3. 如果添加新节点后超过了容量限制，删除双向链表尾部的节点，并在哈希表中移除对应的映射。

(3). 维护双向链表：

添加节点到头部(addToHead)：

将新访问或创建的节点添加到双向链表的头部，表示最近使用。

移除节点(removeNode)：

从双向链表中删除一个节点，无论是因为它被更新、被访问（移动到头部），还是因为缓存逐出（从尾部删除）。

移动节点到头部(moveToHead)：

被访问的节点需要移动到双向链表的头部，表示最近使用。

逐出最久未使用的节点(removeTail)：

当缓存达到容量时，删除双向链表尾部的节点，即最久未使用的缓存项。

关键点

O(1)时间复杂度：

get 和 put 操作都必须在 $O(1)$ 时间内完成。哈希表提供了快速的查找速度，双向链表则允许快速的节点添加、删除操作。

缓存逐出策略：

当缓存达到其容量时，需要逐出最久未使用的项，即双向链表的尾部节点，以确保缓存不会超过设定的容量限制。

代码示例：

Ps: 为了帮助同学们更好地应对算法笔试和面试，老秦这里准备两种风格的代码示例。一方面，**LeetCode** 风格的代码帮助理解和学习算法的基本思想和解题策略；另一方面，**ACM/公司笔试模式**的代码练习则更贴近实际的编程和解题环境，包括如何处理输入输出、如何组织代码结构、以及考虑代码的完整性和独立性。**特别注意：大部分公司笔试算法是 ACM 模式。**

(1).Leetcode 示例代码风格

```
#include <unordered_map>
using namespace std;

// 定义双向链表的节点结构
struct DListNode {
    int key, value;
    DListNode* prev;
    DListNode* next;
    DListNode() : key(0), value(0), prev(nullptr), next(nullptr) {}
    DListNode(int _key, int _value) : key(_key), value(_value),
    prev(nullptr), next(nullptr) {}
};

class LRUCache {
private:
    unordered_map<int, DListNode*> cache;
    DListNode* head;
    DListNode* tail;
    int size;
    int capacity;

    // 在双向链表头部添加节点
    void addToHead(DListNode* node) {
        node->prev = head;
        node->next = head->next;
        head->next->prev = node;
        head->next = node;
    }

    // 删除双向链表中的节点
    void removeNode(DListNode* node) {
        node->prev->next = node->next;
    }
};
```

```

        node->next->prev = node->prev;}

// 移动节点到双向链表头部
void moveToHead(DLinkedNode* node) {
    removeNode(node);
    addToHead(node);}

// 删除双向链表尾部节点，并返回该节点
DLinkedNode* removeTail() {
    DLinkedNode* node = tail->prev;
    removeNode(node);
    return node;}

public:
    LRUCache(int capacity) : capacity(capacity), size(0) {

        // 使用伪头部和伪尾部节点
        head = new DLinkedNode();
        tail = new DLinkedNode();
        head->next = tail;
        tail->prev = head;}

    ~LRUCache() {
        DLinkedNode* curr = head;
        while (curr != nullptr) {
            DLinkedNode* temp = curr;
            curr = curr->next;
            delete temp;}

    int get(int key) {
        if (!cache.count(key)) {
            return -1;}
        DLinkedNode* node = cache[key];

        // 将被访问的节点移动到双向链表头部
        moveToHead(node);
        return node->value;}

    void put(int key, int value) {
        if (!cache.count(key)) {
            DLinkedNode* node = new DLinkedNode(key, value);

            // 添加到哈希表
            cache[key] = node;

            // 添加到双向链表头部
            addToHead(node);

```

```

++size;
if (size > capacity) {
    // 如果超出容量，删除双向链表尾部节点
    DLinkedList* removed = removeTail();

    // 删除哈希表中对应的项
    cache.erase(removed->key);

    // 释放节点空间
    delete removed;
    --size;
}
else {
    DLinkedList* node = cache[key];

    // 更新节点值
    node->value = value;

    // 将节点移动到双向链表头部
    moveToHead(node);
}
}
}

```

(2).ACM 示例代码风格

```

#include <iostream>
#include <unordered_map>
using namespace std;
struct DLinkedList {
    int key, value;
    DLinkedList* prev, * next;
    DLinkedList() : key(0), value(0), prev(nullptr), next(nullptr) {}
    DLinkedList(int _key, int _value) : key(_key), value(_value),
    prev(nullptr), next(nullptr) {}
};
class LRUCache {
private:
    unordered_map<int, DLinkedList*> cache;
    DLinkedList* head, * tail;
    int size;
    int capacity;
    void addNode(DLinkedList* node) {
        node->prev = head;
        node->next = head->next;
        head->next->prev = node;
        head->next = node;
    }
}

```

```

void removeNode(DLinkedNode* node) {
    DLinkedNode* prev = node->prev;
    DLinkedNode* next = node->next;
    prev->next = next;
    next->prev = prev;}
void moveToHead(DLinkedNode* node) {
    removeNode(node);
    addNode(node);}
DLinkedNode* popTail() {
    DLinkedNode* res = tail->prev;
    removeNode(res);
    return res;}
public:
    LRUCache(int capacity) : capacity(capacity), size(0) {
        head = new DLinkedNode();
        tail = new DLinkedNode();
        head->next = tail;
        tail->prev = head;}
    int get(int key) {
        if (cache.find(key) == cache.end()) return -1;
        DLinkedNode* node = cache[key];
        moveToHead(node);
        return node->value;}
    void put(int key, int value) {
        if (cache.find(key) == cache.end()) {
            DLinkedNode* newNode = new DLinkedNode(key, value);
            cache[key] = newNode;
            addNode(newNode);
            ++size;
            if (size > capacity) {
                DLinkedNode* tail = popTail();
                cache.erase(tail->key);
                delete tail;
                --size;}}
        else {
            DLinkedNode* node = cache[key];
            node->value = value;
            moveToHead(node);}}};
int main() {
    LRUCache cache(2); // capacity
    cache.put(1, 1);
    cache.put(2, 2);

```



```
cout << cache.get(1) << endl; // 返回 1

cache.put(3, 3); // 逐出键 2

cout << cache.get(2) << endl; // 返回 -1 (未找到)

cache.put(4, 4); // 逐出键 1

cout << cache.get(1) << endl; // 返回 -1 (未找到)

cout << cache.get(3) << endl; // 返回 3

cout << cache.get(4) << endl; // 返回 4

return 0;}
```

4. 重排链表

题目描述：

给定一个单链表 L 的头节点 head，单链表 L 表示为：

$L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_{n-1} \rightarrow L_n$

请将其重新排列后变为：

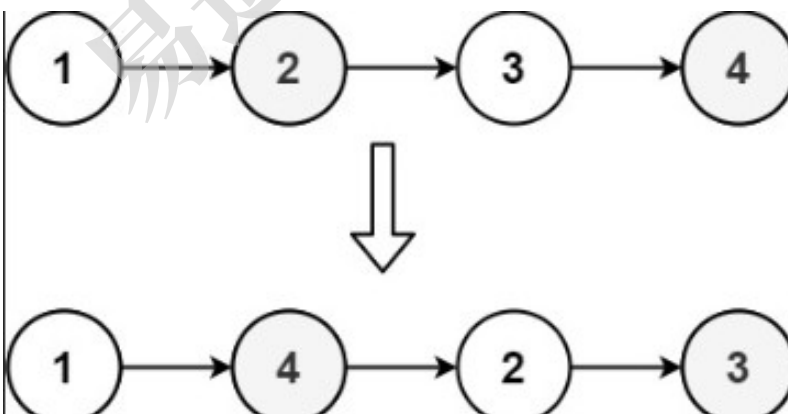
$L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2} \rightarrow \dots$

不能只是单纯的改变节点内部的值，而是需要实际的进行节点交换。

力扣链接(此链接 **ctrl+鼠标左键** 快速跳转)：

<https://leetcode.cn/problems/LGjMqU/description/>

图示：



输入: head = [1,2,3,4]

输出: [1,4,2,3]

Ps:在查看解答之前，先独立尝试解题是老秦非常推荐的做法。通过自己的努力解决问题，

再对照题解进行学习，能显著促进个人技能的提升和深化理解。进一步提高解题效率和能力。

解题思路：

这个题目可以通过两种方法解决：

第一种方法：是借助辅助容器来实现，这种方式比较直观且易于理解。

第二种方法：包括寻找链表的中点、反转链表的后半部分、以及合并链表这几个步骤。这种方法老秦更加推荐，因为它涉及了更多的链表操作技巧，如通过快慢指针寻找链表的中点，使用头插法来反转链表等。这不仅有助于深化对链表结构的理解，还能提升解决复杂问题的能力。

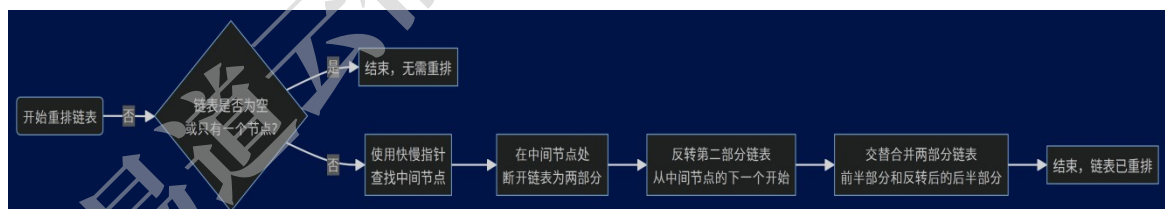
(1). 检查特殊情况：如果链表为空、只有一个节点或只有两个节点，不需要重排，直接返回。

(2). 寻找链表的中点：使用快慢指针法找到链表的中间节点。快指针每次移动两步，慢指针每次移动一步。当快指针无法继续前进时，慢指针所在的位置即为链表的中点。

(3). 反转链表的后半部分：从中点的下一个节点开始，反转链表的后半部分。

(4). 合并链表的前半部分和反转后的后半部分：交替地将前半部分和反转后的后半部分的节点连接起来，完成链表的重排。

代码解题图示



代码示例：

Ps:为了帮助同学们更好地应对算法笔试和面试，老秦这里准备两种风格的代码示例。一方面，**LeetCode** 风格的代码帮助理解和学习算法的基本思想和解题策略；另一方面，**ACM/公司笔试模式**的代码练习则更贴近实际的编程和解题环境，包括如何处理输入输出、如何组织代码结构、以及考虑代码的完整性和独立性。**特别注意：大部分公司笔试算法是 ACM 模式。**

(1).Leetcode 示例代码风格

```
class Solution {  
public:
```

```

void reorderList(ListNode* head) {
    // 特殊情况检查：链表为空、只有一个节点或两个节点，不需要重排
    if (!head || !head->next || !head->next->next) return;

    // Step 1: 查找中点
    ListNode* slow = head;
    ListNode* fast = head;
    while (fast->next && fast->next->next) {
        slow = slow->next; // 慢指针移动一步
        fast = fast->next->next; // 快指针移动两步
    }

    // Step 2: 反转中点之后的链表
    ListNode* prev = nullptr;
    ListNode* curr = slow->next; // 中点的下一个节点作为反转的起点
    while (curr) {
        ListNode* nextTemp = curr->next; // 保存当前节点的下一个节点
        curr->next = prev; // 将当前节点指向前一个节点，完成反转
        prev = curr; // 前移指针
        curr = nextTemp; // 当前节点移至下一个
    }
    slow->next = nullptr; // 断开链表的前半部分和后半部分

    // Step 3: 合并链表
    ListNode* first = head; // 前半部分的起始节点
    ListNode* second = prev; // 反转后的后半部分起始节点
    while (second) {
        ListNode* temp1 = first->next; // 保存前半部分的下一个节点
        first->next = second; // 合并
        second = second->next; // 保存后半部分的下一个节点
        temp1->next = first; // 合并
        first = temp1;
        second = second->next;
    }
}

```

节点

```
first->next = second; // 将前半部分的节点指向后半部分的节点
```

```
second->next = temp1; // 将后半部分的节点指向前半部分的下一个节点
```

一个节点

```
first = temp1; // 前移指针
```

```
second = temp2; // 后移指针}}};
```

(2).ACM 示例代码风格

```
#include <iostream>

// 定义单链表节点结构体
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

class Solution {
public:
    void reorderList(ListNode* head) {
        if (!head || !head->next || !head->next->next) return;

        ListNode* mid = findMiddle(head); // 查找中点
        ListNode* l1 = head;
        ListNode* l2 = mid->next;
        mid->next = nullptr; // 断开链表

        l2 = reverseList(l2); // 反转后半部分

        mergeLists(l1, l2); // 合并链表
    }

    // 查找链表中点
    ListNode* findMiddle(ListNode* head) {
        ListNode* slow = head, * fast = head;
        while (fast->next && fast->next->next) {
            slow = slow->next;
            fast = fast->next->next;
        }
        return slow;
    }
};
```

```

// 反转链表
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr, * curr = head;
    while (curr) {
        ListNode* nextTemp = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nextTemp;
    }
    return prev;
}

// 合并链表
void mergeLists(ListNode* l1, ListNode* l2) {
    while (l1 && l2) {
        ListNode* temp1 = l1->next;
        ListNode* temp2 = l2->next;
        l1->next = l2;

        if (!temp1) break; // 如果 l1 结束，避免添加额外的 l2 节点

        l2->next = temp1;
        l1 = temp1;
        l2 = temp2;
    }
}

int main() {
    // 示例代码: 创建一个链表 1->2->3->4->5
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);
    head->next->next->next = new ListNode(4);
    head->next->next->next->next = new ListNode(5);
    Solution solution;
    solution.reorderList(head);

    // 打印重排后的链表
    ListNode* iter = head;
    while (iter) {
        std::cout << iter->val << " ";
        iter = iter->next;
    }
    std::cout << std::endl;

    // 清理分配的内存
    while (head) {
        ListNode* temp = head->next;

```

```
delete head;
head = temp;}
return 0;}
```

5. 奇偶链表

题目描述：

给定单链表的头节点 `head`，将所有索引为奇数的节点和索引为偶数的节点分别组合在一起，然后返回重新排序的列表。

第一个节点的索引被认为是奇数，第二个节点的索引为偶数，以此类推。

请注意，偶数组和奇数组内部的相对顺序应该与输入时保持一致。

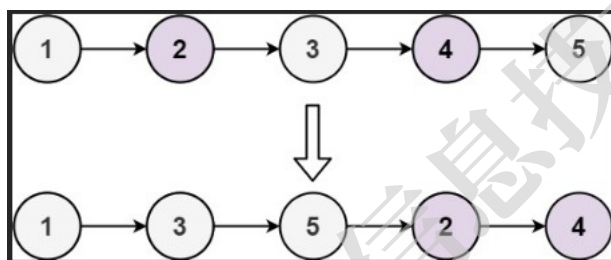
你必须在 $O(1)$ 的额外空间复杂度和 $O(n)$ 的时间复杂度下解决这个问题。

Ps:老秦经验，说明你不能借助其他容器帮你解决这个问题；

力扣链接(此链接 **ctrl+鼠标左键**快速跳转)：

<https://leetcode.cn/problems/odd-even-linked-list/>

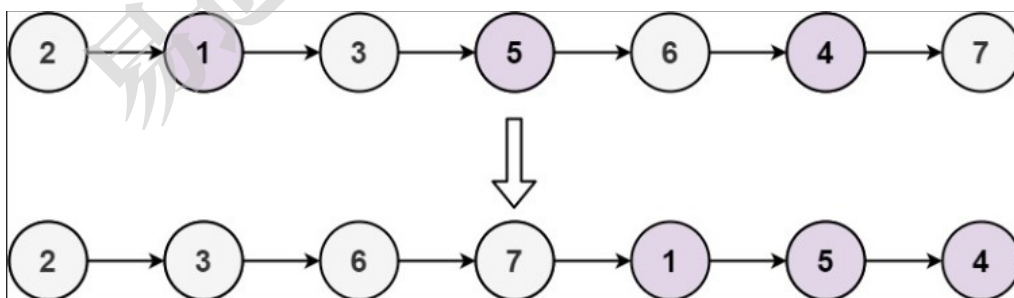
图示 1:



输入: `head = [1,2,3,4,5]`

输出: `[1,3,5,2,4]`

图示 2:



输入: `head = [2,1,3,5,6,4,7]`

输出: `[2,3,6,7,1,5,4]`

Ps:在查看解答之前，先独立尝试解题是老秦非常推荐的做法。通过自己的努力解决问题，再对照题解进行学习，能显著促进个人技能的提升和深化理解。进一步提高解题效率和能力。

解题思路：

解题思路主要分为以下几个步骤，以达到在 $O(1)$ 的额外空间复杂度和 $O(n)$ 的时间复杂度下解决问题的目标：

(1). 初始化指针：

奇数指针 odd：这个指针开始时指向链表的第一个节点，即头节点。它将用于遍历并重新连接链表中所有奇数索引的节点。

偶数指针 even：这个指针开始时指向链表的第二个节点。它将用于遍历并重新连接链表中所有偶数索引的节点。

偶数头指针 evenHead：由于在最后需要将偶数节点链表连接到奇数节点链表的尾部，因此需要一个额外的指针来保存偶数链表的头节点。

(2). 遍历和重连：

使用 odd 和 even 指针遍历整个链表，通过调整 next 指针来重新排列节点。

对于 odd 指针，每次移动都跳过一个节点，即 $odd \rightarrow next = odd \rightarrow next \rightarrow next$ ，这样可以保证 odd 指针始终指向奇数位置的节点。

类似地，对于 even 指针，也是每次跳过一个节点，即 $even \rightarrow next = even \rightarrow next \rightarrow next$ ，保证 even 指针始终指向偶数位置的节点。

在遍历过程中，需要检查 odd->next 和 even->next 是否存在，以防止访问空指针。

(3). 合并链表：

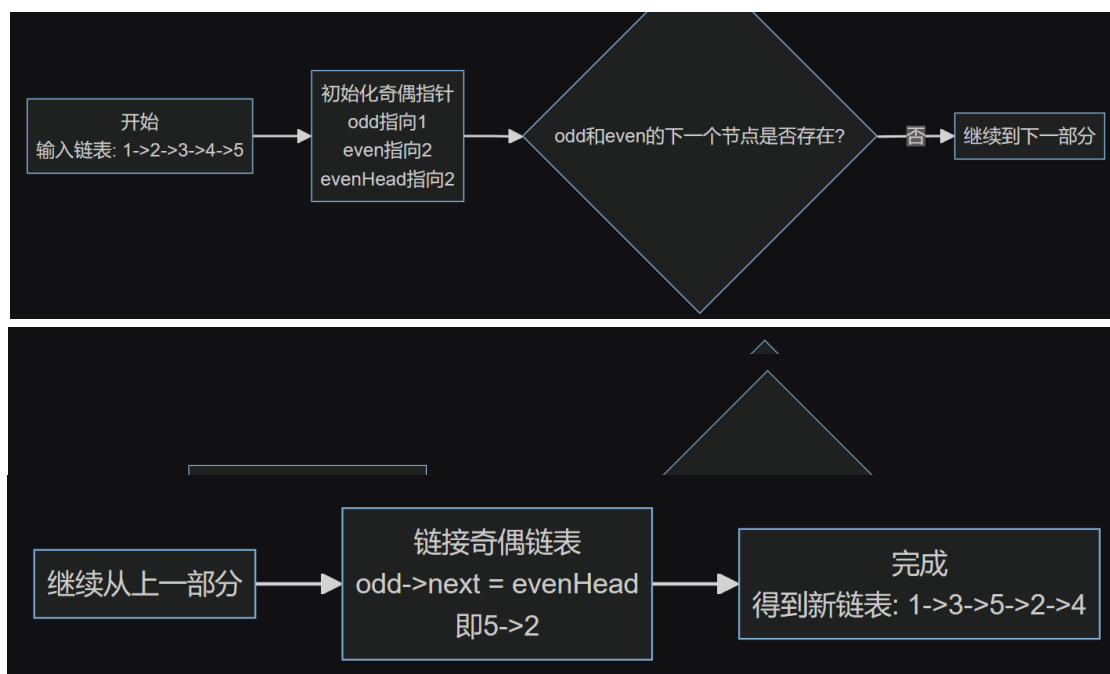
遍历完成后，所有奇数索引的节点已经通过 odd 指针连接在一起，偶数索引的节点通过 even 指针连接在一起。

此时，将奇数链表的尾部 (odd->next) 指向偶数链表的头部 (evenHead)，从而合并两个链表。

(4). 返回结果：

链表的重新排序已完成，返回原始链表的头节点，即 head，因为链表的奇数部分仍然以它开始。

代码解题图示



代码示例：

Ps:为了帮助同学们更好地应对算法笔试和面试，老秦这里准备两种风格的代码示例。一方面，**LeetCode** 风格的代码帮助理解和学习算法的基本思想和解题策略；另一方面，**ACM/公司笔试模式**的代码练习则更贴近实际的编程和解题环境，包括如何处理输入输出、如何组织代码结构、以及考虑代码的完整性和独立性。**特别注意：大部分公司笔试算法是 ACM 模式。**

(1).Leetcode 示例代码风格

```
class Solution {
public:
    ListNode* oddEvenList(ListNode* head) {
        // 如果链表为空，直接返回头指针
        if (head == nullptr) {
            return head;
        }
        // evenHead 保存了链表偶数部分的头节点
        ListNode* evenHead = head->next;
        // odd 和 even 指针分别用来遍历链表的奇数部分和偶数部分
        ListNode* odd = head;
        ListNode* even = evenHead;
        // 遍历链表直到 even 为空或者 even 的下一个节点为空
        // 这确保了我们能够正确处理链表中的每个奇数位置和偶数位置的节点
        while (even != nullptr && even->next != nullptr) {
            // 将 odd 的下一个节点指向 even 的下一个节点（即下一个奇数位置的节点）
            odd->next = even->next;
            // 移动 odd 指针到它的下一个节点
            odd = odd->next;
            // 将 even 的下一个节点指向 odd 的下一个节点（即下一个偶数位置的节点）
            even->next = odd->next;
            // 移动 even 指针到它的下一个节点
```



```
        even = even->next;}

// 在遍历结束后，将奇数部分的最后一个节点连接到偶数部分的头节点
odd->next = evenHead;

// 返回重新排列后的链表的头节点
return head;}};
```

(2).ACM 示例代码风格

```
#include <iostream>
using namespace std;

// 假设 ListNode 结构已经定义
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

// 奇偶重排函数
ListNode* oddEvenList(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode* odd = head, * even = head->next, * evenHead =
even;
    while (even && even->next) {
        odd = odd->next = even->next;
        even = even->next = odd->next;
    }
    odd->next = evenHead;
    return head;
}

// 主函数，用于运行测试案例
int main() {

    // 测试案例和链表构建过程略

    return 0;
}
```

6. 删除链表的倒数第 N 个结点

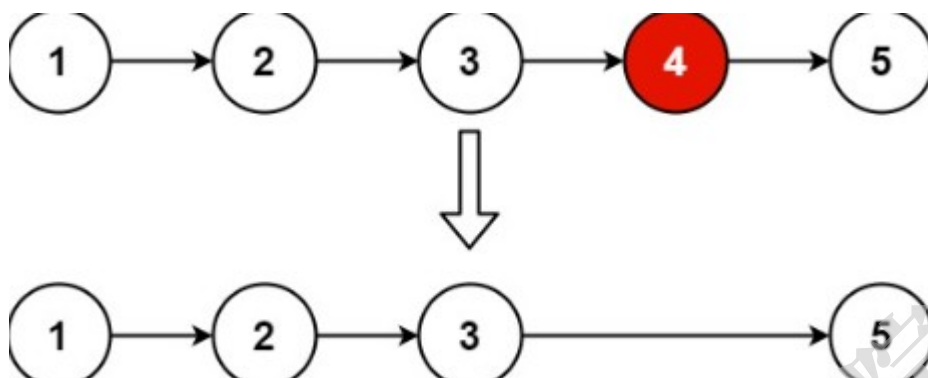
题目描述：

给你一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。

Ps:老秦经验，这个题目很简单，一般人想着两次遍历就可以了，但是一般面试官出这道题，是希望你一次遍历就实现的；

力扣链接(此链接 **ctrl**+鼠标左键快速跳转)：

图示 1:



输入: head = [1,2,3,4,5], n = 2

输出:[1,2,3,5]

解题思路:

解题思路主要分为以下几个步骤，以达到在 $O(n)$ 的时间复杂度下解决问题的目标：

(1). 双指针法

这种方法使用两个指针 **first** 和 **second** 来找到倒数第 n 个节点。具体步骤如下：

步骤 1：初始化两个指针

将 **first** 和 **second** 都指向链表的头节点。

步骤 2：移动 **first** 指针

将 **first** 指针向前移动 n 步。这样，**first** 和 **second** 之间就保持了 n 步的距离。

步骤 3：同时移动两个指针

同时移动 **first** 和 **second** 指针，直到 **first** 指针达到链表末尾。此时，**second** 指针将指向倒数第 $n+1$ 个节点。

步骤 4：删除倒数第 n 个节点

修改 **second** 指针的下一个指针，让它指向倒数第 n 个节点的下一个节点。这样就移除了倒数第 n 个节点。

(2). 特殊情况处理

如果需要删除的是头节点（即倒数第 n 个节点恰好是链表的第一个节点），则在上述过程中，**second** 指针在移动过程中将保持在头节点的位置，因此需要在最后返回 **head->next**，而不是 **head**。

(3). 使用哑节点简化特殊情况

为了避免对头节点的特殊处理，可以在链表的开头添加一个哑节点（**dummy node**），其 **next** 指针指向链表的头节点。这样，即使需要删除头节点，操作也会变得统一。在所有操作完成后，返回哑节点的下一个节点即可，这样可以避免单独处理头节点的情况。

代码示例：

Ps:为了帮助同学们更好地应对算法笔试和面试，老秦这里准备两种风格的代码示例。一方面，**LeetCode** 风格的代码帮助理解和学习算法的基本思想和解题策略；另一方面，**ACM/公司** 笔试模式的代码练习则更贴近实际的编程和解题环境，包括如何处理输入输出、如何组织代码结构、以及考虑代码的完整性和独立性。**特别注意：大部分公司笔试算法是 ACM 模式。**

(1).Leetcode 示例代码风格

```
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode* dummy = new ListNode(0); // 创建哑节点
        dummy->next = head;
        ListNode* first = dummy;
        ListNode* second = dummy;

        // 将 first 指针向前移动 n+1 步
        for (int i = 1; i <= n + 1; i++) {
            first = first->next;
        }

        // 同时移动 first 和 second 指针
        while (first != nullptr) {
            first = first->next;
            second = second->next;
        }

        // 删除倒数第 n 个节点
        second->next = second->next->next;

        ListNode* newHead = dummy->next; // 新的头节点

        delete dummy; // 删除哑节点
        return newHead;
    };
};
```

(2).ACM 示例代码风格

```
#include <iostream>
using namespace std;

// 定义链表的节点结构体
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(NULL) {}
};
```

```

// 函数：删除链表的倒数第 n 个节点，并返回链表的头节点
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode* dummy = new ListNode(0); // 创建一个哑节点
    dummy->next = head;
    ListNode* first = dummy;
    ListNode* second = dummy;

    // 将 first 指针向前移动 n+1 步
    for (int i = 1; i <= n + 1; i++) {
        first = first->next;
    }

    // 同时移动 first 和 second 指针，直到 first 为 NULL
    while (first != NULL) {
        first = first->next;
        second = second->next;
    }

    // 删除倒数第 n 个节点
    ListNode* toBeDeleted = second->next;
    second->next = second->next->next;
    delete toBeDeleted; // 释放内存

    ListNode* newHead = dummy->next;
    delete dummy; // 删除哑节点，释放内存
    return newHead;
}

// 辅助函数：打印链表
void printList(ListNode* head) {
    while (head != NULL) {
        cout << head->val << " ";
        head = head->next;
    }
    cout << endl;
}

// 主函数
int main() {
    // 构造示例链表 1->2->3->4->5
    ListNode* head = new ListNode(1);
    head->next = new ListNode(2);
    head->next->next = new ListNode(3);
    head->next->next->next = new ListNode(4);
}

```

```
head->next->next->next->next = new ListNode(5);
int n;
cout << "Enter n: ";

cin >> n; // 输入要删除的倒数第 n 个节点

// 删除倒数第 n 个节点并打印结果链表

head = removeNthFromEnd(head, n);
printList(head);

// 清理链表内存

while (head != NULL) {
    ListNode* temp = head;
    head = head->next;
    delete temp;
}
return 0;}
```

复杂度分析

时间复杂度： $O(n)$ ，其中 n 是链表的长度。整个方法只需要遍历链表一次。

空间复杂度： $O(1)$ 。我们只使用了常数级别的额外空间。

7. 分隔链表

题目描述：

给你一个链表的头节点 $head$ 和一个特定值 x ，请你对链表进行分隔，使得所有小于 x 的节点都出现在大于或等于 x 的节点之前。

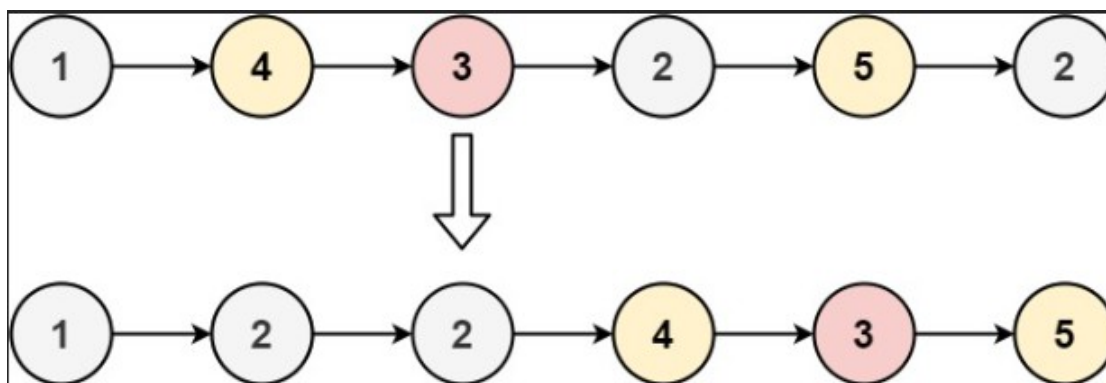
你应当保留两个分区中每个节点的初始相对位置。

Ps:老秦经验：处理这个链表分割的问题时，需要首先意识题目的核心要求——**不仅需要根据一个给定的值 x 将链表分为两部分，还要保持原有的节点顺序不变**；

力扣链接(此链接 **ctrl+鼠标左键**快速跳转)：

<https://leetcode.cn/problems/partition-list/description/>

图示 1:



输入: head = [1,4,3,2,5,2], x = 3

输出: [1,2,2,4,3,5]

解题思路:

要解决这个问题,我们可以使用双指针(或者说双链表)方法,通过创建两个链表:一个用于存放所有小于 x 的节点,另一个用于存放所有大于等于 x 的节点。最后,我们将这两个链表连接起来。

以下是解题步骤的详细说明:

(1).初始化两个链表: lessHead 用于存放小于 x 的节点, greaterHead

用于存放大于等于 x 的节点。同时,我们还需要两个工作指针 lessTail 和 greaterTail 来跟踪这两个链表的末尾,以便我们可以轻松地添加新节点。

(2).遍历原链表: 遍历给定的链表,对于每个节点,根据它的值与 x 的比较结果,将其添加到 lessHead 或 greaterHead 链表的末尾。

(3).连接两个链表: 遍历结束后,我们将小于 x 的链表(如果存在)的末尾连接到大于等于 x 的链表的头部(如果存在)。

(4).处理边界情况: 如果所有节点都小于 x 或所有节点都大于等于 x,我们只需要返回对应的链表即可。

代码示例:

Ps:为了帮助同学们更好地应对算法笔试和面试,老秦这里准备两种风格的代码示例。一方面, **LeetCode 风格**的代码帮助理解和学习算法的基本思想和解题策略;另一方面, **ACM/公司笔试模式**的代码练习则更贴近实际的编程和解题环境,包括如何处理输入输出、如何组织代码结构、以及考虑代码的完整性和独立性。**特别注意:大部分公司笔试算法是 ACM 模式。**

(1).Leetcode 示例代码风格

```
class Solution {
public:
    ListNode* partition(ListNode* head, int x) {
```

```

ListNode lessDummy(0), greaterDummy(0);
ListNode* lessPtr = &lessDummy, * greaterPtr =
&greaterDummy;
while (head) {
    if (head->val < x) {
        lessPtr = lessPtr->next = head;}
    else {
        greaterPtr = greaterPtr->next = head;}
    head = head->next;}

lessPtr->next = greaterDummy.next; // 连接两个链表

greaterPtr->next = nullptr; // 防止形成环

return lessDummy.next;}};

```

(2).ACM 示例代码风格

```

#include <iostream>
using namespace std;
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(nullptr) {}
};

// 分割链表函数
ListNode* partition(ListNode* head, int x) {

    ListNode lessHead(0), greaterHead(0); // 创建两个哑节点
    ListNode* lessTail = &lessHead, * greaterTail = &greaterHead;
    while (head != nullptr) {
        if (head->val < x) {
            lessTail->next = head;
            lessTail = head;}
        else {
            greaterTail->next = head;
            greaterTail = head;}
        head = head->next;}

    lessTail->next = greaterHead.next; // 连接两个部分

    greaterTail->next = nullptr; // 防止形成循环链表

    return lessHead.next;}

// 用于打印链表

```

```

void printList(ListNode* head) {
    while (head != nullptr) {
        cout << head->val << " ";
        head = head->next;
    }
    cout << endl;
}
int main() {
    int n, val, x;

    cin >> n; // 读入节点总数

    cin >> x; // 读入特定值 x

    ListNode* head = nullptr, * tail = nullptr;

    // 构建链表
    for (int i = 0; i < n; ++i) {
        cin >> val;
        if (head == nullptr) {
            head = tail = new ListNode(val);
        }
        else {
            tail->next = new ListNode(val);
            tail = tail->next;
        }
    }

    head = partition(head, x); // 分割链表

    printList(head); // 打印结果链表

    // 释放链表内存
    while (head != nullptr) {
        ListNode* temp = head;
        head = head->next;
        delete temp;
    }
    return 0;
}

```

复杂度分析

时间复杂度： $O(N)$ ，其中 N 是链表中的节点数量。我们只需要遍历整个链表一次。

空间复杂度： $O(1)$ 。我们只使用了固定的额外空间（即两个哑节点和一些临时指针）。尽管我们构建了两个新链表，但这些链表中的节点仍是原链表中的节点，因此没有使用额外的空间。

8. 环形链表

题目描述：

给定一个链表的头节点 $head$ ，返回链表开始入环的第一个节点。如果链表无环，则

返回 null。

如果链表中有某个节点，可以通过连续跟踪 **next** 指针再次到达，则链表中存在环。为了表示给定链表中的环，评测系统内部使用整数 **pos** 来表示链表尾连接到链表中的位置（索引从 0 开始）。如果 **pos** 是 -1，则在该链表中没有环。

注意：**pos** 不作为参数进行传递，仅仅是为了标识链表的实际情况。

不允许修改链表。

Ps:老秦经验：推荐快慢指针，使用哈希表的方法相较于快慢指针法，其优点在于实现简单、直观；

但其缺点也非常明显：

空间复杂度较高：哈希表需要额外的空间来存储已经访问过的节点，最坏的情况下（没有环），空间复杂度为 $O(n)$ ，其中 n 是链表中的节点数。这对于空间敏感的应用可能是不可接受的。

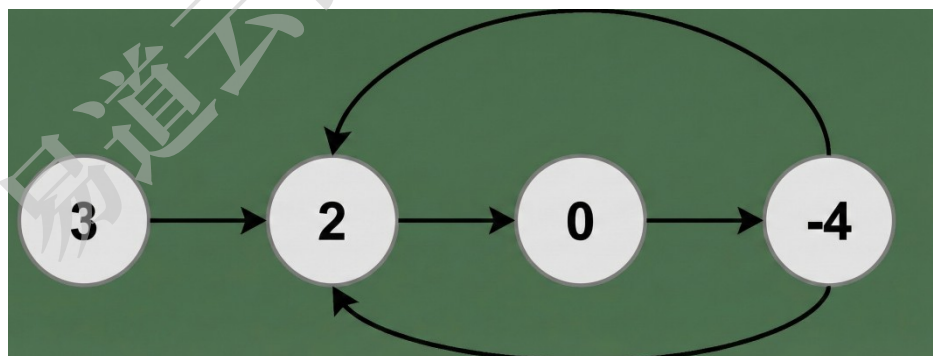
效率问题：虽然哈希表的查找和插入操作平均时间复杂度是 $O(1)$ ，但在极端情况下（如哈希冲突较多时），这些操作的时间复杂度可能会退化。此外，哈希表的管理（如扩容）也会带来额外的时间开销。

当链表不成环时，使用哈希表的方法确实会遍历整个链表直到 **nullptr**，并且最终返回 **nullptr** 以表示链表中没有环。这种方法的直观性和简便性使其在一些场景下非常有用，尤其是当空间复杂度不是主要考虑因素时。

力扣链接(此链接 ctrl+鼠标左键快速跳转)：

<https://leetcode.cn/problems/linked-list-cycle-ii/description/>

图示 1:



输入：head = [3,2,0,-4], pos = 1

输出：返回索引为 1 的链表节点。

解题思路：

使用快慢指针（Floyd's Tortoise and Hare Algorithm）解决链表中的环问题和找到环入口的详细思路分为两个主要阶段：**环的检测**和**找到环入口**。

环的检测

初始化：创建两个指针，slow 和 fast，它们都从链表的头节点 head 开始。

移动指针：在每一轮迭代中，**slow 指针移动一步，fast 指针移动两步**。这个过程一直继续，直到 fast 指针到达链表的尾部（表示链表无环），或者 fast 指针追上 slow 指针（表示链表中存在环）。

检测环：如果 fast 指针和 slow 指针相遇（即 $fast == slow$ ），这意味着链表中存在环。如果 fast 指针到达链表的末尾（ $fast == nullptr$ 或 $fast->next == nullptr$ ），则链表中不存在环。

找到环入口

一旦检测到链表中存在环，就可以使用下面的方法找到环的入口：

重新定位：将 fast（或 slow）指针重新定位回链表的头节点 head，而另一个指针保留在它们首次相遇的位置。

同速移动：然后，以相同的速度（每次一步）移动两个指针，即 fast 和 slow 指针同时各自向前移动。

相遇点即环入口：当这两个指针再次相遇时，它们相遇的节点就是环的入口节点。这是因为从头节点到环入口的距离等于从首次相遇点到环入口的距离。

老秦用一点数学知识解释为什么这个方法有效



这个方法之所以有效，是基于以下数学原理：设链表头到环入口的距离为 A ，环入口到首次相遇点的距离为 B ，首次相遇点回到环入口的距离为 C 。在首次相遇时，慢指针走过的距离是 $A+B$ ，快指针走过的距离是 $A+B+n(B+C)$ （ n 是快指针在环内多转的圈数）。因为快指针走的距离是慢指针的两倍，所以有 $2(A+B) = A+B+n(B+C)$ 。简化这个等式，我们可以得到 $A = C + (n-1)(B+C)$ ，这意味着从链表头到环入口的距离等于首次相遇点绕环 $n-1$ 次后回到环入口的距离。因此，当一个指针从链表头出发，另一个指针从首次相遇点出发，它们以相同的速度移动时，它们相遇的点就是环的入口。

通过这种方法，我们不仅能检测出链表是否有环，还能找到环的确切入口，而且整个过程的空间复杂度为 $O(1)$ ，因为我们只需要两个指针。

代码示例：

Ps:为了帮助同学们更好地应对算法笔试和面试，老秦这里准备两种风格的代码示例。一方面，**LeetCode 风格的代码帮助理解和学习算法的基本思想和解题策略**；另一方面，**ACM/公司笔试模式的代码练习则更贴近实际的编程和解题环境**，包括如何处理输入输出、如何组织代码结构、以及考虑代码的完整性和独立性。**特别注意：大部分公司笔试**

算法是 **ACM** 模式。

(1).Leetcode 示例代码风格

```
class Solution {
public:
    ListNode* detectCycle(ListNode* head) {
        ListNode* slow = head;
        ListNode* fast = head;
        // 1. 寻找相遇点
        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;

            // 如果相遇，说明有环
            if (slow == fast) {
                // 2. 寻找环入口
                ListNode* index1 = fast; // 从相遇点开始
                ListNode* index2 = head; // 从头结点开始
                while (index1 != index2) {
                    index1 = index1->next;
                    index2 = index2->next;
                }
                return index1; // 相遇点即为环入口
            }
        }
        return nullptr; // 无环
    }
};
```

(2).ACM 示例代码风格

```
#include <iostream>
#include <vector>
using namespace std;

struct ListNode {
    int val;
    ListNode* next;
    ListNode(int x) : val(x), next(NULL) {}
};

// 核心算法函数
ListNode* detectCycle(ListNode* head) {
    ListNode* slow = head;
```

```

ListNode* fast = head;
while (fast != nullptr && fast->next != nullptr) {
    slow = slow->next;
    fast = fast->next->next;
    if (slow == fast) {
        ListNode* index1 = fast;
        ListNode* index2 = head;
        while (index1 != index2) {
            index1 = index1->next;
            index2 = index2->next;
        }
        return index1;
    }
}
return nullptr;
}

```

```

int main() {
    int n, pos, val;
    // 输入节点数量
    if (!(cin >> n)) return 0;

    vector<ListNode*> nodes;
    ListNode* head = nullptr;
    ListNode* tail = nullptr;

    // 构建链表
    for (int i = 0; i < n; ++i) {
        cin >> val;
        ListNode* newNode = new ListNode(val);
        nodes.push_back(newNode);
        if (head == nullptr) {
            head = newNode;
            tail = newNode;
        }
        else {
            tail->next = newNode;
            tail = newNode;
        }
    }

    // 输入 pos , 构建环 (pos -1 表示无环)
    cin >> pos;

```

```

if (pos >= 0 && pos < n) {
    tail->next = nodes[pos];
}

ListNode* result = detectCycle(head);

if (result != nullptr) {
    // 输出口节点的值，或者你可以输出由于是第几个节点
    cout << "Tail connects to node index " << pos << " (val: " << result->val << ")"
    << endl;
}
else {
    cout << "no cycle" << endl;
}

// 注意：ACM 模式下带环链表的内存释放比较麻烦，需要避免死循环，此处省略释放代码
return 0;
}

```

复杂度分析

时间复杂度： $O(N)$ ， N 为链表节点数，最多若干次遍历，数量仍为线性。

空间复杂度： $O(1)$ ，仅使用常数个指针。

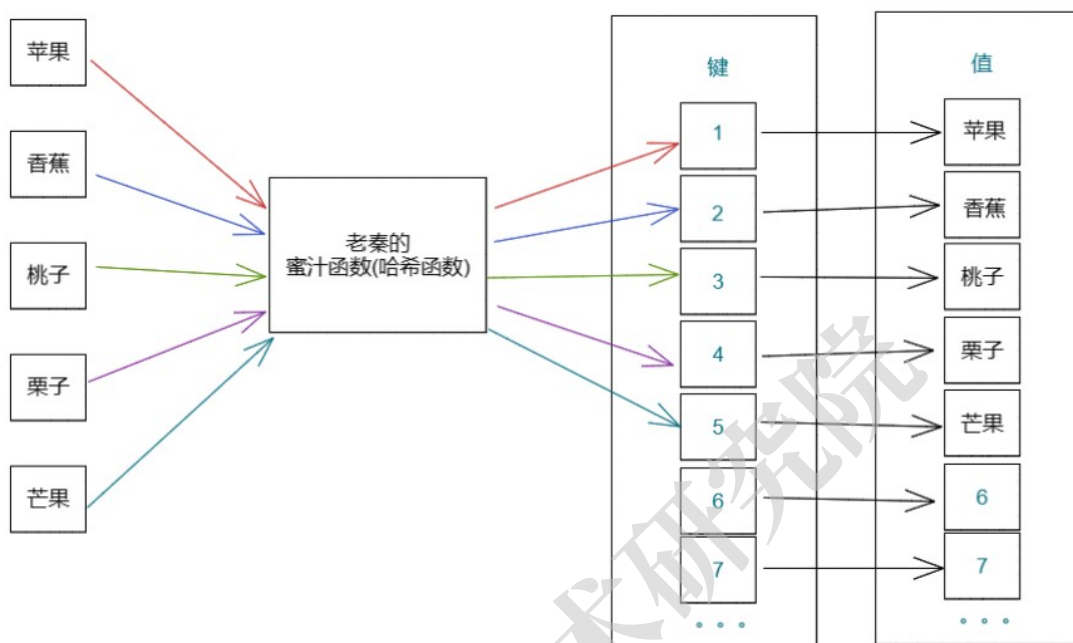
哈希表专题文档阅读指南

哈希表 (Hash Table) 是一种基于键值对 (Key value) 直接访问的数据结构。它通过“映射函数”把关键码值映射到表中一个位置来访问记录，以加快查找的速度。这个映射函数称为哈希函数，存放记录的数组称为哈希表 (或哈希图)。

所有图解链接(<https://note.youdao.com/s/H5rAGncH>)

什么？阿巴阿巴看不懂？没事，老秦给同学们上图解：

图解：



Ps:这个时候可能有小白说了：“咦，这个哈希表不就是脱了裤子放屁吗，多此一举；如果输入的数据本来就是唯一的，使用哈希表不就是多此一举吗？”老秦这里说一下哈，这是一个常见的误解。实际上，哈希表的真正价值不仅仅在于处理唯一性，更重要的是它能够极大提高数据处理的效率。即使数据项本身是唯一的，哈希表依然能通过优化数据的存取路径，显著减少查找时间，特别是在处理大规模数据集时。

详细解释：

哈希函数设计的目的是：将输入（如字符串、数字等）转换成一个整数，这个整数用作在哈希表中的索引。关于哈希函数输出是否为固定长度的索引，这取决于哈希表的实现和哈希函数的设计。以下是一些关键点：

输出范围与哈希表的大小：

哈希函数通常设计为将任意长度的输入映射到一个固定范围的整数值上。这个范围通常与哈希表的槽数（或大小）直接相关。例如，如果哈希表有 N 个槽，那么哈希函数的输出通常是介于 0 和 $N-1$ 之间的整数。

模运算的使用：

在许多实际实现中，哈希函数会生成一个整数，然后通过模运算（例如： $\text{hashValue} \% \text{tableSize}$ ）来确保结果索引落在哈希表的有效范围内。这样的模运算确保了无论哈希函数原始输出如何，最终的索引总是固定长度的，即它总是在哈希表的索引范围内。

固定长度的输出：

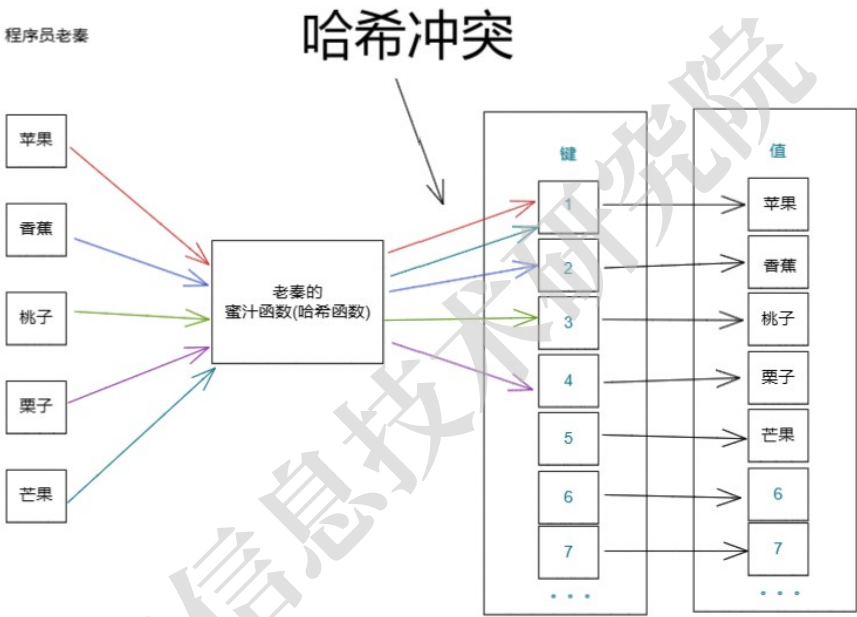
尽管哈希函数处理的输入数据大小可以变化，但输出（即哈希表的索引）的长度是固定的，因为它必须匹配哈希表的当前大小。这意味着无论输入数据的量如何，哈希函数总是输出一个可以直接用作数组索引的整数。

哈希表的核心原理：

哈希函数： 哈希函数将输入（比如一个字符串或者一个数值）转换成一个整数，这个整数表示在哈希表中的索引。

处理冲突： 由于不同的输入可能会产生相同的输出，即所谓的**哈希碰撞**，需要有方法来处理这种情况。常用的方法包括拉链法和开放地址法（如线性探测、二次探测和双重散列）。

话不多说，上图：
图解：



解决哈希碰撞的方法：

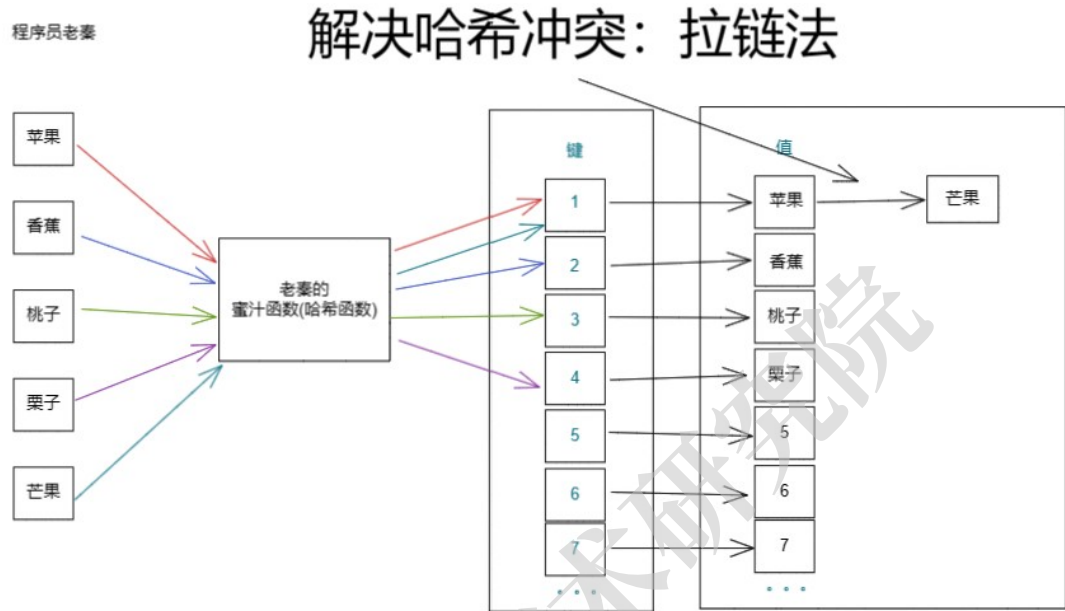
1. 拉链法 (Chaining)： 每个哈希表的槽位关联一个链表，所有映射到同一槽位的元素都会被存储在这个链表中。

拉链法实现方式对比表格

实现方法	插入方式	优点	缺点
链表头插法	将新元素添加到链表的头部。	插入速度快，操作简单。	频繁访问时，较早插入的元素查找效率可能较低。
链表尾插法	将新元素添加到链表的尾部。	保持了元素插入的顺序，可能有利于某些	插入操作涉及到遍历整个链表，效率较低。

	查询。	
--	-----	--

图解：



2. 开放寻址法 (Open Addressing)：当发生碰撞时，寻找下一个空的槽位。常见的策略有线性探测、二次探测和双重散列。

开放寻址法探测技术对比表格

探测技术	描述	优点	缺点
线性探测	逐个检查哈希表的槽位，直到找到空位。	实现简单。	容易产生聚集现象，随着填充率的增加，性能可能显著下降。
二次探测	探测距离按二次方增长，如 1, 4, 9 等。	可以减轻线性探测的聚集问题。	探测距离的增加可能导致空槽位的跳过，降低效率。
双重散列	使用第二个哈希函数来定义探测序列。	减少冲突和聚集，提供更均匀的数据分布。	实现复杂，计算成本高于线性和二次探测。

图解：

假设我们有一个哈希表，共有 7 个槽位，我们将要插入几个元素，这些元素的首次哈希结果指向相同的索引，我们将看到如何使用线性探测来解决这种冲突。

哈希表初始状态

槽位	0	1	2	3	4	5	6
值							

第一步：插入第一个元素 A

哈希函数指定元素 A 应该放在槽位 2。

槽位	0	1	2	3	4	5	6
值			A				

第二步：插入第二个元素 B

哈希函数同样指定元素 B 应该放在槽位 2，但已经被 A 占用，因此使用线性探测方法向后查找下一个空位。

槽位	0	1	2	3	4	5	6
值			A	B			

第三步：插入第三个元素 C

哈希函数再次指定元素 C 应该放在槽位 2，发现 2 和 3 已经被占用，继续线性探测。

槽位	0	1	2	3	4	5	6
值			A	B	C		

第四步：插入第四个元素 D

哈希函数还是指向槽位 2，此时需要继续探测，找到第一个空位。

槽位	0	1	2	3	4	5	6
值			A	B	C	D	

哈希冲突解决方法对比表格

哈希冲突解决方法	优点	缺点
----------	----	----

拉链法	2. 弹性扩展：链表理论上可以无限长，只受限于可用内存，能处理大量冲突。 2. 实现简单：链表的数据结构简单，易于实现和维护。	1. 不一致的查找时间：查找效率取决于链表的长度，在最坏情况下可能变成线性时间，即 $O(n)$ 。 2. 内存使用不均：部分槽位可能对应长链表，而其他槽位则可能空闲。
开放寻址法	2. 内存利用率高：不需要额外空间存储指针或链表。 2. 缓存友好：数据连续存储，有利于缓存优化。	1. 填充因子敏感：填充因子高时，性能显著下降。 2. 删除操作复杂：删除元素后，不能简单地将槽位标记为空，否则会破坏探测序列。通常标记为“已删除”，并在新插入时重用这些槽位。

干货(C++的哈希表底层知识，面试加分)

阈值和负载因子 (Load Factor)

负载因子：哈希表的负载因子定义为当前存储在哈希表中的元素数量与哈希表桶(bucket)数量的比例。负载因子是衡量哈希表满的程度的一个指标。公式为：

$$\text{Load Factor} = \frac{\text{Number of elements in the hash table}}{\text{Number of buckets}}$$

阈值 (Threshold)：是控制哈希表何时扩容的负载因子的最大值。当**哈希表的负载因子超过这个阈值时，哈希表将自动增加桶的数量 (rehashing)**，以分散现有的元素并减少碰撞，维持操作的效率。

扩容 (Rehashing)：当**元素的插入导致负载因子超过预定的阈值时，哈希表需要进行扩容。这个过程包括创建一个更大的哈希表并重新计算每个元素的哈希值，将它们插入到新的哈希表中。**这是一个计算成本较高的操作，但是它有助于维持哈希表的性能，防止由于桶过度填满而导致的效率下降。

配置和影响

设置和获取阈值：在 C++ 中，你可以通过 `load_factor()` 函数来获取当前的负载因子，通过 `max_load_factor()` 函数来获取或设置允许的最大负载因子。例如，如果你认为哈希表的扩容过于频繁，可能会选择增加最大负载因子的值。

性能考量：选择合适的最大负载因子是重要的，较低的负载因子可以减少哈希碰撞，提高查询效率，但会使用更多的内存。较高的负载因子虽然内存使用率更高效，但可能会增加哈希碰撞的频率，降低操作效率。

哈希表的应用：

快速查找：哈希表提供平均时间复杂度为 $O(1)$ 的查找效率，极大提高数据处理速度。

数据去重：使用哈希集合（如 `std::unordered_set`）来快速判断元素是否已经存在于集合中，从而实现去重。

数据索引：哈希表能够快速定位数据，常用于构建数据库和缓存系统的索引。

数据结构选择：

数组：直接通过索引访问元素，适用于元素数量固定且索引已知情况。

Map：如果您需要一个有序的键值对集合，应使用 `std::map`，其底层是红黑树。对于需要高效访问但无序的键值对集合，使用 `std::unordered_map`，其底层是哈希表。

Ps:老秦经验：根据老秦的丰富面试经验，一个常见的面试问题是对比 C++ 中的 **map** 与 **set**。为了帮助大家更好地理解并准备这类问题，老秦精心制作了一份详尽的比较表格，图解地展示了两者的关键差异；

比较表格：

容器类型	底层实现	有序性	重复性	查找效率	增删效率
map	红黑树	有序	不可重复	$O(\log n)$	$O(\log n)$
set	红黑树	有序	不可重复	$O(\log n)$	$O(\log n)$
multimap	红黑树	有序	可重复	$O(\log n)$	$O(\log n)$
multiset	红黑树	有序	可重复	$O(\log n)$	$O(\log n)$
unordered_map	哈希表	无序	不可重复	$O(1)$	$O(1)$
unordered_set	哈希表	无序	不可重复	$O(1)$	$O(1)$

老秦经验总结：

1. 如何记忆容器的对比表格？

1.1 底层结构理解：深入理解容器的底层实现是记忆它们特性的关键。例如，**红黑树**天然具有有序性，因此基于红黑树的容器（如 `std::map` 和 `std::set`）自然是有序的。

1.2 哈希表的优势：哈希表设计用来提高数据访问速度，非常适合快速查找、数据去重和构建高效索引。在处理大量数据时，哈希表通过优化时间复杂度，显著提升数据操作的效率。

2. 数据结构设计中的常见优化策略：

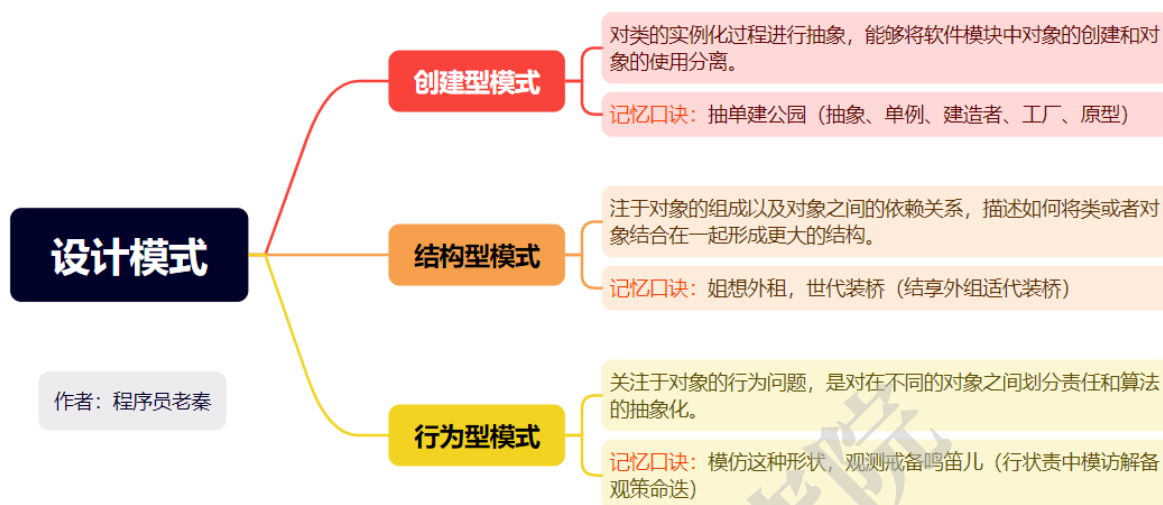
2.1 空间换时间：增加空间消耗以提高处理速度。例如，在哈希表中增加桶的数量，可以有效减少碰撞，从而加快查找速度。这种方法通过使用更多内存来换取更快的执行效率。

2.2 时间换空间：在资源受限的环境下减少空间使用，哪怕这可能意味着性能上的一些妥协。这适用于内存更为宝贵的场景，比如在嵌入式系统或移动设备中，优先考虑内存节省而非最大化运行速度。

PS: 老秦写的本专题聚焦于探讨哈希表在解决算法问题中的应用。详细的 API 使用、具体操作指南以及深入的底层原理分析，这些在面试中经常被提问的主题，将在另一个专题中详尽介绍。老秦准备这些详细内容需要大量的时间和精力，敬请期待老秦的后续深入解析。

第七章 设计模式

23 种设计模式速记



1.创建型模式（Creational Pattern）：

1) 作用：对类的实例化过程进行抽象，能够将软件模块中对象的创建和对对象的使用分离。

2) 记忆口诀：抽单建公园（抽象、单例、建造者、工厂、原型）

3) 包含模式：

工厂模式（Factory Pattern）

抽象工厂模式（Abstract Factory Pattern）

单例模式（Singleton Pattern）

建造者模式（Builder Pattern）

原型模式（Prototype Pattern）

2.结构型模式（Structural Pattern）：

1) 作用：关注于对象的组成以及对象之间的依赖关系，描述如何将类或者对象结合在一起形成更大的结构。

2) 记忆口诀：姐想外租，世代装桥（结享外组适代装桥）

3) 包含模式：

适配器模式（Adapter Pattern）

装饰者模式（Decorator Pattern）

代理模式（Proxy Pattern）

外观模式（Facade Pattern）

桥接模式（Bridge Pattern）

组合模式（Composite Pattern）

享元模式（Flyweight Pattern）

3.行为型模式（Behavioral Pattern）：

1) 作用：关注于对象的行为问题，是对在不同的对象之间划分责任和算法的抽象化。

2) 记忆口诀：模仿这种形状，观测戒备鸣笛儿（行状责中模仿解备观策命迭）

3) 包含模式：

策略模式 (Strategy Pattern)

模板方法模式 (Template Method Pattern)

观察者模式 (Observer Pattern)

迭代器模式 (Iterator Pattern)

责任链模式 (Chain of Responsibility Pattern)

命令模式 (Command Pattern)

备忘录模式 (Memento Pattern)

状态模式 (State Pattern)

访问者模式 (Visitor Pattern)

中介者模式 (Mediator Pattern)

解释器模式 (Interpreter Pattern)

创建型设计模式 (Creational Patterns)

这些模式**关注对象的创建机制**，旨在提供一种更灵活、更复杂的对象创建方式，而**不是直接实例化对象**。常见的创建型模式包括：

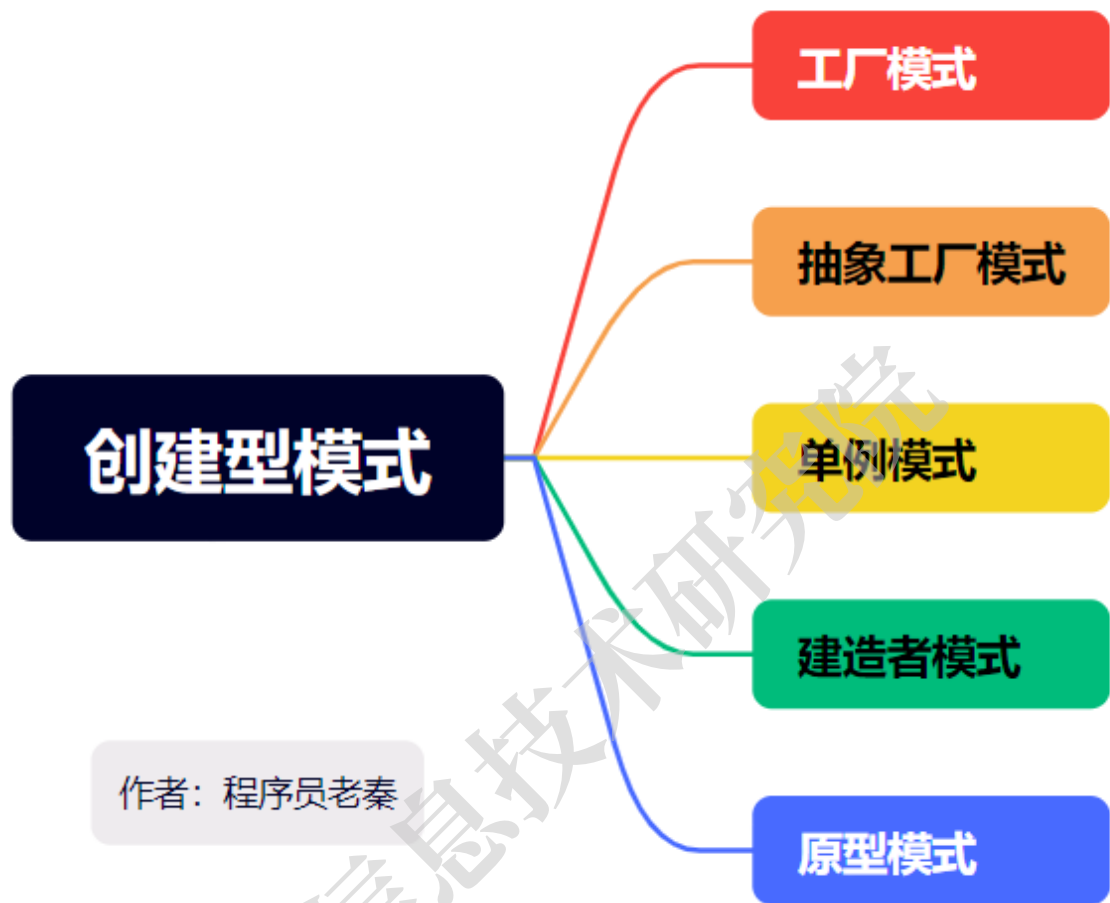
1) **单例模式 (Singleton Pattern)**：确保**一个类只有一个实例**，并提供全局访问点。

2) **工厂方法模式 (Factory Method Pattern)**：定义一个创建对象的接口，但将**对象的实际创建延迟到子类**。

3) **抽象工厂模式 (Abstract Factory Pattern)**：提供一个创建相关对象家族的接口，而**无需指定具体类**。

4) **建造者模式 (Builder Pattern)**：将**一个复杂对象的构建过程分解为多个步骤**，以便更灵活地创建对象。

5) **原型模式 (Prototype Pattern)**：通过**复制现有对象来创建新对象**，而不是从头开始构建。



1. 简单工厂

(1) 核心思想

将对象的创建过程封装在一个独立的工厂类中，客户端只需要通过工厂类来创建所需的对象，而不需要直接实例化具体的类。

(2) 简单工厂的结构

工厂类 (**Factory**) : 负责创建具体对象的类，通常包含一个或多个创建对象的方法。

产品类 (**Product**) : 具体对象的抽象，由工厂类来创建，客户端与产品类进行交互。

(3) 示例：

假设你正在开发一个图形绘制应用程序，需要根据用户的选择创建不同类型的图形对象，如圆形 (Circle) 和矩形 (Rectangle)。你可以使用简单工厂来实现这个场景。

```
#include <iostream>
#include <string>
```

```
// 产品类 - 图形
class Shape {
public:
    virtual void draw() = 0;
    virtual ~Shape() = default; // 确保基类有虚析构函数};

// 具体产品类 - 圆形
class Circle : public Shape {
public:
    void draw() override {
        std::cout << "绘制圆形" << std::endl;};
};

// 具体产品类 - 矩形
class Rectangle : public Shape {
public:
    void draw() override {
        std::cout << "绘制矩形" << std::endl;};
};

// 工厂类
class ShapeFactory {
public:
    // 根据类型创建图形对象
    static Shape* createShape(const std::string& type) {
        if (type == "Circle") {
            return new Circle();}
        else if (type == "Rectangle") {
            return new Rectangle();}
        else {
            return nullptr; // 可以根据需要添加其他图形类型的创建逻辑}}};

int main() {
    // 使用简单工厂创建图形对象
    Shape* shape1 = ShapeFactory::createShape("Circle");
    if (shape1) {
        shape1->draw();
        delete shape1;}
    Shape* shape2 = ShapeFactory::createShape("Rectangle");
    if (shape2) {
        shape2->draw();
        delete shape2;}
    Shape* shape3 = ShapeFactory::createShape("Triangle");
    if (shape3) {
        shape3->draw();
        delete shape3;}
    else {
        std::cout << "无效的图形类型" << std::endl;}
    return 0;}
```


(4)应用场景

- 1) 图形绘制应用程序：根据用户选择的不同图形类型创建相应的图形对象。
- 2) 日志记录器：根据不同的日志级别（信息、警告、错误）创建不同的日志记录器对象。
- 3) 数据库连接管理：根据数据库类型（MySQL、PostgreSQL、Oracle）创建数据库连接对象。
- 4) UI 库控件：根据用户需求创建不同类型的 UI 控件（按钮、文本框、下拉菜单等）。

(5)优点

- 1) 封装对象创建过程：客户端代码与具体类解耦，降低了代码的维护复杂度。
- 2) 统一管理对象创建：通过工厂类统一管理对象的创建逻辑，增强了代码的可维护性。

(6)缺点

- 1) 违反开闭原则：当需要添加新的产品类型时，需要修改工厂类的代码，违反了开闭原则。
- 2) 不适用于频繁变化的需求：如果需要频繁添加新的产品类型，可能需要考虑其他创建型设计模式，如工厂方法模式或抽象工厂模式。

3. 单例设计模式

单例设计模式旨在确保一个类只有一个实例，并提供全局访问点以访问该实例。在单例模式中，有两种常见的实现方式，即饿汉式和懒汉式。它们的主要区别在于实例的创建时间和线程安全性。

(1)饿汉式单例模式

在饿汉式中，实例在类加载时就被创建，不管是否需要使用它。这意味着无论何时引用该类，都会返回相同的实例。

主要特点：

- 1) 线程安全：在多线程环境中，由于实例在类加载时创建，所以不会存在竞态条件，是线程安全的。
- 2) 简单：实现简单，但可能会浪费内存，因为无论是否需要，实例都会创建。

```
class Singleton {
private:
    static Singleton* instance;
    Singleton() {} // 私有构造函数，禁止外部实例化
public:
    static Singleton* getInstance() {
        return instance;
    };
// 静态成员初始化
Singleton* Singleton::instance = new Singleton();
}
```


(2) 懒汉式单例模式

在懒汉式中，**实例在第一次被请求时创建**。这意味着**实例的创建是延迟的**，直到**需要使用时才会创建**。

主要特点：

1) 延迟加载：实例的创建是延迟的，只有在需要时才会创建。

2) 非线程安全：在多线程环境中，多个线程可能同时访问 `getInstance()` 方法，导致创建多个实例。**需要额外的同步措施来保证线程安全。**

```
class Singleton {
private:
    static Singleton* instance;
    Singleton() {} // 私有构造函数，禁止外部实例化
public:
    static Singleton* getInstance() {
        if (instance == nullptr) {
            instance = new Singleton();
        }
        return instance;
    };
    // 静态成员初始化
    Singleton* Singleton::instance = nullptr;
};
```

(3) 实现懒汉式单例设计模式的线程安全

双重检查锁定 (Double-Checked Locking) 是一种用于实现懒汉式单例模式的**常见线程安全技术**。它的目的是在多线程环境下延迟实例化对象，同时确保只有一个实例被创建。这种技术通常用于需要考虑性能的情况下，因为它在绝大多数情况下避免了不必要的锁定操作。

双重检查锁定的基本思想：

首先**检查实例是否已经创建**，如果已经创建，则直接返回实例。如果实例尚未创建，才进行锁定操作。**在锁定的情况下再次检查实例是否已经创建**，以防止其他线程在等待锁定时已经创建了实例。如果实例仍未创建，才创建并返回实例。

以下是一个使用双重检查锁定的懒汉式单例模式示例

```
#include <iostream>
#include <mutex>
class Singleton {
private:
    static Singleton* instance;
    static std::mutex mtx; // 用于线程安全的锁
    Singleton() {} // 私有构造函数，禁止外部实例化
public:
    static Singleton* getInstance() {
        // 第一次检查，如果实例已经存在，直接返回
        if (instance == nullptr) {
            std::lock_guard<std::mutex> lock(mtx); // 加锁
            // 第二次检查，在锁保护下再次检查实例是否存在
            if (instance == nullptr) {
                instance = new Singleton();
            }
        }
        return instance;
    };
};
```

```

        // 第二次检查，防止在等待锁期间其他线程已经创建实例
        if (instance == nullptr) {
            instance = new Singleton(); // 创建实例}}
        return instance;}};
// 静态成员初始化
Singleton* Singleton::instance = nullptr;
std::mutex Singleton::mtx;

```

双重检查锁定确保只有在实例未创建时才会加锁和创建实例，从而提高了性能。

需要注意的是，双重检查锁定需要在多线程环境下使用，因为它涉及到线程同步。另外，C++11 之后，还可以使用更方便的局部静态变量来实现懒汉式单例模式，因为 C++11 引入了线程安全的局部静态变量初始化。

使用局部静态变量的懒汉式单例模式示例：

```

class Singleton {
private:
    Singleton() {} // 私有构造函数，禁止外部实例化
public:
    static Singleton& getInstance() {
        static Singleton instance; // 局部静态变量，线程安全
        return instance;};
}

```

4. 抽象工厂设计模式

(1) 核心思想

抽象工厂设计模式是一种创建型设计模式，它提供了一种创建一系列相关或相互依赖对象的接口，而无需指定其具体类。通过定义一个抽象工厂接口，声明一组用于创建不同产品族的抽象方法，每个具体的工厂类实现这个接口，并负责创建特定的产品族。具体产品类实现了产品接口，每个工厂类创建的产品都属于同一个产品族，但是产品族之间有不同的实现。

抽象工厂模式的核心思想是通过创建一组相关的产品，而不是单个产品。它提供了一种封装对象创建的方式，使得客户端可以从具体实现中解耦。

(2) 结构

- 1) 抽象产品接口 (**Abstract Product**)：定义产品的接口。
- 2) 具体产品类 (**Concrete Product**)：实现抽象产品接口。
- 3) 抽象工厂接口 (**Abstract Factory**)：声明一组创建不同产品的方法。
- 4) 具体工厂类 (**Concrete Factory**)：实现抽象工厂接口，负责创建具体产品。

(3) 示例

假设你正在开发一个图形绘制应用程序，需要根据用户的选择创建不同类型的图形对象，如按钮 (Button) 和文本框 (Textbox)。你可以使用抽象工厂模式来实现这个场景。

```
#include <iostream>
```

```
// 抽象产品接口 - 按钮
class Button {
public:
    virtual void render() = 0;
    virtual ~Button() = default;};

// 具体产品类 - Windows 按钮
class WindowsButton : public Button {
public:
    void render() override {
        std::cout << "Rendering a Windows button." << std::endl;};

// 具体产品类 - Mac 按钮
class MacButton : public Button {
public:
    void render() override {
        std::cout << "Rendering a Mac button." << std::endl;};

// 抽象产品接口 - 文本框
class Textbox {
public:
    virtual void render() = 0;
    virtual ~Textbox() = default;};

// 具体产品类 - Windows 文本框
class WindowsTextbox : public Textbox {
public:
    void render() override {
        std::cout << "Rendering a Windows textbox." << std::endl;};

// 具体产品类 - Mac 文本框
class MacTextbox : public Textbox {
public:
    void render() override {
        std::cout << "Rendering a Mac textbox." << std::endl;};

// 抽象工厂接口
class GUIFactory {
public:
    virtual Button* createButton() = 0;
    virtual Textbox* createTextbox() = 0;
    virtual ~GUIFactory() = default;};

// 具体工厂类 - Windows 工厂
class WindowsFactory : public GUIFactory {
public:
    Button* createButton() override {
        return new WindowsButton();}
    Textbox* createTextbox() override {
        return new WindowsTextbox();};};
```

```

// 具体工厂类 - Mac 工厂
class MacFactory : public GUIFactory {
public:
    Button* createButton() override {
        return new MacButton();}
    Textbox* createTextbox() override {
        return new MacTextbox();}};

int main() {
    // 创建一个 Windows 风格的工厂
    GUIFactory* windowsFactory = new WindowsFactory();
    // 创建一个 Windows 风格的按钮
    Button* windowsButton = windowsFactory->createButton();
    // 渲染按钮
    windowsButton->render(); // Output: Rendering a Windows button.
    // 创建一个 Windows 风格的文本框
    Textbox* windowsTextbox = windowsFactory->createTextbox();
    // 渲染文本框
    windowsTextbox->render(); // Output: Rendering a Windows textbox.
    // 创建一个 Mac 风格的工厂
    GUIFactory* macFactory = new MacFactory();
    // 创建一个 Mac 风格的按钮
    Button* macButton = macFactory->createButton();
    // 渲染按钮
    macButton->render(); // Output: Rendering a Mac button.
    // 创建一个 Mac 风格的文本框
    Textbox* macTextbox = macFactory->createTextbox();
    // 渲染文本框
    macTextbox->render(); // Output: Rendering a Mac textbox.
    // 清理资源
    delete windowsFactory;
    delete windowsButton;
    delete windowsTextbox;
    delete macFactory;
    delete macButton;
    delete macTextbox;
    return 0;}

```

(4)应用场景

1) 图形界面库：图形界面库通常需要创建一组相关的界面元素，如按钮、文本框、标签等。通过抽象工厂模式，可以创建不同风格的界面元素，如 Windows 风格、Mac 风格等，确保界面元素之间的风格一致性。

2) 数据库访问：在数据库访问中，可能需要使用不同的数据库引擎，如 MySQL、Oracle、SQL Server 等。通过抽象工厂模式，可以创建不同数据库引擎的连接

对象、命令对象等，以便在运行时根据需要切换使用不同的数据库引擎。

3) 操作系统适配：在开发跨平台的软件时，可能需要根据不同的操作系统提供不同的实现。通过抽象工厂模式，可以根据操作系统的不同创建对应的工厂对象，从而提供与操作系统相关的实现。

(5)优点

1) 封装对象创建过程：客户端代码与具体类解耦，降低了代码的维护复杂度。

2) 一致的产品族：通过工厂创建一组相关的产品，确保产品之间的一致性和互换性。

3) 符合开闭原则：可以**通过增加新的具体工厂类和具体产品类来扩展系统，而不影响现有代码。**

(6)缺点

1) 增加系统复杂性：由于需要定义多个抽象工厂和具体工厂类，可能会增加系统的复杂性。

2) 不便于单个产品的修改：当产品族中的某个产品需要修改时，可能需要修改所有具体工厂类。

5. 原型设计模式

(1)核心思想

原型设计模式是一种对象创建型模式，它**通过复制现有对象来创建新的对象，而无需显式地调用构造函数。**简单来说，**原型设计模式就是通过克隆已有对象来创建新对象。**

(2)结构

1) 抽象原型类 (Prototype)：定义具有克隆方法的接口。

2) 具体原型类 (Concrete Prototype)：实现抽象原型类的克隆方法。

3) 客户端 (Client)：通过调用原型对象的克隆方法来创建新对象。

(3)示例

假设我们有一个图形库，其中有一个基类 Shape 表示图形，它有一个纯虚函数 draw() 用于绘制图形。现在希望能够复制已有的图形对象来创建新的图形对象，而无需重新构造图形对象。

```
#include <iostream>
#include <string>
// 图形基类
class Shape {
public:
    virtual ~Shape() {}
    virtual void draw() = 0;
    virtual Shape* clone() = 0;};
// 圆形类
class Circle : public Shape {
```

```

private:
    std::string color;
    int radius;
public:
    Circle(std::string color, int radius) : color(color), radius(radius) {}
    void draw() override {
        std::cout << "Drawing a circle with color " << color << " and radius "
<< radius << std::endl;}
    Shape* clone() override {
        return new Circle(*this);};
int main() {
    // 创建一个原型对象
    Shape* circlePrototype = new Circle("red", 5);
    // 克隆原型对象来创建新对象
    Shape* circle1 = circlePrototype->clone();
    Shape* circle2 = circlePrototype->clone();
    // 绘制图形
    circle1->draw(); // Output: Drawing a circle with color red and radius 5
    circle2->draw(); // Output: Drawing a circle with color red and radius 5
    delete circlePrototype;
    delete circle1;
    delete circle2;
    return 0;}

```

(3)应用场景

1) 创建过程复杂且频繁创建相似对象：原型设计模式可以通过复制已有对象来创建新对象，**避免了复杂的创建过程**，提高了创建对象的效率。

2) 需要动态地添加新对象：这些对象的类型在运行时才能确定。原型设计模式通过克隆已有对象来创建新对象，**可以在运行时动态地添加新的对象类型，而无需显式地调用构造函数**。

(4)典型应用场景

1) 图形编辑器：在图形编辑器中，可能需要频繁地创建相似的图形对象，如矩形、圆形等。使用原型设计模式，可以先创建一个原型对象，并通过克隆原型对象来创建新的图形对象，而无需重新构造对象。

2) 缓存系统：在缓存系统中，可能需要频繁地复制已有的缓存对象来创建新的缓存对象。使用原型设计模式，可以通过克隆已有的缓存对象来创建新的缓存对象，而无需重新从数据库或其他地方获取数据。

3) 原始对象的状态保存：有时候需要保存对象的某个特定状态，以便在需要时恢复回来。原型设计模式可以通过克隆对象来保存和恢复对象的状态，而不需要手动记录和恢复每个属性的值。

(5)优点

1) **避免子类的重复创建**：通过克隆现有对象来创建新对象，避免了重复的创建过程。

2) **提高对象创建效率**：通过复制已有对象来创建新对象，避免了复杂的创建过程，提高了创建效率。

3) **简化对象创建**：通过原型模式，可以通过克隆已有对象来创建新对象，简化了对象的创建过程。

(6)缺点

1) **需要实现克隆方法**：所有的具体原型类都需要实现克隆方法，可能会增加代码的复杂度。

2) **深拷贝和浅拷贝问题**：在实现克隆方法时，**需要考虑是进行深拷贝还是浅拷贝**，可能会涉及到一些复杂的逻辑。

6. 建造者设计模式

(1)核心思想

建造者设计模式是一种创建型设计模式，旨在**将对象的构建过程与其表示分离**。该模式允许逐步构建复杂的对象，同时保持构建过程的灵活性。

建造者设计模式的核心思想是将**一个复杂对象的构建过程分解为多个简单的步骤**，**每个步骤由一个具体的建造者类负责实现**。这些步骤按照一定的顺序被调用，最终构建出一个完整的对象。

(2)结构

1) **产品类 (Product)**：要创建的复杂对象。

2) **抽象建造者类 (Builder)**：定义创建产品各个部件的抽象接口。

3) **具体建造者类 (Concrete Builder)**：实现抽象建造者接口，构建和装配各个部件。

4) **指导者类 (Director)**：构建过程中的指挥者，负责按顺序调用建造者的各个部件构建方法。

(3)示例

下面是一个使用 C++ 代码示例来说明建造者设计模式的实现：

```
#include <iostream>
#include <string>
// 产品类
class Pizza {
public:
    void setDough(const std::string& dough) {
        m_dough = dough;
    }
    void setSauce(const std::string& sauce) {
        m_sauce = sauce;
    }
    void setTopping(const std::string& topping) {
        m_topping = topping;
    }
};
```



```

    void showPizza() const {
        std::cout << "Pizza with " << m_dough << " dough, " << m_sauce
        << " sauce, and " << m_topping << " topping." << std::endl; }
private:
    std::string m_dough;
    std::string m_sauce;
    std::string m_topping; };
// 抽象建造者类
class PizzaBuilder {
public:
    virtual ~PizzaBuilder() {}
    virtual void buildDough() = 0;
    virtual void buildSauce() = 0;
    virtual void buildTopping() = 0;
    virtual Pizza* getPizza() = 0; };
// 具体建造者类 A
class HawaiianPizzaBuilder : public PizzaBuilder {
public:
    void buildDough() override {
        m_pizza->setDough("cross"); }
    void buildSauce() override {
        m_pizza->setSauce("mild"); }
    void buildTopping() override {
        m_pizza->setTopping("ham and pineapple"); }
    Pizza* getPizza() override {
        return m_pizza; }
private:
    Pizza* m_pizza = new Pizza(); };
// 具体建造者类 B
class SpicyPizzaBuilder : public PizzaBuilder {
public:
    void buildDough() override {
        m_pizza->setDough("pan baked"); }
    void buildSauce() override {
        m_pizza->setSauce("hot"); }
    void buildTopping() override {
        m_pizza->setTopping("pepperoni and jalapeno"); }
    Pizza* getPizza() override {
        return m_pizza; }
private:
    Pizza* m_pizza = new Pizza(); };
// 指导者类
class PizzaDirector {

```



```

public:
    void setPizzaBuilder(PizzaBuilder* builder) {
        m_builder = builder;}
    void constructPizza() {
        m_builder->buildDough();
        m_builder->buildSauce();
        m_builder->buildTopping();}
private:
    PizzaBuilder* m_builder;};
int main() {
    PizzaDirector director;
    HawaiianPizzaBuilder hawaiianBuilder;
    director.setPizzaBuilder(&hawaiianBuilder);
    director.constructPizza();
    Pizza* hawaiianPizza = hawaiianBuilder.getPizza();
    hawaiianPizza->showPizza(); // Output: Pizza with cross dough, mild
    sauce, and ham and pineapple topping.
    SpicyPizzaBuilder spicyBuilder;
    director.setPizzaBuilder(&spicyBuilder);
    director.constructPizza();
    Pizza* spicyPizza = spicyBuilder.getPizza();
    spicyPizza->showPizza(); // Output: Pizza with pan baked dough, hot
    sauce, and pepperoni and jalapeno topping.
    delete hawaiianPizza;
    delete spicyPizza;
    return 0;}

```

(4)应用场景

1) 创建复杂的对象：当对象的构建过程比较复杂，涉及多个步骤、多个组件或者需要进行一些复杂的计算或处理时，可以使用建造者模式来将构建过程分解为多个简单的步骤，使得构建过程更加清晰和可控。

2) 避免构造函数的参数过多：当一个对象的构造函数需要传递大量的参数时，使用构造函数来创建对象会导致代码难以理解和维护。而使用建造者模式，可以[通过一系列方法来设置对象的属性，使得代码更加清晰易懂。](#)

3) 创建一系列相似的对象：当需要创建一系列相似但属性略有不同的对象时，使用建造者模式可以[通过复用已有的建造者对象来构建这些对象，减少重复的代码。](#)

4) 控制对象的创建过程：建造者模式允许在构建过程中灵活地添加、修改或替换组件，从而更好地控制对象的创建过程。这样可以根据不同的需求，使用不同的建造者来构建具有不同特性的对象。

(5)优点

1) 将复杂的构建过程分解为多个步骤：使得每个步骤更加简单清晰，可以独立变化而不会影响其他部分。

2) 实现了构建过程的灵活性：可以根据不同的需求，通过不同的建造者实现来创建不同的产品。

3) 代码更加清晰可维护：避免了构造函数参数过多的问题，使得代码更加易于理解和维护。

(6)缺点

1) 增加了代码的复杂性：需要定义多个具体的建造者类和指导者类，可能会增加系统的复杂性。

2) 产品的一致性依赖于指导者的正确使用：如果指导者使用不当，可能会导致产品的状态不一致。

结构型设计模式 (Structural Patterns)

这些模式关注如何组合类和对象以形成更大的结构，以解决系统中对象之间的关系。常见的结构型模式包括：

1) 适配器模式 (Adapter Pattern)：允许将一个接口转换为另一个客户端期望的接口。

2) 装饰者模式 (Decorator Pattern)：动态地为对象添加新的功能，而无需修改其源代码。

3) 代理模式 (Proxy Pattern)：提供一个代理对象以控制对其他对象的访问。

4) 组合模式 (Composite Pattern)：允许客户端以统一的方式处理单个对象和对象组合。

5) 桥接模式 (Bridge Pattern)：将抽象部分与其实现分离，以便它们可以独立变化。

6) 外观设计模式 (Facade Design Pattern)：外观模式提供了一个高级别的接口，将多个底层接口组合成一个更简单的接口，以供客户端使用。这有助于隔离客户端代码和复杂子系统之间的耦合关系。

7) 享元模式 (Flyweight Pattern) : 主要用于减少创建对象的数量, 以减少内存占用和提高性能。这种类型的设计模式属于结构型模式, 它提供了减少对象数量从而改善应用所需的对象结构的方式。

7. 适配器设计模式



(1) 核心思想

适配器设计模式 (Adapter Design Pattern) 是一种结构型设计模式, 用于将一个类的接口转换为另一个类的接口, 以满足客户端的需求。适配器模式允许不兼容的类能够合作, 通过适配器将一个类的接口转换为另一个类的接口, 使得两者可以无缝协同工作。

(2) 主要组成部分

- 1) 目标接口 (Target) : 定义客户端所期望的接口形式。
- 2) 适配器 (Adapter) : 将被适配者的接口转换为目标接口的类。
- 3) 被适配者 (Adaptee) : 需要被适配的类, 具有客户端无法直接使用的接口。

(3) 特点

- 1) 接口转换：适配器模式通过转换接口，使得不兼容的类能够协同工作。
- 2) 复用现有类：可以在不修改现有类的情况下复用这些类。
- 3) 提高灵活性：适配器可以使得不同接口的类协同工作，增加代码的灵活性和可扩展性。

(4)示例

下面是一个简单的 C++ 代码示例来说明适配器设计模式的实现：

```
#include <iostream>
// 目标接口
class Target {
public:
    virtual void request() = 0;
};
// 被适配者类
class Adaptee {
public:
    void specificRequest() {
        std::cout << "被适配者的特殊请求" << std::endl;
    }
};
// 类适配器
class Adapter : public Target, private Adaptee {
public:
    void request() override {
        specificRequest();
    }
};
int main() {
    Target* adapter = new Adapter();
    adapter->request();
    delete adapter;
    return 0;
}
```

在上面的示例中，**Target** 是目标接口，定义了客户端所期望的接口形式。**Adaptee** 是被适配者类，具有客户端无法直接使用的特殊接口。**Adapter** 是适配器类，通过继承被适配者类和实现目标接口的方法，将被适配者类的接口转换为目标接口。

在 main 函数中，我们创建了一个适配器对象 adapter，然后调用它的 request 方法来展示适配器将被适配者的特殊请求转换为目标接口的能力。

(5)应用场景

适配器设计模式的应用场景包括但不限于以下情况：

- 1) 老旧接口的使用：当需要使用一些老旧接口的功能，但又希望与现有的代码协同工作时，可以使用适配器模式来将老旧接口转换为现有代码所期望的接口形式。
- 2) 类库的兼容性：当需要使用某个类库，但该类库的接口与当前项目的接口不兼容时，可以使用适配器模式来将类库的接口转换为项目所期望的接口形式。
- 3) 接口的统一：当多个类具有不同的接口，但需要以统一的方式进行操作时，可以使用适配器模式来将它们的接口转换为统一的接口形式。

4) 与第三方组件的集成：当需要将第三方组件集成到自己的系统中，但第三方组件的接口与自己的系统不兼容时，可以使用适配器模式来将第三方组件的接口转换为自己系统所期望的接口形式。

8. 装饰者设计模式

(1) 核心思想

装饰者设计模式 (Decorator Design Pattern) 是一种结构型设计模式，用于动态地给对象添加额外的职责，同时不改变其原始类的结构。装饰者模式通过将对象包装在一个装饰者类中，然后逐层地添加装饰者，从而实现对对象的透明扩展。

(2) 主要组成部分

- 1) 抽象构件 (Component)：**定义了原始对象和装饰者共同的接口。
- 2) 具体构件 (Concrete Component)：**实现了抽象构件的接口，表示原始对象。
- 3) 抽象装饰者 (Decorator)：**装饰者的抽象类或接口，包含一个指向抽象构件的引用。
- 4) 具体装饰者 (Concrete Decorator)：**具体的装饰者类，继承自抽象装饰者，通过对抽象构件进行装饰。

(3) 特点

- 1) 动态扩展：**可以动态地给对象添加新功能，不改变原始类的结构。
- 2) 灵活性高：**可以组合多个装饰者，以实现复杂的功能扩展。
- 3) 遵循开闭原则：**对扩展开放，对修改封闭。

(4) 示例

下面是一个简单的 C++ 代码示例来说明装饰者设计模式的实现：

```
#include <iostream>
// 抽象构件
class Component {
public:
    virtual void operation() = 0;
};
// 具体构件
class ConcreteComponent : public Component {
public:
    void operation() override {
        std::cout << "执行具体构件的操作" << std::endl;
    }
};
// 抽象装饰者
class Decorator : public Component {
protected:
    Component* component;
public:
    Decorator(Component* component) : component(component) {}
};
```

```

        void operation() override {
            if (component != nullptr) {
                component->operation();}
        };
// 具体装饰者 A
class ConcreteDecoratorA : public Decorator {
public:
    ConcreteDecoratorA(Component* component) : Decorator(component) {}
    void operation() override {
        addedBehavior();
        Decorator::operation();}
    void addedBehavior() {
        std::cout << "具体装饰者 A 的附加操作" << std::endl;};
// 具体装饰者 B
class ConcreteDecoratorB : public Decorator {
public:
    ConcreteDecoratorB(Component* component) : Decorator(component) {}
    void operation() override {
        addedBehavior();
        Decorator::operation();}
    void addedBehavior() {
        std::cout << "具体装饰者 B 的附加操作" << std::endl;};
int main() {
    Component* component = new ConcreteComponent();
    Component* decoratorA = new ConcreteDecoratorA(component);
    decoratorA->operation();
    Component* decoratorB = new ConcreteDecoratorB(decoratorA);
    decoratorB->operation();
    delete decoratorB;
    delete decoratorA;
    delete component;
    return 0;
}

```

在上面的示例中，Component 是抽象构件，定义了原始对象和装饰者共同的接口。ConcreteComponent 是具体构件，实现了抽象构件的接口，表示原始对象。Decorator 是抽象装饰者，包含一个指向抽象构件的引用，并实现了抽象构件的接口。ConcreteDecoratorA 和 ConcreteDecoratorB 是具体装饰者，继承自抽象装饰者，通过对抽象构件进行装饰。

在 main 函数中，我们创建了一个具体构件 component，然后分别用具体装饰者 ConcreteDecoratorA 和 ConcreteDecoratorB 对其进行装饰。最终，通过调用装饰者的 operation 方法，实现了对原始对象的透明扩展。

(5)应用场景

装饰者设计模式的应用场景包括但不限于以下情况：

1) 动态增加功能：当需要动态地给对象添加额外的职责时，可以使用装饰者模式。通过创建不同的装饰者类，可以在不改变原始对象的情况下，透明地为对象添加新的功能。

2) 多层次的装饰：当需要对对象进行多层次的装饰时，可以使用装饰者模式。每个具体装饰者可以在上一层装饰的基础上添加新的功能。

3) 组合功能：当需要将多个不同的功能组合在一起使用时，可以使用装饰者模式。通过创建不同的具体装饰者类，可以按需组合功能。

4) 开放封闭原则：当希望在不修改现有代码的情况下扩展功能时，可以使用装饰者模式。通过创建新的具体装饰者类，可以在不修改原始对象的情况下添加新的功能。

5) 单一职责原则：当希望将不同的责任分离到不同的类中，并且可以动态地进行组合时，可以使用装饰者模式。

9. 代理设计模式

(1) 核心思想

代理设计模式 (Proxy Design Pattern) 是一种结构型设计模式，用于在访问对象时提供一种代理，以控制对对象的访问。代理模式通过引入一个代理对象来替代原始对象，从而可以在不改变原始对象的情况下增加额外的功能或控制访问。

(2) 主要组成部分

- 1) 抽象主题 (Subject)：**定义了真实主题和代理的共同接口。
- 2) 真实主题 (Real Subject)：**实际执行业务逻辑的对象。
- 3) 代理 (Proxy)：**一个中间类，通过持有真实主题的引用并实现抽象主题的方法，将客户端的请求委派给真实主题。

(3) 特点

- 1) 控制访问：**代理模式可以控制对原始对象的访问。
- 2) 增强功能：**在不修改原始对象的情况下，通过代理对象增加额外功能。
- 3) 远程代理：**用于处理网络通信，隐藏底层的网络细节。
- 4) 虚拟代理：**延迟对象的创建，只有在需要时才创建真实对象。
- 5) 保护代理：**控制访问权限，确保安全性。

(4) 示例

下面是一个简单的 C++ 代码示例来说明代理设计模式的实现：

```
#include <iostream>
// 抽象主题
class Subject {
public:
    virtual void request() = 0;
};
// 真实主题
class RealSubject : public Subject {
public:
    void request() override {
```



```

        std::cout << "真实主题的请求" << std::endl;});

// 代理
class Proxy : public Subject {
private:
    RealSubject* realSubject;
public:
    Proxy() {
        realSubject = new RealSubject();}
    ~Proxy() {
        delete realSubject;}
    void request() override {
        // 在调用真实主题之前可以添加额外的逻辑
        std::cout << "代理的请求" << std::endl;
        // 委托给真实主题处理
        realSubject->request();});

int main() {
    Subject* proxy = new Proxy();
    proxy->request();
    delete proxy;
    return 0;}

```

在上面的示例中，**Subject** 是抽象主题，定义了真实主题和代理的共同接口。**RealSubject** 是真实主题，实现了抽象主题的方法，用于执行实际的业务逻辑。**Proxy** 是代理类，持有真实主题的引用，并实现抽象主题的方法，通过委托真实主题来处理客户端的请求。

在 **main** 函数中，我们创建了一个代理对象 **proxy**，然后调用它的 **request** 方法来展示代理将请求委托给真实主题的能力。

(5) 应用场景

代理设计模式的应用场景包括但不限于以下情况：

- 1) **远程代理**：当需要访问远程对象时，可以使用代理模式来隐藏底层的网络通信细节，客户端通过代理对象访问远程对象。
- 2) **虚拟代理**：当创建一个对象的成本很高时，可以使用代理模式来延迟对象的实际创建，只有在真正需要时才创建真实对象。
- 3) **安全代理**：当需要对对象的访问进行控制时，可以使用代理模式来添加额外的安全性检查，例如权限验证等。
- 4) **缓存代理**：当需要缓存对象的结果以提高性能时，可以使用代理模式来实现缓存功能，代理对象可以在调用真实对象之前检查缓存，并返回缓存结果。
- 5) **日志记录代理**：当需要记录对象的操作日志时，可以使用代理模式来在调用真实对象之前或之后添加日志记录的功能。

10. 组合设计模式

(1) 核心思想

组合模式允许你将对象组合成树形结构来表示“**部分-整体**”的**层次结构**。组合模式使得客户端对单个对象和组合对象的使用具有一致性。通过组合模式，客户端可以统一地对待单个对象和对象组合，无需关注它们是单个对象还是组合对象。

(2)主要组成部分

1) 抽象组件类 (Component)：定义了组合对象和叶子对象的共同接口或抽象类。

2) 叶子组件类 (Leaf)：表示树形结构中的叶子节点，叶子节点没有子节点。

3) 组合组件类 (Composite)：表示树形结构中的组合节点，组合节点可以包含叶子节点或其他组合节点。

(3)特点

1) 统一处理：客户端可以一致地处理单个对象和组合对象。

2) 层次结构：通过树形结构表示对象之间的层次关系。

3) 简化客户端代码：客户端无需区分单个对象和组合对象，简化了代码复杂性。

(4)示例代码

下面是一个简单的 C++ 代码示例来说明组合设计模式的实现：

```
#include <iostream>
#include <vector>
// 抽象组件类
class Component {
public:
    virtual void operation() = 0;
    virtual ~Component() = default;};
// 叶子组件类
class Leaf : public Component {
public:
    void operation() override {
        std::cout << "叶子组件的操作" << std::endl;};
// 组合组件类
class Composite : public Component {
private:
    std::vector<Component*> components;
public:
    void addComponent(Component* component) {
        components.push_back(component);}
    void operation() override {
        std::cout << "组合组件的操作" << std::endl;
        for (Component* component : components) {
            component->operation();}}
    ~Composite() {
        for (Component* component : components) {
```

```
        delete component;}}};  
int main() {  
    Component* leaf1 = new Leaf();  
    Component* leaf2 = new Leaf();  
    Composite* composite = new Composite();  
    composite->addComponent(leaf1);  
    composite->addComponent(leaf2);  
    composite->operation();  
    delete composite;  
    return 0;}  
}
```

在上面的示例中，Component 是抽象组件类，定义了组件对象的操作方法。Leaf 是叶子组件类，实现了抽象组件类的操作方法。Composite 是组合组件类，可持有多个叶子组件或其他组合组件，并实现了抽象组件类的操作方法。

在 main 函数中，我们首先创建了两个叶子组件对象 leaf1 和 leaf2，以及一个组合组件对象 composite。然后通过 composite 的 addComponent 方法将叶子组件对象添加到组合组件对象中。最后调用组合组件对象的操作方法，会依次调用叶子组件对象的操作方法来展示整个组合对象的行为。

(5)应用场景

1) 树形结构：当需要表示对象的层次结构，且对象之间存在“整体-部分”的关系时，可以使用组合模式来构建树形结构，方便对整个结构进行统一操作。

2) GUI 界面中的布局：在图形用户界面（GUI）的布局中，可以使用组合模式来构建复杂的布局结构，通过将容器组件和子组件组合起来，方便进行整体和部分的布局调整。

3) 文件系统的管理：在文件系统中，文件和文件夹之间存在层次关系，可以使用组合模式来构建文件系统的层次结构，方便对文件和文件夹进行统一的管理和操作。

4) 组织机构的管理：在组织机构中，部门和员工之间存在层次关系，可以使用组合模式来构建组织机构的层次结构，方便对整个组织进行统一的管理和操作。

11. 桥接设计模式

(1)核心思想

桥接模式旨在将一个对象的抽象部分与具体实现部分分离，使得它们可以独立地变化。通过这种方式，系统的抽象层和实现层可以独立进行扩展，而不会相互影响，从而提高系统的灵活性和可扩展性。

(2)主要组成部分

1) 抽象部分 (Abstraction)：定义了抽象接口，并包含一个对实现部分的引用。

2) 具体抽象部分 (Concrete Abstraction)：抽象部分的具体实现，通常是抽象部分的子类。

3) 实现部分 (Implementor)：定义了实现接口，提供基本的操作。

4) 具体实现部分 (Concrete Implementor)：实现部分的具体实现，通常是实现部分的子类。

(3)特点

1) 解耦抽象与实现：通过引入桥接接口，将抽象部分和实现部分分离，允许它们独立地变化和扩展。

2) 提高灵活性：可以动态地切换不同的实现部分，实现更灵活的系统行为。

3) 处理多个变化维度：将每个变化维度抽象成抽象部分和实现部分，灵活地组合不同的变化维度。

(4)示例代码

下面是一个简单的 C++ 代码示例来说明桥接设计模式的实现：

```
#include <iostream>

// 实现部分
class Implementor {
public:
    virtual void operationImpl() = 0;
};

// 具体实现部分 A
class ConcreteImplementorA : public Implementor {
public:
    void operationImpl() override {
        std::cout << "具体实现部分 A 的操作" << std::endl;
    }
};

// 具体实现部分 B
class ConcreteImplementorB : public Implementor {
public:
    void operationImpl() override {
        std::cout << "具体实现部分 B 的操作" << std::endl;
    }
};

// 抽象部分
class Abstraction {
protected:
    Implementor* implementor;
public:
    Abstraction(Implementor* implementor) : implementor(implementor) {}
    virtual void operation() = 0;
};

// 具体抽象部分 A
class ConcreteAbstractionA : public Abstraction {
public:
    ConcreteAbstractionA(Implementor* implementor) :
        Abstraction(implementor) {}
    void operation() override {
        std::cout << "具体抽象部分 A 的操作" << std::endl;
        implementor->operationImpl();
    }
};

// 具体抽象部分 B
class ConcreteAbstractionB : public Abstraction {
public:
```

```

ConcreteAbstractionB(Implementor* implementor) :
Abstraction(implementor) {}
void operation() override {
    std::cout << "具体抽象部分 B 的操作" << std::endl;
    implementor->operationImpl();}
int main() {
    Implementor* implementorA = new ConcreteImplementorA();
    Abstraction* abstractionA = new ConcreteAbstractionA(implementorA);
    abstractionA->operation();
    Implementor* implementorB = new ConcreteImplementorB();
    Abstraction* abstractionB = new ConcreteAbstractionB(implementorB);
    abstractionB->operation();
    delete abstractionB;
    delete implementorB;
    delete abstractionA;
    delete implementorA;
    return 0;
}

```

在上面的示例中：

Implementor 是实现部分，定义了实现的接口。

ConcreteImplementorA 和 ConcreteImplementorB 是具体实现部分，分别实现了 Implementor 的接口。

Abstraction 是抽象部分，定义了抽象接口，并包含一个对实现部分的引用。

ConcreteAbstractionA 和 ConcreteAbstractionB 是具体抽象部分，继承自 Abstraction，实现了抽象部分的接口。

main 函数中，我们创建了具体实现部 ConcreteImplementorA 和 ConcreteImplementorB，然后用具体抽象部 ConcreteAbstractionA 和 ConcreteAbstractionB 对其进行桥接。通过调用抽象部分的 operation 方法，实现了对实现部分的透明调用。

(5)应用场景

1) 解耦抽象与实现：当需要在抽象部分和实现部分之间进行解耦，使它们可以独立地变化时，可以使用桥接模式。

2) 运行时切换实现：当需要在运行时切换不同的实现部分时，可以使用桥接模式。通过更换具体实现部分，可以动态改变系统的行为。

3) 处理多个变化维度：当一个类存在多个变化维度时，可以使用桥接模式。通过将每个变化维度抽象成抽象部分和实现部分，可以灵活地组合不同的变化维度。

4) 独立扩展抽象与实现：当需要对抽象部分和实现部分进行独立扩展时，可以使用桥接模式。通过增加新的具体实现部分或具体抽象部分，可以扩展系统的功能。

12. 外观设计模式的理解

(1)核心思想

外观模式提供了一个高级别的接口，将多个底层接口组合成一个更简单的接口，以供客户端使用。这有助于隔离客户端代码和复杂子系统之间的耦合关系。

(2)主要组成部分

1) 外观类 (Facade) : 提供一个简单的接口，封装了对一个或多个子系统的复杂操作。它知道如何与子系统协调工作以完成特定任务。

2) 子系统类 (Subsystems) : 这些类包含了系统的具体实现，但对客户端是不可见的。客户端通过外观类来与这些子系统进行交互。

特点

1) 简化接口 : 外观模式提供了一个简化的接口，隐藏了子系统的复杂性，使客户端更容易使用。

2) 降低耦合 : 客户端代码与子系统之间的依赖性降低，因为客户端只需与外观对象交互，而不需要直接与多个子系统交互。

3) 改进可维护性 : 外观模式将系统的组件组织得更好，有助于维护和修改系统的不同部分，而不会影响客户端。

(3)示例

假设你正在开发一个多媒体播放器应用程序，其中包含音频播放、视频播放和字幕显示等多个子系统。你可以使用外观模式来创建一个多媒体播放器的外观类，将这些子系统组合起来，并提供一个简化的接口供客户端使用：

```
#include <iostream>
#include <string>
// 外观类 - 多媒体播放器
class MediaPlayer {
private:
    AudioPlayer audioPlayer;
    VideoPlayer videoPlayer;
    SubtitlePlayer subtitlePlayer;
public:
    MediaPlayer() {
        // 初始化子系统
    }
    // 提供一个播放多媒体文件的简化接口
    void playMedia(const std::string& mediaFilePath) {
        audioPlayer.playAudio(mediaFilePath);
        videoPlayer.playVideo(mediaFilePath);
        subtitlePlayer.displaySubtitles(mediaFilePath);
    };
// 子系统类 - 音频播放器
class AudioPlayer {
public:
    void playAudio(const std::string& filePath) {
        std::cout << "播放音频 : " << filePath << std::endl;
        // 实际音频播放逻辑
    }
};
```

```
};
// 子系统类 - 视频播放器
class VideoPlayer {
public:
    void playVideo(const std::string& filePath) {
        std::cout << "播放视频：" << filePath << std::endl;
        // 实际视频播放逻辑}};
// 子系统类 - 字幕显示器
class SubtitlePlayer {
public:
    void displaySubtitles(const std::string& filePath) {
        std::cout << "显示字幕：" << filePath << std::endl;
        // 实际字幕显示逻辑}};
int main() {
    MultimediaPlayer player;
    player.playMedia("example.mp4");
    return 0;}

```

在上面的示例中，MultimediaPlayer 类充当外观，隐藏了音频播放、视频播放和字幕显示等子系统的复杂性，提供了一个简化的接口供客户端使用。

(4)应用场景

外观模式通常在以下情况下使用：

- 1) 系统的多个复杂子系统需要与客户端交互：但客户端不需要了解这些子系统的具体工作方式时，可以使用外观模式简化接口。
- 2) 将系统的组件分层：使客户端可以轻松访问高级别的接口，而不必处理底层的复杂性。
- 3) 隔离客户端代码和底层子系统：提高代码的可维护性和可扩展性。外观模式可以帮助简化大型系统的设计和开发，降低系统的复杂性。

13. 享元设计模式

(1)核心思想

享元设计模式是一种结构型设计模式，旨在通过共享对象来最小化内存使用和提高性能。该模式适用于需要创建大量细粒度对象的情况，通过共享相同的对象实例来减少内存占用。

(2)内部状态与外部状态

- 1) 内部状态 (Intrinsic State) : 对象共享的部分，不会随着外部环境的改变而改变。
- 2) 外部状态 (Extrinsic State) : 对象特定的部分，会随着外部环境的改变而改变。

(3)结构

- 1) 抽象享元类 (**Flyweight**) : 定义享元对象的接口。
- 2) 具体享元类 (**ConcreteFlyweight**) : 实现抽象享元接口，并定义内部状态。
- 3) 享元工厂类 (**FlyweightFactory**) : 用于创建和管理享元对象，确保享元对象的共享。

(4)示例

以下是一个简单的 C++ 代码示例，展示了享元设计模式的实现：

```
#include <iostream>
#include <unordered_map>
// 抽象享元类
class Flyweight {
public:
    virtual void operation(int extrinsicState) = 0;};
// 具体享元类
class ConcreteFlyweight : public Flyweight {
public:
    void operation(int extrinsicState) override {
        std::cout << "具体享元对象，外部状态为：" << extrinsicState <<
std::endl;};};
// 享元工厂类
class FlyweightFactory {
private:
    std::unordered_map<int, Flyweight*> flyweights;
public:
    Flyweight* getFlyweight(int key) {
        if (flyweights.find(key) == flyweights.end()) {
            flyweights[key] = new ConcreteFlyweight();}
        return flyweights[key];}
    ~FlyweightFactory() {
        for (auto& pair : flyweights) {
            delete pair.second;}}};
int main() {
    FlyweightFactory factory;
    Flyweight* flyweight1 = factory.getFlyweight(1);
    flyweight1->operation(100);
    Flyweight* flyweight2 = factory.getFlyweight(2);
    flyweight2->operation(200);
    Flyweight* flyweight3 = factory.getFlyweight(1);
    flyweight3->operation(300);
    return 0;}
```

(6)解释

在上述代码中：

Flyweight 是抽象享元类，定义了享元对象的操作方法。

ConcreteFlyweight 是具体享元类，实现了抽象享元类的方法。

FlyweightFactory 是享元工厂类，用于创建和管理享元对象。

在 main 函数中，通过享元工厂对象 factory 获取具体享元对象，并调用其操作方法来展示对象的外部状态。

(7)应用场景

1) 大量细粒度对象：当需要创建大量细粒度的对象，并且这些对象之间有很多共享的部分时，可以使用享元模式来共享相同的对象实例，减少内存占用。

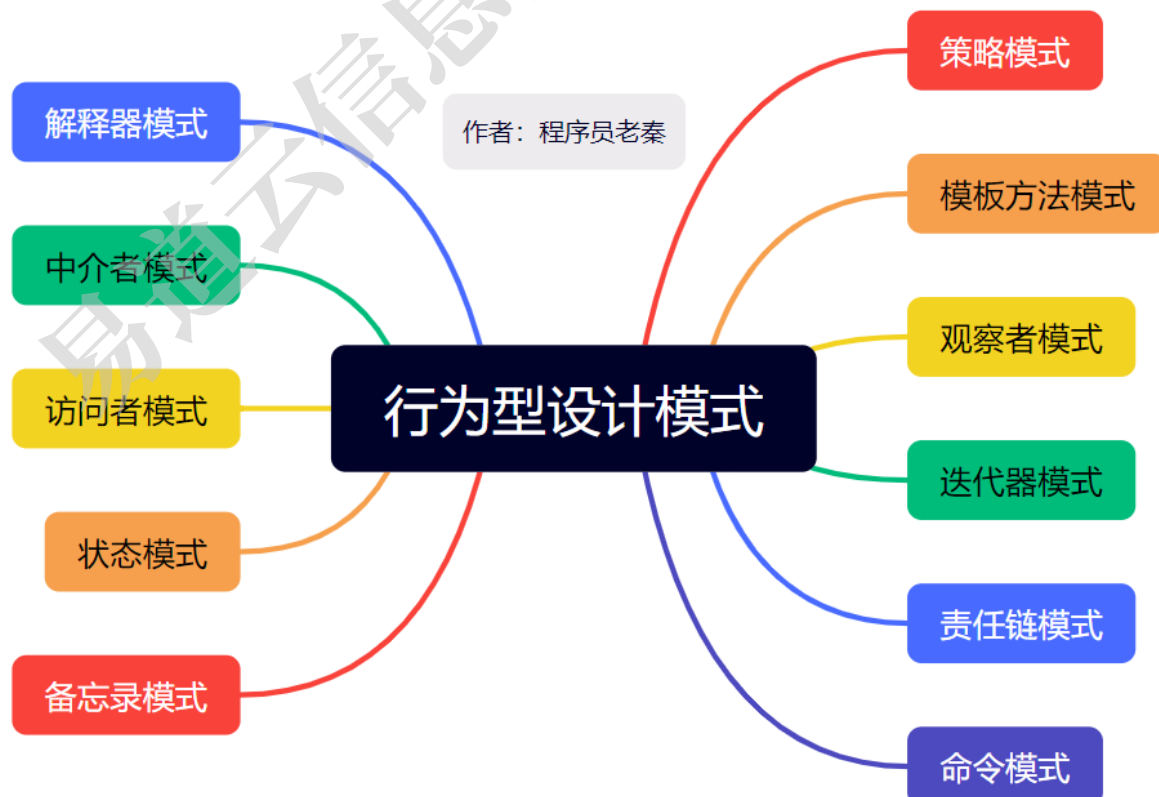
2) 缓存：在需要频繁访问的数据缓存中，使用享元模式可以减少重复创建对象的开销，提高访问速度。

3) 文字处理器：在文字处理器中，每个字符都可以看作一个享元对象，通过共享相同的字符对象实例，可以减少内存使用量。

4) 游戏中的角色和地图：在游戏中，角色和地图等对象可能会有多个实例，使用享元模式可以共享相同的对象实例，提高游戏性能。

行为型设计模式 (Behavioral Patterns)

这些模式关注于对象的行为问题，是**对在不同的对象之间划分责任和算法的抽象化**；不仅仅关注类和对象的结构，而且重点关注它们之间的相互作用。常见的行为型模式包括：



1) 策略模式 (Strategy Pattern)：定义一系列的算法，把它们一个个封装起来，并且使它们可**相互替换**。

2) 模板方法模式 (Template Method Pattern) : 定义一个操作中的算法的骨架, 而将一些步骤延迟到子类中, 使得子类可以不改变一个算法的结构即可重定义该算法的某些特定步骤。

3) 观察者模式 (Observer Pattern) : 定义对象间的一种一对多的依赖关系, 当一个对象的状态发生变化时, 所有依赖于它的对象都得到通知并被自动更新。

4) 迭代器模式 (Iterator Pattern) : 提供一种方法访问一个容器对象中各个元素, 而又不需要暴露该对象的内部细节。

5) 责任链模式 (Chain of Responsibility Pattern) : 使多个对象都有机会处理请求, 从而避免请求的发送者和接收者之间的耦合关系。

6) 命令模式 (Command Pattern) : 将一个请求封装为一个对象, 从而使你可用不同的请求对客户进行参数化。

7) 备忘录模式 (Memento Pattern) : 在不破坏封装性的前提下, 捕获一个对象的内部状态, 并在该对象之外保存这个状态。

8) 状态模式 (State Pattern) : 允许一个对象在其内部状态发生改变时改变其行为。

9) 访问者模式 (Visitor Pattern) : 表示一个作用于某对象结构中的各元素的操作, 使你可以在不改变各元素类的前提下定义作用于这些元素的新操作。

10) 中介者模式 (Mediator Pattern) : 用一个中介对象来封装一系列的对象交互, 使对象不需要显式地相互引用, 从而使其耦合松散。

11) 解释器模式 (Interpreter Pattern) : 给定一个语言, 定义它的文法表示, 并定义一个解释器, 该解释器使用该表示来解释语言中的句子。

14. 解释器设计模式

(1) 核心思想

解释器设计模式 (Interpreter Design Pattern) 是一种行为型设计模式, 它定义了一种语言 and 解释器的结构, 用于解释和执行特定的语法规则。该模式将语言的表达式表示为一个抽象语法树, 并定义了一组解释器来解释和执行这些语法规则。

(2) 主要组成部分

1) 抽象表达式: 一个抽象类或接口, 定义了解释器的抽象方法。

2) 终结符表达式: 表示语言中的终结符, 实现了抽象表达式的解释方法。

3) 非终结符表达式: 表示语言中的非终结符, 通常由多个终结符表达式组成, 并实现了抽象表达式的解释方法。

4) 上下文: 保存了解释器的全局信息, 并提供给解释器访问和操作的接口。

(3) 示例代码

下面是一个简单的 C++ 代码示例来说明解释器设计模式的实现:

```
#include <iostream>
#include <unordered_map>
// 上下文类
class Context {
private:
```

```

    std::unordered_map<std::string, bool> variables;
public:
    bool getVariable(const std::string& name) {
        return variables[name];}
    void setVariable(const std::string& name, bool value) {
        variables[name] = value;}};
// 抽象表达式类
class Expression {
public:
    virtual bool interpret(Context& context) = 0;};
// 终结符表达式类
class TerminalExpression : public Expression {
private:
    std::string variable;
public:
    TerminalExpression(const std::string& variable) : variable(variable) {}
    bool interpret(Context& context) override {
        return context.getVariable(variable);}};
// 非终结符表达式类
class AndExpression : public Expression {
private:
    Expression* expression1;
    Expression* expression2;
public:
    AndExpression(Expression* expression1, Expression* expression2)
        : expression1(expression1), expression2(expression2) {}
    bool interpret(Context& context) override {
        return expression1->interpret(context) && expression2-
>interpret(context);}};
// 非终结符表达式类
class OrExpression : public Expression {
private:
    Expression* expression1;
    Expression* expression2;
public:
    OrExpression(Expression* expression1, Expression* expression2)
        : expression1(expression1), expression2(expression2) {}
    bool interpret(Context& context) override {
        return expression1->interpret(context) || expression2-
>interpret(context);}};
int main() {
    Context context;
    context.setVariable("A", true);

```

```

context.setVariable("B", false);
Expression* expression1 = new TerminalExpression("A");
Expression* expression2 = new TerminalExpression("B");
// A AND B
Expression* andExpression = new AndExpression(expression1,
expression2);
bool result = andExpression->interpret(context);
std::cout << "A AND B = " << result << std::endl;
// A OR B
Expression* orExpression = new OrExpression(expression1, expression2);
result = orExpression->interpret(context);
std::cout << "A OR B = " << result << std::endl;
delete expression1;
delete expression2;
delete andExpression;
delete orExpression;
return 0;}

```

(4)应用场景

1) 特定语言的解释和执行：当需要解释和执行一种特定语言或表达式时，例如解析和执行自定义查询语言、配置文件或规则引擎时。

2) 扩展语言的语法规则：当需要灵活地扩展语言的语法规则时，解释器模式通过定义抽象表达式、终结符表达式和非终结符表达式，使得我们可以轻松地添加新的语法规则，并解释和执行这些规则。

3) 解析和执行复杂逻辑：当需要对复杂的逻辑进行解析和执行时，解释器模式可以将复杂的逻辑分解为简单的语法规则，并通过组合和嵌套表达式来实现逻辑的解释和执行。

15. 中介者设计模式

中介者设计模式是一种行为型设计模式，用于降低多个对象之间的耦合性。**中介者模式通过引入一个中介者对象，将对象间的交互集中管理，从而避免了对象之间的直接通信。**

(1)主要角色

中介者 (Mediator)：定义了与各同事对象通信的接口。

具体中介者 (Concrete Mediator)：实现中介者接口，协调各同事对象之间的通信。

同事类 (Colleague)：每个同事对象知道自己的中介者对象，并通过中介者与其他同事对象通信。

(2)示例代码

下面是一个简单的 C++ 代码示例来说明中介者设计模式的实现：

```
#include <iostream>
```

```

#include <string>
// 抽象中介者类
class Colleague;
class Mediator {
public:
    virtual void sendMessage(const std::string& message, Colleague*
colleague) = 0;};
// 具体中介者类
class ConcreteMediator : public Mediator {
private:
    Colleague* colleague1;
    Colleague* colleague2;
public:
    void setColleague1(Colleague* colleague) {
        colleague1 = colleague;}
    void setColleague2(Colleague* colleague) {
        colleague2 = colleague;}
    void sendMessage(const std::string& message, Colleague* colleague)
override {
        if (colleague == colleague1) {
            colleague2->receiveMessage(message);}
        else if (colleague == colleague2) {
            colleague1->receiveMessage(message);}}};
// 同事类
class Colleague {
protected:
    Mediator* mediator;
public:
    void setMediator(Mediator* mediator) {
        this->mediator = mediator;}
    virtual void sendMessage(const std::string& message) = 0;
    virtual void receiveMessage(const std::string& message) = 0;};
// 具体同事类 A
class ConcreteColleagueA : public Colleague {
public:
    void sendMessage(const std::string& message) override {
        mediator->sendMessage(message, this);}
    void receiveMessage(const std::string& message) override {
        std::cout << "同事类 A 收到消息 : " << message << std::endl;}};
// 具体同事类 B
class ConcreteColleagueB : public Colleague {
public:
    void sendMessage(const std::string& message) override {

```

```

        mediator->sendMessage(message, this);}
    void receiveMessage(const std::string& message) override {
        std::cout << "同事类 B 收到消息 : " << message << std::endl;}};
int main() {
    Mediator* mediator = new ConcreteMediator();
    Colleague* colleague1 = new ConcreteColleagueA();
    Colleague* colleague2 = new ConcreteColleagueB();
    mediator->setColleague1(colleague1);
    mediator->setColleague2(colleague2);
    colleague1->setMediator(mediator);
    colleague2->setMediator(mediator);
    colleague1->sendMessage("Hello, Colleague B!");
    colleague2->sendMessage("Hi, Colleague A!");
    delete mediator;
    delete colleague1;
    delete colleague2;
    return 0;}

```

(3)解释

Mediator : 中介者抽象类，定义了 sendMessage 方法，用于发送消息。

ConcreteMediator : 具体中介者类，维护了两个同事对象的引用，并实现了 sendMessage 方法来协调同事对象之间的通信。

Colleague : 同事抽象类，定义了与中介者进行通信的方法。

ConcreteColleagueA 和 **ConcreteColleagueB** : 具体同事类，实现了 sendMessage 和 receiveMessage 方法，通过中介者发送和接收消息。

在 main 函数中，我们创建了一个具体中介者对象 mediator，以及两个具体同事类对象 colleague1 和 colleague2。通过中介者 mediator 设置同事对象之间的关联，并调用 sendMessage 方法进行通信，显示了中介者模式的工作原理。

(4)应用场景

1) 集中管理和协调对象间的通信 : 当对象间的通信过程需要集中管理和协调时，可以使用中介者模式。中介者模式可以将对象间的交互逻辑集中在中介者中，简化对象之间的通信。

2) 解决对象间的循环依赖 : 当对象间存在循环依赖关系时，可以使用中介者模式。中介者模式可以解决对象间的循环依赖问题，将复杂的关系转化为中介者与各个对象之间的简单关系。

3) 减少对象数量 : 当希望通过一个共享的中介者来减少系统中对象的数量时，可以使用中介者模式。中介者模式将对象间的一对多关系转化为一对一关系，减少了对象之间的直接耦合。

16. 访问者设计模式

访问者设计模式 (Visitor Design Pattern) 是一种行为型设计模式，用于将数据结构与对数据的操作分离。通过访问者模式，可以在不更改数据结构的情况下定义新的操

作。这种模式使得增加新的操作非常容易，且不会影响现有的数据结构。

(1)主要角色

- 1) 访问者 (**Visitor**) : 定义了**对每种具体元素的访问操作**。
- 2) 具体访问者 (**Concrete Visitor**) : 实现了访问者接口，**定义了对具体元素的操作**。
- 3) 元素 (**Element**) : 定义了接受访问者的方法 (accept)，该方法将访问请求传递给访问者。
- 4) 具体元素 (**Concrete Element**) : 实现了元素接口，**定义了具体元素的行为**。
- 5) 对象结构 (**Object Structure**) : 包含元素的集合，并提供遍历元素的方法。

(2)示例代码

下面是一个简单的 C++ 代码示例来说明访问者设计模式的实现：

```
#include <iostream>
#include <vector>
#include <memory>
// 前向声明
class ConcreteElementA;
class ConcreteElementB;
// 访问者接口
class Visitor {
public:
    virtual void visit(ConcreteElementA* element) = 0;
    virtual void visit(ConcreteElementB* element) = 0;};
// 具体访问者 A
class ConcreteVisitorA : public Visitor {
public:
    void visit(ConcreteElementA* element) override {
        std::cout << "具体访问者 A 访问具体元素 A，执行操作 A" << std::endl;}
    void visit(ConcreteElementB* element) override {
        std::cout << "具体访问者 A 访问具体元素 B，执行操作 B" << std::endl;}};
// 具体访问者 B
class ConcreteVisitorB : public Visitor {
public:
    void visit(ConcreteElementA* element) override {
        std::cout << "具体访问者 B 访问具体元素 A，执行操作 C" << std::endl;}
    void visit(ConcreteElementB* element) override {
        std::cout << "具体访问者 B 访问具体元素 B，执行操作 D" << std::endl;}};
// 元素接口
class Element {
public:
    virtual void accept(Visitor* visitor) = 0;};
```

```

// 具体元素 A
class ConcreteElementA : public Element {
public:
    void accept(Visitor* visitor) override {
        visitor->visit(this);}
};

// 具体元素 B
class ConcreteElementB : public Element {
public:
    void accept(Visitor* visitor) override {
        visitor->visit(this);}
};

// 对象结构
class ObjectStructure {
private:
    std::vector<std::unique_ptr<Element>> elements;
public:
    void addElement(std::unique_ptr<Element> element) {
        elements.push_back(std::move(element));}
    void accept(Visitor* visitor) {
        for (const auto& element : elements) {
            element->accept(visitor);}
};

int main() {
    ObjectStructure objectStructure;
    objectStructure.addElement(std::make_unique<ConcreteElementA>());
    objectStructure.addElement(std::make_unique<ConcreteElementB>());
    ConcreteVisitorA visitorA;
    ConcreteVisitorB visitorB;
    objectStructure.accept(&visitorA);
    objectStructure.accept(&visitorB);
    return 0;}

```

(3)解释

Visitor : 访问者接口，定义了对具体元素的访问方法。

ConcreteVisitorA 和 **ConcreteVisitorB** : 具体访问者，实现了访问者接口，分别定义了对具体元素的不同操作。

Element : 元素接口，定义了接受访问者的方法。

ConcreteElementA 和 **ConcreteElementB** : 具体元素，实现了元素接口，并调用访问者的访问方法。

ObjectStructure : 对象结构，维护元素集合，并提供接受访问者的方法。

在 main 函数中，创建了一个对象结构 objectStructure，并向其中添加了具体元素 ConcreteElementA 和 ConcreteElementB。然后创建了具体访问者 ConcreteVisitorA 和 ConcreteVisitorB，并通过调用 objectStructure 的 accept 方法，实现对元素的遍历和操作的执行。

(4)应用场景

1) 处理复杂的对象结构：当存在一个复杂的对象结构，并需要对该结构进行不同的操作时，可以使用访问者模式。通过将操作封装在具体访问者中，可以实现对不同操作的灵活扩展。

2) 避免在元素类中添加新的操作：当不希望元素类中添加新的操作，或者需要对元素进行操作的算法具有不同的变化时，可以使用访问者模式。通过将操作封装在具体访问者中，可以在不更改元素类的情况下定义新的操作。

3) 组合访问和频繁变化的操作：当需要对元素进行组合访问，且元素类的结构稳定，但操作算法的变化频繁时，可以使用访问者模式。通过访问者模式，可以将不同的操作逻辑抽离出来，避免对元素类的修改。

17. 状态设计模式

(1)核心思想

状态模式通过将对象的行为封装在不同的状态类中，并通过上下文类管理状态之间的转换，从而实现对象在不同状态下表现出不同的行为。这样可以避免在对象的行为方法中使用大量的条件语句来处理不同的状态。

(2)组成部分

- 1) 上下文 (Context)：维护一个当前状态的引用，并将请求委托给当前状态处理。
- 2) 状态 (State)：定义了状态的接口，声明了一个或多个在特定状态下的方法。
- 3) 具体状态 (Concrete State)：实现了状态接口中的方法，定义了特定状态下的行为。

(3)示例代码

下面是一个简单的 C++ 代码示例来说明状态设计模式的实现：

```
#include <iostream>
class Context;
// 状态抽象类
class State {
public:
    virtual void handle(Context* context) = 0;
    virtual ~State() = default;
};
// 具体状态类 A
class ConcreteStateA : public State {
public:
    void handle(Context* context) override;
};
// 具体状态类 B
class ConcreteStateB : public State {
public:
    void handle(Context* context) override;
};
```



```

// 上下文类
class Context {
private:
    State* currentState;
public:
    Context(State* initialState) : currentState(initialState) {}
    ~Context() { delete currentState; }
    void setState(State* state) {
        delete currentState;
        currentState = state;}
    void request() {
        currentState->handle(this);};
void ConcreteStateA::handle(Context* context) {
    std::cout << "当前状态是 A，执行操作 A，切换到状态 B" << std::endl;
    context->setState(new ConcreteStateB());}
void ConcreteStateB::handle(Context* context) {
    std::cout << "当前状态是 B，执行操作 B，切换到状态 A" << std::endl;
    context->setState(new ConcreteStateA());}
int main() {
    Context* context = new Context(new ConcreteStateA());
    context->request(); // 输出：当前状态是 A，执行操作 A，切换到状态 B
    context->request(); // 输出：当前状态是 B，执行操作 B，切换到状态 A
    delete context;
    return 0;}

```

(4)解释

State：状态抽象类，定义了状态的接口方法 handle。

ConcreteStateA 和 **ConcreteStateB**：具体状态类，分别实现了在不同状态下的行为。

Context：上下文类，拥有一个当前状态的引用，并将请求委托给当前状态处理。

在 main 函数中，创建了一个上下文对象 context，并将初始状态设置为 ConcreteStateA。然后通过调用 context 的 request 方法来触发状态的改变。

(5)应用场景

1) 对象行为依赖状态：当一个对象的行为取决于其内部状态，并且该对象在运行时需要根据状态改变行为时，可以使用状态模式。

2) 操作有多个实现方式：当一个操作具有多个不同的实现方式，每个实现方式对应一个状态时，可以使用状态模式。状态模式可以将不同的操作实现分离到不同的状态类中，避免了使用大量的条件语句。

3) 复杂状态转换：当对象的行为在不同状态下会发生变化，并且状态转换的规则较为复杂时，可以使用状态模式。状态模式将状态转换的逻辑封装在状态类中，使得状态转换的逻辑更加清晰、可维护。

4) 动态切换状态：当一个对象在不同状态下需要执行不同的操作，并且需要动态地

切换状态时，可以使用状态模式。状态模式可以使对象的状态转换更加灵活，可以根据需要随时切换状态。

18. 备忘录设计模式

备忘录设计模式（Memento Design Pattern）是一种行为型设计模式，用于捕获和保存对象的内部状态，以便在需要时能够恢复到之前的状态。它将对象的状态封装在备忘录对象中，同时提供了对备忘录的创建、恢复和管理。

(1) 核心思想

备忘录模式通过将对象的状态封装在备忘录对象中，实现了状态的保存和恢复。通过备忘录模式，可以在不破坏对象封装性的情况下，实现对对象状态的保存和恢复。

(2) 组成部分

原发器（Originator）：需要保存状态的对象，它可以创建备忘录对象，将当前状态保存到备忘录中，或从备忘录中恢复之前的状态。

备忘录（Memento）：用于保存原发器状态的对象，它提供了访问原发器状态的接口，但不允许其他对象修改备忘录的状态。

负责人（Caretaker）：用于管理备忘录对象的对象，它保存了多个备忘录对象，并提供了对备忘录的管理操作，例如保存备忘录、获取备忘录等。

(3) 示例代码

下面是一个简单的 C++ 代码示例来说明备忘录设计模式的实现：

```
#include <iostream>
#include <string>
// 备忘录类
class Memento {
private:
    std::string state;
public:
    Memento(const std::string& state) : state(state) {}
    std::string getState() const {
        return state;
    }
};
// 原发器类
class Originator {
private:
    std::string state;
public:
    void setState(const std::string& state) {
        this->state = state;
    }
    std::string getState() const {
        return state;
    }
    Memento* createMemento() const {
        return new Memento(state);
    }
};
```

```

        void restoreMemento(const Memento* memento) {
            state = memento->getState();};
// 负责人类
class Caretaker {
private:
    Memento* memento;
public:
    void saveMemento(Memento* memento) {
        this->memento = memento;}
    Memento* getMemento() const {
        return memento;};
int main() {
    Originator originator;
    Caretaker caretaker;
    // 设置初始状态
    originator.setState("State 1");
    std::cout << "Current state: " << originator.getState() << std::endl;
    // 保存备忘录
    caretaker.saveMemento(originator.createMemento());
    // 修改状态
    originator.setState("State 2");
    std::cout << "Current state: " << originator.getState() << std::endl;
    // 恢复到之前的状态
    originator.restoreMemento(caretaker.getMemento());
    std::cout << "Current state: " << originator.getState() << std::endl;
    return 0;}

```

(4)解释

Originator：原发器类，具有一个状态 `state`，并提供了设置状态、获取状态、创建备忘录和恢复备忘录的方法。

Memento：备忘录类，保存了原发器的状态，并提供了获取状态的方法。

Caretaker：负责人类，保存了备忘录对象，并提供了保存备忘录和获取备忘录的方法。

在 `main` 函数中，我们创建了原发器对象 `originator` 和负责人对象 `caretaker`。然后，设置原发器的初始状态，并输出当前状态。接着，保存备忘录，并修改原发器的状态，再次输出当前状态。最后，恢复到之前的状态，并输出当前状态，验证备忘录的恢复功能。

(5)应用场景

1) 保存和恢复对象状态：当需要保存和恢复对象的内部状态时，可以使用备忘录模式。例如，在文本编辑器中，可以使用备忘录模式来实现撤销和恢复操作，以便用户可以回到之前的编辑状态。

2) 快照功能：当需要实现快照功能，保存对象的某个状态，以便后续可以随时恢复

到该状态时，可以使用备忘录模式。例如，在游戏中，可以使用备忘录模式来保存游戏的进度，以便玩家可以在需要时恢复到之前的进度。

3) 事务回滚：当需要实现事务的回滚功能，将一系列操作封装成一个事务，并在失败或取消时能够回滚到事务开始前的状态时，可以使用备忘录模式。例如，在数据库管理系统中，可以使用备忘录模式来实现事务的回滚功能。

4) 多级撤销和恢复：当需要实现多级撤销和恢复操作时，可以使用备忘录模式。备忘录模式可以保存多个备忘录对象，以便实现多级撤销和恢复的功能。

19. 策略设计模式

策略设计模式 (Strategy Design Pattern) 是一种行为型设计模式，它定义了一系列算法，并将每个算法封装到具体的策略类中，使得它们可以互相替换。通过使用策略模式，可以在运行时动态地选择算法，而不需要修改调用算法的代码。

(1) 核心思想

策略模式通过将算法的定义和使用分离，使得算法可以在不同的上下文中复用，同时可以在运行时根据需求选择不同的算法实现。

(2) 组成部分

策略 (Strategy)：策略的抽象类或接口，定义了算法的公共接口。

具体策略 (Concrete Strategy)：策略的具体实现类，实现了算法的具体逻辑。

上下文 (Context)：使用策略的对象，它持有一个策略对象，并在需要时调用策略的方法。

(3) 示例代码

下面是一个简单的 C++ 代码示例来说明策略设计模式的实现：

```
#include <iostream>
// 策略抽象类
class Strategy {
public:
    virtual void execute() = 0;
};
// 具体策略类 A
class ConcreteStrategyA : public Strategy {
public:
    void execute() override {
        std::cout << "使用策略 A 执行算法" << std::endl;
    }
};
// 具体策略类 B
class ConcreteStrategyB : public Strategy {
public:
    void execute() override {
        std::cout << "使用策略 B 执行算法" << std::endl;
    }
};
// 上下文类
class Context {
private:
```

```

    Strategy* strategy;
public:
    void setStrategy(Strategy* strategy) {
        this->strategy = strategy;}
    void executeStrategy() {
        if (strategy) {
            strategy->execute();}}};
int main() {
    Context context;
    // 使用策略 A 执行算法
    ConcreteStrategyA strategyA;
    context.setStrategy(&strategyA);
    context.executeStrategy();
    // 使用策略 B 执行算法
    ConcreteStrategyB strategyB;
    context.setStrategy(&strategyB);
    context.executeStrategy();
    return 0;}

```

在上面的示例中，Strategy 是策略的抽象类，定义了算法的执行接口。ConcreteStrategyA 和 ConcreteStrategyB 是具体策略类，分别实现了不同的算法。Context 是上下文类，用于使用策略执行算法。

在 main 函数中，我们创建了一个上下文对象 context，并通过调用 setStrategy 方法将具体策略对象传给上下文。然后调用 executeStrategy 方法执行算法，实际执行的算法取决于具体策略对象。

(4)应用场景

1) 动态切换算法：当一个系统**需要在不同时间应用不同的算法，并且希望能够灵活地切换算法时，可以使用策略模式**。策略模式允许在运行时动态地选择算法，而不需要修改调用算法的代码。

2) 条件选择算法：当一个系统**需要根据不同的条件选择不同的算法**，并且这些条件可能会发生变化时，可以使用策略模式。策略模式可以通过定义不同的具体策略类来适应不同的条件，实现算法的灵活选择。

3) 解耦和封装复杂算法：当一个系统的算法逻辑复杂，需要**将不同的算法进行解耦和封装时**，可以使用策略模式。策略模式将算法封装在具体策略类中，使得每个策略类只需要关注自己的算法逻辑，提高了代码的可读性和可维护性。

(5)补充说明

在上述示例中，我们可以在 Context 类中添加一个方法 void changeStrategy(Strategy* newStrategy)，用于在运行时切换策略。这样，在程序运行过程中可以动态地改变算法的行为，而不需要修改 Context 类的代码。

另外，策略模式还可以与工厂模式结合使用，通过工厂类来创建具体策略对象，并将其传递给上下文类。这样可以进一步降低上下文类对具体策略类的依赖性，增加系统的灵活性和可拓展性。

20. 模板方法设计模式

模板方法设计模式 (Template Method Pattern) 是一种行为型设计模式，用于定义一个算法的骨架，而将一些步骤的具体实现延迟到子类中。这种模式允许在不改变算法结构的情况下，通过子类重写或扩展某些步骤来定制算法的行为。

(1) 核心思想

模板方法模式将一个算法的步骤模板化，定义在一个抽象的基类中，其中一些步骤由抽象方法或默认方法实现，而其他步骤则由具体子类实现。

(2) 主要组成部分

模板类 (Abstract Class)：定义了算法的骨架，通常包含一个或多个抽象方法，代表算法中的不同步骤。同时，也可以包含具体的方法来处理一些通用的逻辑。

具体子类 (Concrete Subclasses)：继承自模板类，实现模板中的抽象方法，以完成特定步骤的具体实现。每个具体子类可以定制算法的不同部分。

(3) 特点

定义了一个算法的框架，具体步骤由子类实现。

遵循开闭原则，即允许在不修改模板代码的情况下扩展和变化算法的某些部分。

促使代码重用，将通用行为提取到模板方法中。

(4) 示例

假设你正在开发一个烹饪应用程序，其中有不同的食谱，如制作咖啡和茶。你可以使用模板方法模式来定义一个通用的烹饪算法：

```
#include <iostream>
// 模板类 - 食谱
class Recipe {
public:
    // 模板方法，定义烹饪算法的骨架
    void cook() {
        boilWater();
        brew();
        pourInCup();
        if (customerWantsCondiments()) {
            addCondiments();
        }
    }
    // 抽象方法，由子类实现
    virtual void brew() = 0;
    virtual void addCondiments() = 0;
    // 钩子方法，子类可以选择性地覆盖
    virtual bool customerWantsCondiments() {
        return true;
    }
    // 具体方法，通用逻辑
    void boilWater() {
```

```

        std::cout << "烧开水" << std::endl;}
    void pourInCup() {
        std::cout << "倒入杯中" << std::endl;}};
// 具体子类 - 制作咖啡
class CoffeeRecipe : public Recipe {
public:
    void brew() override {
        std::cout << "冲泡咖啡" << std::endl;}
    void addCondiments() override {
        std::cout << "加入糖和牛奶" << std::endl;}};
// 具体子类 - 制作茶
class TeaRecipe : public Recipe {
public:
    void brew() override {
        std::cout << "浸泡茶叶" << std::endl;}
    void addCondiments() override {
        std::cout << "加入柠檬" << std::endl;}};
int main() {
    Recipe* coffee = new CoffeeRecipe();
    coffee->cook();
    Recipe* tea = new TeaRecipe();
    tea->cook();
    delete coffee;
    delete tea;
    return 0;}

```

在上面的示例中，Recipe 类定义了一个通用的烹饪算法框架，而具体的烹饪步骤由子类（CoffeeRecipe 和 TeaRecipe）来实现。这使得你可以轻松添加新的食谱，而不必更改通用的烹饪算法。

(5)应用场景

模板方法模式通常在以下情况下使用：

- 1) 有通用的算法骨架：但不同部分的具体实现可能会变化时。
- 2) 确保一致性：当你想要确保一些步骤在整个算法中保持一致性，而其他步骤由子类提供不同实现时。
- 3) 子类定制算法：当你希望通过子类来定制算法的某些部分，同时保留算法的整体结构。
- 4) 遵循开闭原则：当你需要遵循开闭原则，以便在不修改现有代码的情况下添加新的算法变种。

21. 观察者设计模式

观察者设计模式（Observer Design Pattern）是一种行为型设计模式，用于建立对象之间的一对多依赖关系，当一个对象的状态发生变化时，所有依赖它的对象都会得到通知并自动更新。

(1)主要组成部分

主题 (Subject) : 被观察的对象, 它维护一个观察者列表, 并提供添加、删除和通知观察者的方法。

观察者 (Observer) : 一个抽象类或接口, 定义了一个或多个接收主题通知的方法。

具体观察者 (Concrete Observer) : 观察者的具体实现类, 实现了接收通知并做出相应处理的方法。

(2)特点

实现了对象之间的松耦合。

可以动态地添加和移除观察者。

观察者和主题之间的交互通过接口进行, 增加了系统的灵活性和可扩展性。

(3)示例

下面是一个简单的 C++ 代码示例来说明观察者设计模式的实现：

```
#include <iostream>
#include <vector>
class Observer;
// 主题抽象类
class Subject {
public:
    virtual void attach(Observer* observer) = 0;
    virtual void detach(Observer* observer) = 0;
    virtual void notify() = 0;};
// 观察者抽象类
class Observer {
public:
    virtual void update() = 0;};
// 具体观察者类 A
class ConcreteObserverA : public Observer {
public:
    void update() override {
        std::cout << "具体观察者 A 收到通知并作出响应" << std::endl;};
// 具体观察者类 B
class ConcreteObserverB : public Observer {
public:
    void update() override {
        std::cout << "具体观察者 B 收到通知并作出响应" << std::endl;};
// 具体主题类
class ConcreteSubject : public Subject {
private:
    std::vector<Observer*> observers;
```



```

public:
    void attach(Observer* observer) override {
        observers.push_back(observer);
    }
    void detach(Observer* observer) override {
        for (auto it = observers.begin(); it != observers.end(); ++it) {
            if (*it == observer) {
                observers.erase(it);
                break; } } }
    void notify() override {
        for (auto observer : observers) {
            observer->update(); } } };

int main() {
    ConcreteSubject subject;
    Observer* observerA = new ConcreteObserverA();
    Observer* observerB = new ConcreteObserverB();
    subject.attach(observerA);
    subject.attach(observerB);
    subject.notify(); // 输出：具体观察者 A 收到通知并作出响应，具体观察者 B 收到通知并作出响应
    subject.detach(observerA);
    subject.notify(); // 输出：具体观察者 B 收到通知并作出响应
    delete observerA;
    delete observerB;
    return 0; }

```

在上面的示例中：

Subject 是主题的抽象类，定义了添加、删除和通知观察者的方法。

Observer 是观察者的抽象类，定义了接收通知的方法。

ConcreteObserverA 和 ConcreteObserverB 是具体观察者类，分别实现了接收通知并作出响应的方法。

ConcreteSubject 是具体主题类，维护了一个观察者列表，并在状态变化时通知观察者。

在 main 函数中，我们创建了一个具体主题对象 subject，并创建了两个具体观察者对象 observerA 和 observerB。然后通过调用 subject 的 attach 方法将观察者添加到观察者列表中，再调用 subject 的 notify 方法发送通知。观察者收到通知后会作出相应的响应。

(4)应用场景

1) 对象状态改变时通知其他对象：当一个对象的改变需要通知其他对象，并且不希望这些对象耦合在一起时，可以使用观察者模式。观察者模式可以将主题和观察者解耦，使它们可以独立地进行扩展和维护。

2) 多个对象依赖一个对象的状态：当一个对象的状态改变需要影响其他多个对象，并且这些对象的数量和关系可能动态变化时，可以使用观察者模式。观察者模式可以方便

地管理和维护多个观察者对象，实现对象之间的松耦合。

3) 触发一系列操作：当一个对象需要在特定情况下通知其他对象并触发一系列操作时，可以使用观察者模式。观察者模式可以将通知和操作封装在具体观察者中，实现更加灵活的处理。

4) 动态交互：当一个对象需要与多个对象进行交互，并且对交互的顺序和方式有特定要求时，可以使用观察者模式。观察者模式可以通过定义不同的具体观察者，灵活地管理对象之间的交互。

22. 迭代器设计模式

迭代器设计模式（Iterator Design Pattern）是一种行为型设计模式，用于提供一种统一的方式来访问集合对象中的元素，而无需暴露集合的内部表示。

(1) 主要组成部分

1) 迭代器接口 (Iterator)：定义了访问和遍历集合元素的方法。

2) 具体迭代器 (ConcreteIterator)：实现了迭代器接口中的方法，并维护了一个指向集合中当前元素的指针。

3) 聚合接口 (Aggregate)：定义了获取迭代器的方法。

4) 具体聚合 (ConcreteAggregate)：实现了聚合接口中的方法，并返回一个具体迭代器的实例。

(2) 特点

提供一种统一的方式来访问集合对象中的元素。

隐藏集合的内部表示。

支持多种遍历方式或遍历顺序。

(3) 示例

下面是一个简单的 C++ 代码示例来说明迭代器设计模式的实现：

```
#include <iostream>
#include <vector>
// 迭代器接口
template<class T>
class Iterator {
public:
    virtual bool hasNext() = 0;
    virtual T next() = 0;
};
// 具体迭代器
template<class T>
class ConcreteIterator : public Iterator<T> {
private:
    std::vector<T>& collection;
    int index = 0;
public:
    ConcreteIterator(std::vector<T>& collection) : collection(collection) {}
```

```

    bool hasNext() override {
        return index < collection.size();
    }
    T next() override {
        return collection[index++];
    }
// 聚合接口
template<class T>
class Aggregate {
public:
    virtual Iterator<T>* createIterator() = 0;
// 具体聚合
template<class T>
class ConcreteAggregate : public Aggregate<T> {
private:
    std::vector<T> collection;
public:
    void add(T item) {
        collection.push_back(item);
    }
    Iterator<T>* createIterator() override {
        return new ConcreteIterator<T>(collection);
    }
}
int main() {
    ConcreteAggregate<int> aggregate;
    aggregate.add(1);
    aggregate.add(2);
    aggregate.add(3);
    Iterator<int>* iterator = aggregate.createIterator();
    while (iterator->hasNext()) {
        std::cout << iterator->next() << " ";
    }
    delete iterator;
    return 0;
}

```

在上述示例中：

Iterator 类表示迭代器接口，定义了访问和遍历集合元素的方法。

ConcreteIterator 类表示具体迭代器，实现了迭代器接口，并维护了一个指向集合中当前元素的指针。

Aggregate 类表示聚合接口，定义了获取迭代器的方法。

ConcreteAggregate 类表示具体聚合，实现了聚合接口，并返回一个具体迭代器的实例。

在 main 函数中，我们创建了具体聚合对象 aggregate 并添加了一些元素。然后，我们通过调用 createIterator 方法获取迭代器，并使用迭代器遍历集合中的元素并打印。

(4)应用场景

迭代器设计模式的应用场景包括但不限于以下情况：

1) 统一访问和遍历集合对象中的元素：提供了一种统一的方式来访问集合对象中的元素，而无需关注集合的内部表示和实现方式。

2) 隐藏集合对象的内部结构：提供一种安全的方式来遍历集合中的元素，使得外部代码无法直接访问集合的内部结构。

3) 不同类型集合对象的统一遍历：为每种类型的集合对象提供一个具体的迭代器实现，使得不同类型的集合对象都可以通过相同的方式进行遍历。

4) 支持多种遍历方式或遍历顺序：可以为同一个集合对象提供多个不同的迭代器实现，每个迭代器都可以以不同的方式或顺序遍历集合。

23. 责任链设计模式

责任链设计模式 (Chain of Responsibility Design Pattern) 是一种行为型设计模式，用于将请求的发送者和接收者解耦，并使多个对象都有机会处理该请求。请求沿着对象链进行传递，直到有一个对象能够处理它为止。

(1) 核心思想

责任链模式将请求发送者和接收者解耦，每个处理器都有一个指向下一个处理器的引用。当请求到达处理器时，它可以选择处理请求并结束，或者将请求传递给下一个处理器。这样，请求会依次在处理器链中传递，直到有一个处理器能够处理它。

(2) 主要组成部分

抽象处理器 (Handler)：定义了处理请求的接口和一个指向下一个处理器的引用。

具体处理器 (Concrete Handler)：实现了处理请求的具体逻辑。

客户端 (Client)：负责创建处理器链并发送请求。

(3) 示例

下面是一个简单的 C++ 代码示例来说明责任链设计模式的实现：

```
#include <iostream>
// 抽象处理器
class Handler {
protected:
    Handler* nextHandler;
public:
    Handler() : nextHandler(nullptr) {}
    void setNextHandler(Handler* handler) {
        nextHandler = handler;
    }
    virtual void handleRequest(int request) = 0;
};
// 具体处理器 A
class ConcreteHandlerA : public Handler {
public:
    void handleRequest(int request) override {
        if (request >= 0 && request < 10) {
            std::cout << "具体处理器 A 处理请求：" << request << std::endl;
        }
        else if (nextHandler != nullptr) {
            nextHandler->handleRequest(request);
        }
    }
};
// 具体处理器 B
```

```

class ConcreteHandlerB : public Handler {
public:
    void handleRequest(int request) override {
        if (request >= 10 && request < 20) {
            std::cout << "具体处理器 B 处理请求：" << request << std::endl;
        } else if (nextHandler != nullptr) {
            nextHandler->handleRequest(request);
        }
    }
};

// 具体处理器 C
class ConcreteHandlerC : public Handler {
public:
    void handleRequest(int request) override {
        if (request >= 20 && request < 30) {
            std::cout << "具体处理器 C 处理请求：" << request << std::endl;
        } else if (nextHandler != nullptr) {
            nextHandler->handleRequest(request);
        }
    }
};

int main() {
    Handler* handlerA = new ConcreteHandlerA();
    Handler* handlerB = new ConcreteHandlerB();
    Handler* handlerC = new ConcreteHandlerC();
    handlerA->setNextHandler(handlerB);
    handlerB->setNextHandler(handlerC);
    handlerA->handleRequest(5);
    handlerA->handleRequest(15);
    handlerA->handleRequest(25);
    delete handlerA;
    delete handlerB;
    delete handlerC;
    return 0;
}

```

在上面的示例中：

Handler 是抽象处理器，定义了处理请求的接口和一个指向下一个处理器的引用。

ConcreteHandlerA、ConcreteHandlerB 和 ConcreteHandlerC 是具体处理器，实现了处理请求的具体逻辑。

在 main 函数中，我们创建了具体处理器 handlerA、handlerB 和 handlerC，并通过调用 setNextHandler 方法将它们链接起来形成责任链。然后通过调用 handleRequest 方法来发送请求，并观察请求是如何在责任链中传递和处理的。

(4)应用场景

1) 解耦发送者和接收者：当需要将请求发送者和接收者解耦，并且有多个对象都有机会处理请求时，可以使用责任链模式。通过建立一个处理器链，每个处理器可以选择处理请求或将请求传递给下一个处理器。

2) 动态指定处理请求的对象集合：当希望动态地指定处理请求的对象集合时，可以使用责任链模式。通过动态地调整处理器链，可以实现灵活的请求处理。

3) 顺序处理请求：当需要按顺序处理请求，并且每个处理器都有不同的处理逻辑

时，可以使用责任链模式。

(5)优点

1) 解耦发送者和接收者：责任链模式使得发送者不需要知道请求将由哪个处理器来处理，将请求和处理器解耦。

2) 灵活性和可扩展性：可以动态地调整处理器链，增加或移除处理器，以满足不同的处理需求。

3) 避免紧耦合：将请求发送给处理器链，每个处理器都有机会处理请求，避免了发送者和接收者之间的直接依赖关系。

(6)应用场景

例如，在一个电商网站中，可以使用责任链模式来处理用户的退款请求。假设有三个处理器，分别是财务部门、客服部门和仓储部门。当用户发起退款请求时，请求会依次经过这三个处理器。财务部门负责审核退款申请，客服部门负责与用户沟通并确认退款原因，仓储部门负责处理退货物品。如果某个处理器无法处理该请求，它会将请求传递给下一个处理器，直到有一个处理器能够处理该请求或处理器链结束。

24. 命令设计模式

命令设计模式（Command Design Pattern）是一种行为型设计模式，用于将请求封装成一个对象，从而可以在不同的上下文中使用、传递和操作请求。它将请求的发送者和接收者解耦，使得发送者只需知道如何发送请求，而不需要知道请求是如何被执行和处理的。

(1)核心思想

命令模式将请求封装成一个对象，使得请求可以被参数化、存储、日志记录以及撤销和重做等操作。同时，通过命令对象，可以实现请求的解耦，使发送者和接收者独立变化。

(2)主要组成部分

1) 命令接口（Command）：一个抽象接口，用于声明执行命令的方法。

2) 具体命令（Concrete Command）：命令接口的具体实现，绑定了一个接收者对象和一组操作。

3) 请求者（Invoker）：负责发送命令的对象，持有一个命令对象，并在需要时调用命令对象的执行方法。

4) 接收者（Receiver）：执行命令的对象，实现了具体的操作逻辑，并在接收到命令时执行相应的操作。

(3)示例

下面是一个简单的 C++ 代码示例来说明命令设计模式的实现：

```
#include <iostream>
#include <string>
// 命令接口
class Command {
```

```

public:
    virtual ~Command() {}
    virtual void execute() = 0;};
// 具体命令
class ConcreteCommand : public Command {
private:
    std::string message;
public:
    ConcreteCommand(const std::string& message) : message(message) {}
    void execute() override {
        std::cout << "Executing command: " << message << std::endl;
        // 执行命令的具体操作}};
// 请求者
class Invoker {
private:
    Command* command;
public:
    void setCommand(Command* command) {
        this->command = command;}
    void executeCommand() {
        command->execute();}};
// 接收者
class Receiver {
public:
    void performAction() {
        std::cout << "Performing action" << std::endl;
        // 执行接收者的操作}};
int main() {
    Invoker invoker;
    Receiver receiver;
    Command* command = new ConcreteCommand("Do something");
    // 设置命令
    invoker.setCommand(command);
    // 执行命令
    invoker.executeCommand();
    // 执行接收者的操作
    receiver.performAction();
    delete command;
    return 0;}

```

在上述示例中：

Command 是命令接口，声明了执行命令的方法。

ConcreteCommand 是具体命令，实现了命令接口，并将一个消息绑定在一起。

Invoker 是请求者，持有一个命令对象，并在需要时调用命令对象的执行方法。

Receiver 是接收者，负责实际的操作逻辑。

在 main 函数中，创建了请求者对象 invoker 和接收者对象 receiver，然后创建具体命令对象 command 并将其设置到请求者对象中，最后调用请求者对象的执行命令方法，触发命令的执行。

(4)应用场景

1) 解耦请求发送者和接收者：当需要将请求发送者和接收者解耦，使它们可以独立变化时，可以使用命令模式。

2) 实现撤销、重做、事务等功能：命令模式可以将每个命令封装成一个对象，并保存在历史记录中，以便可以撤销和重做命令，或者将多个命令组合成一个事务进行执行。

3) 日志记录、审计或回放功能：命令模式可以将执行的命令保存下来，以便后续可以进行日志记录、审计或回放操作。

4) 任务调度和命令队列：命令模式可以将命令对象放入队列中，按照顺序执行，从而实现任务调度和命令队列的功能。

(5)优点

1) 解耦发送者和接收者：命令模式将请求封装成一个对象，使得发送者和接收者可以独立变化。

2) 可扩展性：可以很容易地添加新的命令类，而不影响现有代码。

3) 支持撤销和重做：通过将命令保存到历史记录中，可以实现命令的撤销和重做。

4) 支持组合命令：可以将多个命令组合成一个复合命令，从而实现更复杂的操作。